MIT OpenCourseWare
http://ocw.mit.edu

18.085 Computational Science and Engineering I
Fall 2008

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

CHAPTER 1

APPLIED LINEAR ALGEBRA

1.1 FOUR SPECIAL MATRICES

An m by n matrix has m rows and n columns and mn entries. We operate on those rows and columns to solve linear systems $Ax = b$ and eigenvalue problems $Ax = \lambda x$. From inputs A and b (and from software like MATLAB) we get outputs x and λ. A fast stable algorithm is extremely important, and this book includes fast algorithms.

One purpose of matrices is to store information, but another viewpoint is more important for applied mathematics. Often we see the matrix as an "operator." ***A acts on vectors x to produce Ax***. The components of x have a meaning—displacements or pressures or voltages or prices or concentrations. The operator A also has a meaning—in this chapter A takes differences. Then Ax represents pressure differences or voltage drops or price differentials.

Before we turn the problem over to the machine—and also after, when we interpret `A\b` or `eig(A)`—it is the meaning we want, as well as the numbers.

This book begins with **four special families of matrices**—simple and useful, absolutely basic. We look first at the properties of these particular matrices K_n, C_n, T_n, and B_n. (Some properties are obvious, others are hidden.) It is terrific to practice linear algebra by working with genuinely important matrices. Here are K_2, K_3, K_4 in the first family, with -1 and 2 and -1 down the diagonals:

$$K_2 = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \qquad K_3 = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \qquad K_4 = \begin{bmatrix} \mathbf{2} & \mathbf{-1} & 0 & 0 \\ \mathbf{-1} & \mathbf{2} & \mathbf{-1} & 0 \\ 0 & \mathbf{-1} & \mathbf{2} & \mathbf{-1} \\ 0 & 0 & \mathbf{-1} & \mathbf{2} \end{bmatrix}$$

What is significant about K_2 and K_3 and K_4, and eventually the n by n matrix K_n? I will give six answers in the same order that my class gave them—starting with four properties of the K's that you can see immediately.

1. These matrices are **symmetric**. The entry in row i, column j also appears in row j, column i. Thus $K_{ij} = K_{ji}$, on opposite sides of the main diagonal. Symmetry can be expressed by transposing the whole matrix at once: $\boldsymbol{K = K^{\mathrm{T}}}$.

2. The matrices K_n are **sparse**. Most of their entries are zero when n gets large. K_{1000} has a million entries, but only $1000 + 999 + 999$ are nonzero.

3. The nonzeros lie in a "band" around the main diagonal, so each K_n is *banded*. The band has only three diagonals, so these matrices are **tridiagonal**.

Because K is a tridiagonal matrix, $Ku = f$ can be quickly solved. If the unknown vector u has a thousand components, we can find them in a few thousand steps (which take a small fraction of a second). For a full matrix of order $n = 1000$, solving $Ku = f$ would take hundreds of millions of steps. Of course we have to ask if the linear equations have a solution in the first place. That question is coming soon.

4. The matrices have **constant diagonals**. Right away that property wakes up Fourier. It signifies that something is not changing when we move in space or time. The problem is shift-invariant or time-invariant. Coefficients are constant. The tridiagonal matrix is entirely determined by the three numbers $-1, 2, -1$. These are actually "second difference matrices" but my class never says that.

The whole world of Fourier transforms is linked to constant-diagonal matrices. In signal processing, the matrix $D = K/4$ is a "highpass filter." Du picks out the rapidly varying (high frequency) part of a vector u. It gives a *convolution* with $\frac{1}{4}(-1, 2, -1)$. We use these words to call attention to the Fourier part (Chapter 4) of this book.

Mathematicians call K a ***Toeplitz matrix***, and MATLAB uses that name:

The command $K = \mathsf{toeplitz}([\,2\ \ -1\ \ \mathsf{zeros}(1,2)\,])$ **constructs** K_4 **from row 1.**

Actually, Fourier will be happier if we make two small changes in K_n. Insert -1 in the southwest and northeast corners. This completes two diagonals (which circle around). All four diagonals of C_4 wrap around in this "*periodic matrix*" or "*cyclic convolution*" or **circulant matrix**:

Circulant matrix
$$C_4 = \begin{bmatrix} 2 & -1 & 0 & \mathbf{-1} \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ \mathbf{-1} & 0 & -1 & 2 \end{bmatrix} = \mathsf{toeplitz}([\,2\ \ -1\ \ 0\ \ -1\,])\,.$$

This matrix is *singular*. It is *not invertible*. Its determinant is zero. Rather than computing that determinant, it is much better to identify a nonzero vector u that solves $C_4u = 0$. (**If** C_4 **had an inverse, the only solution to** $C_4u = 0$ **would be the zero vector**. We could multiply by C_4^{-1} to find $u = 0$.) For this matrix, the column vector u of all ones (printed as $u = (1, 1, 1, 1)$ with commas) solves $C_4u = 0$.

The columns of C add to the zero column. This vector $u = \mathsf{ones}(4,1)$ is in the *nullspace* of C_4. The nullspace contains all solutions to $Cu = 0$.

Whenever the entries along every row of a matrix add to zero, the matrix is certainly singular. The same all-ones vector u is responsible. Matrix multiplication Cu adds the column vectors and produces zero. The constant vector $u = (1,1,1,1)$ or $u = (c,c,c,c)$ in the nullspace is like the constant C when we integrate a function. In calculus, this "arbitrary constant" is not knowable from the derivative. In linear algebra, the constant in $u = (c,c,c,c)$ is not knowable from $Cu = 0$.

5. All the matrices $K = K_n$ are **invertible**. They are not singular, like C_n. There is a square matrix K^{-1} such that $K^{-1}K = I = \textit{identity matrix}$. And if a square matrix has an inverse on the left, then also $KK^{-1} = I$. This "*inverse matrix*" is also symmetric when K is symmetric. *But K^{-1} is not sparse.*

Invertibility is not easy to decide from a quick look at a matrix. Theoretically, one test is to compute the determinant. There is an inverse except when $\det K = 0$, because the formula for K^{-1} includes a division by $\det K$. But computing the determinant is almost never done in practice! It is a poor way to find $u = K^{-1}f$.

What we actually do is to go ahead with the elimination steps that solve $Ku = f$. Those steps simplify the matrix, to make it triangular. The nonzero pivots on the main diagonal of the triangular matrix show that the original K is invertible. (Important: We don't want or need K^{-1} to find $u = K^{-1}f$. The inverse would be a full matrix, with all positive entries. All we compute is the solution vector u.)

6. The symmetric matrices K_n are **positive definite**. Those words might be new. One goal of Chapter 1 is to explain what this crucial property means (K_4 has it, C_4 doesn't). Allow me to start by contrasting positive definiteness with invertibility, using the words "pivots" and "eigenvalues" that will soon be familiar. *Please notice the Appendix that summarizes linear algebra.*

(***Pivots***) An invertible matrix has n *nonzero* pivots.
A *positive definite* symmetric matrix has n ***positive*** **pivots**.

(***Eigenvalues***) An invertible matrix has n *nonzero* eigenvalues.
A *positive definite* symmetric matrix has n ***positive*** **eigenvalues**.

Positive pivots and eigenvalues are tests for positive definiteness, and C_4 fails those tests because it is singular. Actually C_4 has three positive pivots and eigenvalues, so it almost passes. But its fourth eigenvalue is zero (the matrix is singular). Since no eigenvalue is negative ($\lambda \geq 0$), C_4 is **positive** *semi***definite**.

The pivots appear on the main diagonal in Section 1.3, when solving $Ku = f$ by elimination. The eigenvalues arise in $Kx = \lambda x$. There is also a determinant test for positive definiteness (not just $\det K > 0$). The proper definition of a symmetric positive definite matrix (it is connected to positive energy) will come in Section 1.6.

Changing K_n to T_n

After K_n and C_n, there are two more families of matrices that you need to know. They are symmetric and tridiagonal like the family K_n. But the $(1,1)$ entry in T_n is changed from 2 to 1:

$\mathbf{T_n(1,1) = 1}$ $\qquad T_2 = \begin{bmatrix} \mathbf{1} & -1 \\ -1 & 2 \end{bmatrix}$ and $T_3 = \begin{bmatrix} \mathbf{1} & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$. (1)

That top row (T stands for top) represents a new boundary condition, whose meaning we will soon understand. Right now we use T_3 as a perfect example of elimination. Row operations produce zeros below the diagonal, and the pivots are circled as they are found. **Two elimination steps reduce T to the upper triangular U.**

Step 1. Add row 1 to row 2, which leaves zeros below the first pivot.

Step 2. Add the new row 2 to row 3, which produces U.

$$T = \begin{bmatrix} \textcircled{1} & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \xrightarrow{\text{Step 1}} \begin{bmatrix} \textcircled{1} & -1 & 0 \\ 0 & \textcircled{1} & -1 \\ 0 & -1 & 2 \end{bmatrix} \xrightarrow{\text{Step 2}} \begin{bmatrix} \textcircled{1} & -1 & 0 \\ 0 & \textcircled{1} & -1 \\ 0 & 0 & \textcircled{1} \end{bmatrix} = U\,.$$

All three pivots of T equal 1. We can apply the test for invertibility (three nonzero pivots). T_3 also passes the test for positive definiteness (three *positive* pivots). In fact every T_n in this family is positive definite, with all its pivots equal to 1.

That matrix U has an inverse (which is automatically upper triangular). The exceptional fact for this particular U^{-1} is that *all upper triangular entries are* 1's:

$$U^{-1} = \begin{bmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} = \mathsf{triu}(\mathsf{ones}(3)). \tag{2}$$

This says that the inverse of a 3 by 3 "difference matrix" is a 3 by 3 "**sum matrix**." This neat inverse of U will lead us to the inverse of T in Problem 2. **The product $U^{-1}U$ is the identity matrix I.** U takes differences, and U^{-1} takes sums. Taking differences and then sums will recover the original vector (u_1, u_2, u_3):

Differences from U
$$\begin{bmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix} = \begin{bmatrix} u_1 - u_2 \\ u_2 - u_3 \\ u_3 - 0 \end{bmatrix}$$

Sums from U^{-1}
$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 - u_2 \\ u_2 - u_3 \\ u_3 - 0 \end{bmatrix} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}.$$

Changing T_n to B_n

The fourth family B_n has the last entry also changed from 2 to 1. The new boundary condition is being applied at both ends (B stands for both). These matrices B_n are symmetric and tridiagonal, but you will quickly see that they are *not invertible*. The B_n are positive semidefinite but *not positive definite*:

$\mathbf{B_n(n,n) = 1}$ $\qquad B_2 = \begin{bmatrix} \mathbf{1} & -1 \\ -1 & \mathbf{1} \end{bmatrix}$ and $B_3 = \begin{bmatrix} \mathbf{1} & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & \mathbf{1} \end{bmatrix}$. (3)

Again, elimination brings out the properties of the matrix. The first $n-1$ pivots will all equal 1, because those rows are not changed from T_n. But the change from 2 to 1 in the last entry of B produces a change from 1 to 0 in the last entry of U:

$$B = \begin{bmatrix} \textcircled{1} & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & \mathbf{1} \end{bmatrix} \longrightarrow \begin{bmatrix} \textcircled{1} & -1 & 0 \\ 0 & \textcircled{1} & -1 \\ 0 & -1 & \mathbf{1} \end{bmatrix} \longrightarrow \begin{bmatrix} \textcircled{1} & -1 & 0 \\ 0 & \textcircled{1} & -1 \\ 0 & 0 & \mathbf{0} \end{bmatrix} = U. \quad (4)$$

There are only two pivots. (A pivot must be nonzero.) The last matrix U is certainly not invertible. Its determinant is zero, because its third row is all zeros. The constant vector $(1,1,1)$ is in the **nullspace** of U, and therefore it is in the nullspace of B:

$$\begin{bmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad \text{and also} \quad \begin{bmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

The whole point of elimination was to simplify a linear system like $Bu = 0$, *without changing the solutions*. In this case we could have recognized non-invertibility in the matrix B, because each row adds to zero. Then the sum of its three columns is the zero column. This is what we see when B multiplies the vector $(1,1,1)$.

Let me summarize this section in four lines (all these matrices are symmetric):

K_n and T_n are invertible and (more than that) positive definite.

C_n and B_n are singular and (more than that) positive semidefinite.

The nullspaces of C_n and B_n contain all the constant vectors $u = (c, c, \ldots, c)$. Their columns are dependent.

The nullspaces of K_n and T_n contain only the zero vector $u = (0, 0, \ldots, 0)$. Their columns are independent.

Matrices in MATLAB

It is natural to choose MATLAB for linear algebra, but the reader may select another system. (Octave is very close, and free. Mathematica and Maple are good for symbolic calculation, LAPACK provides excellent codes at no cost in netlib, and there are many other linear algebra packages.) We will construct matrices and operate on them in the convenient language that MATLAB provides.

Our first step is to construct the matrices K_n. For $n = 3$, we can enter the 3 by 3 matrix a row at a time, inside brackets. Rows are separated by a semicolon ;

$$K = [\ 2\ \ -1\ \ 0\ ;\ -1\ \ 2\ \ -1\ ;\ 0\ \ -1\ \ 2\ ]$$

For large matrices this is too slow. We can build K_8 from "eye" and "ones":

$\mathsf{eye}(8) = 8$ by 8 identity matrix $\qquad \mathsf{ones}(7,1) =$ column vector of seven 1's

The diagonal part is $2{*}\mathsf{eye}(8)$. The symbol $*$ means multiplication! The -1's above the diagonal of K_8 have the vector $-\mathsf{ones}(7,1)$ along diagonal 1 of the matrix E:

Superdiagonal of -1's $\qquad E = -\mathsf{diag}(\mathsf{ones}(7,1),1)$

The -1's *below* the diagonal of K_8 lie on the diagonal numbered -1. For those we could change the last argument in E from 1 to -1. Or we can simply transpose E, using the all-important symbol E' for E^{T}. Then K comes from its three diagonals:

Tridiagonal matrix K_8 $\qquad K = 2 * \mathsf{eye}(8) + E + E'$

Note: The zeroth diagonal (main diagonal) is the default with no second argument, so eye(8)= diag(ones(8,1)). And then $\mathsf{diag}(\mathsf{eye}(8)) = \mathsf{ones}(8,1)$.

The constant diagonals make K a Toeplitz matrix. The toeplitz command produces K, when each diagonal is determined by a single number 2 or -1 or 0. Use the zeros vector for the 6 zeros in the first row of K_8:

Symmetric Toeplitz $\quad \mathrm{row1} = [\,2\ \ -1\ \ \mathsf{zeros}(1,6)\,];\ \ K = \mathsf{toeplitz}(\mathrm{row1})$

For an unsymmetric constant-diagonal matrix, use toeplitz(col1, row1). Taking col1 = [1 −1 0 0] and row1 = [1 0 0] gives a 4 by 3 backward difference matrix. It has two nonzero diagonals, 1's and -1's.

To construct the matrices T and B and C from K, just change entries as in the last three lines of this M-file that we have named KTBC.m. Its input is the size n, its output is four matrices of that size. The semicolons suppress display of K, T, B, C:

```
function [K,T,B,C] = KTBC(n)
% Create the four special matrices assuming n>1
K = toeplitz ([2 -1 zeros(1,n-2)]);
T = K; T(1,1) = 1;
B = K; B(1,1) = 1; B(n,n) = 1;
C = K; C(1,n) = -1; C(n,1) = -1;
```

If we happened to want their determinants (we shouldn't!), then with $n = 8$

$$\begin{bmatrix} \det(K) & \det(T) & \det(B) & \det(C) \end{bmatrix} \quad \text{produces the output} \quad \begin{bmatrix} 9 & 1 & 0 & 0 \end{bmatrix}$$

One more point. MATLAB could not store K_n as a dense matrix for $n = 10{,}000$. **The 10^8 entries need about 800 megabytes unless we recognize K as sparse**. The code sparseKTBC.m on the course website avoids storing (and operating on) all the zeros. It has K, T, B, or C and n as its first two arguments. The third argument is 1 for sparse, 0 for dense (default 0 for narg = 2, no third argument).

The input to Sparse MATLAB includes the locations of all nonzero entries. The command $A =$ sparse(i, j, s, m, n) creates an m by n sparse matrix from the vectors i, j, s that list all positions i, j of nonzero entries s. Elimination by lu(A) may produce additional nonzeros (called fill-in) which the software will correctly identify. In the normal "full" option, zeros are processed like all other numbers.

It is best to create the list of triplets i, j, s and then call sparse. Insertions $A(i, j) = s$ or $A(i, j) = A(i, j) + s$ are more expensive. We return to this point in Section 3.6.

The sparse KTBC code on the website uses spdiags to enter the three diagonals. Here is the toeplitz way to form K_8, all made sparse by its sparse vector start:

```
vsp = sparse([2 -1 zeros(1, 6)]) % please look at each output
Ksp = toeplitz(vsp) % sparse format gives the nonzero positions and entries
bsp = Ksp(:, 2)     % colon keeps all rows of column 2, so bsp = column 2 of Ksp
usp = Ksp\bsp       % zeros in Ksp and bsp are not processed, solution: usp(2) = 1
uuu = full(usp)     % return from sparse format to the full uuu = [0 1 0 0 0 0 0 0]
```

Note The open source language Python is also very attractive and convenient.

The next sections will use all four matrices in these basic tasks of linear algebra:

(**1.2**) The finite difference matrices K, T, B, C include boundary conditions

(**1.3**) Elimination produces pivots in D and triangular factors in LDL^{T}

(**1.4**) Point loads produce inverse matrices K^{-1} and T^{-1}

(**1.5**) The eigenvalues and eigenvectors of K, T, B, C involve sines and cosines.

You will see K\f in **1.2**, lu(K) in **1.3**, inv(K) in **1.4**, eig(K) in **1.5**, and chol(K) in **1.6**.

I very much hope that you will come to know and like these special matrices.

■ WORKED EXAMPLES ■

1.1 A $Bu = f$ and $Cu = f$ might be solvable even though B and C are singular!

Show that every vector $f = Bu$ has $f_1 + f_2 + \cdots + f_n = 0$. Physical meaning: **the external forces balance**. Linear algebra meaning: $Bu = f$ is solvable when f is perpendicular to the all-ones column vector $e = (1, 1, 1, 1, \dots) = \mathsf{ones}(n, 1)$.

Solution Bu is a vector of "differences" of u's. Those differences always add to zero:

$$f = Bu = \begin{bmatrix} 1 & -1 & & \\ -1 & 2 & -1 & \\ & -1 & 2 & -1 \\ & & -1 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} = \begin{bmatrix} u_1 - u_2 \\ -u_1 + 2u_2 - u_3 \\ -u_2 + 2u_3 - u_4 \\ -u_3 + u_4 \end{bmatrix}$$

All terms cancel in $(u_1 - u_2) + (-u_1 + 2u_2 - u_3) + (-u_2 + 2u_3 - u_4) + (-u_3 + u_4) = 0$. The dot product with $e = (1, 1, 1, 1)$ is that sum $f^{\mathrm{T}}e = f_1 + f_2 + f_3 + f_4 = 0$.

Dot product $f \cdot e = f^{\mathrm{T}}e = f_1e_1 + f_2e_2 + f_3e_3 + f_4e_4$ (f ′ ∗ e in MATLAB).

A second explanation for $f^{\mathrm{T}}e = 0$ starts from the fact that $Be = 0$. The all-ones vector e is in the nullspace of B. Transposing $f = Bu$ gives $f^{\mathrm{T}} = u^{\mathrm{T}}B^{\mathrm{T}}$, since **the transpose of a product has the individual transposes in reverse order**. This matrix B is symmetric so $B^{\mathrm{T}} = B$. Then

$$f^{\mathrm{T}}e = u^{\mathrm{T}}B^{\mathrm{T}}e \quad \text{is equal to} \quad u^{\mathrm{T}}Be = u^{\mathrm{T}}0 = 0.$$

Conclusion $Bu = f$ is only solvable when f is perpendicular to the all-ones vector e. (*The same is true for* $Cu = f$. Again the differences cancel out.) **The external forces balance** when the f's add to zero. The command B\f will produce Inf because B is square and singular, but the "pseudoinverse" u = pinv(B) ∗ f will succeed. (Or add a zero row to B and f before the command B\f, to make the system rectangular.)

1.1 B The "fixed-free" matrix H changes the *last entry of* K from 2 to 1. Connect H to the "free-fixed" T (*first entry* $= 1$) by using the reverse identity matrix J:

$$H = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & \mathbf{1} \end{bmatrix} \quad \begin{matrix} \text{comes from} \\ JTJ \text{ via the} \\ \textit{reverse identity } J \end{matrix} \quad J = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

Chapter 2 shows how T comes from a *tower* structure (free at the top). H comes from a *hanging* structure (free at the bottom). Two MATLAB constructions are

$H =$ toeplitz([2 −1 0]); $H(3,3) = 1$ **or** $J =$ fliplr(eye(3)); $H = J * T * J$

Solution JT reverses the rows of T. Then JTJ reverses the columns to give H:

$$T = \begin{bmatrix} \mathbf{1} & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix} \qquad \begin{matrix} JT = \\ (\text{rows}) \end{matrix} \begin{bmatrix} 0 & -1 & 2 \\ -1 & 2 & -1 \\ \mathbf{1} & -1 & 0 \end{bmatrix} \qquad \begin{matrix} (JT)J = H\,. \\ (\textit{columns too}) \end{matrix}$$

We could reverse columns first by TJ. Then $J(TJ)$ would be the same matrix H as $(JT)J$. **The parentheses never matter in** $(AB)C = A(BC)$!

Any permutation matrix like J has the rows of the identity matrix I in some order. There are six 3 by 3 permutation matrices because there are six orders for the numbers $1, 2, 3$. *The inverse of every permutation matrix is its transpose.* This particular J is symmetric, so it has $J = J^{\mathrm{T}} = J^{-1}$ as you can easily check:

$$H = JTJ \quad \text{so} \quad H^{-1} = J^{-1}T^{-1}J^{-1} \quad \text{which is} \quad H^{-1} = JT^{-1}J. \tag{5}$$

With back = 3:−1:1, reordering to JTJ is H = T(back, back) in MATLAB.

Problem Set 1.1

Problems 1–4 are about T^{-1} and Problems 5–8 are about K^{-1}.

1 The inverses of T_3 and T_4 (with $T_{11} = 1$ in the top corner) are

$$T_3^{-1} = \begin{bmatrix} 3 & 2 & 1 \\ 2 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \text{and} \quad T_4^{-1} = \begin{bmatrix} 4 & 3 & 2 & 1 \\ 3 & 3 & 2 & 1 \\ 2 & 2 & 2 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}.$$

Guess T_5^{-1} and multiply by T_5. Find a simple formula for the entries of T_n^{-1} on and below the diagonal ($i \geq j$), and then on and above the diagonal ($i \leq j$).

2 Compute T_3^{-1} in three steps, using U and U^{-1} in equation (2):

1. Check that $T_3 = U^{\mathrm{T}}U$, where U has 1's on the main diagonal and -1's along the diagonal above. Its transpose U^{T} is lower triangular.
2. Check that $UU^{-1} = I$ when U^{-1} has 1's on and above the main diagonal.
3. Invert $U^{\mathrm{T}}U$ to find $T_3^{-1} = (U^{-1})(U^{-1})^{\mathrm{T}}$. *Inverses come in reverse order*!

3 The difference matrix $U = U_5$ in MATLAB is eye(5)−diag(ones(4,1),1). Construct the sum matrix S from triu(ones(5)). (This keeps the upper triangular part of the 5 by 5 all-ones matrix.) Multiply $U * S$ to verify that $S = U^{-1}$.

4 For every n, $S_n = {U_n}^{-1}$ is upper triangular with ones on and above the diagonal. For $n = 4$ check that SS^{T} produces the matrix ${T_4}^{-1}$ predicted in Problem 1. Why is SS^{T} certain to be a symmetric matrix?

5 The inverses of K_3 and K_4 (please also invert K_2) have fractions $\frac{1}{\det} = \frac{1}{4}, \frac{1}{5}$:

$$K_3^{-1} = \frac{1}{4}\begin{bmatrix} 3 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 3 \end{bmatrix} \quad \text{and} \quad K_4^{-1} = \frac{1}{5}\begin{bmatrix} 4 & 3 & 2 & 1 \\ 3 & 6 & 4 & 2 \\ 2 & 4 & 6 & 3 \\ 1 & 2 & 3 & 4 \end{bmatrix}.$$

First *guess* the determinant of $K = K_5$. Then compute $\mathsf{det}(K)$ and $\mathsf{inv}(K)$ and $\mathsf{det}(K) * \mathsf{inv}(K)$—any software is allowed.

6 (Challenge problem) Find a *formula* for the i, j entry of K_4^{-1} *below the diagonal* ($i \geq j$). Those entries grow linearly along every row and up every column. (Section 1.4 will come back to these important inverses.) Problem 7 below is developed in the Worked Example of Section 1.4.

7 **A column u times a row v^{T} is a rank-one matrix** uv^{T}. All columns are multiples of u, and all rows are multiples of v^{T}. ${T_4}^{-1} - {K_4}^{-1}$ has rank 1:

$$T_4^{-1} - K_4^{-1} = \frac{1}{5}\begin{bmatrix} 16 & 12 & 8 & 4 \\ 12 & 9 & 6 & 3 \\ 8 & 6 & 4 & 2 \\ 4 & 3 & 2 & 1 \end{bmatrix} = \frac{1}{5}\begin{bmatrix} 4 \\ 3 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 4 & 3 & 2 & 1 \end{bmatrix}$$

Write $K_3 - T_3$ in this special form uv^{T}. Predict a similar formula for $T_3^{-1} - K_3^{-1}$.

8 (a) Based on Problem 7, predict the i, j entry of $T_5^{-1} - K_5^{-1}$ below the diagonal.

(b) Subtract this from your answer to Problem 1 (the formula for T_5^{-1} when $i \geq j$). This gives the not-so-simple formula for K_5^{-1}.

9 Following Example **1.1 A** with C instead of B, show that $e = (1, 1, 1, 1)$ is perpendicular to each column of C_4. Solve $Cu = f = (1, -1, 1, -1)$ with the singular matrix C by $\mathsf{u} = \mathsf{pinv(C)} * \mathsf{f}$. Try $\mathsf{u} = \mathsf{C \backslash e}$ and $\mathsf{C \backslash f}$, before and after adding a fifth equation $0 = 0$.

10 The "hanging matrix" H in Worked Example **1.1 B** changes the last entry of K_3 to $H_{33} = 1$. Find the inverse matrix from $H^{-1} = JT^{-1}J$. Find the inverse also from $H = UU^{\mathrm{T}}$ (check upper times lower triangular!) and $H^{-1} = (U^{-1})^{\mathrm{T}}U^{-1}$.

11 Suppose U is any upper triangular matrix and J is the reverse identity matrix in **1.1 B**. Then JU is a "southeast matrix". What geographies are UJ and JUJ? By experiment, a southeast matrix times a northwest matrix is ____.

12 Carry out elimination on the 4 by 4 circulant matrix C_4 to reach an upper triangular U (or try $[L, U] = \mathsf{lu}(C)$ in MATLAB). Two points to notice: The last entry of U is ____ because C is singular. The last column of U has new nonzeros. Explain why this "fill-in" happens.

13 By hand, can you factor the circulant C_4 (with three nonzero diagonals, allowing wraparound) into circulants L times U (with two nonzero diagonals, allowing wraparound so not truly triangular)?

14 Gradually reduce the diagonal $2, 2, 2$ in the matrix K_3 until you reach a singular matrix M. This happens when the diagonal entries reach ____. Check the determinant as you go, and find a nonzero vector that solves $Mu = 0$.

Questions 15–21 bring out important facts about matrix multiplication.

15 How many individual multiplications to create Ax and A^2 and AB?

$$A_{n \times n}\, x_{n \times 1} \qquad A_{n \times n}\, A_{n \times n} \qquad A_{m \times n}\, B_{n \times p} = (AB)_{m \times p}$$

16 You can multiply Ax by rows (the usual way) or **by columns** (more important). Do this multiplication both ways:

$$\textbf{By rows} \quad \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} \text{inner product using row 1} \\ \text{inner product using row 2} \end{bmatrix}$$

$$\textbf{By columns} \quad \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = 1 \begin{bmatrix} 2 \\ 4 \end{bmatrix} + 2 \begin{bmatrix} 3 \\ 5 \end{bmatrix} = \begin{bmatrix} \textbf{combination} \\ \textbf{of columns} \end{bmatrix}$$

17 The product Ax is a **linear combination of the columns of** A. The equations $Ax = b$ have a solution vector x exactly when b is a ____ of the columns.

Give an example in which b is *not in the column space* of A. There is no solution to $Ax = b$, because b is not a combination of the columns of A.

18 Compute $C = AB$ by multiplying the matrix A times each column of B:

$$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 8 & * \\ 14 & * \end{bmatrix}.$$

Thus, $\mathsf{A} * \mathsf{B}(:,\mathsf{j}) = \mathsf{C}(:,\mathsf{j})$.

19 You can also compute AB by multiplying each row of A times B:

$$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 2 * \text{row } 1 + 3 * \text{row } 2 \\ 4 * \text{row } 1 + 5 * \text{row } 2 \end{bmatrix} = \begin{bmatrix} 8 & 16 \\ * & * \end{bmatrix}.$$

A solution to $Bx = 0$ is also a solution to $(AB)x = 0$. Why? From

$$Bx = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \text{how do we know} \quad ABx = \begin{bmatrix} 8 & 16 \\ * & * \end{bmatrix} \begin{bmatrix} -2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}?$$

20 The four ways to find AB give numbers, columns, rows, and **matrices**:

1 ***(rows of A) times (columns of B)***	C(i,j) = A(i,:) * B(:,j)
2 ***A times (columns of B)***	C(:,j) = A * B(:,j)
3 ***(rows of A) times B***	C(i,:) = A(i,:) * B
4 **(columns of A) times (rows of B)**	for k = 1:n, C = C + A(:,k) * B(k,:); end

Finish these 8 multiplications for **columns times rows**. How many for n by n?

$$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} + \begin{bmatrix} 3 \\ 5 \end{bmatrix} \begin{bmatrix} 2 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 4 & 8 \end{bmatrix} + \begin{bmatrix} * & * \\ * & * \end{bmatrix} = \begin{bmatrix} 8 & * \\ * & * \end{bmatrix}.$$

21 Which *one* of these equations is true for all n by n matrices A and B?

$$AB = BA \qquad (AB)A = A(BA) \qquad (AB)B = B(BA) \qquad (AB)^2 = A^2B^2\,.$$

22 Use $n = 1000$; $e = \mathsf{ones}(n, 1)$; $K = \mathsf{spdiags}([-e, 2 * e, -e], -1\!:\!1, n, n)$; to enter K_{1000} as a sparse matrix. Solve the sparse equation $Ku = e$ by $\mathsf{u} = \mathsf{K}\backslash\mathsf{e}$. Plot the solution by $\mathsf{plot}(u)$.

23 Create 4-component vectors u, v, w and enter $A = \mathsf{spdiags}([u, v, w], -1\!:\!1, 4, 4)$. Which components of u and w are left out from the -1 and 1 diagonals of A?

24 Build the sparse identity matrix $I = \mathsf{sparse}(i, j, s, 100, 100)$ by creating vectors i, j, s of positions i, j with nonzero entries s. (You could use a for loop.) In this case $\mathsf{speye}(100)$ is quicker. Notice that $\mathsf{sparse}(\mathsf{eye}(10000))$ would be a disaster, since there isn't room to store $\mathsf{eye}(10000)$ before making it sparse.

25 The only solution to $Ku = 0$ or $Tu = 0$ is $u = 0$, so K and T are invertible. For proof, suppose u_i is the largest component of u. If $-u_{i-1} + 2u_i - u_{i+1}$ is zero, this forces $u_{i-1} = u_i = u_{i+1}$. Then the next equations force every $u_j = u_i$. At the end, when the boundary is reached, $-u_{n-1} + 2u_n$ only gives zero if $u = 0$.

Why does this "diagonally dominant" argument fail for B and C?

26 For which vectors v is $\mathsf{toeplitz}(v)$ a circulant matrix (cyclic diagonals) ?

27 (Important) Show that the 3 by 3 matrix K comes from $A_0^{\mathrm{T}} A_0$:

$$A_0 = \begin{bmatrix} -1 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 1 \end{bmatrix} \quad \text{is a "}\textbf{difference matrix}\text{"}$$

Which column of A_0 would you remove to produce A_1 with $T = A_1^{\mathrm{T}} A_1$? Which column would you remove next to produce A_2 with $B = A_2^{\mathrm{T}} A_2$? The difference matrices A_0, A_1, A_2 have $0, 1, 2$ boundary conditions. So do the "second differences" K, T, and B.

MIT OpenCourseWare
http://ocw.mit.edu

18.085 Computational Science and Engineering I
Fall 2008

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

CHAPTER 4

FOURIER SERIES AND INTEGRALS

4.1 FOURIER SERIES FOR PERIODIC FUNCTIONS

This section explains three Fourier series: **sines, cosines, and exponentials e^{ikx}**. Square waves (1 or 0 or -1) are great examples, with delta functions in the derivative. We look at a spike, a step function, and a ramp—and smoother functions too.

Start with $\sin x$. It has period 2π since $\sin(x + 2\pi) = \sin x$. It is an odd function since $\sin(-x) = -\sin x$, and it vanishes at $x = 0$ and $x = \pi$. Every function $\sin nx$ has those three properties, and Fourier looked at *infinite combinations of the sines*:

Fourier sine series
$$S(x) = b_1 \sin x + b_2 \sin 2x + b_3 \sin 3x + \cdots = \sum_{n=1}^{\infty} b_n \sin nx \quad (1)$$

If the numbers $b_1, b_2, \ldots$ drop off quickly enough (we are foreshadowing the importance of the decay rate) then the sum $S(x)$ will inherit all three properties:

Periodic $S(x + 2\pi) = S(x)$ **Odd** $S(-x) = -S(x)$ $S(0) = S(\pi) = 0$

200 years ago, Fourier startled the mathematicians in France by suggesting that *any function* $S(x)$ with those properties could be expressed as an infinite series of sines. This idea started an enormous development of Fourier series. Our first step is to compute from $S(x)$ the number b_k that multiplies $\sin kx$.

Suppose $S(x) = \sum b_n \sin nx$. *Multiply both sides by* $\sin kx$. *Integrate from* 0 *to* π:

$$\int_0^{\pi} S(x) \sin kx \, dx = \int_0^{\pi} b_1 \sin x \sin kx \; dx + \cdots + \int_0^{\pi} \boldsymbol{b_k \sin kx \; \sin kx \, dx} + \cdots \quad (2)$$

On the right side, all integrals are zero except the highlighted one with $n = k$. This property of "**orthogonality**" will dominate the whole chapter. The sines make 90° angles in function space, when their inner products are integrals from 0 to π:

Orthogonality
$$\int_0^{\pi} \sin nx \; \sin kx \, dx = 0 \quad \text{if} \quad n \neq k\,. \quad (3)$$

Zero comes quickly if we integrate $\int \cos mx\, dx = \left[\frac{\sin mx}{m}\right]_0^\pi = 0 - 0$. So we use this:

Product of sines
$$\sin nx\ \sin kx = \frac{1}{2}\cos(n-k)x - \frac{1}{2}\cos(n+k)x\,. \tag{4}$$

Integrating $\cos mx$ with $m = n - k$ and $m = n + k$ proves orthogonality of the sines.

The exception is when $n = k$. Then we are integrating $(\sin kx)^2 = \frac{1}{2} - \frac{1}{2}\cos 2kx$:

$$\int_0^\pi \sin kx\ \sin kx\, dx = \int_0^\pi \frac{1}{2}\, dx - \int_0^\pi \frac{1}{2}\cos 2kx\, dx = \frac{\pi}{2}\,. \tag{5}$$

The highlighted term in equation (2) is $\boldsymbol{b_k\pi/2}$. Multiply both sides of (2) by $2/\pi$:

Sine coefficients
$S(-x) = -S(x)$
$$b_k = \frac{2}{\pi}\int_0^\pi S(x)\sin kx\, dx = \frac{1}{\pi}\int_{-\pi}^\pi S(x)\sin kx\, dx. \tag{6}$$

Notice that $S(x)\sin kx$ is *even* (equal integrals from $-\pi$ to 0 and from 0 to π).

I will go immediately to the most important example of a Fourier sine series. $S(x)$ is an **odd square wave** with $SW(x) = 1$ for $0 < x < \pi$. It is drawn in Figure 4.1 as an odd function (with period 2π) that vanishes at $x = 0$ and $x = \pi$.

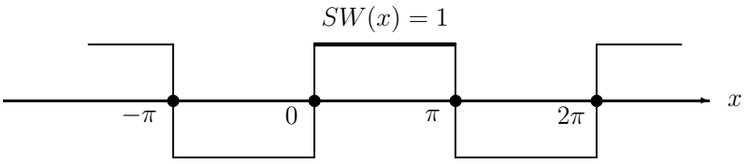


Figure 4.1: The odd square wave with $SW(x + 2\pi) = SW(x) = \{1 \text{ or } 0 \text{ or } -1\}$.

Example 1 Find the Fourier sine coefficients b_k of the square wave $SW(x)$.

Solution For $k = 1, 2, \ldots$ use the first formula (6) with $S(x) = 1$ between 0 and π:

$$b_k = \frac{2}{\pi}\int_0^\pi \sin kx\, dx = \frac{2}{\pi}\left[\frac{-\cos kx}{k}\right]_{\mathbf{0}}^{\boldsymbol{\pi}} = \frac{2}{\pi}\left\{\frac{\mathbf{2}}{\mathbf{1}}, \frac{0}{2}, \frac{\mathbf{2}}{\mathbf{3}}, \frac{0}{4}, \frac{\mathbf{2}}{\mathbf{5}}, \frac{0}{6}, \ldots\right\} \tag{7}$$

The even-numbered coefficients b_{2k} are all zero because $\cos 2k\pi = \cos 0 = 1$. The odd-numbered coefficients $b_k = 4/\pi k$ decrease at the rate $1/k$. We will see that same $1/k$ decay rate for all functions formed from *smooth pieces and jumps*.

Put those coefficients $4/\pi k$ and zero into the Fourier sine series for $SW(x)$:

Square wave
$$\boldsymbol{SW(x)} = \frac{4}{\pi}\left[\frac{\sin x}{\mathbf{1}} + \frac{\sin 3x}{\mathbf{3}} + \frac{\sin 5x}{\mathbf{5}} + \frac{\sin 7x}{\mathbf{7}} + \cdots\right] \tag{8}$$

Figure 4.2 graphs this sum after one term, then two terms, and then five terms. You can see the all-important **Gibbs phenomenon** appearing as these "partial sums"

include more terms. Away from the jumps, we safely approach $SW(x) = 1$ or -1. At $x = \pi/2$, the series gives a beautiful alternating formula for the number π:

$$1 = \frac{4}{\pi}\left[\frac{1}{1} - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots\right] \quad \text{so that} \quad \boldsymbol{\pi = 4\left[\frac{1}{1} - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots\right]}. \tag{9}$$

The Gibbs phenomenon is the overshoot that moves closer and closer to the jumps. Its height approaches 1.18 . . . and it does not decrease with more terms of the series! Overshoot is the one greatest obstacle to calculation of all discontinuous functions (like shock waves in fluid flow). We try hard to avoid Gibbs but sometimes we can't.

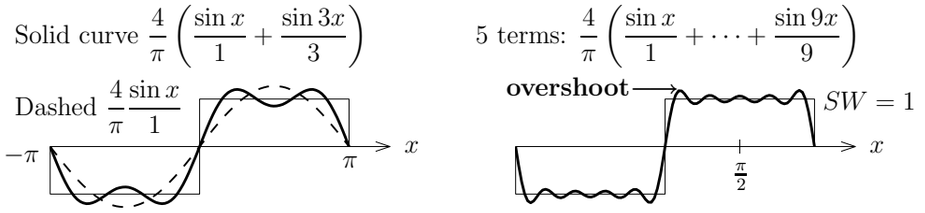


Figure 4.2: **Gibbs phenomenon**: Partial sums $\sum_1^N b_n \sin nx$ overshoot near jumps.

Fourier Coefficients are Best

Let me look again at the first term $b_1 \sin x = (4/\pi)\sin x$. This is the **closest possible approximation** to the square wave SW, by any multiple of $\sin x$ (closest in the least squares sense). To see this optimal property of the Fourier coefficients, minimize the error over all b_1:

$$\text{The error is } \int_0^\pi (SW - b_1 \sin x)^2\, dx \quad \text{The } b_1 \text{ derivative is } -2\int_0^\pi (SW - b_1 \sin x)\sin x\, dx.$$

The integral of $\sin^2 x$ is $\pi/2$. So the derivative is zero when $b_1 = (2/\pi)\int_0^\pi S(x)\sin x\, dx$. This is exactly equation (6) for the Fourier coefficient.

Each $b_k \sin kx$ is as close as possible to $SW(x)$. We can find the coefficients b_k one at a time, *because the sines are orthogonal*. The square wave has $b_2 = 0$ because all other multiples of $\sin 2x$ increase the error. Term by term, we are "projecting the function onto each axis $\sin kx$."

Fourier Cosine Series

The cosine series applies to *even functions* with $C(-x) = C(x)$:

$$\textbf{Cosine series} \quad C(x) = a_0 + a_1 \cos x + a_2 \cos 2x + \cdots = a_0 + \sum_{n=1}^{\infty} a_n \cos nx. \tag{10}$$

Every cosine has period 2π. Figure 4.3 shows two even functions, the **repeating ramp** $RR(x)$ and the **up-down train** $UD(x)$ of delta functions. That sawtooth ramp RR is the integral of the square wave. The delta functions in UD give the derivative of the square wave. (For sines, the integral and derivative are cosines.) RR and UD will be valuable examples, one smoother than SW, one less smooth.

First we find formulas for the cosine coefficients a_0 and a_k. The constant term a_0 is the *average value* of the function $C(x)$:

a_0 = Average
$$a_0 = \frac{1}{\pi}\int_0^\pi C(x)\,dx = \frac{1}{2\pi}\int_{-\pi}^{\pi} C(x)\,dx. \tag{11}$$

I just integrated every term in the cosine series (10) from 0 to π. On the right side, the integral of a_0 is $a_0\pi$ (divide both sides by π). All other integrals are zero:

$$\int_0^\pi \cos nx\,dx = \left[\frac{\sin nx}{n}\right]_0^\pi = 0 - 0 = 0. \tag{12}$$

In words, the constant function 1 is orthogonal to $\cos nx$ over the interval $[0, \pi]$.

The other cosine coefficients a_k come from the *orthogonality of cosines*. As with sines, we multiply both sides of (10) by $\cos kx$ and integrate from 0 to π:

$$\int_0^\pi C(x)\cos kx\,dx = \int_0^\pi a_0\cos kx\,dx + \int_0^\pi a_1\cos x\cos kx\,dx + \cdots + \int_0^\pi \boldsymbol{a_k(\cos kx)^2\,dx} + \cdots$$

You know what is coming. On the right side, only the highlighted term can be nonzero. Problem 4.1.1 proves this by an identity for $\cos nx\cos kx$—now (4) has a plus sign. The bold nonzero term is $\boldsymbol{a_k\pi/2}$ and we multiply both sides by $2/\pi$:

Cosine coefficients
$C(-x) = C(x)$
$$a_k = \frac{2}{\pi}\int_0^\pi C(x)\cos kx\,dx = \frac{1}{\pi}\int_{-\pi}^{\pi} C(x)\cos kx\,dx\,. \tag{13}$$

Again the integral over a full period from $-\pi$ to π (also 0 to 2π) is just doubled.

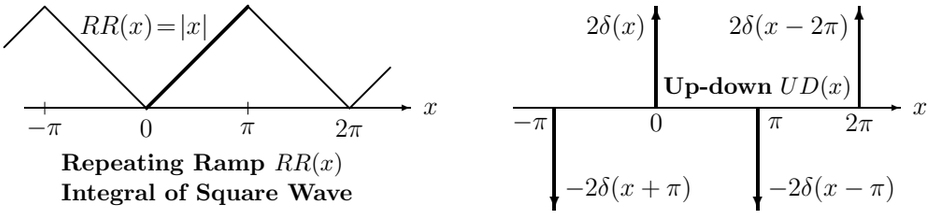


Figure 4.3: The repeating ramp RR and the up-down UD (periodic spikes) are even. The derivative of RR is the odd square wave SW. **The derivative of SW is UD.**

Example 2 Find the cosine coefficients of the ramp $RR(x)$ and the up-down $UD(x)$.

Solution The simplest way is to start with the sine series for the square wave:

$$SW(x) = \frac{4}{\pi}\left[\frac{\sin x}{1} + \frac{\sin 3x}{3} + \frac{\sin 5x}{5} + \frac{\sin 7x}{7} + \cdots\right].$$

Take the derivative of every term to produce cosines in the up-down delta function:

Up-down series $$UD(x) = \frac{4}{\pi}\left[\cos x + \cos 3x + \cos 5x + \cos 7x + \cdots\right]. \tag{14}$$

Those coefficients don't decay at all. The terms in the series don't approach zero, so officially the series cannot converge. Nevertheless it is somehow correct and important. Unofficially this sum of cosines has all 1's at $x = 0$ and all -1's at $x = \pi$. Then $+\infty$ and $-\infty$ are consistent with $2\delta(x)$ and $-2\delta(x - \pi)$. The true way to recognize $\delta(x)$ is by the test $\int \delta(x) f(x)\, dx = f(0)$ and Example 3 will do this.

For the repeating ramp, we integrate the square wave series for $SW(x)$ and add the average ramp height $a_0 = \pi/2$, halfway from 0 to π:

Ramp series $$\boldsymbol{RR(x)} = \frac{\pi}{2} - \frac{\pi}{4}\left[\frac{\cos x}{\mathbf{1^2}} + \frac{\cos 3x}{\mathbf{3^2}} + \frac{\cos 5x}{\mathbf{5^2}} + \frac{\cos 7x}{\mathbf{7^2}} + \cdots\right]. \tag{15}$$

The constant of integration is a_0. *Those coefficients a_k drop off like $1/k^2$.* They could be computed directly from formula (13) using $\int x \cos kx\, dx$, but this requires an integration by parts (or a table of integrals or an appeal to *Mathematica* or *Maple*). It was much easier to integrate every sine separately in $SW(x)$, which makes clear the crucial point**:** Each "degree of smoothness" in the function is reflected in a faster decay rate of its Fourier coefficients a_k and b_k.

No decay	**Delta** functions (with spikes)
$\mathbf{1/k}$ **decay**	**Step** functions (with jumps)
$\mathbf{1/k^2}$ **decay**	**Ramp** functions (with corners)
$\mathbf{1/k^4}$ **decay**	**Spline** functions (jumps in f''')
$\boldsymbol{r^k}$ **decay with** $\boldsymbol{r < 1}$	**Analytic** functions like $1/(2 - \cos x)$

Each integration divides the kth coefficient by k. So the decay rate has an extra $1/k$. The "Riemann-Lebesgue lemma" says that a_k and b_k approach zero for any continuous function (in fact whenever $\int |f(x)| dx$ is finite). Analytic functions achieve a new level of smoothness—they can be differentiated forever. Their Fourier series and Taylor series in Chapter 5 converge **exponentially fast**.

The poles of $1/(2 - \cos x)$ will be complex solutions of $\cos x = 2$. Its Fourier series converges quickly because r^k decays faster than any power $1/k^p$. Analytic functions are ideal for computations—the Gibbs phenomenon will never appear.

Now we go back to $\delta(x)$ for what could be the most important example of all.

Example 3 Find the (cosine) coefficients of the *delta function* $\delta(x)$, made 2π-periodic.

Solution The spike occurs at the start of the interval $[0, \pi]$ so safer to integrate from $-\pi$ to π. We find $a_0 = 1/2\pi$ and the other $a_k = 1/\pi$ (cosines because $\delta(x)$ is even):

$$\textbf{Average} \quad a_0 = \frac{1}{2\pi}\int_{-\pi}^{\pi} \delta(x)\,dx = \boldsymbol{\frac{1}{2\pi}} \qquad \textbf{Cosines} \quad a_k = \frac{1}{\pi}\int_{-\pi}^{\pi} \delta(x)\cos kx\,dx = \boldsymbol{\frac{1}{\pi}}$$

Then the series for the delta function has all cosines in equal amounts:

Delta function
$$\delta(x) = \frac{1}{2\pi} + \frac{1}{\pi}\left[\cos x + \cos 2x + \cos 3x + \cdots\right]. \tag{16}$$

Again this series cannot truly converge (its terms don't approach zero). But we can graph the sum after $\cos 5x$ and after $\cos 10x$. Figure 4.4 shows how these "partial sums" are doing their best to approach $\delta(x)$. They oscillate faster and faster away from $x = 0$.

Actually there is a neat formula for the partial sum $\delta_N(x)$ that stops at $\cos Nx$. Start by writing each term $2\cos\theta$ as $e^{i\theta} + e^{-i\theta}$:

$$\delta_N = \frac{1}{2\pi}\left[1 + 2\cos x + \cdots + 2\cos Nx\right] = \frac{1}{2\pi}\left[1 + e^{ix} + e^{-ix} + \cdots + e^{iNx} + e^{-iNx}\right].$$

This is a geometric progression that starts from e^{-iNx} and ends at e^{iNx}. We have powers of the same factor e^{ix}. The sum of a geometric series is known:

Partial sum up to $\cos Nx$
$$\delta_N(x) = \frac{1}{2\pi}\frac{e^{i(N+\frac{1}{2})x} - e^{-i(N+\frac{1}{2})x}}{e^{ix/2} - e^{-ix/2}} = \frac{1}{2\pi}\frac{\sin(N+\frac{1}{2})x}{\sin\frac{1}{2}x}. \tag{17}$$

This is the function graphed in Figure 4.4. We claim that for any N the area underneath $\delta_N(x)$ is 1. (Each cosine integrated from $-\pi$ to π gives zero. The integral of $1/2\pi$ is 1.) The central "lobe" in the graph ends when $\sin(N+\frac{1}{2})x$ comes down to zero, and that happens when $(N+\frac{1}{2})x = \pm\pi$. I think the area under that lobe (marked by bullets) approaches the same number $1.18\ldots$ that appears in the Gibbs phenomenon.

In what way does $\delta_N(x)$ approach $\delta(x)$? The terms $\cos nx$ in the series jump around at each point $x \neq 0$, not approaching zero. At $x = \pi$ we see $\frac{1}{2\pi}[1 - 2 + 2 - 2 + \cdots]$ and the sum is $1/2\pi$ or $-1/2\pi$. The bumps in the partial sums don't get smaller than $1/2\pi$. The right test for the delta function $\delta(x)$ is to multiply by a smooth $f(x) = \sum a_k \cos kx$ and integrate, because *we only know* $\delta(x)$ *from its integrals* $\int \delta(x)f(x)\,dx = f(0)$:

Weak convergence of $\delta_N(x)$ to $\delta(x)$
$$\int_{-\pi}^{\pi} \boldsymbol{\delta_N(x) f(x)\,dx = a_0 + \cdots + a_N \to f(0)}. \tag{18}$$

In this integrated sense (*weak sense*) the sums $\delta_N(x)$ do approach the delta function! The convergence of $a_0 + \cdots + a_N$ is the statement that at $x = 0$ the Fourier series of a smooth $f(x) = \sum a_k \cos kx$ converges to the number $f(0)$.

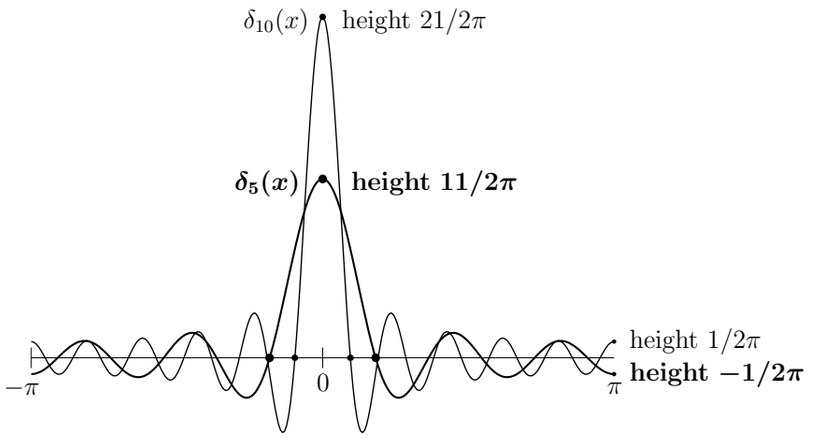


Figure 4.4: The sums $\delta_N(x) = (1 + 2\cos x + \cdots + 2\cos Nx)/2\pi$ try to approach $\delta(x)$.

Complete Series: Sines and Cosines

Over the half-period $[0, \pi]$, the sines are not orthogonal to all the cosines. In fact the integral of $\sin x$ times 1 is not zero. So for functions $F(x)$ that are not odd or even, we move to the complete series (sines plus cosines) on the full interval. Since our functions are periodic, that "full interval" can be $[-\pi, \pi]$ or $[0, 2\pi]$:

Complete Fourier series
$$F(x) = a_0 + \sum_{n=1}^{\infty} a_n \cos nx + \sum_{n=1}^{\infty} b_n \sin nx\,. \tag{19}$$

On every "2π interval" all sines and cosines are mutually orthogonal. We find the Fourier coefficients a_k and b_k in the usual way: **Multiply** (19) **by** 1 **and** $\cos kx$ **and** $\sin kx$, **and integrate both sides from** $-\pi$ **to** π:

$$\boldsymbol{a_0} = \frac{1}{2\pi}\int_{-\pi}^{\pi} F(x)\,dx \quad \boldsymbol{a_k} = \frac{1}{\pi}\int_{-\pi}^{\pi} F(x)\cos kx\,dx \quad \boldsymbol{b_k} = \frac{1}{\pi}\int_{-\pi}^{\pi} F(x)\sin kx\,dx. \tag{20}$$

Orthogonality kills off infinitely many integrals and leaves only the one we want.

Another approach is to split $F(x) = C(x) + S(x)$ into an even part and an odd part. Then we can use the earlier cosine and sine formulas. The two parts are

$$C(x) = F_{\mathbf{even}}(x) = \frac{F(x) + F(-x)}{2} \qquad S(x) = F_{\mathbf{odd}}(x) = \frac{F(x) - F(-x)}{2}. \tag{21}$$

The even part gives the a's and the odd part gives the b's. Test on a short square pulse from $x = 0$ to $x = h$—this one-sided function is not odd or even.

Example 4 Find the a's and b's if $F(x) = \textbf{square pulse} = \begin{cases} \mathbf{1} & \text{for } 0 < x < h \\ \mathbf{0} & \text{for } h < x < 2\pi \end{cases}$

Solution The integrals for a_0 and a_k and b_k stop at $x = h$ where $F(x)$ drops to zero. The coefficients decay like $1/k$ because of the jump at $x = 0$ and the drop at $x = h$:

Coefficients of square pulse $\qquad a_0 = \dfrac{1}{2\pi}\displaystyle\int_0^h 1\,dx = \dfrac{\boldsymbol{h}}{\boldsymbol{2\pi}} = \textbf{average}$

$$a_k = \frac{1}{\pi}\int_0^h \cos kx\,dx = \frac{\boldsymbol{\sin kh}}{\boldsymbol{\pi k}} \qquad b_k = \frac{1}{\pi}\int_0^h \sin kx\,dx = \frac{\boldsymbol{1-\cos kh}}{\boldsymbol{\pi k}}. \tag{22}$$

If we divide $F(x)$ by h, its graph is a tall thin rectangle: height $\frac{1}{h}$, base h, and area $= 1$.

When h approaches zero, $F(x)/h$ is squeezed into a very thin interval. *The tall rectangle approaches (weakly) the delta function* $\delta(x)$. The average height is area$/2\pi = 1/2\pi$. Its other coefficients a_k/h and b_k/h approach $1/\pi$ and 0, already known for $\delta(x)$:

$$\frac{\boldsymbol{F(x)}}{\boldsymbol{h}} \to \boldsymbol{\delta(x)} \qquad \frac{a_k}{h} = \frac{1}{\pi}\frac{\sin kh}{kh} \to \frac{1}{\pi} \quad \text{and} \quad \frac{b_k}{h} = \frac{1-\cos kh}{\pi kh} \to 0 \text{ as } h \to 0. \tag{23}$$

When the function has a jump, its Fourier series picks the halfway point. This example would converge to $F(0) = \frac{1}{2}$ and $F(h) = \frac{1}{2}$, halfway up and halfway down.

The Fourier series converges to $F(x)$ at each point where the function is smooth. This is a highly developed theory, and Carleson won the 2006 Abel Prize by proving convergence for every x except a set of measure zero. If the function has finite energy $\int |F(x)|^2\,dx$, he showed that the Fourier series converges "almost everywhere."

Energy in Function = Energy in Coefficients

There is an extremely important equation (***the energy identity***) that comes from integrating $(F(x))^2$. When we square the Fourier series of $F(x)$, and integrate from $-\pi$ to π, all the "cross terms" drop out. The only nonzero integrals come from 1^2 and $\cos^2 kx$ and $\sin^2 kx$, multiplied by a_0^2 and a_k^2 and b_k^2:

$$\textbf{Energy in } \boldsymbol{F(x)} = \int_{-\pi}^{\pi}(\boldsymbol{a_0} + \textstyle\sum \boldsymbol{a_k}\cos \boldsymbol{kx} + \sum \boldsymbol{b_k}\sin \boldsymbol{kx})^2 \boldsymbol{dx}$$
$$\int_{-\pi}^{\pi}(\boldsymbol{F(x)})^2\boldsymbol{dx} = \boldsymbol{2\pi a_0^2 + \pi(a_1^2 + b_1^2 + a_2^2 + b_2^2 + \cdots)}. \tag{24}$$

The energy in $F(x)$ equals the energy in the coefficients. The left side is like the length squared of a vector, except *the vector is a function.* The right side comes from an infinitely long vector of a's and b's. The lengths are equal, which says that the Fourier transform from function to vector is like an orthogonal matrix. Normalized by constants $\sqrt{2\pi}$ and $\sqrt{\pi}$, we have an *orthonormal basis in function space.*

What is this function space? It is like ordinary 3-dimensional space, except the "vectors" are functions. Their length $\|f\|$ comes from integrating instead of adding: $\|f\|^2 = \int |f(x)|^2 dx$. These functions fill **Hilbert space**. The rules of geometry hold:

Length $\|f\|^2 = (f, f)$ comes from the inner product $(f, g) = \int f(x)g(x)\,dx$
Orthogonal functions $(f, g) = 0$ produce a right triangle: $\|f + g\|^2 = \|f\|^2 + \|g\|^2$

I have tried to draw Hilbert space in Figure 4.5. It has infinitely many axes. *The energy identity* (24) *is exactly the Pythagoras Law in infinite-dimensional space.*

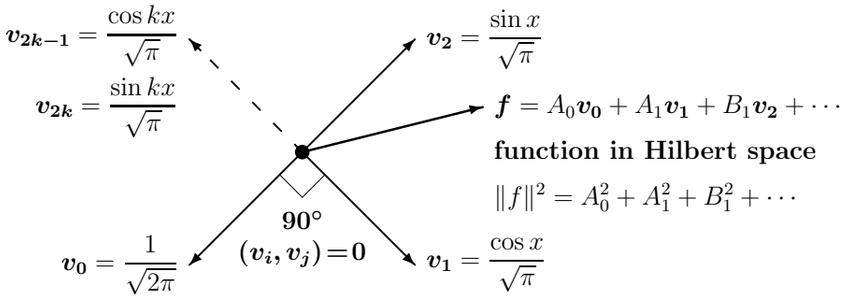


Figure 4.5: The Fourier series is a combination of orthonormal v's (sines and cosines).

Complex Exponentials $c_k e^{ikx}$

This is a small step and we have to take it. In place of separate formulas for a_0 and a_k and b_k, we will have *one formula* for all the complex coefficients c_k. And the function $F(x)$ might be complex (as in quantum mechanics). The Discrete Fourier Transform will be much simpler when we use N complex exponentials for a vector. We practice in advance with the complex infinite series for a 2π-periodic function:

Complex Fourier series
$$F(x) = c_0 + c_1 e^{ix} + c_{-1} e^{-ix} + \cdots = \sum_{n=-\infty}^{\infty} c_n e^{inx} \quad (25)$$

If every $c_n = c_{-n}$, we can combine e^{inx} with e^{-inx} into $2\cos nx$. Then (25) is the cosine series for an even function. If every $c_n = -c_{-n}$, we use $e^{inx} - e^{-inx} = 2i \sin nx$. Then (25) is the sine series for an odd function and the c's are pure imaginary.

To find c_k, multiply (25) by e^{-ikx} (not e^{ikx}) **and integrate from $-\pi$ to π**:

$$\int_{-\pi}^{\pi} F(x)e^{-ikx}dx = \int_{-\pi}^{\pi} c_0 e^{-ikx}dx + \int_{-\pi}^{\pi} c_1 e^{ix}e^{-ikx}dx + \cdots + \int_{-\pi}^{\pi} \boldsymbol{c_k e^{ikx} e^{-ikx} dx} + \cdots$$

The complex exponentials are orthogonal. Every integral on the right side is zero, except for the highlighted term (when $n = k$ and $e^{ikx}e^{-ikx} = 1$). The integral of 1 is 2π. That surviving term gives the formula for c_k:

Fourier coefficients
$$\int_{-\pi}^{\pi} F(x)e^{-ikx}\,dx = 2\pi c_k \quad \text{for} \quad k = 0, \pm 1, \ldots \quad (26)$$

Notice that $c_0 = a_0$ is still the average of $F(x)$, because $e^0 = 1$. The orthogonality of e^{inx} and e^{ikx} is checked by integrating, as always. But the complex inner product (F, G) takes the *complex conjugate* $\overline{G}$ *of* G. Before integrating, change e^{ikx} to e^{-ikx}:

$$\textbf{Complex inner product} \quad (F,G) = \int_{-\pi}^{\pi} F(x)\overline{G(x)}\,dx \qquad \textbf{Orthogonality of } \boldsymbol{e^{inx}} \textbf{ and } \boldsymbol{e^{ikx}} \quad \int_{-\pi}^{\pi} e^{i(n-k)x}dx = \left[\frac{e^{i(n-k)x}}{i(n-k)}\right]_{-\pi}^{\pi} = 0\,. \tag{27}$$

Example 5 Add the complex series for $1/(2-e^{ix})$ and $1/(2-e^{-ix})$. These geometric series have exponentially fast decay from $1/2^k$. The functions are analytic.

$$\left(\frac{1}{2} + \frac{e^{ix}}{4} + \frac{e^{2ix}}{8} + \cdot\cdot\right) + \left(\frac{1}{2} + \frac{e^{-ix}}{4} + \frac{e^{-2ix}}{8} + \cdot\cdot\right) = 1 + \frac{\cos x}{2} + \frac{\cos 2x}{4} + \frac{\cos 3x}{8} + \cdot\cdot$$

When we add those functions, we get a real analytic function:

$$\frac{1}{2-e^{ix}} + \frac{1}{2-e^{-ix}} = \frac{(2-e^{-ix}) + (2-e^{ix})}{(2-e^{ix})(2-e^{-ix})} = \frac{4-2\cos x}{5-4\cos x} \tag{28}$$

This ratio is the infinitely smooth function whose cosine coefficients are $1/2^k$.

Example 6 Find c_k for the 2π-periodic shifted pulse $F(x) = \begin{cases} 1 & \text{for } s \le x \le s+h \\ 0 & \text{elsewhere in } [-\pi, \pi] \end{cases}$

Solution The integrals (26) from $-\pi$ to π become integrals from s to $s+h$:

$$c_k = \frac{1}{2\pi}\int_s^{s+h} 1 \cdot e^{-ikx}\,dx = \frac{1}{2\pi}\left[\frac{e^{-ikx}}{-ik}\right]_s^{s+h} = \boldsymbol{e^{-iks}}\left(\frac{1-e^{-ikh}}{2\pi ik}\right). \tag{29}$$

Notice above all the simple effect of the shift by s. *It "modulates" each* c_k *by* e^{-iks}. The energy is unchanged, the integral of $|F|^2$ just shifts, and all $|e^{-iks}| = 1$:

$$\textbf{Shift} \quad F(x) \quad \text{to} \quad F(x-s) \longleftrightarrow \textbf{Multiply} \quad c_k \quad \text{by} \quad e^{-iks}. \tag{30}$$

Example 7 **Centered pulse** with shift $s = -h/2$. The square pulse becomes centered around $x = 0$. This even function equals 1 on the interval from $-h/2$ to $h/2$:

$$\textbf{Centered by } \boldsymbol{s = -\frac{h}{2}} \qquad c_k = e^{ikh/2}\frac{1-e^{-ikh}}{2\pi ik} = \frac{\mathbf{1}}{\boldsymbol{2\pi}}\frac{\boldsymbol{\sin(kh/2)}}{\boldsymbol{k/2}}\,.$$

Divide by h for a tall pulse. The ratio of $\sin(kh/2)$ to $kh/2$ is the **sinc function**:

$$\textbf{Tall pulse} \qquad \frac{F_{\text{centered}}}{h} = \frac{1}{2\pi}\sum_{-\infty}^{\infty} \operatorname{sinc}\left(\frac{kh}{2}\right) e^{ikx} = \begin{cases} 1/h & \text{for } -h/2 \le x \le h/2 \\ 0 & \text{elsewhere in } [-\pi, \pi] \end{cases}$$

That division by h produces area $= 1$. **Every coefficient approaches $\boldsymbol{\frac{1}{2\pi}}$ as $\boldsymbol{h \to 0}$.** The Fourier series for the tall thin pulse again approaches the Fourier series for $\delta(x)$.

Hilbert space can contain vectors $c = (c_0, c_1, c_{-1}, c_2, c_{-2}, \cdots)$ instead of functions $F(x)$. The length of c is $2\pi \sum |c_k|^2 = \int |F|^2 dx$. The function space is often denoted by L^2 and the vector space is ℓ^2. The energy identity is trivial (but deep). Integrating the Fourier series for $F(x)$ times $\overline{F(x)}$, orthogonality kills every $c_n \overline{c_k}$ for $n \neq k$. This leaves the $c_k \overline{c_k} = |c_k|^2$:

$$\int_{-\pi}^{\pi} |F(x)|^2 dx = \int_{-\pi}^{\pi} (\sum c_n e^{inx})(\sum \overline{c_k} e^{-ikx}) dx = 2\pi(|c_0|^2 + |c_1|^2 + |c_{-1}|^2 + \cdots). \quad (31)$$

This is Plancherel's identity: The energy in x-space equals the energy in k-space.

Finally I want to emphasize the three big rules for operating on $F(x) = \sum c_k e^{ikx}$:

1. **The derivative $\dfrac{dF}{dx}$ has Fourier coefficients ikc_k** (energy moves to high k).
2. **The integral of $F(x)$ has Fourier coefficients $\dfrac{c_k}{ik}, k \neq 0$** (faster decay).
3. **The shift to $F(x-s)$ has Fourier coefficients $e^{-iks}c_k$** (no change in energy).

Application: Laplace's Equation in a Circle

Our first application is to Laplace's equation. The idea is to construct $u(x, y)$ as an infinite series, choosing its coefficients to match $u_0(x, y)$ along the boundary. Everything depends on the shape of the boundary, and we take a circle of radius 1.

Begin with the simple solutions 1, $r\cos\theta$, $r\sin\theta$, $r^2\cos 2\theta$, $r^2\sin 2\theta$, ... to Laplace's equation. Combinations of these special solutions give all solutions in the circle:

$$u(r, \theta) = a_0 + a_1 r\cos\theta + b_1 r\sin\theta + a_2 r^2\cos 2\theta + b_2 r^2 \sin 2\theta + \cdots \quad (32)$$

It remains to choose the constants a_k and b_k to make $u = u_0$ on the boundary. For a circle $u_0(\theta)$ is periodic, since θ and $\theta + 2\pi$ give the same point:

$$\text{Set } r = 1 \qquad u_0(\theta) = a_0 + a_1\cos\theta + b_1\sin\theta + a_2\cos 2\theta + b_2\sin 2\theta + \cdots \quad (33)$$

This is exactly the Fourier series for u_0. **The constants a_k and b_k must be the Fourier coefficients of $u_0(\theta)$.** Thus the problem is completely solved, if an infinite series (32) is acceptable as the solution.

Example 8 **Point source** $u_0 = \delta(\theta)$ at $\theta = 0$ The whole boundary is held at $u_0 = 0$, except for the source at $x = 1$, $y = 0$. Find the temperature $u(r, \theta)$ inside.

Fourier series for δ $$u_0(\theta) = \frac{1}{2\pi} + \frac{1}{\pi}(\cos\theta + \cos 2\theta + \cos 3\theta + \cdots) = \frac{1}{2\pi}\sum_{-\infty}^{\infty} e^{in\theta}$$

Inside the circle, each $\cos n\theta$ is multiplied by r^n:

Infinite series for u $$u(r,\theta) = \frac{1}{2\pi} + \frac{1}{\pi}(r\cos\theta + r^2\cos 2\theta + r^3\cos 3\theta + \cdots) \quad (34)$$

Poisson managed to sum this infinite series! It involves a series of powers of $re^{i\theta}$. So we know the response at every (r,θ) to the point source at $r = 1$, $\theta = 0$:

Temperature inside circle $$u(r,\theta) = \frac{1}{2\pi}\frac{1-r^2}{1+r^2-2r\cos\theta} \quad (35)$$

At the center $r = 0$, this produces the average of $u_0 = \delta(\theta)$ which is $a_0 = 1/2\pi$. On the boundary $r = 1$, this produces $u = 0$ except at the point source where $\cos 0 = 1$:

On the ray $\theta = 0$ $$u(r,\theta) = \frac{1}{2\pi}\frac{1-r^2}{1+r^2-2r} = \frac{1}{2\pi}\frac{1+r}{1-r}. \quad (36)$$

As r approaches 1, the solution becomes infinite as the point source requires.

Example 9 Solve for any boundary values $u_0(\theta)$ by integrating over point sources.

When the point source swings around to angle φ, the solution (35) changes from θ to $\theta - \varphi$. Integrate this "Green's function" to solve in the circle:

Poisson's formula $$u(r,\theta) = \frac{1}{2\pi}\int_{-\pi}^{\pi} u_0(\varphi)\,\frac{1-r^2}{1+r^2-2r\cos(\theta-\varphi)}\,d\varphi \quad (37)$$

Ar $r = 0$ the fraction disappears and u is the average $\int u_0(\varphi)d\varphi/2\pi$. The steady state temperature at the center is the average temperature around the circle.

Poisson's formula illustrates a key idea. Think of any $u_0(\theta)$ as a circle of point sources. The source at angle $\varphi = \theta$ produces the solution inside the integral (37). Integrating around the circle adds up the responses to all sources and gives the response to $u_0(\theta)$.

Example 10 $u_0(\theta) = 1$ on the top half of the circle and $u_0 = -1$ on the bottom half.

Solution The boundary values are the square wave $SW(\theta)$. Its sine series is in (8):

Square wave for $u_0(\theta)$ $$SW(\theta) = \frac{4}{\pi}\left[\frac{\sin\theta}{1} + \frac{\sin 3\theta}{3} + \frac{\sin 5\theta}{5} + \cdots\right] \quad (38)$$

Inside the circle, multiplying by r, r^2, r^3,... gives fast decay of high frequencies:

Rapid decay inside $$u(r,\theta) = \frac{4}{\pi}\left[\frac{r\sin\theta}{1} + \frac{r^3\sin 3\theta}{3} + \frac{r^5\sin 5\theta}{5} + \cdots\right] \quad (39)$$

Laplace's equation has smooth solutions, even when $u_0(\theta)$ is not smooth.

■ WORKED EXAMPLE ■

A hot metal bar is moved into a freezer (zero temperature). The sides of the bar are coated so that heat only escapes at the ends. *What is the temperature* $u(x,t)$ along the bar at time t? It will approach $u = 0$ as all the heat leaves the bar.

Solution The heat equation is $u_t = u_{xx}$. At $t = 0$ the whole bar is at a constant temperature, say $u = 1$. The ends of the bar are at zero temperature for all time $t > 0$. This is an **initial-boundary value problem**:

Heat equation $u_t = u_{xx}$ with $u(x,0) = 1$ and $u(0,t) = u(\pi,t) = 0.$ (40)

Those zero boundary conditions suggest a sine series. Its coefficients depend on t:

Series solution of the heat equation
$$u(x,t) = \sum_{1}^{\infty} b_n(t)\, \sin nx. \tag{41}$$

The form of the solution shows **separation of variables**. In a comment below, we look for products $A(x)\,B(t)$ that solve the heat equation and the boundary conditions. What we reach is exactly $A(x) = \sin nx$ and the series solution (41).

Two steps remain. First, choose each $b_n(t)\sin nx$ to satisfy the heat equation:

Substitute into $u_t = u_{xx}$ $b_n'(t)\sin nx = -n^2 b_n(t)\, \sin nx$ $\quad b_n(t) = e^{-n^2 t}\, b_n(0).$

Notice $b_n' = -n^2 b_n$. Now determine each $b_n(0)$ from the initial condition $u(x,0) = 1$ on $(0,\pi)$. Those numbers are the Fourier sine coefficients of $SW(x)$ in equation (38):

Box function/square wave $\displaystyle\sum_{1}^{\infty} b_n(0)\, \sin nx = 1$ $\quad b_n(0) = \frac{4}{\pi n}$ for odd n

This completes the series solution of the initial-boundary value problem:

Bar temperature
$$u(x,t) = \sum_{\text{odd } n} \frac{4}{\pi n}\, e^{-n^2 t}\, \sin nx. \tag{42}$$

For large n (high frequencies) the decay of $e^{-n^2 t}$ is very fast. The dominant term $(4/\pi)e^{-t}\sin x$ for large times will come from $n = 1$. This is typical of the heat equation and all diffusion, that the solution (the temperature profile) becomes very smooth as t increases.

Numerical difficulty I regret any bad news in such a beautiful solution. To compute $u(x,t)$, we would probably truncate the series in (42) to N terms. When that finite series is graphed on the website, serious bumps appear in $u_N(x,t)$. You ask if there is a physical reason but there isn't. The solution should have maximum temperature at the midpoint $x = \pi/2$, and decay smoothly to zero at the ends of the bar.

Those unphysical bumps are precisely the **Gibbs phenomenon**. The initial $u(x,0)$ is 1 on $(0,\pi)$ but its odd reflection is -1 on $(-\pi,0)$. That jump has produced the slow $4/\pi n$ decay of the coefficients, with Gibbs oscillations near $x=0$ and $x=\pi$. The sine series for $u(x,t)$ is not a success numerically. Would finite differences help?

Separation of variables We found $b_n(t)$ as the coefficient of an eigenfunction $\sin nx$. Another good approach is to put $u = A(x)\,B(t)$ directly into $u_t = u_{xx}$:

$$\textbf{Separation} \quad A(x)\,B'(t) = A''(x)\,B(t) \text{ requires } \frac{A''(x)}{A(x)} = \frac{B'(t)}{B(t)} = \textbf{constant}. \tag{43}$$

A''/A is constant in space, B'/B is constant in time, and they are equal:

$$\frac{A''}{A} = -\lambda \text{ gives } A = \sin\sqrt{\lambda}\,x \text{ and } \cos\sqrt{\lambda}\,x \qquad \frac{B'}{B} = -\lambda \text{ gives } B = e^{-\lambda t}$$

The products $AB = e^{-\lambda t}\sin\sqrt{\lambda}\,x$ and $e^{-\lambda t}\cos\sqrt{\lambda}\,x$ solve the heat equation for any number λ. But the boundary condition $u(0,t) = 0$ eliminates the cosines. Then $u(\pi,t) = 0$ requires $\lambda = n^2 = 1, 4, 9, \ldots$ to have $\sin\sqrt{\lambda}\,\pi = 0$. Separation of variables has recovered the functions in the series solution (42).

Finally $u(x,0) = 1$ determines the numbers $4/\pi n$ for odd n. We find zero for even n because $\sin nx$ has $n/2$ positive loops and $n/2$ negative loops. For odd n, the extra positive loop is a fraction $1/n$ of all loops, giving slow decay of the coefficients.

Heat bath (the opposite problem) The solution on the website is $1 - u(x,t)$, because it solves a different problem. **The bar is initially frozen at** $U(x,0) = 0$. It is placed into a heat bath at the fixed temperature $U = 1$ (or $U = T_0$). The new unknown is U and its boundary conditions are no longer zero.

The heat equation and its boundary conditions are solved first by $U_B(x,t)$. In this example $U_B \equiv 1$ is constant. Then the difference $V = U - U_B$ has zero boundary values, and its initial values are $V = -1$. Now the eigenfunction method (or separation of variables) solves for V. (The series in (42) is multiplied by -1 to account for $V(x,0) = -1$.) Adding back U_B solves the heat bath problem: $U = U_B + V = 1 - u(x,t)$.

Here $U_B \equiv 1$ is the *steady state* solution at $t = \infty$, and V is the *transient* solution. The transient starts at $V = -1$ and decays quickly to $V = 0$.

Heat bath at one end The website problem is different in another way too. The Dirichlet condition $u(\pi,t) = 1$ is replaced by the Neumann condition $u'(1,t) = 0$. Only the left end is in the heat bath. Heat flows down the metal bar and out at the far end, now located at $x = 1$. How does the solution change for fixed-free?

Again $U_B = 1$ is a steady state. The boundary conditions apply to $V = 1 - U_B$:

$$\textbf{Fixed-free eigenfunctions} \quad V(0) = 0 \text{ and } V'(1) = 0 \text{ lead to } A(x) = \sin\left(n + \frac{1}{2}\right)\pi x. \tag{44}$$

Those eigenfunctions give a new form for the sum of $B_n(t)\,A_n(x)$:

Fixed-free solution
$$V(x,t) = \sum_{\text{odd } n} B_n(0)\, e^{-(n+\frac{1}{2})^2\pi^2 t} \sin\left(n+\frac{1}{2}\right)\pi x. \tag{45}$$

All frequencies shift by $\frac{1}{2}$ and multiply by π, because $A'' = -\lambda A$ has a free end at $x = 1$. The crucial question is: **Does orthogonality still hold for** these new eigenfunctions $\sin\left(n+\frac{1}{2}\right)\pi x$ **on** $[0,1]$? The answer is *yes* because this fixed-free "Sturm–Liouville problem" $A'' = -\lambda A$ is still symmetric.

Summary The series solutions all succeed but the truncated series all fail. We can see the overall behavior of $u(x,t)$ and $V(x,t)$. But their exact values close to the jumps are not computed well until we improve on Gibbs.

We could have solved the fixed-free problem on $[0,1]$ with the fixed-fixed solution on $[0,2]$. That solution will be symmetric around $x = 1$ so its slope there is zero. Then rescaling x by 2π changes $\sin(n+\frac{1}{2})\pi x$ into $\sin(2n+1)x$. I hope you like the graphics created by Aslan Kasimov on the cse website.

Problem Set 4.1

1 Find the Fourier series on $-\pi \le x \le \pi$ for

(a) $f(x) = \sin^3 x$, an odd function
(b) $f(x) = |\sin x|$, an even function
(c) $f(x) = x$
(d) $f(x) = e^x$, using the complex form of the series.

What are the even and odd parts of $f(x) = e^x$ and $f(x) = e^{ix}$?

2 From Parseval's formula the square wave sine coefficients satisfy
$$\pi(b_1^2 + b_2^2 + \cdots) = \int_{-\pi}^{\pi} |f(x)|^2\, dx = \int_{-\pi}^{\pi} 1\, dx = 2\pi.$$
Derive the remarkable sum $\pi^2 = 8(1 + \frac{1}{9} + \frac{1}{25} + \cdots)$.

3 If a square pulse is centered at $x = 0$ to give
$$f(x) = 1 \quad \text{for} \quad |x| < \frac{\pi}{2}, f(x) = 0 \quad \text{for} \quad \frac{\pi}{2} < |x| < \pi,$$
draw its graph and find its Fourier coefficients a_k and b_k.

4 Suppose f has period T instead of $2x$, so that $f(x) = f(x+T)$. Its graph from $-T/2$ to $T/2$ is repeated on each successive interval and its real and complex Fourier series are
$$f(x) = a_0 + a_1 \cos\frac{2\pi x}{T} + b_1 \sin\frac{2\pi x}{T} + \cdots = \sum_{-\infty}^{\infty} c_k\, e^{ik2\pi x/T}$$
Multiplying by the right functions and integrating from $-T/2$ to $T/2$, find a_k, b_k, and c_k.

5 Plot the first three partial sums and the function itself:

$$x(\pi - x) = \frac{8}{\pi}\left(\frac{\sin x}{1} + \frac{\sin 3x}{27} + \frac{\sin 5x}{125} + \cdots\right), 0 < x < \pi.$$

Why is $1/k^3$ the decay rate for this function? What is the second derivative?

6 What constant function is closest in the least square sense to $f = \cos^2 x$? What multiple of $\cos x$ is closest to $f = \cos^3 x$?

7 Sketch the 2π-periodic half wave with $f(x) = \sin x$ for $0 < x < \pi$ and $f(x) = 0$ for $-\pi < x < 0$. Find its Fourier series.

8 (a) Find the lengths of the vectors $u = (1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \ldots)$ and $v = (1, \frac{1}{3}, \frac{1}{9}, \ldots)$ in Hilbert space and test the Schwarz inequality $|u^{\mathrm{T}}v|^2 \le (u^{\mathrm{T}}u)(v^{\mathrm{T}}v)$.

(b) For the functions $f = 1 + \frac{1}{2}e^{ix} + \frac{1}{4}e^{2ix} + \cdots$ and $g = 1 + \frac{1}{3}e^{ix} + \frac{1}{9}e^{2ix} + \cdots$ use part (a) to find the numerical value of each term in

$$\left|\int_{-\pi}^{\pi} \overline{f}(x)\, g(x)\, dx\right|^2 \le \int_{-\pi}^{\pi} |f(x)|^2\, dx \int_{-\pi}^{\pi} |g(x)|^2\, dx.$$

Substitute for f and g and use orthogonality (or Parseval).

9 Find the solution to Laplace's equation with $u_0 = \theta$ on the boundary. Why is this the imaginary part of $2(z - z^2/2 + z^3/3 \cdots) = 2\log(1 + z)$? Confirm that on the unit circle $z = e^{i\theta}$, the imaginary part of $2\log(1 + z)$ agrees with θ.

10 If the boundary condition for Laplace's equation is $u_0 = 1$ for $0 < \theta < \pi$ and $u_0 = 0$ for $-\pi < \theta < 0$, find the Fourier series solution $u(r, \theta)$ inside the unit circle. What is u at the origin?

11 With boundary values $u_0(\theta) = 1 + \frac{1}{2}e^{i\theta} + \frac{1}{4}e^{2i\theta} + \cdots$, what is the Fourier series solution to Laplace's equation in the circle? Sum the series.

12 (a) Verify that the fraction in Poisson's formula satisfies Laplace's equation.

(b) What is the response $u(r, \theta)$ to an impulse at the point $(0, 1)$, at the angle $\varphi = \pi/2$?

(c) If $u_0(\varphi) = 1$ in the quarter-circle $0 < \varphi < \pi/2$ and $u_0 = 0$ elsewhere, show that at points on the horizontal axis (and especially at the origin)

$$u(r, 0) = \frac{1}{2} + \frac{1}{2\pi}\tan^{-1}\left(\frac{1 - r^2}{-2r}\right) \quad \text{by using}$$

$$\int \frac{d\varphi}{b + c\cos\varphi} = \frac{1}{\sqrt{b^2 - c^2}}\tan^{-1}\left(\frac{\sqrt{b^2 - c^2}\,\sin\varphi}{c + b\cos\varphi}\right).$$

13 When the centered square pulse in Example 7 has width $h = \pi$, find

(a) its energy $\int |F(x)|^2\,dx$ by direct integration

(b) its Fourier coefficients c_k as specific numbers

(c) the sum in the energy identity (31) or (24)

If $h = 2\pi$, why is $c_0 = 1$ the only nonzero coefficient? What is $F(x)$?

14 In Example 5, $F(x) = 1 + (\cos x)/2 + \cdots + (\cos nx)/2^n + \cdots$ is infinitely smooth:

(a) If you take 10 derivatives, what is the Fourier series of $d^{10}F/dx^{10}$?

(b) Does that series still converge quickly? Compare n^{10} with 2^n for n^{1024}.

15 (*A touch of complex analysis*) The analytic function in Example 5 blows up when $4\cos x = 5$. This cannot happen for real x, but equation (28) shows blowup if $e^{ix} = 2$ or $\frac{1}{2}$. In that case we have *poles at* $x = \pm i \log 2$. Why are there also poles at all the complex numbers $x = \pm i \log 2 + 2\pi n$?

16 (*A second touch*) Change 2's to 3's so that equation (28) has $1/(3 - e^{ix}) + 1/(3 - e^{-ix})$. Complete that equation to find the function that gives fast decay at the rate $1/3^k$.

17 (*For complex professors only*) Change those 2's and 3's to 1's:

$$\frac{1}{1 - e^{ix}} + \frac{1}{1 - e^{-ix}} = \frac{(1 - e^{-ix}) + (1 - e^{ix})}{(1 - e^{ix})(1 - e^{-ix})} = \frac{2 - e^{ix} - e^{-ix}}{2 - e^{ix} - e^{-ix}} = 1\,.$$

A constant! What happened to the pole at $e^{ix} = 1$? Where is the dangerous series $(1 + e^{ix} + \cdots) + (1 + e^{-ix} + \cdots) = 2 + 2\cos x + \cdots$ involving $\delta(x)$?

18 Following the Worked Example, solve the heat equation $u_t = u_{xx}$ from a point source $u(x, 0) = \delta(x)$ with free boundary conditions $u'(\pi, t) = u'(-\pi, t) = 0$. Use the infinite cosine series for $\delta(x)$ with time decay factors $b_n(t)$.

MIT OpenCourseWare
http://ocw.mit.edu

18.085 Computational Science and Engineering I
Fall 2008

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

LINEAR ALGEBRA IN A NUTSHELL

One question always comes on the first day of class. "*Do I have to know linear algebra*?" My reply gets shorter every year: "*You soon will.*" This section brings together many important points in the theory. It serves as a quick primer, not an official part of the applied mathematics course (like Chapter 1 and 2).

This summary begins with two lists that use most of the key words of linear algebra. The first list applies to invertible matrices. That property is described in 14 different ways. The second list shows the contrast, when A is singular (not invertible). There are more ways to test invertibility of an n by n matrix than I expected.

Nonsingular	Singular
A is invertible	A is not invertible
The columns are independent	The columns are dependent
The rows are independent	The rows are dependent
The determinant is not zero	The determinant is zero
$Ax = 0$ has one solution $x = 0$	$Ax = 0$ has infinitely many solutions
$Ax = b$ has one solution $x = A^{-1}b$	$Ax = b$ has no solution or infinitely many
A has n (nonzero) pivots	A has $r < n$ pivots
A has full rank	A has rank $r < n$
The reduced row echelon form is $R = I$	R has at least one zero row
The column space is all of $\mathbf{R}^n$	The column space has dimension $r < n$
The row space is all of $\mathbf{R}^n$	The row space has dimension $r < n$
All eigenvalues are nonzero	Zero is an eigenvalue of A
$A^{\mathrm{T}}A$ is symmetric positive definite	$A^{\mathrm{T}}A$ is only semidefinite
A has n (positive) singular values	A has $r < n$ singular values

Now we take a deeper look at linear equations, without proving every statement we make. The goal is to discover what $Ax = b$ really means. One reference is my textbook *Introduction to Linear Algebra*, published by Wellesley-Cambridge Press. That book has a much more careful development with many examples (you could look at the course page, with videos of the lectures, on ocw.mit.edu or web.mit.edu/18.06).

The key is to think of every multiplication Ax, a matrix A times a vector x, as a *combination of the columns of* A:

Matrix Multiplication by Columns

$$\begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} C \\ D \end{bmatrix} = C \begin{bmatrix} 1 \\ 3 \end{bmatrix} + D \begin{bmatrix} 2 \\ 6 \end{bmatrix} = \textbf{combination of columns}.$$

Multiplying by rows, the first component $C + 2D$ comes from 1 and 2 in the first row of A. But I strongly recommend to think of Ax **a column at a time**. Notice how

$x = (1, 0)$ and $x = (0, 1)$ will pick out single columns of A:

$$\begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \text{ first column} \qquad \begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \text{ last column}.$$

Suppose A is an m by n matrix. Then $Ax = 0$ has at least one solution, the all-zeros vector $x = 0$. There are certainly other solutions in case $n > m$ (more unknowns than equations). Even if $m = n$, there might be nonzero solutions to $Ax = 0$; then A is square but not invertible. It is the number r of *independent* rows and columns that counts. That number r is the **rank** of A ($r \leq m$ and $r \leq n$).

> The **nullspace** of A is the set of all solutions x to $Ax = 0$. This nullspace $\boldsymbol{N}(A)$ contains only $x = 0$ when the columns of A are **independent**. In that case the matrix A has full column rank $r = n$: independent columns.

For our 2 by 2 example, the combination with $C = 2$ and $D = -1$ produces the zero vector. Thus $x = (2, -1)$ is in the nullspace, with $Ax = 0$. The columns $(1, 3)$ and $(2, 6)$ are "linearly dependent." One column is a multiple of the other column. *The rank is* $r = 1$. The matrix A has a whole line of vectors $cx = c(2, -1)$ in its nullspace:

Nullspace is a line $\quad \begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ and also $\begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} \boldsymbol{2c} \\ \boldsymbol{-c} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$.

If $Ax = 0$ and $Ay = 0$, then every combination $cx + dy$ is in the nullspace. Always $Ax = 0$ asks for a combination of the columns of A that produces the zero vector:

x in nullspace $\quad x_1\,(\text{column } 1) + \cdots + x_n\,(\text{column } n) = \text{ zero vector}$

When those columns are independent, the only way to produce $Ax = 0$ is with $x_1 = 0$, $x_2 = 0$, ..., $x_n = 0$. Then $x = (0, \ldots, 0)$ is the only vector in the nullspace of A. Often this will be our requirement (independent columns) for a good matrix A. In that case, $A^{\mathrm{T}}A$ also has independent columns. The square n by n matrix $A^{\mathrm{T}}A$ is then invertible and symmetric and positive definite. If A is good then $A^{\mathrm{T}}A$ is even better.

I will extend this review (*still optional*) to the geometry of $Ax = b$.

Column Space and Solutions to Linear Equations

$Ax = b$ **asks for a linear combination of the columns that equals** b. In our 2 by 2 example, the columns go in the same direction! Then b does too:

Column space $\quad Ax = \begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} C \\ D \end{bmatrix}$ is always on the line through $\begin{bmatrix} 1 \\ 3 \end{bmatrix}$.

We can only solve $Ax = b$ when the vector b is on that line. For $b = (1, 4)$ there is no solution, it is off the line. For $b = (5, 15)$ there are many solutions (5 times column 1 gives b, and this b is on the line). The big step is to look at a space of vectors:

Definition: **The *column space* contains all combinations of the columns.**

In other words, $\boldsymbol{C}(A)$ contains all possible products A times x. Therefore $Ax = b$ is **solvable** exactly when the vector b is in the column space $\boldsymbol{C}(A)$.

For an m by n matrix, the columns have m components. The column space of A is in m-dimensional space. The word **"space"** indicates that the key operation of linear algebra is allowed: *Any combination of vectors in the space stays in the space.* The zero combination is allowed, so the vector $x = 0$ is in every space.

How do we write down all solutions, when b belongs to the column space of A? Any one solution to $Ax = b$ is a **particular solution $\boldsymbol{x_p}$**. Any vector $\boldsymbol{x_n}$ in the nullspace solves $Ax = 0$. Adding $Ax_p = b$ to $Ax_n = 0$ gives $A(x_p + x_n) = b$. **The complete solution to $Ax = b$ has this form $x = x_p + x_n$**:

Complete solution $x = x_{\text{particular}} + x_{\text{nullspace}} = (\text{one } x_p) + (\text{all } x_n)$.

In the example, $b = (5, 15)$ is 5 times the first column, so one particular solution is $x_p = (5, 0)$. To find all other solutions, add to x_p any vector x_n in the nullspace—which is the line through $(2, -1)$. Here is $x_p + (\text{all } x_n)$:

$$\begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} C \\ D \end{bmatrix} = \begin{bmatrix} 5 \\ 15 \end{bmatrix} \quad \text{gives} \quad x_{\text{complete}} = \begin{bmatrix} C \\ D \end{bmatrix} = \begin{bmatrix} 5 \\ 0 \end{bmatrix} + \begin{bmatrix} 2c \\ -c \end{bmatrix}.$$

This line of solutions is drawn in Figure A1. *It is not a subspace.* It does not contain $(0, 0)$, because it is shifted over by the particular solution $(5, 0)$. We only have a "space" of solutions when b is zero (then the solutions fill the nullspace).

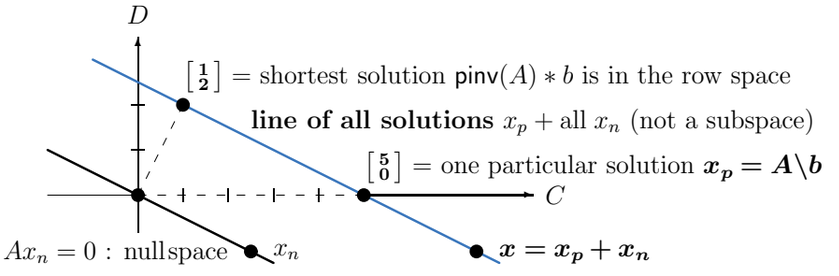


Figure A1: Parallel lines of solutions to $Ax_n = 0$ and $\left[\begin{smallmatrix} 1 & 2 \\ 3 & 6 \end{smallmatrix}\right](\boldsymbol{x_p + x_n}) = \left[\begin{smallmatrix} 5 \\ 15 \end{smallmatrix}\right]$

May I collect three important comments on linear equations $Ax = b$.

1. Suppose A is a square *invertible* matrix (the most common case in practice). Then the nullspace only contains $x_n = 0$. The particular solution $x_p = A^{-1}b$ is the only solution. The complete solution $x_p + x_n$ is $A^{-1}b + 0$. Thus $x = A^{-1}b$.

2. $Ax = b$ has infinitely many solutions in Figure A1. The shortest x always lies in the "row space" of A. That particular solution $(1, 2)$ is found by the *pseudo-inverse* pinv (A). The backslash $A\backslash b$ finds an x with at most m nonzeros.

3. Suppose A is tall and thin ($m > n$). The n columns are likely to be independent. But if b is not in the column space, $Ax = b$ has no solution. The least squares method minimizes $\|b - Ax\|^2$ by solving $A^{\mathrm{T}}A\widehat{x} = A^{\mathrm{T}}b$.

The Four Fundamental Subspaces

The nullspace $\boldsymbol{N}(A)$ contains all solutions to $Ax = 0$. The column space $\boldsymbol{C}(A)$ contains all combinations of the columns. When A is m by n, $\boldsymbol{N}(A)$ is a subspace of $\mathbf{R}^n$ and $\boldsymbol{C}(A)$ is a subspace of $\mathbf{R}^m$.

The other two fundamental spaces come from the transpose matrix A^{T}. They are $\boldsymbol{N}(A^{\mathrm{T}})$ and $\boldsymbol{C}(A^{\mathrm{T}})$. We call $\boldsymbol{C}(A^{\mathrm{T}})$ the "row space of A" because the rows of A are the columns of A^{T}. What are those spaces for our 2 by 2 example?

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 6 \end{bmatrix} \quad \text{transposes to} \quad A^{\mathrm{T}} = \begin{bmatrix} 1 & 3 \\ 2 & 6 \end{bmatrix}.$$

Both columns of A^{T} are in the direction of $(1, 2)$. The line of all vectors $(c, 2c)$ is $\boldsymbol{C}(A^{\mathrm{T}})$ = row space of A. The nullspace of A^{T} is in the direction of $(3, -1)$:

Nullspace of $\boldsymbol{A^{\mathrm{T}}}$ $\quad A^{\mathrm{T}}y = \begin{bmatrix} 1 & 3 \\ 2 & 6 \end{bmatrix} \begin{bmatrix} E \\ F \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ gives $\begin{bmatrix} E \\ F \end{bmatrix} = \begin{bmatrix} 3c \\ -c \end{bmatrix}$.

The four subspaces $\boldsymbol{N}(A)$, $\boldsymbol{C}(A)$, $\boldsymbol{N}(A^{\mathrm{T}})$, $\boldsymbol{C}(A^{\mathrm{T}})$ combine beautifully into the big picture of linear algebra. Figure A2 shows how the nullspace $\boldsymbol{N}(A)$ is perpendicular to the row space $\boldsymbol{C}(A^{\mathrm{T}})$. Every input vector x splits into a row space part x_r and a nullspace part x_n. Multiplying by A always(!) produces a vector in the column space. Multiplication goes from left to right in the picture, from x to $Ax = b$.

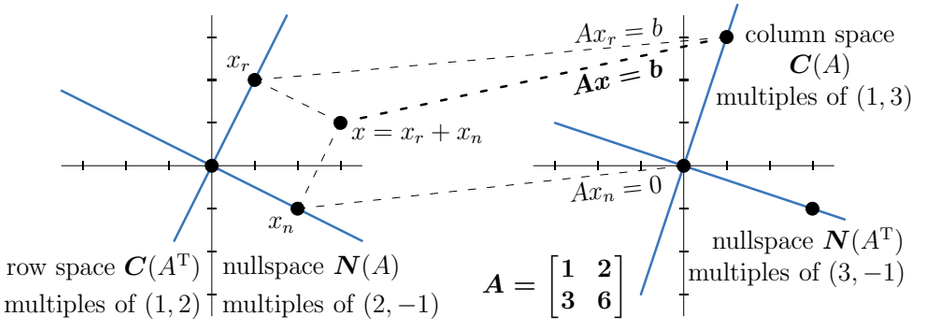


Figure A2: The four fundamental subspaces (lines) for the singular matrix A.

On the right side are the column space $\boldsymbol{C}(A)$ and the fourth space $\boldsymbol{N}(A^{\mathrm{T}})$. **Again they are perpendicular**. The columns are multiples of $(1,3)$ and the y's are multiples of $(3,-1)$. If A were an m by n matrix, its columns would be in m-dimensional space $\mathbf{R}^m$ and so would the solutions to $A^{\mathrm{T}}y = 0$. Our singular 2 by 2 example has $m = n = 2$, and all four fundamental subspaces in Figure A2 are lines in $\mathbf{R}^2$.

This figure needs more words. Each subspace contains infinitely many vectors, or only the zero vector $x = 0$. If u is in a space, so are $10u$ and $-100u$ (and most importantly $0u$). We measure the ***dimension*** of a space not by the number of vectors, which is infinite, but by *the number of independent vectors*. In this example each dimension is 1. A line has one independent vector but not two

Dimension and Basis

A full set of independent vectors is a "**basis**" for a space. This idea is important. The basis has as many independent vectors as possible, and their combinations fill the space. **A basis has not too many vectors, and not too few**:

1. The basis vectors are **linearly independent**.

2. Every vector in the space is a **unique combination** of those basis vectors.

Here are particular bases for $\mathbf{R}^n$ among all the choices we could make:

Standard basis	=	columns of the **identity** matrix
General basis	=	columns of any **invertible** matrix
Orthonormal basis	=	columns of any **orthogonal** matrix

The "dimension" of the space is the number of vectors in a basis.

Difference Matrices

Difference matrices with boundary conditions give exceptionally good examples of the four subspaces (and there is a physical meaning behind them). We choose forward and backward differences that produce 2 by 3 and 3 by 2 matrices:

Forward $\boldsymbol{\Delta_+}$
Backward $\boldsymbol{-\Delta_-}$

$$A = \begin{bmatrix} -1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix} \quad \text{and} \quad A^{\mathrm{T}} = \begin{bmatrix} -1 & 0 \\ 1 & -1 \\ 0 & 1 \end{bmatrix}.$$

A is imposing no boundary conditions (no rows are chopped off). Then A^{T} must impose two boundary conditions and it does: $+1$ disappeared in the first row and -1 in the third row. $A^{\mathrm{T}}w = f$ builds in the boundary conditions $w_0 = 0$ and $w_3 = 0$.

The nullspace of A contains $x = (1,1,1)$. Every constant vector $x = (c,c,c)$ solves $Ax = 0$, and *the nullspace* $\boldsymbol{N}(A)$ *is a line* in three-dimensional space. The row space of A is the plane through the rows $(-1,1,0)$ and $(0,-1,1)$. Both vectors are perpendicular to $(1,1,1)$ so **the whole row space is perpendicular to the nullspace**. Those two spaces are on the left side (the 3D side) of Figure A3.

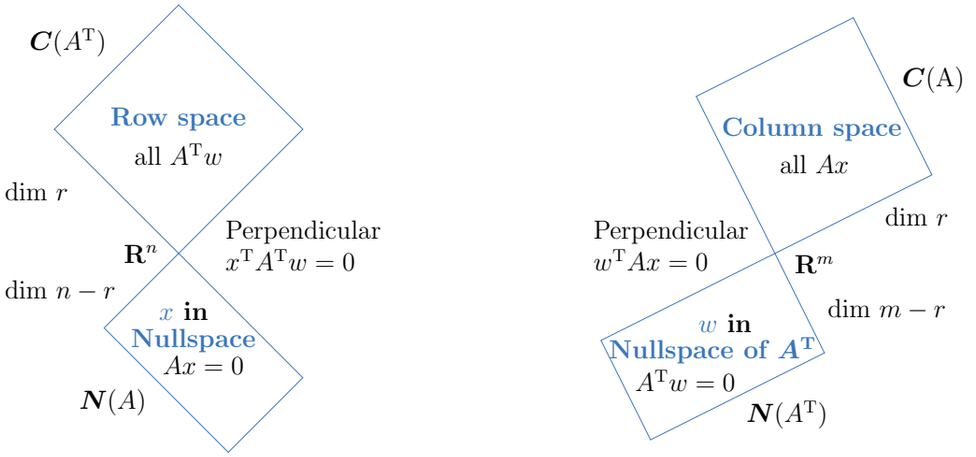


Figure A3: Dimensions and orthogonality for any m by n matrix A of rank r.

Figure A3 shows the **Fundamental Theorem of Linear Algebra**:

1. The row space in $\mathbf{R}^n$ and column space in $\mathbf{R}^m$ have the same dimension r.

2. The nullspaces $\boldsymbol{N}(A)$ and $\boldsymbol{N}(A^{\mathrm{T}})$ have dimensions $n-r$ and $m-r$.

3. $\boldsymbol{N}(A)$ is perpendicular to the row space $\boldsymbol{C}(A^{\mathrm{T}})$.

4. $\boldsymbol{N}(A^{\mathrm{T}})$ is perpendicular to the column space $\boldsymbol{C}(A)$.

The dimension r of the column space is the "rank" of the matrix. It equals the number of (nonzero) pivots in elimination. The matrix has full column rank when $r = n$ and the columns are linearly independent; the nullspace only contains $x = 0$. Otherwise some nonzero combination x of the columns produces $Ax = 0$.

The dimension of the nullspace is $n-r$. There are n unknowns in $Ax = 0$, and there are really r equations. Elimination leaves $n-r$ columns without pivots. The corresponding unknowns are free (give them any values). This produces $n-r$ independent solutions to $Ax = 0$, a basis for the nullspace.

A good basis makes scientific computing possible:

1 Sines and cosines **2** Finite elements **3** Splines **4** Wavelets

A basis of eigenvectors is often the best.

MIT OpenCourseWare
http://ocw.mit.edu

18.085 Computational Science and Engineering I
Fall 2008

For information about citing these materials or our Terms of Use, visit: http://ocw.mit.edu/terms.

Gradient and Divergence / Parallel Table

Gradient $v = \operatorname{grad} u = \nabla u$	**Divergence** $\operatorname{div} w = \nabla \cdot w = 0$
Potential $u(x,y)$: $v_1 = \dfrac{\partial u}{\partial x}, v_2 = \dfrac{\partial u}{\partial y}$	Stream function $s(x,y)$: $w_1 = \dfrac{\partial s}{\partial y}, w_2 = -\dfrac{\partial s}{\partial x}$
Test on v: $\dfrac{\partial v_2}{\partial x} - \dfrac{\partial v_1}{\partial y} = 0$	Test on w: $\dfrac{\partial w_1}{\partial x} + \dfrac{\partial w_2}{\partial y} = 0$
Irrotational: zero vorticity	Solenoidal: zero source
Zero circulation around loops: $\int v \cdot t \, ds = \int v_1 \, dx + v_2 \, dy = 0$	Zero flux through loops: $\int w \cdot n \, ds = \int w_1 \, dy - w_2 \, dx = 0$
Kirchhoff's Voltage Law	Kirchhoff's Current Law
Equipotentials $u(x,y) =$ constant	Streamlines $s(x,y) =$ constant
v is perpendicular to equipotentials	w is tangent to streamlines

---

Green-Gauss Formula $\displaystyle\iint w \cdot \operatorname{grad} u \, dx \, dy = \iint u(-\operatorname{div} w) \, dx \, dy + \int u \, w \cdot n \, ds$

$(\mathbf{grad})^{\mathbf{T}} = -\mathbf{div}$ from integration by parts: $(\boldsymbol{Au})^{\mathbf{T}}\boldsymbol{w} = \boldsymbol{u}^{\mathbf{T}}(\boldsymbol{A}^{\mathbf{T}}\boldsymbol{w})$

Connections when $(\boldsymbol{v_1}, \boldsymbol{v_2}) = (\boldsymbol{w_1}, \boldsymbol{w_2})$

1. Equipotentials are perpendicular to streamlines
2. Laplace's equation $\operatorname{div}(\operatorname{grad} u) = \dfrac{\partial}{\partial x}\left(\dfrac{\partial u}{\partial x}\right) + \dfrac{\partial}{\partial y}\left(\dfrac{\partial u}{\partial y}\right) = \nabla \cdot \nabla u = 0$
3. Cauchy-Riemann equations $\dfrac{\partial u}{\partial x} = \dfrac{\partial s}{\partial y}$ and $\dfrac{\partial u}{\partial y} = -\dfrac{\partial s}{\partial x}$ connecting u to s
4. Laplace's equation for s $\dfrac{\partial^2 s}{\partial x^2} + \dfrac{\partial^2 s}{\partial y^2} = -\dfrac{\partial^2 u}{\partial x \partial y} + \dfrac{\partial^2 u}{\partial y \partial x} = 0$
5. Zero vorticity and zero source: Ideal potential flow
6. In two dimensions: $u(x,y) + is(x,y)$ is a function $f(x+iy)$

©2006 Gilbert Strang

3.5 Finite Differences and Fast Poisson Solvers

It is extremely unusual to use eigenvectors to solve a linear system $KU = F$. You need to know all the eigenvectors of K, and (much more than that) the eigenvector matrix S must be especially fast to work with. Both S and S^{-1} are required, because $K^{-1} = S\Lambda^{-1}S^{-1}$. The eigenvalue matrices Λ and Λ^{-1} are diagonal and quick. When the derivatives in Poisson's equation $-u_{xx} - u_{yy} = f(x, y)$ are replaced by second differences, we do know the eigenvectors (discrete sines in the columns of S). Then the Fourier transform quickly inverts and multiplies by S.

On a square mesh those differences have $-1, 2, -1$ in the x-direction and $-1, 2, -1$ in the y-direction (divided by h^2, where h = meshwidth). Figure 3.20 shows how the second differences combine into a "5-*point molecule*" for the discrete Laplacian. Boundary values $u = u_0(x, y)$ are assumed to be given along the sides of a unit square.

The square mesh has N interior points in each direction ($N = 5$ in the figure). In this case there are $n = N^2 = 25$ unknown mesh values U_{ij}. When the molecule is centered at position (i, j), the discrete Poisson equation gives a row of $K2D\,U = F$:

$$\boldsymbol{K2D\,U = F} \qquad 4u_{ij} - u_{i,\,j-1} - u_{i-1,\,j} - u_{i+1,\,j} - u_{i,\,j+1} = h^2 f(ih, jh) = F_{ij}\,. \quad (1)$$

The inside rows of $K2D$ have five nonzero entries $4, -1, -1, -1, -1$. When (i, j) is next to a boundary point of the square, the known value of u_0 at that neighboring boundary point moves to the right side of equation (1). It becomes part of the vector F, and a -1 drops out of the corresponding row of K. So $K2D$ has five nonzeros on inside rows and fewer nonzeros on next-to-boundary rows.

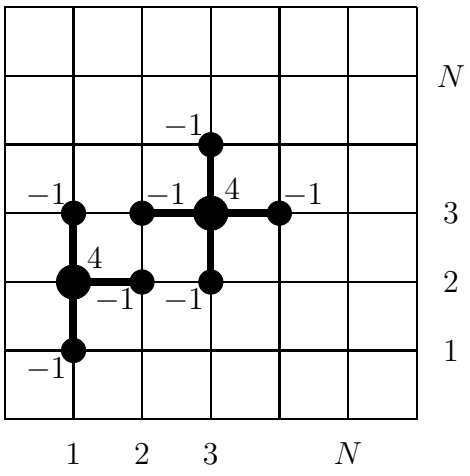


Figure 3.20: 5-point molecules at inside points (fewer -1's next to boundary).

This matrix $K2D$ is sparse. Using blocks of size N, we can create the 2D matrix from the familiar N by N second difference matrix K. Number the nodes of the square a row at a time (this "natural numbering" is not necessarily best). Then the -1's for the neighbor above and the neighbor below are N positions away from the

main diagonal of $\boldsymbol{K2D}$. The 2D matrix is *block tridiagonal with tridiagonal blocks*:

$$K = \begin{bmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & \cdot & \cdot & \cdot \\ & & -1 & 2 \end{bmatrix} \qquad \boldsymbol{K2D} = \begin{bmatrix} K+2I & -I & & \\ -I & K+2I & -I & \\ & \cdot & \cdot & \cdot \\ & & -I & K+2I \end{bmatrix} \tag{2}$$

Size N **Size $n = N^2$** **Bandwidth $w = N$**

Time N **Space $nw = N^3$** **Time $nw^2 = N^4$**

The matrix $K2D$ has 4's down the main diagonal in equation (1). Its bandwidth $w = N$ is the distance from the main diagonal to the nonzeros in $-I$. Many of the spaces in between are filled during elimination! This is discussed in Section 6.1.

Kronecker product One good way to create $K2D$ from K and I is by the `kron` command. When A and B have size N by N, the matrix $\texttt{kron}(A, B)$ is N^2 by N^2. *Each number a_{ij} is replaced by the block $a_{ij}B$.*

To take second differences in all rows at the same time, $\texttt{kron}(I, K)$ produces a block diagonal matrix of K's. In the y-direction, $\texttt{kron}(K, I)$ multiplies -1 and 2 and -1 by I (dealing with a column of meshpoints at a time). Add x and y differences:

$$\boldsymbol{K2D} = \texttt{kron}(I, K) + \texttt{kron}(K, I) = \begin{bmatrix} K & & \\ & K & \\ & & \cdot \end{bmatrix} + \begin{bmatrix} 2I & -I & \cdot \\ -I & 2I & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} \tag{3}$$

This sum agrees with the 5-point matrix in (2). The computational question is how to work with $K2D$. We will propose three methods:

1. **Elimination in a good order** (not using the special structure of $K2D$)
2. **Fast Poisson Solver** (applying the FFT = Fast Fourier Transform)
3. **Odd-Even Reduction** (since $K2D$ is block tridiagonal).

The novelty is in the Fast Poisson Solver, which uses the known eigenvalues and eigenvectors of K and $K2D$. It is strange to solve linear equations $KU = F$ by expanding F and U in eigenvectors, but here it is extremely successful.

Elimination and Fill-in

For most two-dimensional problems, elimination is the way to go. The matrix from a partial differential equation is sparse (like $K2D$). It is banded but the bandwidth is not so small. (Meshpoints cannot be numbered so that all five neighbors in the molecule receive nearby numbers.) Figure 3.21 has points $1, \ldots, N$ along the first row, then a row at a time going up the square. *The neighbors above and below point j have numbers $j - N$ and $j + N$.*

©2006 Gilbert Strang

Ordering by rows produces the -1's in $K2D$ that are N places away from the diagonal. The matrix $K2D$ has bandwidth N. **The key point is that elimination fills in the zeros inside the band.** We add row 1 (times $\frac{1}{4}$) to row 2, to eliminate the -1 in position $(2,1)$. But the last -1 in row 1 produces a new $-\frac{1}{4}$ in row 2. A zero inside the band has disappeared. As elimination continues from A to U, virtually the whole band is filled in.

In the end, U has about 5 times 25 nonzeros (this is N^3, the space needed to store U). There will be about N nonzeros next to the pivot when we reach a typical row, and N nonzeros below the pivot. Row operations to remove those nonzeros will require up to N^2 multiplications, and there are N^2 pivots. So the count of multiplications is about 25 times 25 (this is N^4, for elimination in 2D).

Figure 3.21: Typical rows of $K2D$ have 5 nonzeros. Elimination fills in the band.

Section 6.1 will propose a different numbering of the meshpoints, to reduce the fill-in that we see in U. This reorders the rows of $K2D$ by a permutation matrix P, and the columns by P^{T}. The new matrix $P(K2D)P^{\mathrm{T}}$ is still symmetric, but elimination (with fill-in) proceeds in a completely different order. The MATLAB command `symamd`(K) produces a nearly optimal choice of P.

Elimination is fast in two dimensions (but a Fast Poisson Solver is faster!). In three dimensions the matrix size is N^3 and the bandwidth is N^2. By numbering the nodes a plane at a time, vertical neighbors are N^2 nodes apart. The operation count for elimination becomes N^7, which can be seriously large. Chapter 6 on *Solving Large Systems* will introduce badly needed alternatives to elimination in 3D.

Solvers Using Eigenvalues

Our matrices K and $K2D$ are extremely special. We know the eigenvalues and eigenvectors of the second-difference matrix K. The eigenvalues have the special form $\lambda = 2-2\cos\theta$, and the eigenvectors are discrete sines. There will be a similar pattern for $K2D$, which is formed in a neat way from K (by Kronecker product). The Poisson Solver uses those eigenvalues and eigenvectors to solve $(K2D)(U2D) = (F2D)$. On a square mesh it is much faster than elimination.

Here is the idea, first in one dimension. The matrix K has eigenvalues $\lambda_1, \ldots, \lambda_N$ and eigenvectors $y_1, \ldots, y_N$. There are three steps to the solution of $KU = F$:

1. Expand F as a combination $F = a_1 y_1 + \cdots + a_N y_N$ of the eigenvectors

2. Divide each a_k by λ_k

3. Recombine eigenvectors into $U = (a_1/\lambda_1)\, y_1 + \cdots + (a_N/\lambda_N)\, y_N$.

©2006 Gilbert Strang

The success of the method depends on the speed of steps 1 and 3. Step 2 is fast.

To see that U in step 3 is correct, *multiply it by the matrix* K. Every eigenvector gives $Ky = \lambda y$. That cancels the λ in each denominator. The result is that KU agrees with the vector F in step 1.

Now look at the calculation required in each step, using matrices. Suppose S is the **eigenvector matrix**, with the eigenvectors $y_1, \dots, y_N$ of K in its columns. Then the coefficients $a_1, \dots, a_N$ come by solving $Sa = F$:

$$\textbf{Step 1} \quad \textbf{Solve } \boldsymbol{Sa = F} \qquad \begin{bmatrix} y_1 & \cdots & y_N \end{bmatrix} \begin{bmatrix} a_1 \\ \vdots \\ a_N \end{bmatrix} = a_1 y_1 + \cdots + a_N y_N = F\,. \quad (4)$$

Thus $a = S^{-1}F$. Then step 2 divides the a's by the λ's to find $\Lambda^{-1}a = \Lambda^{-1}S^{-1}F$. (The **eigenvalue matrix** Λ is just the diagonal matrix of λ's.) Step 3 uses those coefficients a_k/λ_k in recombining the eigenvectors into the solution vector U:

$$\textbf{Step 3} \qquad U = \begin{bmatrix} y_1 & \cdots & y_N \end{bmatrix} \begin{bmatrix} a_1/\lambda_1 \\ \vdots \\ a_N/\lambda_N \end{bmatrix} = \boldsymbol{S\Lambda^{-1}a = S\Lambda^{-1}S^{-1}F}\,. \quad (5)$$

You see the matrix $K^{-1} = S\Lambda^{-1}S^{-1}$ appearing in that last formula. We have $K^{-1}F$ because K itself is $S\Lambda S^{-1}$—the usual diagonalization of a matrix. The eigenvalue method is using this $S\Lambda S^{-1}$ factorization instead of $K = LU$ from elimination.

The speed of steps 1 and 3 depends on multiplying quickly by S^{-1} and S. Those are full matrices, not sparse like K. Normally they both need N^2 operations in one dimension (where the matrix size is N). But the "sine eigenvectors" in S give the Discrete Sine Transform, and the *Fast Fourier Transform* executes S and S^{-1} in $N \log_2 N$ steps. In one dimension this is not as fast as cN from elimination, but in two dimensions $N^2 \log(N^2)$ easily wins.

Column k of the matrix S contains the eigenvector y_k. The number $S_{jk} = \sin \frac{jk\pi}{N+1}$ is the jth component of that eigenvector. For our example with $N = 5$ and $N+1 = 6$, you could list the sines of every multiple of $\pi/6$. Here are those numbers:

$$\textbf{Sines } \frac{1}{2}, \frac{\sqrt{3}}{2}, 1, \frac{\sqrt{3}}{2}, \frac{1}{2}, 0, \ \textbf{(repeat with minus signs)} \ \ \textbf{(repeat 12 numbers forever)}\,.$$

The kth column of S (kth eigenvector y_k) takes every kth number from that list:

$$y_1 = \begin{bmatrix} 1/2 \\ \sqrt{3}/2 \\ 1 \\ \sqrt{3}/2 \\ 1/2 \end{bmatrix} \quad y_2 = \begin{bmatrix} \sqrt{3}/2 \\ \sqrt{3}/2 \\ 0 \\ -\sqrt{3}/2 \\ -\sqrt{3}/2 \end{bmatrix} \quad y_3 = \begin{bmatrix} 1 \\ 0 \\ -1 \\ 0 \\ 1 \end{bmatrix} \quad y_4 = \begin{bmatrix} \sqrt{3}/2 \\ -\sqrt{3}/2 \\ 0 \\ \sqrt{3}/2 \\ -\sqrt{3}/2 \end{bmatrix} \quad y_5 = \begin{bmatrix} 1/2 \\ -\sqrt{3}/2 \\ 1 \\ -\sqrt{3}/2 \\ 1/2 \end{bmatrix}$$

Those eigenvectors are orthogonal! This is guaranteed by the symmetry of K. All these eigenvectors have length $3 = (N+1)/2$. Dividing each column by $\sqrt{3}$, we have *orthonormal eigenvectors*. $S/\sqrt{3}$ is an orthogonal matrix Q, with $Q^{\mathrm{T}}Q = I$.

©2006 Gilbert Strang

In this special case, S and Q are also symmetric. So $Q^{-1} = Q^{\mathrm{T}} = Q$.

Notice that y_k has $k-1$ changes of sign. It comes from k loops of the sine curve. The eigenvalues are increasing: $\lambda = 2-\sqrt{3},\ 2-1,\ 2-0,\ 2+1,\ 2+\sqrt{3}$. Those eigenvalues add to 10, which is the sum down the diagonal (the *trace*) of K_5. The product of the 5 eigenvalues (easiest by pairs) confirms that $\det(K_5) = 6$.

Fast Poisson Solvers

To extend this eigenvalue method to two dimensions, we need the eigenvalues and eigenvectors of $K2D$. The key point is that the N^2 eigenvectors of $K2D$ are *separable*. Each eigenvector y_{kl} separates into a product of sines:

Eigenvectors y_{kl} The (i,j) component of y_{kl} is $\sin\dfrac{ik\pi}{N+1}\sin\dfrac{jl\pi}{N+1}$. (6)

When you multiply that eigenvector by $K2D$, you have second differences in the x-direction and y-direction. The second differences of the first sine (x-direction) produce a factor $\lambda_k = 2-2\cos\frac{k\pi}{N+1}$. This is the eigenvalue of K in 1D. The second differences of the other sine (y-direction) produce a factor $\lambda_l = 2-2\cos\frac{l\pi}{N+1}$. So the eigenvalue in two dimensions is the sum $\lambda_k + \lambda_l$ of one-dimensional eigenvalues:

$$\boldsymbol{(K2D)y_{kl} = \lambda_{kl}y_{kl}} \qquad \lambda_{kl} = (2-2\cos\tfrac{k\pi}{N+1}) + (2-2\cos\tfrac{l\pi}{N+1})\,. \tag{7}$$

Now the solution of $\boldsymbol{K2D\,U = F}$ comes by a *two-dimensional sine transform*:

$$F_{i,j} = \sum\sum a_{kl}\sin\frac{ik\pi}{N+1}\sin\frac{jl\pi}{N+1} \qquad U_{i,j} = \sum\sum \frac{a_{kl}}{\lambda_{kl}}\sin\frac{ik\pi}{N+1}\sin\frac{jl\pi}{N+1} \tag{8}$$

Again we find the a's, divide by the λ's, and build U from the eigenvectors in S:

Step 1 $a = S^{-1}F$ **Step 2** $\Lambda^{-1}a = \Lambda^{-1}S^{-1}F$ **Step 3** $U = S\Lambda^{-1}S^{-1}F$

Swartztrauber [SIAM Review **19** (1977) 490] gives the operation count $2N^2\log_2 N$. This uses the Fast Sine Transform (based on the FFT) to multiply by S^{-1} and S. The Fast Fourier Transform is explained in Section 4.3.

Note We take this chance to notice the good properties of a Kronecker product $C = \mathtt{kron}(A,B)$. Suppose A and B have their eigenvectors in the columns of S_A and S_B. The eigenvalues are in Λ_A and Λ_B. Then we know S_C and Λ_C:

$$\begin{gathered}\textit{The eigenvectors of } \mathtt{kron}(\boldsymbol{A,B}) \textit{ are in } \mathtt{kron}(\boldsymbol{S_A,S_B}).\\ \textit{The eigenvalues are in } \mathtt{kron}(\boldsymbol{\Lambda_A,\Lambda_B}).\end{gathered} \tag{9}$$

The diagonal blocks in $\mathtt{kron}(\Lambda_A,\Lambda_B)$ are entries $\lambda_k(A)$ times the diagonal matrix Λ_B. So the eigenvalues λ_{kl} of C are just the products $\lambda_k(A)\lambda_l(B)$.

In our case A and B were I and K. The matrix $K2D$ added the two products $\texttt{kron}(I,K)$ and $\texttt{kron}(K,I)$. Normally we cannot know the eigenvectors and eigenvalues of a matrix sum—except when the matrices commute. Since all our matrices are formed from K, *these Kronecker products do commute.* This gives the separable eigenvectors and eigenvalues in (6) and (7).

Cyclic Odd-Even Reduction

There is an entirely different (and very simple) approach to $\mathbf{KU} = \mathbf{F}$. I will start in one dimension, by writing down three rows of the usual second difference equation:

$$\begin{array}{lrcl} \text{Row } i-1 & -U_{i-2} + 2U_{i-1} - U_i & = & F_{i-1} \\ \text{Row } i & -U_{i-1} + 2U_i - U_{i+1} & = & F_i \\ \text{Row } i+1 & -U_i + 2U_{i+1} - U_{i+2} & = & F_{i+1} \end{array} \tag{10}$$

Multiply the middle equation by 2, and add. *This eliminates U_{i-1} and U_{i+1}*:

Odd-even reduction in 1D $$-U_{i-2} + 2U_i - U_{i+2} = F_{i-1} + 2F_i + F_{i+1}\,. \tag{11}$$

Now we have a **half-size system**, involving only half of the U's (with even indices). The new system (11) has the same tridiagonal form as before. When we repeat, *cyclic reduction produces a quarter-size system.* Eventually we can reduce $KU = F$ to a very small problem, and then cyclic back-substitution produces the whole solution.

How does this look in two dimensions? The big matrix $K2D$ is block triangular:

$$\boldsymbol{K2D} = \begin{bmatrix} A & -I & & \\ -I & A & -I & \\ & \cdot & \cdot & \cdot \\ & & -I & A \end{bmatrix} \quad \text{with} \quad A = K + 2I \quad \text{from equation (4)}\,. \tag{12}$$

The three equations in (10) become *block equations for whole rows of N mesh values.* We are taking the unknowns $\mathbf{U}_i = (U_{i1}, \ldots, U_{iN})$ a row at a time. If we write three rows of (10), the block A replaces the number 2 in the scalar equation. The block $-I$ replaces the number -1. To reduce $(K2D)(U2D) = (F2D)$ to a half-size system, multiply the middle equation (with i even) by A and add the three block equations:

Reduction in 2D $$-I\mathbf{U}_{i-2} + (A^2 - 2I)\mathbf{U}_i - I\mathbf{U}_{i+2} = \mathbf{F}_{i-1} + A\mathbf{F}_i + \mathbf{F}_{i+1}\,. \tag{13}$$

The new half-size matrix is still block tridiagonal. *The diagonal blocks that were previously A in* (12) *are now $A^2 - 2I$*, with the same eigenvectors. The unknowns are the $\frac{1}{2}N^2$ values $U_{i,j}$ at meshpoints with even indices i.

The bad point is that each new diagonal block $A^2 - 2I$ has *five diagonals*, where the original block $A = K + 2I$ had three diagonals. This bad point gets worse as cyclic reduction continues. At step r, the diagonal blocks become $A_r = A_{r-1}^2 - 2I$ and their bandwidth doubles. We could find tridiagonal factors $(A - \sqrt{2}I)(A + \sqrt{2}I) = A^2 - 2I$,

©2006 Gilbert Strang

but the number of factors grows quickly. Storage and computation and roundoff error are increasing too rapidly with more reduction steps.

Stable variants of cyclic reduction were developed by Buneman and Hockney. The clear explanation by Buzbee, Golub, and Nielson [SIAM Journal of Numerical Analysis **7** (1970) 627] allows other boundary conditions and other separable equations, coming from polar coordinates or convection terms like $C\partial u/\partial x + D\partial u/\partial y$. After m steps of cyclic reduction, Hockney went back to a Fast Poisson Solver.

This combination FACR(m) is widely used, and the optimal number of cyclic reduction steps (before the FFT takes over) is small. For $N = 128$ a frequent choice is $m = 2$. Asymptotically m_{opt} grows like $\log\log N$ and the operation count for FACR(m_{opt}) is $3N^2 \log\log N$. In practical scientific computing with N^2 unknowns (or with N^3 unknowns in three dimensions), a Fast Poisson Solver is a winner.

******** Add code for Fast Poisson *********

Problem Set 3.5

Problems 1– are for readers who get enthusiastic about `kron`.

1 Why is the transpose of $C = \texttt{kron}(A, B)$ equal to $\texttt{kron}(A^{\mathrm{T}}, B^{\mathrm{T}})$? Why is the inverse equal to $C^{-1} = \texttt{kron}(A^{-1}, B^{-1})$? You have to transpose each block $a_{ij}B$ of the Kronecker product C, and then patiently multiply CC^{-1} by blocks.

C is symmetric (or orthogonal) when A and B are symmetric (or orthogonal).

2 Why is the matrix $C = \texttt{kron}(A, B)$ times the matrix $D = \texttt{kron}(S, T)$ equal to $CD = \texttt{kron}(AS, BT)$? This needs even more patience with block multiplication. The inverse $CC^{-1} = \texttt{kron}(I, I) = I$ is a special case.

Note Suppose S and T are eigenvector matrices for A and B. From $AS = S\Lambda_A$ and $BT = T\Lambda_B$ we have $CD = \texttt{kron}(AS, BT) = \texttt{kron}(S\Lambda_A, T\Lambda_B)$. Then $CD = D\,\texttt{kron}(\Lambda_A, \Lambda_B) = D\Lambda_C$. So $D = \texttt{kron}(S, T)$ is the eigenvector matrix for C.

Chapter 5

Initial Value Problems

5.1 Finite Difference Methods

We don't plan to study highly complicated nonlinear differential equations. Our first goal is to see why a difference method is successful (or not). The crucial questions of **stability and accuracy** can be clearly understood for linear equations. Then we can construct difference approximations to a great variety of practical problems.

Another property we need is **computational speed**. That depends partly on the complexity of the equation $u' = f(u,t)$. Often we measure speed by the number of times that $f(u,t)$ has to be computed in each time step (that number could be one). When we turn to *implicit* methods and *predictor-corrector* methods, to improve stability, the cost per step goes up but we gain speed with a larger step Δt.

This chapter begins with basic methods (forward Euler, backward Euler) and then improves. Each time, we test stability on $u' = a\,u$. When a is negative, Δt is often limited by $-a\,\Delta t \le C$. This has an immediate effect: the equation with $a = -99$ requires a much smaller Δt than $a = -1$. Let me organize the equations as scalar and vector, nonlinear in general or linear with constant coefficients a and A:

1 equation	$u' = f(u,t)$	$u' = au$	$\boldsymbol{a \approx \partial f/\partial u}$	$\boldsymbol{a \le 0}$
$\boldsymbol{N}$ **equations**	$u_i' = f_i(u,t)$	$u' = Au$	$\boldsymbol{A_{ij} \approx \partial f_i/\partial u_j}$	$\boldsymbol{\mathrm{Re}\,\lambda(A) \le 0}$

For an ordinary differential equation $u' = f(u,t)$, good codes will increase the accuracy (and keep stability) far beyond the $O(\Delta t)$ error in Euler's methods. You can rely on freely available software like `ode45` to make the two crucial decisions:

1. to choose an accurate difference method (and change the formula adaptively)

2. to choose a stable time step (and change Δt adaptively).

We will introduce accurate methods, and find the stability limits on Δt.

©2006 Gilbert Strang

Stiff Differential Equations

First we call attention to one extra question: ***Is the equation stiff?*** Let me begin with a made-up example to introduce stiffness and its effects:

$$\begin{array}{rcl} v(t) = e^{-t} & + & e^{-99t} \\ \uparrow & & \uparrow \\ \text{controls decay} & & \text{controls } \Delta t \end{array} \qquad \begin{array}{l} \text{The step } \Delta t \text{ is 99 times} \\ \text{smaller because of } e^{-99t} \\ \text{that disappears anyway} \end{array}$$

Those decay rates -1 and -99 could be the eigenvalues of A, as in Example 1.

Example 1

$$\frac{d}{dt}\begin{bmatrix} v \\ w \end{bmatrix} = \begin{bmatrix} -50 & 49 \\ 49 & -50 \end{bmatrix}\begin{bmatrix} v \\ w \end{bmatrix} \quad \text{with} \quad \begin{bmatrix} v(0) \\ w(0) \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}. \tag{1}$$

The solution has $v(t) = e^{-t} + e^{-99t}$ and $w(t) = e^{-t} - e^{-99t}$. The time scales are different by a factor of 99 (the condition number of A). **The solution will decay at the slow time scale of e^{-t}, but computing e^{-99t} may require a very small Δt for stability.** It is frustrating to have Δt controlled by the component that is decaying so fast.

Any explicit method will have a requirement $99\Delta t \le C$. We will see how this happens and how an implicit method (like `ode15s` and `od23t` in MATLAB) can avoid it.

Trefethen [ ] points to these applications where stiffness comes with the problem:

1. **Chemical kinetics** (reactions go at very different speeds)
2. **Control theory** (probably the largest application area for MATLAB)
3. **Circuit simulation** (components respond at widely different time scales)
4. **Method of Lines** (large systems come from partial differential equations).

Example 2 The N by N second difference matrix K produces a large stiff system:

$$\textbf{Method of Lines} \qquad \frac{du}{dt} = \frac{-Ku}{(\Delta x)^2} \quad \text{has} \quad \frac{du_i}{dt} = \frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta x)^2}. \tag{2}$$

This comes from the heat equation $\partial u/\partial t = \partial^2 u/\partial x^2$, by discretizing only the space derivative. Example 1 had eigenvalues -1 and -99, while Example 2 has N eigenvalues—but the difficulty is essentially the same! The most negative eigenvalue here is about $a = -4/(\Delta x)^2$. So a small Δx (for accuracy) will require a *very small* Δt (for stability).

This "semidiscrete" method of lines is an important idea. Discretizing the space variables first produces a large system that can be given to an ODE solver. (We have ordinary differential equations in time.) If it wants to, that solver can vary the time step Δt and even the discretization formula as $u(t)$ speeds up or slows down.

This method splits the approximation of a PDE into two parts. Finite differences/finite elements in earlier chapters produce the first part (discrete in space). The upcoming stability-accuracy analysis applies to the second part (discrete in time). This idea is very simple and useful, even if it misses the good methods later in this chapter that take full advantage of space-time. For the heat equation $u_t = u_{xx}$, the useful fact $u_{tt} = u_{xxxx}$ allows us to cancel space errors with time errors—which we won't notice when they are separated in the semidiscrete method of lines.

Forward Euler and Backward Euler

The equation $u' = f(u,t)$ starts from an initial value $u(0)$. The key point is that the rate of change u' is determined by the current state u at any moment t. This model of reality, where all the history is contained in the current state $u(t)$, is a tremendous success throughout science and engineering. (It makes calculus almost as important as linear algebra.) But for a computer, continuous time has to change to discrete time. One differential equation allows many difference equations!

The simplest method (Euler is pronounced "Oiler") uses a forward difference:

$$\textbf{Forward Euler} \quad \frac{\boldsymbol{U_{n+1} - U_n}}{\boldsymbol{\Delta t}} = \boldsymbol{f(U_n, t_n)} \quad \text{is} \quad U_{n+1} = U_n + \Delta t\, f_n\,. \tag{3}$$

Over each Δt interval, the slope of U *doesn't change*. Figure 5.1 shows how the correct solution to $u' = au$ follows a smooth curve, while $U(t)$ is only piecewise linear. A better method (higher accuracy) will stay much closer to the curve by using more information than the one slope $f_n = f(U_n, t_n)$ at the start of the step.

replacements

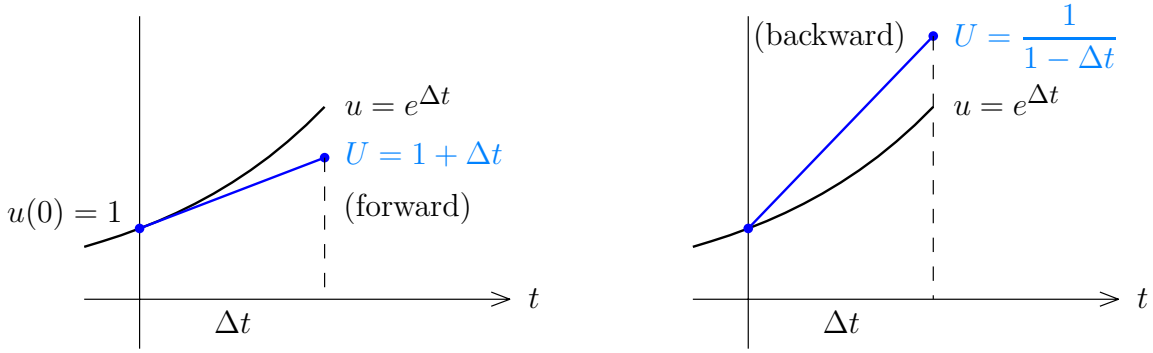


Figure 5.1: Forward Euler and Backward Euler for $u' = u$. One-step errors $\approx \frac{1}{2}(\Delta t)^2$.

Backward Euler comes from using f_{n+1} *at the end of the step*, when $t = t_{n+1}$:

$$\textbf{Backward Euler} \quad \frac{\boldsymbol{U_{n+1} - U_n}}{\boldsymbol{\Delta t}} = \boldsymbol{f(U_{n+1}, t_{n+1})} \quad \text{is} \quad U_{n+1} - \Delta t\, f_{n+1} = U_n\,. \tag{4}$$

This is an ***implicit method***. To find U_{n+1}, we have to solve equation (4). When f is linear in u, we are solving a linear system at each step (and the cost is low if

the matrix is tridiagonal, like $I - \Delta t\, K$ in Example 2). We will comment later on iterations like Newton's method or predictor-corrector in the nonlinear case.

The first example to study is the linear scalar equation $u' = au$. Compare forward and backward Euler, for one step and for n steps:

Forward $U_{n+1} = (1 + a\,\Delta t)U_n$ leads to $\boldsymbol{U_n = (1 + a\,\Delta t)^n U_0}$. (5)

Backward $(1 - a\,\Delta t)U_{n+1} = U_n$ leads to $\boldsymbol{U_n = (1 - a\,\Delta t)^{-n} U_0}$. (6)

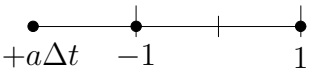
Forward Euler is like compound interest, at the rate a. Each step starts with U_n and adds the interest $a\,\Delta t\, U_n$. As the stepsize Δt goes to zero and we need $T/\Delta t$ steps to reach time T, this discrete compounding approaches continuous compounding. The discrete solution U_n approaches the continuous solution of $u' = au$:

$$(1 + a\,\Delta t)^{T/\Delta t} \quad \text{approaches} \quad e^{aT} \quad \text{as} \quad \Delta t \to 0\,.$$

This is the convergence of U to u that we will prove below, more generally. It holds for backward Euler too, because $(1 - a\,\Delta t)^{-1} = 1 + a\Delta t + \text{higher order terms}$.

The stability question arises for very negative a. The true solution $e^{-at}u(0)$ is extremely stable, approaching zero. (If this is your own money, you should change banks and get $a > 0$.) Backward Euler will be stable because it divides by $1 - a\,\Delta t$ (which is greater than 1 when a is negative). *Forward Euler will explode if* $1 + a\,\Delta t$ *is smaller than* -1*, because its powers grow exponentially*:

Instability $1 + a\,\Delta t < -1$ when $a\,\Delta t < -2$ (7)



That is a pretty sharp borderline between stability and instability, at $-a\,\Delta t = 2$. For $u' = -20u$ which has $a = -20$, the borderline is $\Delta t = \frac{2}{20} = \frac{1}{10}$. Compare the results at time $T = 2$ from $\Delta t = \frac{1}{11}$ (22 steps) and $\Delta t = \frac{1}{9}$ (18 steps):

$$\text{Stable } \Delta t = \frac{1}{11} \qquad (1 + a\,\Delta t)^{22} = \left(-\frac{9}{11}\right)^{22} \approx .012$$

$$\text{Unstable } \Delta t = \frac{1}{9} \qquad (1 + a\,\Delta t)^{18} = \left(-\frac{11}{9}\right)^{18} \approx 37.043$$

I would describe backward Euler as absolutely stable (**A-stable**) because it is stable whenever Re $a < 0$. Only an implicit method can be A-stable. Forward Euler is a **stable method**(!) because it succeeds as $\Delta t \to 0$. For small enough Δt, it is on the stable side of the borderline.

In this example a *good quality approximation* requires more than stability (even $\Delta t = \frac{1}{11}$ is too big). Those powers of $-\frac{9}{11}$ alternate in sign, while e^{-20t} stays positive.

Accuracy and Convergence

Since forward and backward differences are *first order accurate*, it is no surprise that the errors from forward and backward Euler are $O(\Delta t)$. This error $e = u - U$ is

replacements

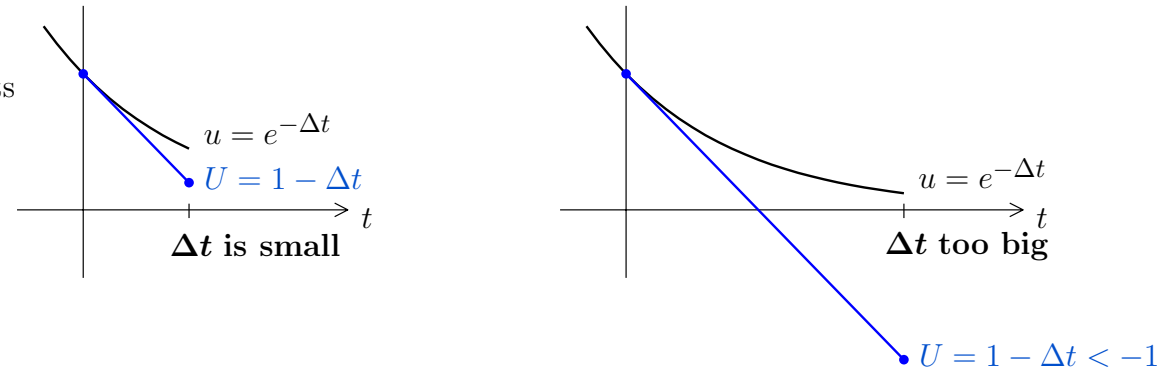


Figure 5.2: Forward Euler (stable and unstable, with $-a\Delta t$ going above 2).

measured at a fixed time T. As Δt decreases, so does the new error added at each step. But the number of steps to reach that time increases, keeping $n\,\Delta t = T$.

To see why the error $u(T) - U(T)$ is $O(\Delta t)$, the key is stability. We need to know that *earlier errors don't increase as they are carried forward to time T*. Forward Euler is the simplest difference equation, so it is the perfect example to follow through first. (The next sections will apply the same idea to partial differential equations.) Euler's difference equation for $u' = f(u,t) = au$ is

$$U_{n+1} = U_n + \Delta t\, f(U_n, t_n) = U_n + a\,\Delta t\, U_n\,. \tag{8}$$

The true solution at time $n\,\Delta t$ satisfies (8) except for a **discretization error DE**:

$$u_{n+1} = u_n + \Delta t\, u_n' + \mathrm{DE} = u_n + a\,\Delta t\, u_n + \mathrm{DE}_{n+1}\,. \tag{9}$$

That error DE is of order $(\Delta t)^2$ because the second-order term is missing (it should be $\frac{1}{2}(\Delta t)^2 u_n''$, but Euler didn't keep it). Subtracting (8) from (9) gives a difference equation for the error $e = u - U$, propagating forward in time:

Error equation
$$e_{n+1} = e_n + a\,\Delta t\, e_n + \mathrm{DE}_{n+1}\,. \tag{10}$$

You could think of this one-step error DE_{n+1} as a deposit like $(\Delta t)^2$ into your account. Once the deposit is made, it will grow or decay according to the error equation. To reach time $T = N\,\Delta t$, each error DE_k at step k has $N - k$ more steps to go:

$$e_N = (1 + a\,\Delta t)^{N-1}\,\mathrm{DE}_1 + \cdots + (1 + a\,\Delta t)^{N-k}\,\mathrm{DE}_k + \cdots + \mathrm{DE}_N\,. \tag{11}$$

Now stability plays its part. If a is negative, those powers are all less than 1 (in absolute value)—*provided $1 + a\,\Delta t$ does not go below -1*. If a is positive, those powers of $1 + a\,\Delta t$ are all less than $(e^{a\,\Delta t})^N = e^{aT}$. Then the error e_N has N terms in (11), and every term is less than $c(\Delta t)^2$ for a fixed constant c:

Error bound
$$|\boldsymbol{e_N}| = |\boldsymbol{u_N - U_N}| \le \boldsymbol{N\,c\,(\Delta t)^2 = c\,T\,\Delta t}\,. \tag{12}$$

The errors along the way, of size $(\Delta t)^2$, combined after $N = T/\Delta t$ steps into an overall Δt error. This depended on stability—the local errors didn't explode.

Notice that the error growth follows the difference equation in (10), not the differential equation. The stiff example with $a\,\Delta t = (-20)(\frac{1}{9})$ gave a large $1 + a\,\Delta t$, even when $e^{a\,\Delta t}$ was small. We still call forward Euler a **stable method**, because as soon as Δt is small enough the danger has passed. Backward Euler also gives $|e_N| = O(\Delta t)$. The problem with these first-order methods is their low accuracy.

The local discretization error DE tells us the accuracy. For Euler that error DE $\approx \frac{1}{2}(\Delta t)^2 u''$ is the first term that Euler misses in the Taylor series for $u(t + \Delta t)$. Better methods will capture that term exactly, and miss on a higher-order term. The discretization error (we find it by comparing $u(t+\Delta t)$ with U_{n+1}, assuming $u(t)$ agrees with U_n) begins with a power $(\Delta t)^{p+1}$:

Local discretization error $$\text{DE} \approx c(\Delta t)^{p+1}\frac{d^{p+1}u}{dt^{p+1}}\,. \tag{13}$$

The method decides c and $p + 1$. With stability, $T/\Delta t$ steps give a global error of order $(\Delta t)^p$. The derivative of u shows whether the problem is hard or easy.

Error estimates like (13) appear everywhere in numerical analysis. The $1, -2, 1$ second difference has error $\frac{1}{12}(\Delta x)^4 u''''$. Partial differential equations (Laplace, wave, heat) produce similar terms. In one line we find $c = -\frac{1}{2}$ for *backward Euler*:

$$(u_{n+1} - u_n) - \Delta t\, u'_{n+1} \approx \left(\Delta t\, u'_n + \frac{(\Delta t)^2}{2}\, u''_n\right) - \Delta t(u'_n + \Delta t\, u''_n) \approx -\frac{(\Delta t)^2}{2}\, u''_n. \tag{14}$$

The global estimate $|u - U| \le C\,\Delta t \max|u''|$ shows no error when u is linear and u'' is zero (Euler's approximation of constant slope becomes true). For nonlinear equations, the key is in the subtraction of (8) from (9). A "Lipschitz" bound L on $\partial f/\partial u$ replaces the number a in the error equation:

$$|f(u,t) - f(U,t)| \le L|u - U| \quad \text{gives} \quad e_{n+1} \le (1 + L\Delta t)\, e_n + \text{DE}_{n+1}\,. \tag{15}$$

Second-Order Methods

Here is a first way to increase the accuracy. We could center the equation at the midpoint $(n + \frac{1}{2})\Delta t$ of the step, by averaging $f(U_n, t_n)$ and $f(U_{n+1}, t_{n+1})$. The result is an *implicit second-order method* use in MATLAB's `ode23t`:

Trapezoidal rule/Crank-Nicolson $$\frac{\boldsymbol{U_{n+1} - U_n}}{\boldsymbol{\Delta t}} = \frac{\mathbf{1}}{\mathbf{2}}(\boldsymbol{f_{n+1} + f_n})\,. \tag{16}$$

So $U_{n+1} - \frac{1}{2}\,\Delta t\, f_{n+1} = U_n + \frac{1}{2}\,\Delta t\, f_n$. For our model $u' = f(u) = au$, this is

$$(1 - \frac{1}{2}a\,\Delta t)\, U_{n+1} = (1 + \frac{1}{2}a\,\Delta t)\, U_n \quad \text{which gives} \quad U_{n+1} = \left(\frac{1 + \frac{1}{2}a\,\Delta t}{1 - \frac{1}{2}a\,\Delta t}\right) U_n\,. \tag{17}$$

The true solution has $u_{n+1} = e^{a\Delta t}u_n$. Problem 1 will find DE $\approx (\Delta t)^3$. The equation is stable. So $N = T/\Delta t$ steps produce $|e_N| = |u_N - U_N| \le cT(\Delta t)^2$.

How to improve forward Euler and still keep it explicit? We don't want U_{n+1} on the right side of the equation, but we are happy with U_{n-1} (from the previous step!). Here is a combination that gives second-order accuracy in a **two-step method**:

$$\textbf{"Adams-Bashforth"} \quad \frac{\boldsymbol{U_{n+1} - U_n}}{\boldsymbol{\Delta t}} = \frac{\mathbf{3}}{\mathbf{2}}\boldsymbol{f(U_n, t_n)} - \frac{\mathbf{1}}{\mathbf{2}}\boldsymbol{f(U_{n-1}, t_{n-1})}\,. \tag{18}$$

Remember that $f(U_{n-1}, t_{n-1})$ is already computed in the previous step, going from $n-1$ to n. So this **explicit multistep method** requires no extra work, and improves the accuracy. To see how $(\Delta t)^2$ comes out correctly, write u' for f:

$$\frac{3}{2}u'_n - \frac{1}{2}u'_{n-1} \approx \frac{3}{2}u'_n - \frac{1}{2}(u'_n - \Delta t\, u''_n) = u'_n + \frac{1}{2}\Delta t\, u''_n\,.$$

Multiplied by Δt in (18), that new term $\frac{1}{2}(\Delta t)^2 u''_n$ is exactly what Euler missed. *Each extra term in the difference equation can increase the accuracy by* 1.

A third possibility uses the already computed value U_{n-1} (instead of the slope f_{n-1}). With $\frac{3}{2}, -\frac{4}{2}, \frac{1}{2}$ chosen for second-order accuracy, we get an implicit **backward difference method**:

$$\textbf{Backward differences} \quad \frac{\boldsymbol{3U_{n+1} - 4U_n + U_{n-1}}}{\boldsymbol{2\Delta t}} = \boldsymbol{f(U_{n+1}, t_{n+1})}\,. \tag{19}$$

What about stability? The implicit method (17) is stable even in the stiff case, when a is very negative. $1 - \frac{1}{2}a\,\Delta t$ (left side) will be larger than $1 + \frac{1}{2}a\,\Delta t$ (right side). (19) is also stable. The explicit method (18) will be stable if Δt is small enough, but there is always a limit on Δt for stiff systems.

Here is a quick way to find the stability limit in (18) when a is real. The limit occurs when the growth factor is exactly $G = -1$. ***Set $U_{n+1} = -1$ and $U_n = 1$ and $U_{n-1} = -1$ in (18). Solve for a when $f(u,t) = au$:***

$$\textbf{Stability limit in (18)} \quad \frac{-2}{\Delta t} = \frac{3}{2}a + \frac{1}{2}a \quad \text{gives} \quad a\,\Delta t = -1\,. \quad \text{So} \quad C = 1\,. \tag{20}$$

Those second-order methods (17)–(18)–(19) are definitely useful! The reader might suggest including both U_{n-1} and f_{n-1} to increase the accuracy to $p = 3$. Sadly, this method is violently unstable (Problem 4). We may use older values of U in backward differences or $f(U)$ in Adams formulas, but including both U and $f(U)$ for even higher accuracy produces instability for all Δt.

Multistep Methods: Explicit and Implicit

By using p older values of either U or $f(U)$ (already computed at previous steps!), the accuracy can be increased to order p. Each ∇U is $U(t) - U(t - \Delta t)$ and $p = 2$

is (19):

Backward differences
$$\left(\nabla + \frac{1}{2}\nabla^2 + \cdots + \frac{1}{p}\nabla^p\right) U_{n+1} = \Delta t\, f(U_{n+1}, t_{n+1})\,. \quad (21)$$

MATLAB's stiff code `ode15s` varies from $p = 1$ to $p = 5$ depending on the local error.

Using older values of the right side $f(U)$ gives an **Adams-Bashforth method**:

$$U_{n+1} - U_n = \Delta t(b_1\, f_n + \cdots + b_p\, f_{n-p+1}) \quad \text{with} \quad f_n = f(U_n, t_n)\,. \quad (22)$$

The table shows the numbers b up to $p = 4$, starting with Euler for $p = 1$.

order of accuracy	b_1	b_2	b_3	b_4	limit on $a\Delta t$ for stability	constant c in error DE
$p = 1$	1				-2	$1/2$
$p = 2$	$3/2$	$-1/2$			-1	$5/12$
$p = 3$	$23/12$	$-16/12$	$5/12$		$-6/11$	$3/8$
$p = 4$	$55/24$	$-59/24$	$37/24$	$-9/24$	$-3/10$	$251/720$

The fourth-order method is often a good choice, although astronomers go above $p = 8$. At the stability limit $G = -1$ as in (20). The local error DE $\approx c(\Delta t)^{p+1}u^{(p+1)}$ is a problem of step control. Whether it is amplified by later steps is a problem of stability control.

Implicit methods have an extra term $c_0 f_{n+1}$ at the new time level. Properly chosen, that adds one extra order of accuracy—as it did for the trapezoidal rule, which is the second method in the new table. Backward Euler is the first:

order of accuracy	c_0	c_1	c_2	c_3	limit on $a\Delta t$ for stability	constant c in error DE
$p = 1$	1				$-\infty$	$-1/2$
$p = 2$	$1/2$	$1/2$			$-\infty$	$-1/12$
$p = 3$	$5/12$	$8/12$	$-1/12$		-6	$-1/24$
$p = 4$	$9/24$	$19/24$	$-5/24$	$1/24$	-3	$-19/720$

Every row of both tables adds to 1, so $u' = 1$ is solved exactly.

You see that the error constants and stability are all in favor of implicit methods. So is the user, except when solving for U_{n+1} becomes expensive. Then there is a simple and successful ***predictor-corrector*** method, or else ***Newton's method***.

Predictor-Corrector

P: Use the explicit formula to *predict* a new U^*_{n+1}
E: Use u^*_{n+1} to *evaluate* the right side f^*_{n+1}
C: Use f^*_{n+1} in the implicit formula to *correct* to a new U_{n+1}.

The stability is much improved if another E step evaluates f_{n+1} with the corrected u_{n+1}. In principle we should continue the corrector to convergence. Frequently 1–2 corrections are enough, and the extra calculation can be put to use. By comparing the predicted U^*_{n+1} and corrected U_{n+1} the code can estimate the local error and change Δt:

$$\text{local error DE} \approx \frac{c}{c^* - c}(U_{n+1} - U^*_{n+1}), \tag{23}$$

where c^* and c are the error constants in the tables for the predictor and corrector.

Implicit methods often have a **Newton loop** inside each time step, to compute U_{n+1}. The kth iteration in the Newton loop is a linear system, with the Jacobian matrix $A^{(k)}_{ij} = \partial f_i/\partial u_j$ evaluated at the latest approximation $U^{(k)}_{n+1}$ with $t = t_{n+1}$.

Here is the kth i to solve $U_{n+1} - c_0 f(U_{n+1}, t_{n+1}) =$ old values:

$$\textbf{Newton iteration } (I - c_0 A^{(k)})(U^{(k+1)}_{n+1} - U^{(k)}_{n+1}) = c_0 f(U^{(k)}_{n+1}, t_{n+1}) + \text{old values}. \tag{24}$$

When $f(u) = Au$ is linear, you only need one iteration ($k = 0$ starts with $U^{(0)}_{n+1} = U_n$). For nonlinear $f(u)$, Newton's rapid convergence squares the error at each step (when it gets close). The price is a new evaluation of $f(u)$ and its matrix $A^{(k)}$ of derivatives.

Runge-Kutta Methods

A-stable methods have no stability limit. Multistep methods achieve high accuracy with one or two evaluations of f. If more evaluations are not too expensive, ***Runge-Kutta methods*** are definitely competitive (and these are self-starting). They are compound one-step methods, using Euler's $U_n + \Delta t\, f_n$ ***inside f:***

$$\textbf{Simplified Runge-Kutta} \qquad \frac{U_{n+1} - U_n}{\Delta t} = \frac{1}{2}\big[f_n + f(U_n + \Delta t\, f_n, t_{n+1})\big]. \tag{25}$$

You see the compounding of f. For the model equation $u' = au$ the result is

$$u_{n+1} = u_n + \frac{1}{2}\Delta t\big[au_n + a(u_n + \Delta t\, au_n)\big] = \big(1 + a\Delta t + \frac{1}{2}a^2\Delta t^2\big)u_n = G\, u_n\,. \tag{26}$$

This confirms the second-order accuracy; the growth factor G agrees with $e^{a\Delta t}$ through $(\Delta t)^2$. There will be a stability threshold $a\Delta t = C$, where $|G| = 1$:

$$\textbf{Stability limit} \qquad |G| = \Big|1 + C + \frac{1}{2}C^2\Big| = 1 \quad (\text{for complex } C = a + ib)$$

The complete stability limit is not a point but a closed curve in the complex plane. Figure 5.____ shows all the complex numbers C at which $|G| = 1$.

The famous version of Runge-Kutta is compounded *four times* and achieves $p = 4$:

Runge-Kutta
$$\frac{U_{n+1} - U_n}{\Delta t} = \frac{1}{3}(k_1 + 2k_2 + 2k_3 + k_4) \tag{27}$$

$$k_1 = \frac{1}{2} f(U_n, t_n) \qquad k_3 = \frac{1}{2} f(U_n + \Delta t\, \boldsymbol{k_2}, t_{n+1/2})$$
$$k_2 = \frac{1}{2} f(U_n + \Delta t\, \boldsymbol{k_1}, t_{n+1/2}) \qquad k_4 = \frac{1}{2} f(U_n + 2\Delta t\, \boldsymbol{k_3}, t_{n+1})$$

For this one-step method, no special starting instructions are necessary. It is simple to change Δt as you go. The growth factor reproduces $e^{a\Delta t}$ through $\frac{1}{24} a^4 \Delta t^4$. The error constant is the next coefficient $\frac{1}{120}$. Among highly accurate methods, Runge-Kutta is especially easy to code and run—probably the easiest there is. MATLAB's workhorse is `ode45`.

To prove that the stability threshold $a\Delta t = -2.78$ is genuine, we reproduce the solution of $u' = -100u + 100 \sin t$. With $u(0) = 0$, Runge-Kutta gives

$$\begin{aligned} U_{120} &= 0.151 = u(3) \quad \text{with } \Delta t = \frac{3}{120} \quad \text{and } a\Delta t = -2.5 \\ U_{100} &= 670{,}000{,}000{,}000 \quad \text{with } \Delta t = \frac{3}{100} \quad \text{and } a\Delta t = -3 \end{aligned} \tag{28}$$

Differential-Algebraic Equations

Our system of equations might not come in the form $u' = f(u, t)$. The more general form $F(u', u, t) = 0$ appears in many applications. It may not be easy (or possible) to solve this system explicitly for u'. For large electrical networks (VLSI on chips), transistors produce a highly nonlinear dependence on u' and u.

Here is a simple model in which solving for the vector u' is impossible:

$$Mu' = f(u, t) \quad \text{with a singular matrix } M \text{ (rank } r < N). \tag{29}$$

Suppose we change variables from u to v, so as to diagonalize M. In the new variables we have r true differential equations and $N - r$ *algebraic equations* (*no derivatives*). The system becomes a ***differential-algebraic equation*** (**DAE**):

DAE
$$\begin{aligned} \frac{dv_i}{dt} &= f_i(v_1, \ldots, v_N, t) \quad i = 1, \ldots, r \\ 0 &= f_i(v_1, \ldots, v_N, t) \quad i = r+1, \ldots, N. \end{aligned} \tag{30}$$

The $N - r$ algebraic equations restrict v to a surface in N-dimensional space. The r differential equations move the vector $v(t)$ along that surface.

Petzold [46] developed the DASSL package for solving DAE's in the general form $F(u', u, t) = 0$, replacing u' by a backward difference of u. The best recommendation we can make is to experiment with this software!

 ©2006 Gilbert Strang

Problem Set 5.1

1 The error in the trapezoidal (Crank-Nicolson) method ( ) comes from the difference

$$e^{a\Delta t} - \left[\frac{1+(a\Delta t/2)}{1-(a\Delta t/2)}\right] = e^{a\Delta t} - \left[\left(1+\frac{a\Delta t}{2}\right)\left(1+\frac{a\Delta t}{2}+\left(\frac{a\Delta t}{2}\right)^2+\cdots\right)\right]$$

This involves the two series that everyone should learn: the exponential series for $e^{a\Delta t}$ and the geometric series $1+x+x^2+\cdots$ for $\frac{1}{1-x}$.

Multiply inside the brackets to produce the correct $\frac{1}{2}(a\Delta t)^2$. Show that the $(a\Delta t)^3$ term is wrong by $c=\frac{1}{12}$. Then the error is DE $\approx \frac{1}{12}(\Delta t)^3\, u'''$.

2 Try Runge-Kutta on $u' = -100u + 100\sin t$ with $\Delta t = -.0275$ and $-.028$. Those are close to the stability limit $-.0278$.

3 For the backward difference error in (19), expand $\frac{1}{2}(3e^{a\Delta t}-4+e^{-a\Delta t})-a\Delta t\, e^{a\Delta t}$ into a series in $a\Delta t$. Show that the leading error is $-\frac{1}{3}(a\Delta t)^3$ so that $c=-\frac{1}{3}$.

4 Stability in (19).

©2006 Gilbert Strang

5.2 Accuracy and Stability for $u_t = c\,u_x$

This section begins a major topic in scientific computing: **Initial-value problems for partial differential equations**. Naturally we start with linear equations that involve only one space dimension x (and time t). The exact solution is $u(x,t)$ and its discrete approximation on a space-time grid has the form $U_{j,n} = U(j\Delta x, n\Delta t)$. We want to know if U is near u—how close they are and how stable U is.

Begin with the simplest wave equation (first-order, linear, constant coefficient):

$$\textbf{One-way wave equation} \qquad \frac{\partial u}{\partial t} = c\,\frac{\partial u}{\partial x}\,. \tag{1}$$

We are given $u(x,0)$ at time $t = 0$. We want to find $u(x,t)$ for all $t > 0$. For simplicity, these functions are defined on the whole line $-\infty < x < \infty$. There are no difficulties with boundaries (where waves could change direction and bounce back).

The solution $u(x,t)$ will have the typical feature of *hyperbolic equations*: *signals travel at finite speed*. Unlike the second-order wave equation $u_{tt} = c^2 u_{xx}$, this first-order equation $u_t = c\,u_x$ sends signals in one direction only.

Solution for $u(x,0) = e^{ikx}$

Throughout this chapter I will solve for a pure exponential $u(x,0) = e^{ikx}$. **At every time t, the solution remains a multiple Ge^{ikx}.** The growth factor G will depend on the frequency k and the time t, but different frequencies do not mix. Substituting $u = G(k,t)\,e^{ikx}$ into $u_t = c\,u_x$ yields a simple ordinary differential equation for G, because we can cancel e^{ikx}. The derivative of e^{ikx} produces the factor ik:

$$u_t = c\,u_x \quad \text{is} \quad \frac{dG}{dt}e^{ikx} = ikc\,Ge^{ikx} \quad \text{or} \quad \frac{dG}{dt} = ikcG\,. \tag{2}$$

The growth factor is $G(k,t) = e^{ikct}$. The initial value is $G = 1$.

$$\textbf{An exponential solution to } \frac{\partial u}{\partial t} = c\,\frac{\partial u}{\partial x} \textbf{ is } u(x,t) = e^{ikct}e^{ikx} = \boldsymbol{e^{ik(x+ct)}}\,. \tag{3}$$

Immediately we see two important features of this solution:

1. The growth factor $G = e^{ikct}$ has absolute value $|G| = 1$.
2. The initial function e^{ikx} moves to the left with fixed velocity c, to $e^{ik(x+ct)}$.

The initial value at the origin is $u(0,0) = e^{ik0} = 1$. This value $u = 1$ appears at all points on the line $x + ct = 0$. **The initial data propagates along the characteristic lines $x + ct =$ *constant***, in Figure 5.3. Right now we know this fact for the special solutions $e^{ik(x+ct)}$. Soon we will know it for all solutions.

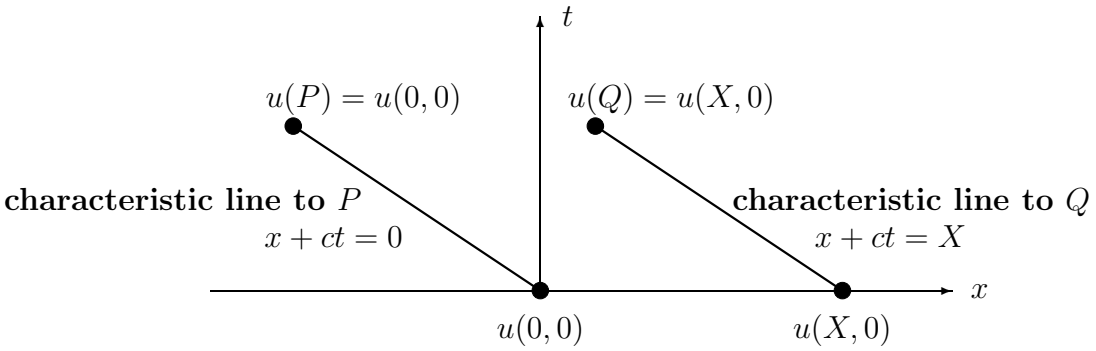


Figure 5.3: The solution $u(x,t)$ moves left with speed c, along characteristic lines.

Figure 5.3 shows the travel path of the solution in the x-t plane (we are introducing the characteristic lines). Figure 5.4 will graph the solution itself at times 0 and t. That step function combines exponentials $e^{ik(x+ct)}$ for different frequencies k. By linearity we can add those solutions.

Solution for Every $u(x,0)$

In almost all partial differential equations, the solution changes shape as it travels. Here the shape stays the same. All pure exponentials travel at the same velocity c, so *every* initial function moves with that velocity. We can write down the solution:

General solution $\dfrac{\partial u}{\partial t} = c\dfrac{\partial u}{\partial x}$ is solved by $\boldsymbol{u(x,t) = u(x+ct,0)}$. (4)

The solution is a function only of $x+ct$. That makes it constant along characteristic lines, where $x+ct$ is constant. This dependence on $x+ct$ also makes it satisfy the equation $u_t = c\,u_x$, by the chain rule. If we take $u = (x+ct)^n$ as an example, the extra factor c appears in $\partial u/\partial t$:

$$\frac{\partial u}{\partial x} = n\,(x+ct)^{n-1} \quad \text{and} \quad \frac{\partial u}{\partial t} = cn\,(x+ct)^{n-1} \quad \text{which is} \quad c\frac{\partial u}{\partial x}.$$

A Taylor series person would combine those powers (different n) to produce a large family of solutions. A Fourier series person combines exponentials (different k) to produce an even larger family. In fact *all* solutions are functions of $x+ct$ alone.

Here are two important initial functions—a light flashes or a dam breaks.

Example 1 $u(x,0) =$ delta function $\delta(x) =$ **flash of light** at $x = 0, t = 0$

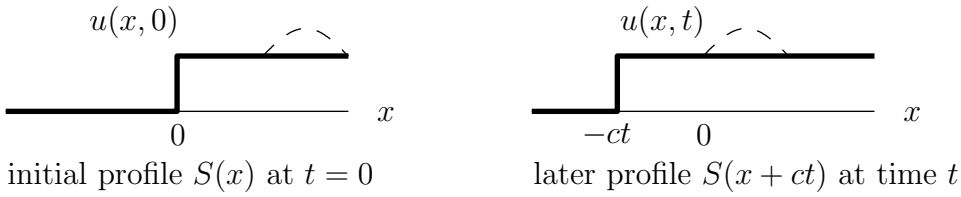
By our formula (4), the solution is $u(x,t) = \delta(x+ct)$. The light flash reaches the point $x = -c$ at the time $t = 1$. It reaches $x = -2c$ at the time $t = 2$. The impulse is traveling to the left at speed $|dx/dt| = c$. In this example all frequencies k are present in equal amounts, because the Fourier transform of a delta function is a constant.

©2006 Gilbert Strang

Notice that a point goes dark again as soon as the flash passes through. This is the Huygens principle in 1 and 3 dimensions. If we lived in two or four dimensions, the wave would not pass all at once and we wouldn't see clearly.

Example 2 $u(x,0) =$ step function $S(x) =$ **wall of water** at $x = 0, t = 0$

The solution $S(x+ct)$ is the moving step function in Figure 5.4. The wall of water travels to the left (*one-way wave*). At time t, the "tsunami" reaches the point $x = -ct$. The flash of light will get there first, because its speed c is greater than the tsunami speed. That is why a warning is possible for an approaching tsunami.



initial profile $S(x)$ at $t = 0$ later profile $S(x + ct)$ at time t

Figure 5.4: The wall travels left with velocity c (all waves e^{ikx} do too).

An actual tsunami is described by the nonlinear "shallow water equations" that come later. The feature of *finite speed* still holds.

Finite Difference Methods for $u_t = c\,u_x$

The one-way wave equation is a perfect example for creating and testing finite difference approximations. We can replace $\partial u/\partial t$ by a forward difference with step Δt. Here are four choices for the discrete form of $\partial u/\partial x$ at meshpoint $i\Delta x$:

1. **Forward** $= \dfrac{U_{i+1} - U_i}{\Delta x} =$ **upwind**: Low accuracy, conditionally stable for $c > 0$.
2. **Centered** $= \dfrac{U_{i+1} - U_{i-1}}{2\Delta x}$: Unstable after a few steps as we will prove!
3. **Lax-Friedrichs**: (20) has low accuracy, conditionally stable also for $c < 0$.
4. **Lax-Wendroff**: (14) has extra accuracy, conditionally stable also for $c < 0$.

The list doesn't end there. We have reached a central problem of scientific computing, to construct approximations that are *stable* and *accurate* and *fast*. That topic can't be developed on one page, especially when we move to nonlinear equations.

Conditionally stable means that the time step Δt is restricted. The need for this restriction was noticed by Courant, Friedrichs, and Lewy. When the space difference

reaches no further than $x + \Delta x$, there is an automatic stability restriction:

CFL requirement for stability $$r = c\,\frac{\Delta t}{\Delta x} \le 1\,. \tag{5}$$

That number $c\,\Delta t/\Delta x$ is often called the *Courant number*. (It was really Lewy who recognized that $r \le 1$ is necessary for stability and convergence.) The reasoning is straightforward, based on using the initial value that controls $u(x,t)$:

> The true solution at (x,t) equals the initial value $u(x+ct,0)$. Taking n discrete steps to reach $t = n\,\Delta t$ uses information on the initial values as far out as $x + n\,\Delta x$. If $x + ct$ is further than $x + n\,\Delta x$, the method can't work:

CFL condition $$x + c\,t \le x + n\,\Delta x \quad \text{or} \quad c\,n\,\Delta t \le n\,\Delta x \quad \text{or} \quad r = c\,\frac{\Delta t}{\Delta x} \le 1\,. \tag{6}$$

If the difference equation uses $U(x + 2\Delta x, t)$, then CFL relaxes to $r \le 2$.

A particular finite difference equation might require a tighter restriction on Δt for stability. It might even be unstable for all ratios r (we hope not). The only route to *unconditional stability* for all Δt is an *implicit method*, which computes x-differences at the new time $t + \Delta t$. This will be useful later for diffusion terms like u_{xx}. For advection terms (first derivatives), explicit methods with a CFL limitation are usually accepted because a much larger Δt would lose accuracy as well as stability.

To repeat, if $r > 1$ then the finite difference solution at x, t does not use initial value information near the correct point $x^* = x + ct$. Hopeless.

Accuracy of the Upwind Difference Equation

Linear problems with constant coefficients are the ones to understand first. Exactly as for differential equations, we can follow each pure exponential e^{ikx}. After a single time step, there will be a growth factor in $U(x, \Delta t) = Ge^{ikx}$. That growth factor $G(k, \Delta t, \Delta x)$ may have magnitude $|G| < 1$ or $|G| > 1$. This will control stability or instability. The order of accuracy (if we compute in the k-ω domain) comes from comparing G with the true factor $e^{ikc\Delta t}$ from the differential equation.

We now determine that the order of accuracy is $p = 1$ for the *upwind method*.

Forward differences $$\frac{U(x,t+\Delta t) - U(x,t)}{\Delta t} = c\,\frac{U(x+\Delta x,t) - U(x,t)}{\Delta x}. \tag{7}$$

We will test the accuracy in the x-t domain and then the k-ω domain. Either way we use Taylor series to check the leading terms. Substituting the true solution $u(x,t)$ in place of $U(x,t)$, its forward differences are

Time $$\frac{1}{\Delta t}\,[u(x,t+\Delta t) - u(x,t)] \;=\; u_t + \frac{1}{2}\Delta t\,u_{tt} + \cdots \tag{8}$$

Space $$\frac{c}{\Delta x}\,[u(x+\Delta x,t) - u(x,t)] \;=\; c\,u_x + \frac{1}{2}c\,\Delta x\,u_{xx} + \cdots \tag{9}$$

On the right side, $u_t = c\,u_x$ is good. One more derivative gives $u_{tt} = c\,u_{xt} = c^2\,u_{xx}$. Notice c^2. Then $\Delta t\,u_{tt}$ matches $c\,\Delta x\,u_{xx}$ *only in the special case* $c\Delta t = \Delta x$:

©2006 Gilbert Strang

$$\frac{1}{2}\Delta t\, c^2\, u_{xx} \quad \text{equals} \quad \frac{1}{2} c\, \Delta x\, u_{xx} \quad \text{only if} \quad r = \frac{c\Delta t}{\Delta x} = 1.$$

For any ratio $r \neq 1$, the difference between (8) and (9) has a ***first-order error***. Let me show this also in the k-ω Fourier picture and then improve to second-order.

Fix the ratio $r = c\Delta t/\Delta x$ as $\Delta x \to 0$ and $\Delta t \to 0$. In the difference equation (7), write each new value at time $t + \Delta t$ as a combination of two old values of U:

$$\textbf{Difference equation} \quad U(x, t + \Delta t) = (\mathbf{1} - \boldsymbol{r})\, U(x, t) + \boldsymbol{r}\, U(x + \Delta x, t)\,. \tag{10}$$

Starting from $U(x, 0) = e^{ikx}$ we quickly find the growth factor $\boldsymbol{G}$ at time Δt:

$$\textbf{After 1 step} \quad (1 - r)e^{ikx} + r\, e^{ik(x+\Delta x)} = \left[\mathbf{1} - \boldsymbol{r} + \boldsymbol{r}\, \boldsymbol{e^{ik\Delta x}}\right] e^{ikx} = \boldsymbol{G}\, e^{ikx}\,. \tag{11}$$

To test the accuracy, compare this $G = G_{\textbf{approx}}$ to the exact growth factor $e^{ick\Delta t}$. Use the power series $1 + x + x^2/2! + \cdots$ for any e^x:

$$\textbf{Accuracy} G_{\textbf{approx}} = 1 - r + r\, e^{ik\Delta x} = (1 - r) + r + r(ik\Delta x) + \frac{1}{2} r\, (ik\Delta x)^2 + \cdots$$

$$G_{\textbf{exact}} = e^{ick\Delta t} = e^{irk\Delta x} = 1 + irk\Delta x + \frac{1}{2}(irk\Delta x)^2 + \cdots \tag{12}$$

The first terms agree as expected. Forward differences replaced derivatives, and the method is ***consistent***. We saw $u_t = c\, u_x$ in comparing (8) with (9). The next terms do *not* agree unless $r = r^2$:

$$\text{Compare } \frac{1}{2} r(ik\Delta x)^2 \text{ with } \frac{1}{2} r^2 (ik\Delta x)^2. \quad \textbf{Single-step error of order } \boldsymbol{(k\Delta t)^2}. \tag{13}$$

After $1/\Delta t$ steps, those errors of order $k^2(\Delta t)^2$ give a final error $O(k^2\Delta t)$. Forward differences are only first order accurate, and so is the whole method.

The special case $r = 1$ means $c\Delta t = \Delta x$. The difference equation is *exactly correct*. The true and approximate solutions at $(x, \Delta t)$ are both $u(x + \Delta x, 0)$. We are *on the characteristic line* in Figure 5.3. This is an interesting special case (the golden Δt, but hard to repeat in scientific computing when c varies).

Conclusion Except when $r = r^2$, ***the upwind method is first-order accurate.***

Higher Accuracy for Lax-Wendroff

To upgrade the accuracy, we need to match the $\frac{1}{2}\Delta t\, u_{tt}$ error term in the forward time difference by an additional space difference that gives $\frac{1}{2}\Delta t\, c^2 u_{xx}$. This is achieved by the **Lax-Wendroff method**:

$$\frac{U(x, t + \Delta t) - U(x, t)}{\Delta t} = c\, \frac{U(x + \Delta x, t) - U(x - \Delta x, t)}{2\Delta x} + \frac{\Delta t}{2}\, c^2 \left(\frac{U(x + \Delta x, t) - 2U(x, t) + U(x - \Delta x, t)}{(\Delta x)^2} \right). \tag{14}$$

Substituting the true solution, that second difference produces $\frac{1}{2} c^2 \Delta t\, u_{xx}$ plus higher order terms. This cancels the $\frac{1}{2}\Delta t\, u_{tt}$ error term in the time difference, computed

in equation (8). (Remember $u_{tt} = cu_{xt} = c^2 u_{xx}$. The centered difference has no Δx term.) Thus Lax-Wendroff has **second-order accuracy**.

To see this in the k-ω frequency domain, rewrite the LW difference equation (14):

$$U(x,t+\Delta t) = (1-r^2)U(x,t) + \frac{1}{2}(r^2+r)U(x+\Delta x,t) + \frac{1}{2}(r^2-r)U(x-\Delta x,t)\,. \quad (15)$$

Substitute $U(x,t) = e^{ikx}$ to find the one-step growth factor G at time $t+\Delta t$:

Growth factor for LW $$G = (\mathbf{1}-\boldsymbol{r^2}) + \frac{\mathbf{1}}{\mathbf{2}}(\boldsymbol{r^2}+\boldsymbol{r})e^{ik\Delta x} + \frac{\mathbf{1}}{\mathbf{2}}(\boldsymbol{r^2}-\boldsymbol{r})e^{-ik\Delta x}.$$

Expanding $e^{ik\Delta x}$ and $e^{-ik\Delta x}$ in powers of $ik\,\Delta x$, this becomes

$$G = 1 + r(ik\Delta x) + \frac{1}{2}r^2(ik\Delta x)^2 + O(k\Delta x)^3\,. \quad (16)$$

Comparing with $G_{\mathbf{exact}} = e^{irk\Delta x}$ in equation (12), three terms agree. So the one-step error is of order $(k\Delta x)^3$. After $1/\Delta t$ steps the second-order accuracy of Lax-Wendroff is confirmed.

Figure 5.5 shows by actual computation the improvement in accuracy. For a first-order method, the "wall of water" is smeared out. High frequencies have growth factors $|G(k)|$ much smaller than 1. There is too much dissipation. For the first-order Lax-Friedrichs method, the dissipation is even worse (Problem 2). The second-order Lax-Wendroff method stays much closer to the discontinuity. But it's not perfect—those oscillations are not good.

For an ideal difference equation, we want to add enough dissipation very *close to the shock*, to avoid that oscillation (the Gibbs phenomenon). A lot of thought has gone into high resolution methods, to capture shock waves cleanly.

Greater accuracy is achievable by including more terms in the difference equation. If we go from the three terms in Lax-Wendroff to five terms, we can reach fourth-order accuracy. If we use *all* values $U(j\Delta x, n\Delta t)$ at every time step, which requires more work, we can achieve **spectral accuracy**. Then the error decreases faster than any power of Δx, provided $u(x,t)$ is smooth enough to allow derivatives of all orders. Section ____ gives a separate discussion of this **spectral method**.

Stability of the Four Finite Difference Methods

Now we turn from accuracy to stability. Accuracy requires G to stay close to the true $e^{ick\Delta t}$. Stability requires G to stay *inside the unit circle*. We need $|G| \le 1$ for all frequencies k or the finite difference approximation $G^n e^{ikx}$ will blow up.

We now check whether or not $|G| \le 1$, in the four methods.

1. Forward differences in space and time: $\boldsymbol{\Delta U/\Delta t = c\,\Delta U/\Delta x}$ (upwind).

©2006 Gilbert Strang

Recall from equation (11) that $G = 1 - r + re^{ik\Delta x}$. If the Courant number r is between 0 and 1, the triangle inequality gives $|G| \le 1$:

$$\textbf{Stability for } \mathbf{0 \le r \le 1} \qquad |G| \le |1 - r| + |re^{ik\Delta x}| = 1 - r + r = 1\,. \tag{17}$$

This sufficient condition $0 \le c\,\Delta t/\Delta x \le 1$ is exactly the same as the Courant-Friedrichs-Lewy necessary condition! They reasoned that $U(x, n\Delta t)$ depends on the initial values between x and $x + n\Delta x$. That **domain of dependence** must include the point $x + c\,n\Delta t$. (Otherwise, changing the initial value at the point $x + c\,n\Delta t$ would change the true solution u but not the approximation U.) Then $c\,n\Delta t$ must lie between 0 and $n\Delta x$, which means that $0 \le r \le 1$.

Figure 5.5 shows G in the stable case $r = \frac{2}{3}$ and the unstable case $r = \frac{4}{3}$ (when Δt is too large). As k varies, and $e^{ik\Delta x}$ goes around a unit circle, the complex number $G = 1 - r + re^{ik\Delta x}$ goes in a circle of radius r. The center is $1 - r$. Always $G = 1$ at zero frequency (constant solution, no growth).

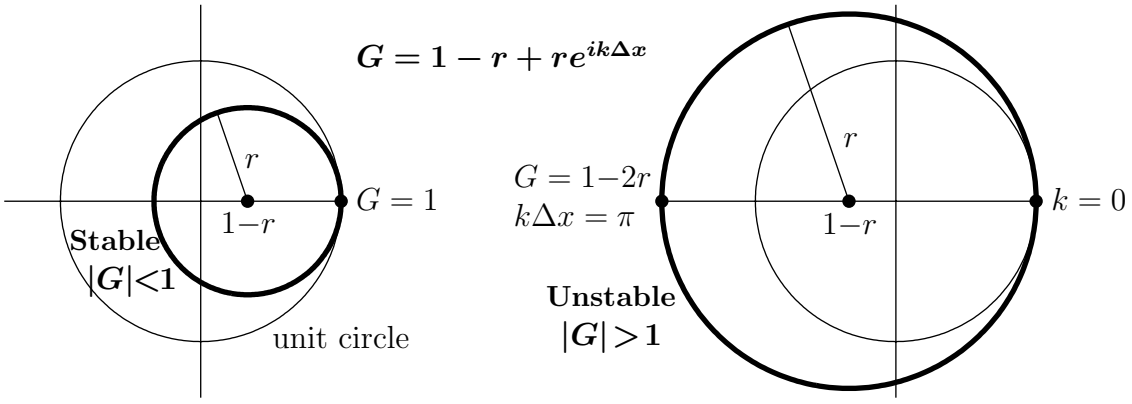


Figure 5.5: Stable (upwind) and unstable (centered): CFL numbers $r = \frac{2}{3}$ and $r = \frac{4}{3}$.

2. Forward difference in time, centered difference in space.

This combination is never stable! The shorthand $U_{j,n}$ will stand for $U(j\Delta x, n\Delta t)$:

$$\frac{U_{j,n+1} - U_{j,n}}{\Delta t} = c\,\frac{U_{j+1,n} - U_{j-1,n}}{2\Delta x} \quad \text{or} \quad U_{j,n+1} = U_{j,n} + \frac{r}{2}\,(U_{j+1,n} - U_{j-1,n})\,. \tag{18}$$

Those coefficients 1 and $r/2$ and $-r/2$ go into the growth factor G, when the solution is a pure exponential and e^{ikx} is factored out:

$$\textbf{Unstable: } \mathbf{|G| > 1} \qquad G = 1 + \frac{r}{2}e^{ik\Delta x} - \frac{r}{2}e^{-ik\Delta x} = \mathbf{1 + ir\,\sin k\Delta x}\,. \tag{19}$$

The real part is 1. The magnitude is $|G| \ge 1$. Its graph is on the left side of Figure 5.6.

3. Lax-Friedrichs Method (upwind-downwind).

We can recover stability for centered differences by changing the time difference. Replace $U_{j,n}$ by the average $\frac{1}{2}(U_{j+1,n} + U_{j-1,n})$ of its neighbors:

$$\textbf{Lax-Friedrichs} \qquad \frac{U_{j,n+1} - \frac{1}{2}(U_{j+1,n} + U_{j-1,n})}{\Delta t} = c\,\frac{U_{j+1,n} - U_{j-1,n}}{2\Delta x}\,. \tag{20}$$

Two old values $U_{j+1,n}$ and $U_{j-1,n}$ produce each new value $U_{j,n+1}$. Moving terms to the right-hand side, the coefficients are $\frac{1}{2}(1+r)$ and $\frac{1}{2}(1-r)$. The growth factor is

$$G = \frac{1+r}{2}\,e^{ik\Delta x} + \frac{1-r}{2}\,e^{-ik\Delta x} = \boldsymbol{\cos k\Delta x + ir \sin k\Delta x}\,. \tag{21}$$

The absolute value is $|G|^2 = (\cos k\Delta x)^2 + r^2(\sin k\Delta x)^2$. In Figure 5.6, $|G| \le 1$ when $r^2 \le 1$. This stability condition agrees again with the CFL condition.

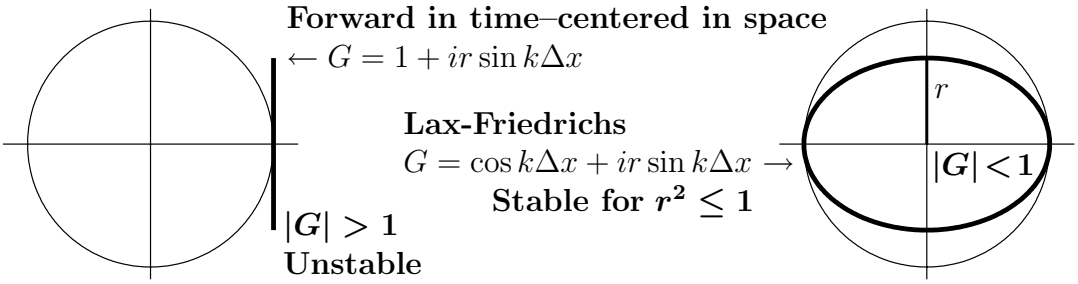


Figure 5.6: Equation (18) is unstable for all r. Equation (20) is stable for $r^2 \le 1$.

Notice that c and r can be negative. The wave can go either way! This will be useful for the two-way wave equation, but the accuracy is still first-order. The Lax-Friedrichs G matches the next term in the exact growth factor only if $r^2 = 1$:

$$\begin{aligned} G &= \cos k\Delta x + ir \sin k\Delta x = 1 + irk\Delta x - \frac{1}{2}(k\Delta x)^2 + \cdots \\ G_{\textbf{exact}} &= e^{ikr\Delta x} = 1 + irk\,\Delta x + \frac{1}{2}i^2r^2(k\,\Delta x)^2 + \cdots \end{aligned} \tag{22}$$

In the exceptional cases $r = 1$ and $r = -1$, G agrees with G_{exact}. Staying exactly on the characteristic line, $U_{j,n+1}$ matches the true $u(j\Delta x, t + \Delta t)$. For $r^2 < 1$, Lax-Friedrichs has an important advantage and disadvantage:

Good Each new $U_{j,n+1}$ is a **positive** combination of old values.

Not good The accuracy is only **first-order**.

Problem 6 will show that second-order is impossible with positive coefficients.

©2006 Gilbert Strang

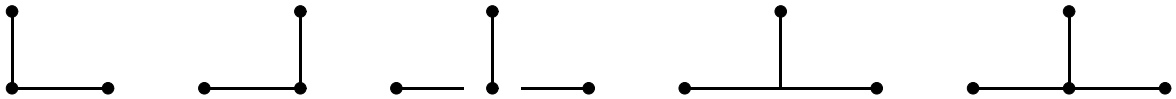
4. Lax-Wendroff Method (second-order accurate).

The LW difference equation (14) combines $U_{j,n}$ and $U_{j-1,n}$ and $U_{j+1,n}$ to compute the new value $U_{j,n+1}$. The coefficients of these three old values go into G:

$$\textbf{Lax-Wendroff} \qquad G = (1 - r^2) + \frac{1}{2}(r^2 + r)e^{ik\Delta x} + \frac{1}{2}(r^2 - r)e^{-ik\Delta x}. \tag{23}$$

This is $G = 1 - r^2 + r^2 \cos k\Delta x + ir \sin k\Delta x$. At the dangerous frequency $k\Delta x = \pi$, the growth factor is $1 - 2r^2$. That stays above -1 if $r^2 \le 1$.

Problem 5 shows that $|G| \le 1$ for every $k\Delta x$. **Lax-Wendroff is stable whenever the CFL condition $r^2 \le 1$ is satisfied**. Again the wave can go either way (or both ways) since c and r can be negative. This is the most accurate of the five methods in Figure 5.7.



upwind **stable** if $r \le 1$ | wrong way **unstable** all Δt | centered **unstable** all Δt | Lax-Friedrichs **stable** if $|r| \le 1$ | Lax-Wendroff **stable** if $|r| \le 1$

Figure 5.7: Difference methods for the one-way wave equation $u_t = cu_x$.

Equivalence of Stability and Convergence

Does the discrete solution U approach the true solution u as $\Delta t \to 0$? The expected answer is yes. But there are two requirements for convergence, and one of them—*stability*—is by no means automatic. The other requirement is *consistency*—the discrete problem must approximate the correct continuous problem. The fact that these two properties are sufficient for convergence, and also *necessary* for convergence, is the ***fundamental theorem of numerical analysis***:

Lax equivalence theorem	Stability is equivalent to convergence, for a consistent approximation to a well-posed linear problem.

Lax proved the equivalence theorem for initial-value problems. The rate of convergence is given in (26). The theorem is equally true for boundary-value problems, and for the approximation of functions, and for the approximation of integrals. It applies to every discretization, when the given problem $Lu = f$ is replaced by $L_h U_h = f_h$. Assuming the inputs f and f_h are close, we will prove that u and U_h are close—provided L_h is stable. The key points of the proof take only a few lines when the equation is linear, and you will see the essence of this fundamental theorem.

Suppose f is changed to f_h and L is replaced by L_h. The requirements are

Consistency:	$f_h \to f$ and $L_h u \to Lu$ for smooth solutions u.
Well-posed:	The inverse of L is bounded: $\|u\| = \|L^{-1}f\| \le C\|f\|$.
Stability:	The inverses L_h^{-1} remain uniformly bounded: $\|L_h^{-1}f_h\| \le C\|f_h\|$.

Under those conditions, the approximation $U_h = L_h^{-1}f_h$ will approach u as h goes to zero. We subtract and add $L_h^{-1}Lu = L_h^{-1}f$ when u is smooth:

$$\textbf{Convergence} \qquad u - U_h = L_h^{-1}(L_h u - Lu) + L_h^{-1}(f - f_h) \to 0\,. \tag{24}$$

Consistency controls the quantities in parentheses (they go to zero). Stability controls the operators L_h^{-1} that act on them. Well-posedness controls the approximation of all solutions by smooth solutions. Then always U_h converges to u.

If stability fails, there will be an input for which the approximations $U_h = L_h^{-1}f$ are not bounded. The *uniform boundedness theorem produces this bad* f, from the inputs f_h on which instability gives $\|L_h^{-1}f_h\| \to \infty$. Convergence fails for this f.

A perfect equivalence theorem goes a little further, after careful definitions:

$$\textbf{Consistency + Stability} \Longleftrightarrow \textbf{Well-posedness + Convergence}\,.$$

Our effort will now concentrate on initial-value problems, to estimate the error (the convergence rate) in $u - U_h$. The parameter h becomes Δt. We take n steps.

The Rate of Convergence

Consistency means that the error at each time step goes to zero as the mesh is refined. Our Taylor series estimates have done more: **The order of accuracy gives the rate** that this one-step error goes to zero. The problem is to extend this local rate to a global rate of convergence, accumulating the errors over n time steps.

Let me write S for a single finite difference step, so $\boldsymbol{U(t + \Delta t) = S\,U(t)}$. The corresponding step for the differential equation will be $\boldsymbol{u(t + \Delta t) = R\,u(t)}$. Then consistency means that Su is close to Ru, and the order of accuracy p tells how close:

Accuracy of discretization	$\|Su - Ru\| \le C_1(\Delta t)^{p+1}$ for smooth solutions u.
Well-posed problem	$\|R^n u\| \le C_2\|u\|$ for $n\,\Delta t \le T$.
Stable approximations	$\|S^n U\| \le C_3\|U\|$ for $n\,\Delta t \le T$.

The difference between $U = S^n u(0)$ and the true $u = R^n u(\quad)$ is $(S^n - R^n)u(\quad)$. The key idea is a "telescoping identity" that involves n single-step differences $S - R$:

$$S^n - R^n = S^{n-1}(S - R) + S^{n-2}(S - R)R + \cdots + (S - R)R^{n-1}\,. \tag{25}$$

Each of those n terms has a clear meaning. First, a power R^k carries $u(0)$ to the true solution $u(k\,\Delta t)$. Then $(S - R)u(k\,\Delta t)$ gives the error at step k of order $(\Delta t)^{p+1}$. Then powers of S carry that one-step error forward to time $n\Delta t$. By stability, this

©2006 Gilbert Strang

amplifies the error by no more than C_3. There are $n \le T/\Delta t$ steps. **The final rate of convergence for smooth solutions is $(\Delta t)^p$**:

$$\|U(n\,\Delta t) - u(n\,\Delta t)\| = \|(S^n - R^n)u(0)\| \le C_1C_2C_3\,\frac{T}{\Delta t}(\Delta t)^{p+1} = C_1C_2C_3\,T(\Delta t)^p\,. \tag{26}$$

Notice how smoothness was needed in the Taylor series (8) and (9), when Δt and Δx multiplied u_{tt} and u_{xx}. That first-order accuracy would not apply if u or u_t or u_x had a jump. Still the order of accuracy p gives a practical estimate of the overall approximation error $u - U$. The problem of scientific computing is to get beyond $p = 1$ while maintaining stability and speed.

Problem Set 5.2

1 Integrate $u_t = c\,u_x$ from $-\infty$ to ∞ to prove that mass is conserved: $dM/dt = 0$. Multiply by u and integrate $uu_t = c\,uu_x$ to prove that energy is also conserved:

$$M(t) = \int_{-\infty}^{\infty} u(x,t)\,dx \quad \text{and} \quad E(t) = \tfrac{1}{2}\int_{-\infty}^{\infty} (u(x,t))^2\,dx \quad \text{stay constant in time.}$$

2 Substitute the true $u(x,t)$ into the Lax-Friedrichs method (21) and use $u_t = cu_x$ and $u_{tt} = c^2u_{xx}$ to find the coefficient of the *numerical dissipation* u_{xx}.

3 The difference equation $U_{j,n+1} = \sum a_m U_{j+m,n}$ has growth factor $G = \sum a_m e^{imk\Delta x}$. Show *consistency* with $e^{ick\Delta t}$ (first-order accuracy at least) when $\sum a_m = 1$ and $\sum m a_m = c\Delta t/\Delta x = r$.

4 The condition for second-order accuracy is $\sum m^2 a_m = r^2$, from the Taylor series. Check this for Lax-Wendroff with $a_0 = 1-r^2$, $a_1 = \frac{1}{2}(r^2+r)$, $a_{-1} = \frac{1}{2}(r^2-r)$. With *nonnegative coefficients* $a_m \ge 0$, the Schwarz inequality $(\sum m\sqrt{a_m}\sqrt{a_m})^2 \le (\sum m^2 a_m)(\sum a_m)$ becomes an equality $r^2 = r^2$. This equality only happens if $m\sqrt{a_m} = (\text{constant})\sqrt{a_m}$. *Second-order is impossible with* $a_m \ge 0$, unless the difference equation has only one term $U_{j,n+1} = U_{j+m,n}$.

5 The Lax-Wendroff method has $G = 1 - r^2 + r^2\cos k\Delta x + ir\sin kx$. Square the real and imaginary parts to get (eventually!) $|G|^2 = 1 - (r^2 - r^4)(1-\cos k\Delta x)^2$. Prove stability, that $|G|^2 \le 1$ if $r^2 \le 1$.

6 Suppose the coefficients in a linear differential equation change as t changes. The one-step solution operators become S_k and R_k, for the step from $k\,\Delta t$ to $(k+1)\Delta t$. After n steps, products replace the powers S^n and R^n in U and u:

$$U(n\,\Delta t) = S_{n-1}S_{n-2}\ldots S_1S_0\,u(0) \quad \text{and} \quad u(n\,\Delta t) = R_{n-1}R_{n-2}\ldots R_1R_0\,u(0)\,.$$

Change the telescoping formula (25) *to produce this* $U - u$. Which parts are controlled by stability? Which parts by well-posedness (= stability of the differential equation)? Consistency still controls $S_k - R_k$.

7 Even an unstable method will converge to the true solution $u = e^{ick\Delta t}e^{ikx}$ for each separate frequency k. Consistency assures that the single-step growth factor G is $1 + ick\,\Delta t + O(\Delta t)^2$. Then for $t = n\,\Delta t$,

$$G^n = \left(1 + \frac{ickt}{n} + O(\frac{1}{n^2})\right)^n \longrightarrow e^{ickt} \quad \text{which is convergence.}$$

How can we have convergence for each $u(0) = e^{ikx}$ and still prove divergence for a combination of frequencies $u(0) = \sum_{-\infty}^{\infty} c_k\, e^{ikx}$?

8 The upwind method with $r > 1$ is unstable because the CFL condition fails. By Problem 3, it does converge to $e^{ik(x+ct)}$ based on values of $u(x,0) = e^{ikx}$ *that do not reach as far as* $x + ct$. The method must be finding a "correct" extrapolation of e^{ikx}. So propose an initial $u(x,0)$ for which convergence fails.

©2006 Gilbert Strang

5.3 The Wave Equation and Staggered Leapfrog

This section focuses on the ***second-order wave equation*** $\boldsymbol{u_{tt} = c^2 u_{xx}}$. We find the exact solution $u(x,t)$. Accuracy and stability are confirmed for the leapfrog method (centered second differences in t and x). This two-step method requires that we rethink the growth factor G, which was clear for a single step. The result will be $p = 2$ for the order of accuracy, and $c\Delta t/\Delta x \le 1$ for stability.

It is useful to rewrite the wave equation as a *first-order system*. The components v_1 and v_2 of the vector unknown can be $\partial u/\partial t$ and $c\,\partial u/\partial x$. Then we are back to a single-step growth factor, but G is now a 2 by 2 matrix.

Second-order accuracy extends to this system $v_t = Av_x$ if we use a ***staggered mesh***. The mesh for v_2 lies in between the mesh for v_1. This has become the standard method in computational electromagnetics (solving Maxwell's equations). The physical laws relating the electric field $\boldsymbol{E}$ and the magnetic field $\boldsymbol{H}$ are beautifully copied by the difference equations on a staggered mesh. The mesh becomes especially important in more space dimensions (x-y and x-y-z), and in finite volume methods.

This section goes beyond the one-way wave equation in at least five ways:

1. *Two characteristic lines* $x+ct = C_{\mathsf{left}}$ and $x-ct = C_{\mathsf{right}}$ go through each (x,t).

2. The *leapfrog method* involves three time levels $t+\Delta t$, t, and $t-\Delta t$.

3. *First-order systems* have vector unknowns $v(x,t)$ and growth matrices G.

4. *Staggered grids* give the much-used FDTD method for Maxwell's equations.

5. *More space dimensions* lead to new CFL and vN stability conditions on Δt.

With $-\infty < x < \infty$, we don't yet have boundary conditions in space. And we are not facing real problems like $u_{tt} = c^2(x)\,u_{xx} + Fe^{ikx}$, with a high frequency forcing terms ($k >> 1$) and a varying speed $c(x)$.

Solution of the Wave Equation

Exactly as for the one-way equation $u_t = cu_x$, we solve the two-way wave equation $u_{tt} = c^2 u_{xx}$ for each pure exponential. That allows us to ***separate the variables***. The space variable is in e^{ikx}, and we look for solutions $\boldsymbol{u(x,t) = G(t)e^{ikx}}$:

$$\textbf{Each } \boldsymbol{k} \qquad \frac{\partial^2 u}{\partial t^2} = c^2\,\frac{\partial^2 u}{\partial x^2} \quad \text{becomes} \quad \frac{d^2G}{dt^2}\,e^{ikx} = i^2c^2k^2G\,e^{ikx}\,. \tag{1}$$

Thus $G_{tt} = i^2c^2k^2G$. This second-order equation has two solutions, $G_{\mathsf{left}} = e^{ickt}$ and $G_{\mathsf{right}} = e^{-ickt}$. So there are two waves with speed c:

$$\textbf{Pure waves} \quad \boldsymbol{u_{\mathsf{left}}(x,t) = e^{ik(x+ct)}} \quad \text{and} \quad \boldsymbol{u_{\mathsf{right}}(x,t) = e^{ik(x-ct)}}\,. \tag{2}$$

Combinations of left-going waves $e^{ik(x+ct)}$ will give a general function $F_1(x+ct)$. Combinations of $e^{ik(x-ct)}$ give $F_2(x-ct)$. The complete solution includes both:

$$\boldsymbol{u(x,t) = u_{\text{left}}(x,t) + u_{\text{right}}(x,t) = F_1(x+ct) + F_2(x-ct)\,.} \tag{3}$$

We need those two functions to match an initial shape $u(x,0)$ and velocity $u_t(x,0)$:

$$\textbf{At } \boldsymbol{t=0} \quad u(x,0) = F_1(x) + F_2(x) \quad \text{and} \quad u_t(x,0) = c\,F_1'(x) - c\,F_2'(x)\,. \tag{4}$$

Solving for F_1 and F_2 gives the unique solution that matches $u(x,0)$ and $u_t(x,0)$:

$$\textbf{Solution} \qquad u(x,t) = \frac{u(x+ct,0)+u(x-ct,0)}{2} + \frac{1}{2c}\int_{x-ct}^{x+ct} u_t(x,0)\,dx\,. \tag{5}$$

The "domain of dependence" of $u(x,t)$ includes the initial values from $x-ct$ to $x+ct$. That domain is on the left side of Figure 5.9, bounded by the characteristic lines.

Example 1 Starting with zero velocity, $u_t(x,0)=0$, the integrated term in formula (5) is zero. A step function $S(x)$ (*wall of water*) will travel left and right along characteristic lines, as in Figure 5.8. It reaches the points $x=1$ and $x=-1$ at time $t=1/c$:

$$\textbf{Two walls} \qquad u(x,t) = \frac{1}{2}\,S(x+ct) + \frac{1}{2}\,S(x-ct) = \left\{0 \text{ or } \frac{1}{2} \text{ or } 1\right\}.$$

By time t, the initial jump at $x=0$ affects the solution between $x=-ct$ and $x=ct$. That is the "domain of influence" of the point $x=0, t=0$.

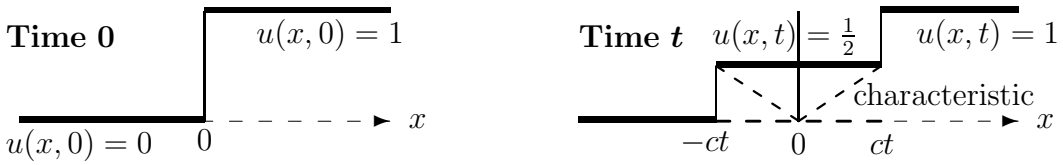


Figure 5.8: Two-wall solution to the wave equation starting from a step function.

The Semidiscrete Wave Equation

Let me start by discretizing only the space derivative u_{xx}. The second difference $U_{j+1} - 2U_j + U_{j-1}$ is the natural choice, divided by $(\Delta x)^2$. For the approximations $U_j(t)$ at the meshpoints $x = j\Delta x$, we have a family of ODEs in the time direction (*method of lines*):

$$\textbf{Semidiscrete } \boldsymbol{u_{tt} = c^2 u_{xx}} \qquad U_j'' = \frac{c^2}{(\Delta x)^2}(U_{j+1} - 2U_j + U_{j-1}\,. \tag{6}$$

©2006 Gilbert Strang

Again we follow every exponential, looking for $U_j(t) = G(t)e^{ikj\Delta x}$. Substitute into (6) and cancel the common factor $e^{ikj\Delta x}$. Instead of $G_{tt} = -c^2k^2G$ we have

$$\textbf{Growth equation } G_{tt} = \frac{c^2}{(\Delta x)^2}(e^{ik\Delta x} - 2 + e^{-ik\Delta x})G = -\frac{c^2}{(\Delta x)^2}(2 - 2\cos k\Delta x)G. \tag{7}$$

The correct right side $-c^2k^2G$ is multiplied by a factor F^2. This F^2 turns up so often that we need to recognize it! Use $2 - 2\cos\theta = 4\sin^2(\theta/2)$:

$$\textbf{Sinc squared } \quad F^2 = \frac{2 - 2\cos k\Delta x}{k^2(\Delta x)^2} = \frac{4\sin^2(k\Delta x/2)}{k^2(\Delta x)^2} = \left(\frac{\sin(k\Delta x/2)}{k\Delta x/2}\right)^2. \tag{8}$$

The "*sinc function*" is defined as $\sin\theta$ divided by θ. When $\theta = k\Delta x$ is small, this is $1 + 0(\theta^2)$. Then F^2 near 1 and equation (7) is close to the correct $G_{tt} = -c^2k^2G$.

For every $k\Delta x$, the growth equation (7) has two exponential solutions:

$$\textbf{Semidiscrete growth} \qquad G_{tt} = -c^2F^2k^2G \quad \text{gives} \quad G(t) = e^{\pm icFkt}. \tag{9}$$

The wave speed c is multiplied by F to give the numerical "phase velocity" cF. *Notice that F depends on k.* Different frequencies e^{ikx} are traveling at different speeds $cF(k)$. This is ***dispersion*** and we will see it again in Section 5.____.

I will mention that the "group velocity"—the derivative of cFk with respect to k—is a more important quantity than the phase velocity cF.

The semidiscrete form suggests a good algorithm for the wave equation, if we have boundary conditions (say $u = 0$ along the lines $t = 0$ and $t = \pi$). If $h = \Delta x = \frac{\pi}{n+1}$, this interval has interior meshpoints. The n by n second difference matrix is the special K from earlier chapters (but now we have $-K$):

$$\textbf{Semidiscrete with boundaries} \qquad U''(t) = \frac{c^2}{(\Delta x)^2}\,KU\,. \tag{10}$$

This is just the equation $MU'' + KU = 0$ of oscillating springs in Section 2.2.

The n eigenvalues of K are positive numbers $2 - 2\cos j\Delta x$. The only change from the equation on an infinite line is that j takes only the values $1, 2, \ldots, n$. The oscillations go on forever as in (8), the energy is conserved, and now the waves bounce back from the boundaries instead of continuing out to $x = \pm\infty$.

Leapfrog from Centered Differences

A fully discrete method also approximates u_{tt} by a centered differences. This time difference "*leaps over*" the space difference at $t = n\Delta t$:

$$\textbf{Leapfrog method} \qquad \frac{U_{j,n+1} - 2U_{j,n} + U_{j,n-1}}{(\Delta t)^2} = c^2\,\frac{U_{j+1,n} - 2U_{j,n} + U_{j-1,n}}{(\Delta x)^2}\,. \tag{11}$$

This has two key differences from the 5-point molecule for $u_{xx} + u_{yy} = 0$ (Laplace). First, $u_{tt} - c^2u_{xx}$ has a minus sign. Second, we have two conditions at $t = 0$ and no

conditions at a later time. We are marching forward in time (marching with Laplace's equation would be totally unstable). A separate calculation for the first time step computes $U_{j,1}$ from the initial shape $u(x,0)$ and the velocity $u_t(x,0)$.

The accuracy of leapfrog is normally second-order. Substitute the true $u(x,t)$ into (11), and use the Taylor series for second differences (Section 1.2). The first terms in the local error give consistency.

$$\textbf{Second-order} \qquad u_{tt} + \frac{1}{12}(\Delta t)^2 u_{tttt} + \cdots = c^2\left(u_{xx} + \frac{1}{12}(\Delta x)^2 u_{xxxx} + \cdots\right). \tag{12}$$

In this case $u_{tttt} = c^2 u_{xxtt} = c^2 u_{ttxx} = c^4 u_{xxxx}$. The two sides of (12) differ by

$$\textbf{Local discretization error} \qquad \frac{1}{12}[(\Delta t)^2 c^4 - (\Delta x)^2 c^2] u_{xxxx} + \cdots \tag{13}$$

Again $c\Delta t = \Delta x$ is the golden time step that follows the characteristic exactly. The two triangles in Figure 5.9 become exactly the same in this borderline case $r = 1$. The CFL reasoning shows instability for $r > 1$. We now show that $r \le 1$ is stable.

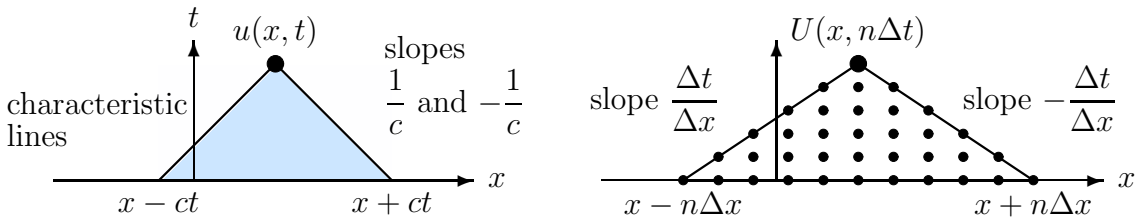


Figure 5.9: Domains of dependence: u from wave equation and U from leapfrog.

Stability of the Leapfrog Method

A difference equation must use the initial conditions in this whole interval, to have a chance of converging to $u(x,t)$. The domain of dependence for U must include the domain of dependence for u. The slopes must have $\Delta t/\Delta x \le 1/c$. Since convergence requires stability, we have a Courant-Friedrichs-Lewy condition on Δt:

CFL condition The leapfrog method will require $r = c\,\Delta t/\Delta x \le 1$.

For a double-step difference equation, we still look for pure solutions $U(x, n\Delta t) = G^n e^{ikx}$, separating time from space. In the leapfrog equation (11) this gives

$$\left[\frac{G^{n+1} - 2G^n + G^{n-1}}{(\Delta t)^2}\right] e^{ikx} = c^2 G^n \left[\frac{e^{ik\Delta x} - 2 + e^{-ik\Delta x}}{(\Delta x)^2}\right] e^{ikx}.$$

Set $r = c\Delta t/\Delta x$ and cancel $G^{n-1}e^{ikx}$. This leaves a quadratic equation for G:

$$G^2 - 2G + 1 = r^2\, G\,(2\cos k\Delta x - 2). \tag{14}$$

©2006 Gilbert Strang

The two-step leapfrog equation allows two G's (of course!). For stability, both must satisfy $|G| \le 1$ for all frequencies k. Rewrite equation (14) for G:

Growth factor equation $$G^2 - 2\left[1 - r^2 + r^2 \cos k\Delta x\right] G + 1 = 0\,. \tag{15}$$

The roots of $G^2 - 2aG + 1 = 0$ are $G = a \pm \sqrt{a^2 - 1}$. Everything depends on that square root giving an imaginary number, when $a^2 = [1 - r^2 + r^2 \cos k\Delta x]^2 \le 1$:

$$\text{If} \quad a^2 \le 1 \quad \text{then} \quad G = a \pm i\sqrt{1 - a^2} \quad \text{has} \quad |G|^2 = a^2 + (1 - a^2) = 1\,.$$

The CFL condition $r \le 1$ does produce $a^2 \le 1$ and the leapfrog method is stable:

Stability If $r \le 1$ then $|a| = |1 - r^2 + r^2 \cos k\Delta x| \le (1 - r^2) + r^2 = 1\,.$ (16)

An unstable $r > 1$ would produce $|a| = |1 - 2r^2| > 1$ at the dangerous $k\Delta x = \pi$. Then both G's are real, and their product is 1, and one of them has $|G| > 1$.

Note 1 Suppose r is exactly 1, so that $\boldsymbol{c\Delta t = \Delta x}$. At this "golden ratio" we expect perfect accuracy. The middle terms $-2U_{j,n}$ and $-2r^2U_{j,n}$ cancel in the leapfrog equation (11), leaving a complete leap over the center points at (j, n) when $r = 1$:

Exact leapfrog $$U_{j,n+1} + U_{j,n-1} = U_{j+1,n} + U_{j-1,n}\,. \tag{17}$$

The difference equation is satisfied by $u(x,t)$, because it is satisfied by every wave $U(x + ct)$ and $U(x - ct)$. Take $U_{j,n} = U(j\Delta x + cn\Delta t)$ and use $c\Delta t = \Delta x$:

$U_{j,n+1}$ and $U_{j+1,n}$	are both equal to	$U(j\Delta x + cn\Delta t + \Delta x)$
$U_{j,n-1}$ and $U_{j-1,n}$	are both equal to	$U(j\Delta x + cn\Delta t - \Delta x)$

So (17) is satisfied by all traveling waves $U(x + ct)$, and similarly by $U(x - ct)$.

Note 2 You could also apply leapfrog to the *one-way* equation $u_t = c\,u_x$:

One-way leapfrog $$U_{j,n+1} - U_{j,n-1} = \frac{c\Delta t}{\Delta x}(U_{j+1,n} - U_{j-1,n}). \tag{18}$$

Now the growth factor equation is $G^2 - 2(ir \sin k\Delta x)G - 1 = 0$. Problem ____ confirms that the stability condition is again $r \le 1$. In that stable case, one growth factor is sensible and the other is strange:

$$G_1 = e^{ir \sin k\Delta x} \approx e^{ick\Delta t} \quad \text{and} \quad G_2 = -e^{-ir \sin k\Delta x} \approx -1\,. \tag{19}$$

G_1 and G_2 are *exactly on the unit circle*. With $|G| = 1$ there is no room to move. Numerical diffusion $\alpha(U_{j+1,n} - 2U_{j,n} + U_{j-1,n})$ usually adds extra stability, but not here. So leapfrog for first-order equations can be dangerous.

Section 5.4 will study the convection-diffusion equation $u_t = c\,u_x + d\,u_{xx}$.

©2006 Gilbert Strang

Wave Equation in Higher Dimensions

The wave equation extends to three-dimensional space (with speed set at $c = 1$):

3D Wave equation $$\boldsymbol{u_{tt} = u_{xx} + u_{yy} + u_{zz}}\,. \tag{20}$$

Waves go in all directions, and the solution is a superposition of pure harmonics. These plane waves now have three wave numbers k, ℓ, m, and frequency w:

Exponential solutions $$u(x,y,z,t) = \boldsymbol{e^{i(\omega t + kx + \ell y + mz)}}\,.$$

Substituting into the wave equation gives $\omega^2 = k^2 + \ell^2 + m^2$. So there are two frequencies $\pm\omega$. These exponential solutions combine to match the initial wave height $u(x,y,z,0)$ and its velocity $u_t(x,y,z,0)$.

Suppose the initial velocity is a ***three-dimensional delta function*** $\boldsymbol{\delta(x,y,z)}$:

$$\delta(x,y,z) = \delta(x)\delta(y)\delta(z) \quad \text{gives} \quad \iiint f(x,y,z)\,\delta(x,y,z)\,dV = f(0,0,0)\,. \tag{21}$$

The resulting $u(x,y,z,t)$ will be the *fundamental solution* of the wave equation. It is the response to the delta function, which gives equal weight to all harmonics. Rather than computing that superposition we find it from the wave equation itself. Spherical symmetry greatly simplifies $u_{xx} + u_{yy} + u_{zz}$, when u depends only on r and t:

Symmetry produces $\boldsymbol{u(r,t)}$ $$\frac{\partial^2 u}{\partial t^2} = \frac{\partial^2 u}{\partial r^2} + \frac{2}{r}\frac{\partial u}{\partial r}\,. \tag{22}$$

Multiplying by r, this is a one-dimensional equation $(ru)_{tt} = (ru)_{rr}$! Its solutions ru will be functions of $r - t$ and $r + t$. Starting from a delta function is like sound going out from a bell, or light from a point source. *The solution is nonzero only on the sphere $r = t$.* So every point hears the bell only once, as the sound wave passes by. An impulse in 3D produces a sharp response (this is Huygen's principle).

In 2D, the solution does *not* return to zero for $t > r$. We couldn't hear or see clearly in Flatland. You might imagine a point source in two dimensions as a *line source* in the z-direction in three dimensions. The solution is independent of z, so it satisfies $u_{tt} = u_{xx} + u_{yy}$. But in three dimensions, spheres starting from sources along the line continue to hit the listener. They come from further and further away, so the solution decays—but it is not zero. The wave front passes, but waves keep coming.

EXERCISE ON EQ.(26)

Leapfrog Method in Higher Dimensions

In one dimension, two characteristics go out from each point $(x, 0)$. In 2D and 3D, a *characteristic cone* goes out from $(x, y, 0)$ and $(x, y, z, 0)$. It is essential to see how the stability condition changes from $r = c\Delta t/\Delta x \le 1$.

©2006 Gilbert Strang

The leapfrog method replaces u_{xx} and u_{yy} by centered differences at time $n\Delta t$:

Leapfrog for $u_{tt} = u_{xx} + u_{yy}$
$$\frac{U_{n+1} - 2U_n + U_{n-1}}{(\Delta t)^2} = \frac{\Delta_x^2 U_n}{(\Delta x)^2} + \frac{\Delta_y^2 U_n}{(\Delta y)^2}.$$

U_0 and U_1 come from the given initial conditions $u(x, y, 0)$ and $u_t(x, y, 0)$. We look for a solution $U_n = G^n e^{ikx} e^{i\ell y}$ with **separation of variables**. Substituting into the leapfrog equation and canceling $G^{n-1} e^{ikx} e^{i\ell y}$ produces the 2D equation for two G's:

Growth factor
$$\frac{G^2 - 2G + 1}{(\Delta t)^2} = G\frac{(2\cos k\,\Delta x - 2)}{(\Delta x)^2} + G\frac{(2\cos \ell\,\Delta y - 2)}{(\Delta y)^2}. \quad (23)$$

Again this has the form $G^2 - 2aG + 1 = 0$. You can see a in brackets:

$$G^2 - 2\left[1 - \left(\frac{\Delta t}{\Delta x}\right)^2 (1 - \cos k\,\Delta x) - \left(\frac{\Delta t}{\Delta y}\right)^2 (1 - \cos \ell\,\Delta y)\right] G + 1 = 0. \quad (24)$$

Both roots must have $|G| = 1$ for stability. This still requires $-1 \le a \le 1$. When the cosines are -1 (the dangerous value) we find the stability condition for leapfrog:

Stability
$$-1 \le 1 - 2\left(\frac{\Delta t}{\Delta x}\right)^2 - 2\left(\frac{\Delta t}{\Delta y}\right)^2 \text{ needs } \left(\frac{\boldsymbol{\Delta t}}{\boldsymbol{\Delta x}}\right)^2 + \left(\frac{\boldsymbol{\Delta t}}{\boldsymbol{\Delta y}}\right)^2 \le \mathbf{1}. \quad (25)$$

For $\Delta x = \Delta y$ on a square grid, this is $\Delta t \le \Delta x/\sqrt{2}$. In three dimensions it would be $\Delta t \le \Delta x/\sqrt{3}$. Those also come from the CFL condition, that the characteristic cone must lie inside the pyramid that gives the leapfrog domain of dependence. Figure 5.10 shows the cone and pyramid just touching, when $\Delta t = \Delta x/\sqrt{2}$.

Cone has circular base for $u_{tt} = u_{xx} + u_{yy}$
Pyramid has diamond base for leapfrog
Cone and pyramid go up to $(0, 0, \Delta t)$

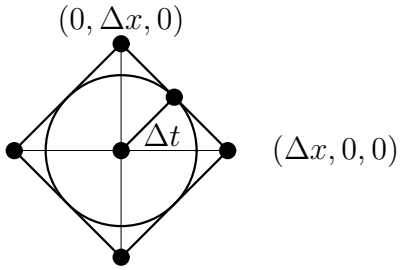


Figure 5.10: The pyramid contains and touches the cone when $(\Delta t)^2 = (\Delta x)^2/2$.

An Equivalent First-order System

I can display a system of two equations $v_t = Av_x$ that is equivalent to $u_{tt} = c^2 u_{xx}$:

First-order system
$$\frac{\boldsymbol{\partial}}{\boldsymbol{\partial t}}\begin{bmatrix} \boldsymbol{u_t} \\ \boldsymbol{cu_x} \end{bmatrix} = \begin{bmatrix} \mathbf{0} & \boldsymbol{c} \\ \boldsymbol{c} & \mathbf{0} \end{bmatrix} \frac{\boldsymbol{\partial}}{\boldsymbol{\partial x}}\begin{bmatrix} \boldsymbol{u_t} \\ \boldsymbol{cu_x} \end{bmatrix}. \quad (26)$$

The first equation recovers $u_{tt} = c^2 u_{xx}$. The second is the identity $cu_{xt} = cu_{tx}$. Notice that the 2 by 2 matrix is symmetric and its eigenvalues are the wave velocities $\pm c$.

©2006 Gilbert Strang

This "symmetric hyperbolic" form $v_t = Av_x$ is useful in theory and practice. The energy $E(t) = \int \frac{1}{2}\|v(x,t)\|^2\,dx$ is automatically constant in time! Here is the proof for any equation $v_t = Av_x$ with a symmetric matrix A:

$$\frac{\partial}{\partial t}\left(\frac{1}{2}\|v\|^2\right) = \sum v_i \frac{\partial v_i}{\partial t} = v^{\mathrm{T}} v_t = v^{\mathrm{T}} A v_x = \frac{\partial}{\partial x}\left(\frac{1}{2} v^{\mathrm{T}} A v\right). \tag{27}$$

When you integrate over all x, the left side is $\partial E/\partial t$. The right side is $\frac{1}{2}v^{\mathrm{T}}Av$ at the limits $x = \pm\infty$. Those limits give zero (no signal has reached that far). So the derivative of $E(t)$ is zero, and $E(t)$ stays constant.

The Euler equations of compressible flow are also a first-order system, but not linear. In physics and engineering, a linear equation deals with a small disturbance. Something from outside acts to change the equilibrium, but not by much:

in acoustics it is a slowly moving body

in aerodynamics it is a slender wing

in elasticity it is a small load

in electromagnetism it is a small source.

Below some level, the cause-effect relation is very close to linear. In acoustics, the sound speed is steady when pressure is nearly constant. In elasticity, Hooke's law holds until the geometry changes or the material begins to break down. In electromagnetism, nonlinearity comes with relativistic and quantum effects.

The case to understand has $A =$ constant matrix, with n real eigenvalues λ and eigenvectors w. The vector equation $v_t = Av_x$ will split into n scalar one-way wave equations $U_t = \lambda U_x$. When $Aw = \lambda w$ we look for $v(x,t) = U(x,t)w$:

$$v_t = Av_x \quad \text{becomes} \quad \frac{\partial U}{\partial t} w = A\frac{\partial U}{\partial x} w = \lambda \frac{\partial U}{\partial x} w \quad \text{so} \quad \frac{\partial U}{\partial t} = \lambda \frac{\partial U}{\partial x}. \tag{28}$$

The complete solution vector is $v(x,t) = U_1(x+\lambda_1 t)w_1 + \cdots + U_n(x+\lambda_n t)w_n$. ***The problem $v_t = Av_x$ has n signal speeds λ_i and it sends out n waves.***

There are n characteristic lines $x + \lambda_i t =$ constant. The wave equation has $\left[\begin{smallmatrix} \mathbf{0} & \mathbf{c} \\ \mathbf{c} & \mathbf{0} \end{smallmatrix}\right]$ and two eigenvalues $\lambda = c$ and $\lambda = -c$. The eigenvectors w are $(\frac{1}{2}, -\frac{1}{2})$ and $(\frac{1}{2}, \frac{1}{2})$. Then the two scalar equations $U_t = \lambda U_x$ produce left and right waves:

$$\begin{aligned} \boldsymbol{\lambda_1 = \quad c} \qquad & \frac{\partial}{\partial t}(u_t + c\,u_x) = \quad c\,\frac{\partial}{\partial x}(u_t + c\,u_x) \\ \boldsymbol{\lambda_2 = -c} \qquad & \frac{\partial}{\partial t}(u_t - c\,u_x) = -c\,\frac{\partial}{\partial x}(u_t - c\,u_x). \end{aligned} \tag{29}$$

Each equation agrees with $u_{tt} = c^2 u_{xx}$. The one-way waves are $U(x,t) = U(x+\lambda t, 0)$. The vector solution $v(x,t)$ is recovered from $U_1 w_1 + U_2 w_2$:

$$v = \begin{bmatrix} u_t \\ c\,u_x \end{bmatrix} = (u_t + c\,u_x)\begin{bmatrix} \frac{1}{2} \\ -\frac{1}{2} \end{bmatrix} + (u_t - c\,u_x)\begin{bmatrix} \frac{1}{2} \\ \frac{1}{2} \end{bmatrix}. \tag{30}$$

A stable difference method for $v_t = Av_x$ comes from a stable method for $u_t = \pm c u_x$. Just replace c by A in Lax-Friedrichs and Lax-Wendroff, or go to leapfrog.

©2006 Gilbert Strang

Leapfrog on a Staggered Grid

The discrete case should copy the continuous case. The two-step leapfrog difference equation should reduce to a pair of one-step equations. But if we don't keep the individual equations centered, they will lose second-order accuracy. **The way to center both first-order equations is to use a staggered grid** (Figure 5.11).

Please allow me to name the two components $v_1 = \boldsymbol{E}$ and $v_2 = \boldsymbol{H}$. Then the staggered grid for the wave equation matches *Yee's method for Maxwell's equations*. Yee's idea transformed the whole subject of computational electromagnetics (it is now called the FDTD method: finite differences in the time domain). Previously the moment method, which is Galerkin's method, had been dominant—but staggered grids are so natural for $\boldsymbol{E}$ and $\boldsymbol{H}$. We stay with the wave equation here, copying (26):

$$\begin{array}{ll} \textbf{Maxwell in 1D} & \boldsymbol{\partial E/\partial t = c\,\partial H/\partial x} \\ \textbf{(normalized)} & \boldsymbol{\partial H/\partial t = c\,\partial E/\partial x} \end{array} \quad \text{becomes} \quad \begin{array}{l} \Delta_t E/\Delta t = c\Delta_x H/\Delta x \\ \Delta_t H/\Delta t = c\Delta_x E/\Delta x\,. \end{array} \tag{31}$$

Those first derivatives of E and H are replaced by first differences. I will put E on the standard grid and H on the staggered (half-integer) grid. *Notice how all the differences are centered in Figure 5.11a. This gives second-order accuracy.*

The identities $E_{tx} = E_{xt}$ and $H_{tx} = H_{xt}$ lead to wave equations for E *and* H:

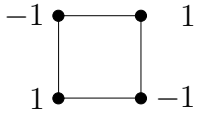
$$\begin{array}{rcl} E_t &=& cH_x \\ H_t &=& cE_x \end{array} \quad \text{becomes} \quad \begin{array}{rcccccc} E_{tt} &=& cH_{xt} &=& cH_{tx} &=& c^2E_{xx} \\ H_{tt} &=& cE_{xt} &=& cE_{tx} &=& c^2H_{xx} \end{array} \tag{32}$$

In the discrete case, the identity is $\Delta_x(\Delta_t) = \Delta_t(\Delta_x)$. Differences copy derivatives.

When we eliminate $\boldsymbol{H}$, we get the two-step leapfrog equation for $\boldsymbol{E}$. And eliminating E gives the leapfrog equation for H. This all comes from the finite difference analogue of the cross-derivative identity $u_{tx} = u_{xt}$:

$$\boldsymbol{\frac{\partial}{\partial x}\left(\frac{\partial u}{\partial t}\right) = \frac{\partial}{\partial t}\left(\frac{\partial u}{\partial x}\right)} \quad \textit{corresponds to} \quad \boldsymbol{\frac{\Delta_x(\Delta_t U)}{(\Delta x)(\Delta t)} = \frac{\Delta_t(\Delta_x U)}{(\Delta t)(\Delta x)}}\,. \tag{33}$$

With equal denominators, we only need to check the numerators. On any grid, the same 1's and -1's appear both ways in $\Delta x \Delta t$ and $\Delta t \Delta x$!



$$\begin{array}{rcccc} \Delta_x(\Delta_t U) &=& (U_{n+1,j+1} - U_{n,j+1}) &-& (U_{n+1,j} - U_{n,j}) \\ \Delta_t(\Delta_x U) &=& (U_{n+1,j+1} - U_{n+1,j}) &-& (U_{n,j+1} - U_{n,j}) \end{array}$$

You could compare ( ) with the Cauchy-Riemann equations $u_x = s_y$ and $u_y = -s_x$ for the potential $u(x, y)$ and stream function $s(x, y)$. (Those solve Laplace's equation and not the wave equation.) It would be natural to discretize the Cauchy-Riemann equations on a staggered grid.

May I emphasize that these grids are useful for many other equations too. We will see the "half-point" grid values in Section 5.____ for the flux F in the conservation law $u_t + F(u)_x = 0$, which is a nonlinear extension of the one-way wave equation. Half-point values are centrally important throughout the ***finite volume method***. Maxwell's equations in **integral form** lead to the finite integration technique [ ].

©2006 Gilbert Strang

Maxwell's Equations

For electrodynamics, the number c in the wave equation is the *speed of light*. It is the same large number that appears in Einstein's $e = mc^2$. The CFL stability condition $c^2(\Delta t)^2 \le (\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2$ for the leapfrog method might require very small time steps (on the scale of ordinary life). But we all know that the wavelength for light is nothing like a meter or a centimeter. The leapfrog method is entirely appropriate, and we write Maxwell's equations without source terms:

Maxwell's equations in free space
$$\frac{\partial E}{\partial t} = \frac{1}{\epsilon}\,\text{curl}\,H \quad \text{and} \quad \frac{\partial H}{\partial t} = -\frac{1}{\mu}\,\text{curl}\,E \tag{34}$$

An important application is the reflection of a radar signal by an airplane. The region of interest is ***exterior*** to the plane. In principle that region extends infinitely far in all directions. In practice we compute inside a large box, and choose boundary conditions that don't reflect waves back into the box from its artificial boundary (which is a computational region and not physical).

Those ***absorbing boundary conditions*** [ ] are crucial to a good discretization. Chapter 7 of [ ] describes how a "*perfectly matched layer*" can select coefficients so that waves go through the boundary with very little reflection. Applications of Maxwell's equations range all the way from the Earth's electromagnetic environment to cell phones (safety of the user) to micron-scale lasers and photonics.

The first of Maxwell's six equations in ( ) involves the electric field component E_x:
$$\frac{\partial}{\partial t}E_x = \frac{1}{\epsilon}\left[\frac{\partial}{\partial y}H_z - \frac{\partial}{\partial z}H_y\right]. \tag{35}$$

Yee's difference equation computes E_x at the new time $(n+1)\Delta t$ from E_x at time $n\Delta t$ and the space differences of H_z and H_y ***at time*** $\boldsymbol{(n+\frac{1}{2})\Delta t}$. Figure 5.11 shows how those components of the magnetic field $\mathbf{H}$ are on a grid that is staggered with respect to the grid for $\mathbf{E}$. We have six differential equations like (35) and six difference equations, to produce E_x, E_y, E_z at time $(n+1)\Delta t$ and then H_x, H_y, H_z at time $(n+1.5)\Delta t$.

The stability condition $c^2(\Delta t)^2 \le (\Delta x)^2+(\Delta y)^2+(\Delta z)^2$ is acceptable. Perhaps the greatest drawback is the rectangular grid (finite elements are always more adaptable). But the FDTD method has been used with 10^9 meshpoints, which we cannot afford on an unstructured mesh. Finite differences also have numerical dispersion—the discrete wave speeds depend on the wave number $\boldsymbol{k} = (k_x, k_y, k_z)$. Those speeds don't exactly match c. We will have phase factors like F in equation ( ), extended to include Δy and Δz. When the dispersion creates significant errors, we can upgrade the spatial differences to fourth-order accuracy (using more mesh values). But those wider difference methods can go across material interfaces and external boundaries. This produces the ever-present give and take of numerical analysis: higher accuracy brings greater complexity. We can take larger steps Δt but every step is slower (and harder to code).

©2006 Gilbert Strang

FIGURE TO COME...

Figure 5.11: $\dfrac{\Delta E}{\Delta t} = c\dfrac{\Delta H}{\Delta x}$ and $\dfrac{\Delta H}{\Delta t} = c\dfrac{\Delta E}{\Delta x}$ are staggered but centered.

Problem Set 5.3

1 Write the equation $u_{tt} = u_{xx} + u_{yy}$ as a first-order system $v_t = Av_x + Bv_y$ with the vector unknown $v = (u_t, u_x, u_y)$. The matrices A and B should be symmetric. Then the energy $E(t) = \frac{1}{2}\int (u_t^2 + u_x^2 + u_y^2)\, dx$ is *constant*.

2 How was the symmetry of A used in the final step $v^{\mathrm{T}}Av_x = (\frac{1}{2}v^{\mathrm{T}}Av)_x$ in equation (27) ? You could write out $v^{\mathrm{T}}Av = \sum\sum a_{ij}v_i(x)v_j(x)$ and take the derivative of each term by the product rule.

3 Add Gauss law $\operatorname{div} D = 0$ and $\operatorname{div} B = 0$ with $D = \epsilon E$ and $B = \mu H$.

5.4 The Heat Equation and Convection-Diffusion

The wave equation conserves energy. The heat equation $u_t = u_{xx}$ dissipates energy. The starting conditions for the wave equation can be recovered by going backward in time. The starting conditions for the heat equation can never be recovered. Compare $u_t = cu_x$ with $u_t = u_{xx}$, and look for pure exponential solutions $u(x,t) = G(t)\,e^{ikx}$:

Wave equation: $\boldsymbol{G' = ickG}$	$G(t) = e^{ickt}$	has $\lvert G\rvert = 1$	(conserving energy)
Heat equation: $\boldsymbol{G' = -k^2 G}$	$G(t) = e^{-k^2 t}$	has $G < 1$	(dissipating energy)

Discontinuities are immediately smoothed out by the heat equation, since G is exponentially small when k is large. This section solves $u_t = u_{xx}$ first analytically and then by finite differences. The key to the analysis is the beautiful ***fundamental solution*** starting from a point source (delta function). We will show in equation (7) that this special solution is a bell-shaped curve:

$$\boldsymbol{u(x,t) = \frac{1}{\sqrt{4\pi t}}\, e^{-x^2/4t}} \text{ comes from the initial condition } u(x,0) = \delta(x)\,. \tag{1}$$

Notice that $u_t = cu_x + du_{xx}$ has convection and diffusion at the same time. The wave is smoothed out as it travels. This is a much simplified linear model of the nonlinear Navier-Stokes equations for fluid flow. The relative strength of convection by cu_x and diffusion by du_{xx} will be given below by the Peclet number.

The **Black-Scholes equation** for option pricing in mathematical finance also has this form. So do the key equations of environmental and chemical engineering.

For difference equations, explicit methods have stability conditions like $\Delta t \le \frac{1}{2}(\Delta x)^2$. This very short time step is more expensive than $c\Delta t \le \Delta x$. **Implicit methods** can avoid that stability condition by computing the space difference $\Delta^2 U$ at the new time level $n+1$. This requires solving a linear system at each time step.

We can already see two major differences between the heat equation and the wave equation (and also one conservation law that applies to both):

1. ***Infinite signal speed.*** The initial condition at a single point *immediately* affects the solution at all points. The effect far away is not large, because of the very small exponential $e^{-x^2/4t}$ in the fundamental solution. But it is not zero. (A wave produces no effect at all until the signal arrives, with speed c.)

2. ***Dissipation of energy.*** The energy $\frac{1}{2}\int (u(x,t))^2\,dx$ is a *decreasing* function of t. For proof, multiply the heat equation $u_t = u_{xx}$ by u. Integrate uu_{xx} by parts with $u(\infty) = u(-\infty) = 0$ to produce the integral of $-(u_x)^2$:

$$\textbf{Energy decay} \qquad \frac{d}{dt}\int_{-\infty}^{\infty} \frac{1}{2}u^2\,dx = \int_{-\infty}^{\infty} uu_{xx}\,dx = -\int_{-\infty}^{\infty} (u_x)^2\,dx \le 0\,. \tag{2}$$

©2006 Gilbert Strang

3. *Conservation of heat* (analogous to conservation of mass):

$$\textbf{Heat is conserved} \qquad \frac{d}{dt}\int_{-\infty}^{\infty} u(x,t)\,dx = \int_{-\infty}^{\infty} u_{xx}\,dx = \Big[u_x(x,t)\Big]_{x=-\infty}^{\infty} = 0\,. \tag{3}$$

Analytic Solution of the Heat Equation

Start with **separation of variables** to find solutions to the heat equation:

$$\text{Assume } u(x,t) = G(t)E(x). \quad \text{Then } u_t = u_{xx} \text{ gives } G'E = GE'' \text{ and } \frac{G'}{G} = \frac{E''}{E}\,. \tag{4}$$

The ratio G'/G depends only on t. The ratio E''/E depends only on x. Since equation (4) says they are equal, they must be constant. This produces a useful family of solutions to $u_t = u_{xx}$:

$$\frac{E''}{E} = \frac{G'}{G} \quad \text{is solved by} \quad \boldsymbol{E(x) = e^{ikx}} \text{ and } \boldsymbol{G(t) = e^{-k^2 t}}\,.$$

Two x-derivatives produce the same $-k^2$ as one t-derivative. We are led to exponential solutions of $e^{ikx}e^{-k^2t}$ and to their linear combinations (integrals over different k):

$$\textbf{General solution} \qquad \boldsymbol{u(x,t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} \widehat{u}_0(k) e^{ikx} e^{-k^2 t}\,dx.} \tag{5}$$

At $t = 0$, formula (5) recovers the initial condition $u(x,0)$ because it inverts the Fourier transform $\widehat{u}_0$ (Section 4.4.) So we have the analytical solution to the heat equation—not necessarily in an easily computable form! This form usually requires two integrals, one to find the transform $\widehat{u}_0(k)$ of $u(x,0)$, and the other to find the inverse transform of $\widehat{u}_0(k)e^{-k^2t}$ in (5).

Example 1 Suppose the initial function is a bell-shaped Gaussian $u(x,0) = e^{-x^2/2\sigma}$. Then the solution remains a Gaussian. The number σ that measures the width of the bell increases to $\sigma + 2t$ at time t, as heat spreads out. This is one of the few integrals involving e^{-x^2} that we can do exactly. Actually, we don't have to do the integral.

That function $e^{-x^2/2\sigma}$ is the impulse response (fundamental solution) at time $t = 0$ to a delta function $\delta(x)$ that occurred earlier at $t = -\frac{1}{2}\sigma$. So the answer we want (at time t) is the result of starting from that $\delta(x)$ and going forward a total time $\frac{1}{2}\sigma + t$:

$$\textbf{Widening Gaussian} \qquad u(x,t) = \frac{\sqrt{\pi(2\sigma)}}{\sqrt{\pi(2\sigma+4t)}}\, e^{-x^2/(2\sigma+4t)}\,. \tag{6}$$

This has the right start at $t = 0$ and it satisfies the heat equation.

©2006 Gilbert Strang

The Fundamental Solution

For a delta function $u(x,0) = \delta(x)$ at $t = 0$, the Fourier transform is $\widehat{u}_0(k) = 1$. Then the inverse transform in (5) produces $u(x,t) = \frac{1}{2\pi}\int e^{ikx}e^{-k^2t}\,dk$ One computation of this u uses a neat integration by parts for $\partial u/\partial x$. It has three -1's, from the integral of ke^{-k^2t} and the derivative of ie^{ikx} and integration by parts itself:

$$\frac{\partial u}{\partial x} = \frac{1}{2\pi}\int_{-\infty}^{\infty}(e^{-k^2t}k)(ie^{ikx})\,dk = -\frac{1}{4\pi t}\int_{-\infty}^{\infty}(e^{-k^2t})(xe^{ikx})\,dk = -\frac{xu}{2t}\,. \tag{7}$$

This linear equation $\partial u/\partial x = -xu/2t$ is solved by $u = ce^{-x^2/4t}$. The constant $c = 1/\sqrt{4\pi t}$ is determined by the requirement $\int u(x,t)\,dx = 1$. (This conserves the heat $\int u(x,0)\,dx = \int \delta(x)\,dx = 1$ that we started with. It is the area under a bell-shaped curve.) The solution (1) for diffusion from a point source is confirmed:

Fundamental solution from $u(x,0) = \delta(x)$

$$u(x,t) = \frac{1}{\sqrt{4\pi t}}\,e^{-x^2/4t}\,. \tag{8}$$

In two dimensions, we can separate x from y and solve $u_t = u_{xx} + u_{yy}$:

Fundamental solution from $u(x,y,0) = \delta(x)\delta(y)$

$$u(x,y,t) = \left(\frac{1}{\sqrt{4\pi t}}\right)^2 e^{-x^2/4t}\,e^{-y^2/4t}\,. \tag{9}$$

With patience you can verify that $u(x,t)$ and $u(x,y,t)$ do solve the 1D and 2D heat equations (Problem ____). The zero initial conditions away from the origin are correct as $t \to 0$, because $e^{-c/t}$ goes to zero much faster than $1/\sqrt{t}$ blows up. And since the total heat remains at $\int u\,dx = 1$ or $\iint u\,dx\,dy = 1$, we have a valid solution.

If the source is at another point $x = s$, then the response just shifts by s. The exponent becomes $-(x-s)^2/4t$ instead of $-x^2/4t$. If the initial $u(x,0)$ is a *combination* of delta functions, then by linearity the solution is the same combination of responses. But every $u(x,0)$ is an integral $\int \delta(x-s)\,u(s,0)\,ds$ of point sources! So the solution to $u_t = u_{xx}$ is an integral of the responses to $\delta(x-s)$. Those responses are fundamental solutions starting from all points $x = s$:

Solution from any $u(x,0)$

$$u(x,t) = \frac{1}{\sqrt{4\pi t}}\int_{-\infty}^{\infty} u(s,0)\,e^{-(x-s)^2/4t}\,ds\,. \tag{10}$$

Now the formula is reduced to one infinite integral—but still not simple. And for a problem with boundary conditions at $x = 0$ and $x = 1$ (the temperature on a finite interval, much more realistic), we have to think again. Similarly for an equation $u_t = (c(x)u_x)_x$ with variable conductivity or diffusivity. That thinking probably leads us to finite differences.

I see the solution $u(x,t)$ in (10) as the **convolution** of the initial function $u(x,0)$ with the fundamental solution. Three important properties are immediate:

©2006 Gilbert Strang

1. ***If $u(x,0) \geq 0$ for all x then $u(x,t) \geq 0$ for all x and t.*** Nothing in formula (10) will be negative.

2. ***The solution is infinitely smooth.*** The Fourier transform $\widehat{u}_0(k)$ in (5) is multiplied by $e^{-k^2 t}$. In (10), we can take all the x and t derivatives we want.

3. ***The scaling matches x^2 with t.*** A diffusion constant d in the equation $u_t = du_{xx}$ will lead to the same solution with t replaced by dt, when we write the equation as $\partial u/\partial(dt) = \partial^2 u/\partial x^2$. The fundamental solution has $e^{-x^2/4dt}$ and its Fourier transform has $e^{-dk^2 t}$.

Example 2 Suppose the initial temperature is a step function $u(x,0) = 0$. Then for negative x and $u(x,0) = 1$ for positive x. The discontinuity is smoothed out immediately, as heat flows to the left. The integral in formula (10) is zero up to the jump:

$$u(x,t) = \frac{1}{\sqrt{4\pi t}} \int_0^\infty e^{-(x-s)^2/4t}\, ds\,. \tag{11}$$

No luck with this integral! We can find the area under a complete bell-shaped curve (or half the curve) but there is no elementary formula for the area under a piece of the curve. No elementary function has the derivative e^{-x^2}. That is unfortunate, since those integrals give *cumulative probabilities* and statisticians need them all the time. So they have been normalized into the **error function** and tabulated to high accuracy:

Error function
$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-s^2}\, ds\,. \tag{12}$$

The integral from $-x$ to 0 is also $\operatorname{erf}(x)$. The normalization by $2/\sqrt{\pi}$ gives $\operatorname{erf}(\infty) = 1$.

We can produce this error function from the heat equation integral (11) by setting $S = (s-x)/\sqrt{4t}$. Then $s = 0$ changes to $S = -x/\sqrt{4t}$ as the lower limit on the integral, and $dS = ds/\sqrt{4t}$. Split into an integral from 0 to ∞, and from $-x/\sqrt{4t}$ to 0:

$$u(x,t) = \frac{\sqrt{4t}}{\sqrt{4\pi t}} \int_{-x/\sqrt{4t}}^\infty e^{-S^2}\, dS = \frac{1}{2}\left(1 + \operatorname{erf}\left(\frac{x}{\sqrt{4t}}\right)\right). \tag{13}$$

Good idea to check that this gives $u = \frac{1}{2}$ at $x = 0$ (where the error function is zero). This is the only temperature we know exactly, by symmetry between left and right.

Explicit Finite Differences

The simplest finite differences are *forward* for $\partial u/\partial t$ and *centered* for $\partial^2 u/\partial x^2$:

Explicit method
$$\frac{\boldsymbol{\Delta_t U}}{\boldsymbol{\Delta t}} = \frac{\boldsymbol{\Delta_x^2 U}}{\boldsymbol{(\Delta x)^2}} \qquad \frac{U_{j,n+1} - U_{j,n}}{\Delta t} = \frac{U_{j+1,n} - 2U_{j,n} + U_{j-1,n}}{(\Delta x)^2}. \tag{14}$$

Each new value $U_{j,n+1}$ is given explicitly by $U_{j,n} + R(U_{j+1,n} - 2U_{j,n} + U_{j,n-1})$. The crucial ratio for the heat equation $u_t = u_{xx}$ is now $\boldsymbol{R = \Delta t/(\Delta x)^2}$.

We substitute $U_{j,n} = G^n\, e^{ikj\Delta x}$ to find the growth factor $G = G(k, \Delta t, \Delta x)$:

One-step growth factor
$$G = 1 + R(e^{ik\Delta x} - 2 + e^{-ik\Delta x}) = \mathbf{1 + 2R(\cos k\Delta x - 1)}\,. \quad (15)$$

G is real, just as the exact one-step factor $e^{-k^2\Delta t}$ is real. Stability requires $|G| \le 1$. Again the most dangerous case is when the cosine equals -1 at $k\Delta x = \pi$:

Stability condition
$$|G| = |1 - 4R| \le 1 \text{ which requires } \boldsymbol{R = \frac{\Delta t}{(\Delta x)^2}} \le \frac{1}{2}\,. \quad (16)$$

In many cases we accept that small time step Δt and use this simple method. The accuracy from forward Δ_t and centered Δ_x^2 is $|U - u| = O(\Delta t + (\Delta x)^2)$. Those two error terms are comparable when R is fixed.

We could improve this one-step method to a **multistep method**. The "**method of lines**" calls an ODE solver for the system of differential equations (continuous in time, discrete in space). There is one equation for every meshpoint $x = jh$:

Method of Lines
$$\frac{\boldsymbol{dU}}{\boldsymbol{dt}} = \frac{\boldsymbol{\Delta_x^2 U}}{\boldsymbol{(\Delta x)^2}} \qquad \frac{dU_j}{dt} = \frac{U_{j+1} - 2U_j + U_{j-1}}{(\Delta x)^2}\,. \quad (17)$$

This is a **stiff system**, because its matrix $-K$ (second difference matrix) has a large condition number: $\lambda_{\max}(K)/\lambda_{\min}(K) \approx N^2$. We could choose a stiff solver like `ode15s` in MATLAB.

Implicit Finite Differences

A fully implicit method for $u_t = u_{xx}$ computes $\Delta_x^2 U$ at the new time $(n+1)\Delta t$:

Implicit
$$\frac{\boldsymbol{\Delta_t U_n}}{\boldsymbol{\Delta t}} = \frac{\boldsymbol{\Delta_x^2 U_{n+1}}}{\boldsymbol{(\Delta x)^2}} \qquad \frac{U_{j,n+1} - U_{j,n}}{\Delta t} = \frac{U_{j+1,n+1} - 2U_{j,n+1} + U_{j-1,n+1}}{(\Delta x)^2}\,. \quad (18)$$

The accuracy is still first-order in time and second-order in space. But stability no longer depends on the ratio $R = \Delta t/(\Delta x)^2$. We have *unconditional* stability, with a growth factor $0 < G \le 1$ for all k. Substituting $U_{j,n} = G^n e^{ijk\Delta x}$ into (18) and then canceling those terms from both sides leaves an extra G on the right side:

$$G = 1 + RG(e^{ik\Delta x} - 2 + e^{-ik\Delta x}) \quad \text{leads to} \quad G = \frac{1}{1 + 2R(1 - \cos k\Delta x)}\,. \quad (19)$$

The denominator is at least 1, which ensures that $0 < G \le 1$. The time step is controlled by accuracy, because stability is no longer a problem.

©2006 Gilbert Strang

There is a simple way to improve to second-order accuracy. *Center everything at step* $n + \frac{1}{2}$. Average an explicit $\Delta_x^2 U_n$ with an implicit $\Delta_x^2 U_{n+1}$. This produces the famous ***Crank-Nicolson method*** (like the trapezoidal rule):

$$\textbf{Crank-Nicolson} \qquad \frac{U_{j,n+1} - U_{j,n}}{\Delta t} = \frac{1}{2(\Delta x)^2}(\Delta_x^2 U_{j,n} + \Delta_x^2 U_{j,n+1})\,. \tag{20}$$

Now the growth factor G, by substituting $U_{j,n} = G^n e^{ijk\Delta x}$ into (20), solves

$$\frac{G-1}{\Delta t} = \frac{G+1}{2(\Delta x)^2}\,(2\cos k\Delta x - 2)\,. \tag{21}$$

Separate out the part involving G, write R for $\Delta t/(\Delta x)^2$, and cancel the 2's:

$$\textbf{Unconditional stability} \qquad G = \frac{1 + R(\cos k\Delta x - 1)}{1 - R(\cos k\Delta x - 1)} \quad \text{has} \quad \boldsymbol{|G| \le 1}\,. \tag{22}$$

The numerator is smaller than the denominator, since $\cos k\Delta x \le 1$. We do notice that $\cos k\Delta x = 1$ whenever $k\Delta x$ is a multiple of 2π. Then $G = 1$ at those frequencies, so Crank-Nicolson does not give the strict decay of the fully implicit method. We could weight the implicit $\Delta_x^2 U_{n+1}$ by $a > \frac{1}{2}$ and the explicit $\Delta_x^2 U_n$ by $1 - a < \frac{1}{2}$, to give a whole range of unconditionally stable methods (Problem ____).

Numerical example

Finite Intervals with Boundary Conditions

We introduced the heat equation on the whole line $-\infty < x < \infty$. But a physical problem will be on a **finite interval** like $0 \le x \le 1$. We are back to Fourier series (not Fourier integrals) for the solution $u(x,t)$. And second differences bring back the great matrices K, T, B, C that depend on the boundary conditions:

Absorbing boundary at $x = 0$: The temperature is held at $\boldsymbol{u(0,t) = 0}$.

Insulated boundary: No heat flows through the left boundary if $\boldsymbol{u_x(0,t) = 0}$.

If both boundaries are held at zero temperature, the solution will approach $u(x,t) = 0$ everywhere as t increases. If both boundaries are insulated as in a freezer, the solution will approach $u(x,t) = \textit{constant}$. No heat can escape, and it is evenly distributed as $t \to \infty$. This case still has the conservation law $\int_0^1 u(x,t)\,dx = \text{constant}$.

Example 3 (Fourier series solution) We know that e^{ikx} is multiplied by $e^{-k^2 t}$ to give a solution of the heat equation. Then $u = e^{-k^2 t}\sin kx$ is another solution (combining $+k$ with $-k$). With zero boundary conditions $u(0,t) = u(1,t) = 0$, the only allowed

frequencies are $k = n\pi$ (then $\sin n\pi x = 0$ at both ends $x = 0$ and $x = 1$). The complete solution is a combination of these exponential solutions with $k = n\pi$:

Complete solution
$$u(x,t) = \sum_{n=1}^{\infty} b_n\, e^{-n^2\pi^2 t} \sin n\pi x\,. \tag{23}$$

The Fourier sine coefficients b_n are chosen to match $u(x,0) = \sum b_n \sin n\pi x$ at $t = 0$.

You can expect cosines to appear for insulated boundaries, where the slope (not the temperature) is zero. This gives exact solutions to compare with finite difference solutions. For finite differences, *absorbing boundary conditions produce the matrix* K (not B or C). The choice between explicit and implicit decides whether we have second differences $-KU$ at time level n or level $n+1$:

Explicit method
$$U_{n+1} - U_n = -RKU_n \tag{24}$$

Fully implicit
$$U_{n+1} - U_n = -RKU_{n+1} \tag{25}$$

Crank-Nicolson
$$U_{n+1} - U_n = -\tfrac{1}{2}RK(U_n + U_{n+1})\,. \tag{26}$$

The explicit stability condition is again $R \le \frac{1}{2}$ (Problem ____). Both implicit methods are unconditionally stable (in theory). The reality test is to try them in practice.

An insulated boundary at $x = 0$ changes K to T. Two insulated boundaries produce B. Periodic conditions will produce C. The fact that B and C are singular no longer stops the computations. In the fully implicit method $(I + RB)U_{n+1} = U_n$, the extra identity matrix makes $I + RB$ invertible.

The **two-dimensional heat equation** describes the temperature distribution in a plate. For a square plate with absorbing boundary conditions, the difference matrix K changes to $\boldsymbol{K2D}$. The bandwidth jumps from 1 (triangular matrix) to N (when meshpoints are ordered a row at a time). Each time step of the implicit method now requires a serious computation. So implicit methods pay an increased price for stability, to avoid the explicit restriction $\Delta t \le \frac{1}{4}(\Delta x)^2 + \frac{1}{4}(\Delta y)^2$.

Convection-Diffusion

Put a chemical into flowing water. It diffuses while it is carried along by the flow. A diffusion term $d\,u_{xx}$ appears together with a convection term $c\,u_x$. This is the simplest model for one of the most important differential equations in engineering:

Convection-diffusion equation
$$\frac{\partial u}{\partial t} = c\,\frac{\partial u}{\partial x} + d\,\frac{\partial^2 u}{\partial x^2}\,. \tag{27}$$

On the whole line $-\infty < x < \infty$, the flow and the diffusion don't interact. If the velocity is c, convection just carries along the diffusing solution to $h_t = d\,h_{xx}$:

Diffusing traveling wave
$$\boldsymbol{u(x,t) = h(x + ct, t)\,.} \tag{28}$$

©2006 Gilbert Strang

Substituting into equation (27) confirms that this is the solution (correct at $t = 0$):

Chain rule
$$\frac{\partial u}{\partial t} = c\frac{\partial h}{\partial x} + \frac{\partial h}{\partial t} = c\frac{\partial h}{\partial x} + d\frac{\partial^2 h}{\partial x^2} = c\frac{\partial u}{\partial x} + d\frac{\partial^2 u}{\partial x^2}. \tag{29}$$

Exponentials also show this separation of convection e^{ikct} from diffusion e^{-dk^2t}:

Starting from e^{ikx}
$$\boldsymbol{u(x,t) = e^{-dk^2t}\, e^{ik(x+ct)}}. \tag{30}$$

Convection-diffusion is a terrific model problem, and the constants c and d clearly have different units. We take this small step into *dimensional analysis*:

Convection coefficient c: $\dfrac{\textbf{distance}}{\textbf{time}}$ Diffusion coefficient d: $\dfrac{\textbf{(distance)}^{\mathbf{2}}}{\textbf{time}}$ (31)

Suppose L is a typical length scale in the problem. ***The Peclet number $Pe = cL/d$*** is dimensionless. It measures the relative importance of convection and diffusion. This Peclet number for the linear equation (27) corresponds to the *Reynolds number* for the nonlinear Navier-Stokes equations (Section).

In the difference equation, the ratios $r = c\Delta t/\Delta x$ and $2R = 2d\Delta t/(\Delta x)^2$ are also dimensionless. That is why the stability conditions $r \le 1$ and $2R \le 1$ were natural for the wave and heat equations. The new problem combines convection and diffusion, and the ***cell Peclet number P*** uses $\Delta x/2$ as the length scale in place of L:

Cell Peclet Number
$$P = \frac{r}{2R} = \frac{c\,\Delta x}{2d}. \tag{32}$$

We still don't have agreement on the best finite difference approximation! Here are three natural candidates (you may have an opinion after you try them):

1. **Forward in time, centered convection, centered diffusion**
2. **Forward in time, upwind convection, centered diffusion**
3. **Explicit convection** (*centered or upwind*)**, with implicit diffusion.**

Each method will show the effects of r and R and P (we can replace $r/2$ by RP):

1. Centered explicit
$$\frac{U_{j,n+1} - U_{j,n}}{\Delta t} = c\frac{U_{j+1,n} - U_{j-1,n}}{2\Delta x} + d\frac{\Delta_x^2 U_{j,n}}{(\Delta x)^2}. \tag{33}$$

Every new value $U_{j,n+1}$ is a combination of three known values at time n:

$$U_{j,n+1} = (\mathbf{1 - 2R})U_{j,n} + (\boldsymbol{R + RP})U_{j+1,n} + (\boldsymbol{R - RP})U_{j-1,n}. \tag{34}$$

Those three coefficients add to 1, and $U =$ constant certainly solves equation (33). ***If all three coefficients are positive, the method is surely stable.*** More than that, *oscillations cannot appear*. Positivity of the middle coefficient requires $R \le \frac{1}{2}$,

as usual for diffusion. Positivity of the other coefficients requires $|\boldsymbol{P}| \le \mathbf{1}$. Of course P will be small when Δx is small (so we have convergence as $\Delta x \to 0$). In avoiding oscillations, the actual cell size Δx is crucial to the quality of U.

Figure 5.12 was created by Strikwerda [59] and Persson to show the oscillations for $P > 1$ and the smooth approximations for $P < 1$. Notice how the initial hat function is smoothed and spread and shrunk by diffusion. Problem ____ finds the exact solution, which is moved along by convection. Strictly speaking, even the oscillations might pass the stability test $|G| \le 1$ (Problem ____). But they are unacceptable.

Figure 5.12: Convection-diffusion with and without numerical oscillations: $R =$ ____, $r =$ ____ and ____.

2. Upwind convection
$$\frac{U_{j,n+1} - U_{j,n}}{\Delta t} = c\,\frac{U_{j+1,n} - U_{j,n}}{\Delta x} + d\,\frac{\Delta_x^2 U_{j,n}}{(\Delta x)^2}. \tag{35}$$

The accuracy in space has dropped to first order. But the oscillations are eliminated whenever $r + 2R \le 1$. That condition ensures three positive coefficients when (35) is solved for the new value $U_{j,n+1}$:

$$U_{j,n+1} = (\boldsymbol{RP} + \boldsymbol{R})U_{j+1,n} + (\mathbf{1} - \boldsymbol{RP} - \mathbf{2}\boldsymbol{R})U_{j,n} + \boldsymbol{R}U_{j-1,n}\,. \tag{36}$$

Arguments are still going, comparing the centered method 1 and the upwind method 2. The difference between the two convection terms, **upwind minus centered**, is actually a diffusion term hidden in (35)!

Extra diffusion
$$\frac{U_{j+1} - U_j}{\Delta x} - \frac{U_{j+1} - U_{j-1}}{2\Delta x} = \left(\frac{\Delta x}{2}\right)\frac{U_{j+1} - 2U_j + U_{j-1}}{(\Delta x)^2}\,. \tag{37}$$

So the upwind method has this extra numerical diffusion or "***artificial viscosity***" to kill oscillations. It is a non-physical damping. If the upwind approximation were included in Figure 5.12, it would be *distinctly below* the exact solution. Nobody is perfect.

3. Implicit diffusion
$$\frac{U_{j,n+1} - U_{j,n}}{\Delta t} = c\,\frac{U_{j+1,n} - U_{j,n}}{\Delta x} + d\,\frac{\Delta_x^2 U_{j,\boldsymbol{n+1}}}{(\Delta x)^2}\,. \tag{38}$$

MORE TO DO

Problem Set 5.4

1 Solve the heat equation starting from a combination $u(x,0) = \delta(x+1) - 2\delta(x) + \delta(x-1)$ of three delta functions. What is the total heat $\int u(x,t)\,dx$ at time t? Draw a graph of $u(x,1)$ by hand or by MATLAB.

2 Integrating the answer to Problem 1 gives another solution to the heat equation:

$$\text{Show that} \quad w(x,t) = \int_0^x u(X,t)\,dX \quad \text{solves} \quad w_t = w_{xx}\,.$$

Graph the initial function $w(x,0)$ and sketch the solution $w(x,1)$.

©2006 Gilbert Strang

3 Integrating once more solves the heat equation $h_t = h_{xx}$ starting from $h(x,0) = \int w(X,0)\,dX = \textit{hat function}$. Draw the graph of $h(x,0)$. Figure 5.12 shows the graph of $h(x,t)$, shifted along by convection to $h(x+ct,t)$.

4 In convection-diffusion, compare the condition $R \le \frac{1}{2}, P \le 1$ (for positive coefficients in the centered method) with $r + 2R \le 1$ (for the upwind method). For which c and d is the upwind condition less restrictive, in avoiding oscillations?

5 The eigenvalues of the n by n second difference matrix K are $\lambda_k = 2 - 2\cos\frac{k\pi}{n+1}$. The eigenvectors y_k in Section 1.5 are discrete samples of $\sin k\pi x$. Write the general solutions to the fully explicit and fully implicit equations (14) and (18) after N steps, as combinations of those discrete sines y_k times powers of λ_k.

6 Another exact integral involving $e^{-x^2/4t}$ is

$$\int_0^\infty x\,e^{-x^2/4t}\,dx = \Big[-2t\,e^{-x^2/4t}\Big]_0^\infty = 2t\,.$$

From (17), show that the temperature is $u = \sqrt{t}$ at the center point $x = 0$ starting from a ramp $u(x,0) = \max(0,x)$.

7 A ramp is the integral of a step function. So the solution of $u_t = u_{xx}$ starting from a ramp (Problem 6) is the integral of the solution starting from a step function (Example 2 in the text). Then $\sqrt{t}$ must be the total amount of heat that has crossed from $x > 0$ to $x < 0$ in Example 2 by time t. Explain each of those three sentences.

 ©2006 Gilbert Strang

5.5 Difference Matrices and Eigenvalues

This brief section collects together useful notes on finite difference matrices, and also finite element matrices. Certainly those special matrices K, T, B, C from the start of the book are the building blocks for approximations to u_{xx} and u_{yy}. Second derivatives and fourth derivatives lead to symmetric matrices. *First derivatives are antisymmetric.* They present more difficulties.

A symmetric matrix has orthogonal eigenvectors. For those special matrices, the eigenvectors are discrete samples of sines and cosines and e^{ikx}. The eigenvalues are real, and they often involve $e^{ik\Delta x} - 2 + e^{-ik\Delta x}$. That is the discrete factor $2\cos k\Delta x - 2$. Divided by $(\Delta x)^2$, it is close for small k to the factor $-k^2$ from the second derivative of e^{ikx}. *The von Neumann approach using e^{ikx} matches the eigenvectors of these matrices*, and the growth factors G match the eigenvalues.

For a one-sided (upwind) difference, the matrix eigenvalues are not always reliable. For a centered difference they do follow von Neumann. Compare

$$\boldsymbol{\Delta^+} = \begin{bmatrix} -1 & 1 & & \\ & -1 & 1 & \\ & & -1 & 1 \\ & & & -1 \end{bmatrix} \qquad \boldsymbol{\Delta^0} = \frac{1}{2}\begin{bmatrix} 0 & 1 & & \\ -1 & 0 & -1 & \\ & -1 & 0 & -1 \\ & & -1 & 0 \end{bmatrix}$$

The eigenvalues of the triangular upwind matrix Δ^+ are all -1 (*useless*). The eigenvalues of the antisymmetric Δ^0 are guaranteed to be imaginary like the factor ik from the derivative of e^{ikx}. The eigenvalues $\lambda = -1$ for Δ^+ do not make upwind differences useless. They only mean that the von Neumann test, which produces $e^{ik\Delta x} - 1$, is better than relying on eigenvalues.

As it stands, Δ^+ is exactly in "*Jordan form.*" The matrix has only one line of eigenvectors, not n. It is an extreme example of a nondiagonalizable (and somehow degenerate) matrix. If the diagonals of -1's and 1's are extended to infinity, then Fourier and von Neumann produce vectors with components $e^{ikj\Delta x}$ and with eigenvalues $e^{ik\Delta x} - 1$. In summary: For normal matrices, eigenvalues are a reliable guide. For other constant-diagonal matrices, better to rely on von Neumann.

Briefly, the discrete growth factors G are exactly the eigenvalues when the matrices are called "*normal*" and the test is $AA^{\mathrm{T}} = A^{\mathrm{T}}A$ (for complex matrices take the conjugate transpose A^*). This test is passed by all symmetric and antisymmetric and orthogonal matrices.

Options for First Differences

Upwind elements
Streamline diffusion
DG
Boundary conditions
Convection-diffusion

©2006 Gilbert Strang

5.6 Nonlinear Flow and Conservation Laws

Nature is nonlinear. The coefficients in the equation *depend on the solution* u. In place of $u_t = c\,u_x$ we will study $u_t + uu_x = 0$ and more generally $u_t + f(u)_x = 0$. These are "conservation laws" and the conserved quantity is the integral of u.

The first part of this book emphasized the **balance equation**: forces balance and currents balance. For steady flow this was Kirchhoff's Current Law: *flow in equals flow out*. The net flow was zero. Now the flow is *unsteady*—the "mass inside" is changing. So a new $\partial/\partial t$ term will enter the conservation law.

There is "flux" through the boundaries. In words, **the rate of change of mass inside a region equals that incoming flux**. For an interval $[a, b]$, the incoming flux is the difference in fluxes at the endpoints a and b:

Integral form
$$\frac{d}{dt}\int_a^b u(x,t)\,dx = \boldsymbol{f(u(a,t)) - f(u(b,t))}\,. \tag{1}$$

In applications, u can be a *density* (of cars along a highway). The integral of u gives the mass (number of cars) between a and b. This number changes with time, as cars flow in at point a and out at point b. The flux is **density $\boldsymbol{u}$ times velocity $\boldsymbol{v}$**.

The integral form is fundamental. We can get a differential form by allowing b to approach a. Suppose $b - a = \Delta x$. If $u(x,t)$ is a smooth function, its integral over a distance Δx will have leading term $\Delta x\, u(a,t)$. So if we divide equation (1) by Δx, the limit as Δx approaches zero is $\partial u/\partial t = -\partial f(u)/\partial x$:

Differential form
$$\frac{\partial u}{\partial t} + \frac{\partial}{\partial x} f(u) = \frac{\partial u}{\partial t} + f'(u)\,\frac{\partial u}{\partial x} = 0 \tag{2}$$

When $f(u)$ = density u times velocity $v(u)$, we can solve this single conservation law. For traffic flow, the velocity $v(u)$ can be measured (it will decrease as density increases). In gas dynamics there are also conservation laws for momentum and energy. The velocity v becomes another unknown, along with the pressure p. The **Euler equations** for gas dynamics in one space dimension include two additional equations:

Conservation of momentum
$$\frac{\partial}{\partial t}(uv) + \frac{\partial}{\partial x}(uv^2 + p) = 0 \tag{3}$$

Conservation of energy
$$\frac{\partial}{\partial t}(E) + \frac{\partial}{\partial x}(Ev + Ep) = 0\,. \tag{4}$$

Systems of conservation laws are more complicated, but our scalar equation (2) already has the possibility of **shocks**. A shock is a discontinuity in the solution $u(x,t)$, where the differential form breaks down and we need the integral form (1).

The other outstanding example, together with traffic flow, is **Burger's equation**, for u = velocity. **The flux $f(u)$ is $\frac{1}{2}u^2$**. The "inviscid" form has no u_{xx}:

Burger's equation
$$\frac{\partial u}{\partial t} + \frac{\partial}{\partial x}\left(\frac{u^2}{2}\right) = \frac{\partial u}{\partial t} + u\,\frac{\partial u}{\partial x} = 0\,.$$

When both the density and velocity are unknowns, these examples combine into *conservation of mass and conservation of momentum*. Typically we change density to ρ. For small disturbances of a uniform density ρ_0, we could linearize the conservation laws and reach the wave equation (Problem). But the Euler and Navier-Stokes equations are truly nonlinear, and we begin the task of solving them.

We will approach conservation laws (and these examples) in three ways:

1. By following characteristics until trouble arrives: they separate or collide

2. By a special formula ( )

3. By finite difference and finite volume methods, which are the practical choice.

Characteristics

The one-way wave equation $u_t = c\,u_x$ is solved by $u(x,t) = u(x+ct,0)$. Every initial value u_0 is carried along a characteristic line $x + ct = x_0$. Those lines are parallel when the velocity c is a constant.

The conservation law $u_t = +u\,u_x = 0$ will be solved by $u(x,t) = u(x-ut,0)$. Every initial value $u_0 = u(x_0,0)$ is carried along a characteristic line $\boldsymbol{x - u_0 t = x_0}$. Those lines are *not* parallel because their slopes depend on the initial value u_0.

Notice that the formula $u(x,t) = u(x-ut,0)$ involves u on both sides. It gives the solution "implicitly." If the initial function is $u(x,0) = 1 - x$, for example, the formula must be solved for u:

$$u = 1 - (x - ut) \quad \text{gives} \quad (1-t)u = 1 - x \quad \text{and} \quad u = \frac{1-x}{1-t}\,. \tag{5}$$

This does solve Burger's equation, since the time derivative $u_t = (1-x)/(1-t)^2$ is equal to $-uu_x$. The characteristic lines (with different slopes) can meet. This is an extreme example, where all characteristics meet at the same point:

$$x - u_0 t = x_0 \quad \text{or} \quad x - (1-x_0)t = x_0 \quad \text{which goes through } x = 1, t = 1 \tag{6}$$

You see how the solution $u = (1-x)/(1-t)$ becomes $0/0$ at that point $x = 1, t = 1$. Beyond their meeting point, the characteristics cannot completely decide $u(x,t)$.

A more fundamental example is the **Riemann problem**, which starts from two constant values $u = A$ and $u = B$. Everything depends on whether $A > B$ or $A < B$. On the left side of Figure 5.13, with $A > B$, *the characteristics meet*. On the right

©2006 Gilbert Strang

side, with $A < B$, *the characteristics separate*. Both cases present a new (nonlinear) problem, when we don't have a single characteristic that is safely carrying the correct initial value to the point. This Riemann problem has *two* characteristics through the point, or *none*:

Shock	Characteristics *collide* (light goes red: speed drops from 60 to 0)
Fan	Characteristics *separate* (light goes green: speed up from 0 to 60)

The problem is how to connect $u = 60$ to $u = 0$, when the characteristics don't give the answer. A shock will be sharp breaking (drivers only see the car ahead in this model). A fan will be gradual acceleration.

TO DO...

Figure 5.13: A shock when characteristics collide, a fan when they separate.

For the conservation law $u_t + f(u)_x = 0$, the characteristics are $x - f'(u_0)t = x_0$. That line has the right slope to carry the constant value $u = u_0$:

$$\frac{d}{dt}\,u(x_0 + St, t) = S\,\frac{\partial u}{\partial x} + \frac{\partial u}{\partial t} = 0 \quad \text{when} \quad S = f'(u)\,. \tag{7}$$

The solution until trouble arrives is $u(x,t) = u(x - f'(u)t, 0)$.

Shocks

After trouble arrives, it will be the *integral form* that guides the choice of the correct solution u. If there is a jump in u (*a shock*), that integral from tells where the jump must occur. Suppose u has different values u_L and u_R at points x_L and x_R on the left and right sides of the shock:

Integral form
$$\frac{d}{dt}\int_{x_L}^{x_R} u\,dx + f(u_R) - f(u_L) = 0\,. \tag{8}$$

If the position of the shock is $x = X(t)$, we take x_L and x_R very close to X. The values of $u(x,t)$ inside the integral are close to the constants u_L and u_R:

$$\frac{d}{dt}\big[(x - x_L)\,u_L + (x_R - X)\,u_R\big] + f(u_R) - f(u_L) \approx 0\,.$$

This gives the speed $s = dX/dt$ of the shock curve:

Jump condition
$$s\,u_L - s\,u_R + f(u_R) - f(u_L) = 0$$

shock speed
$$= \frac{\boldsymbol{f(u_R) - f(u_L)}}{\boldsymbol{u_R - u_L}} = \frac{[\,\boldsymbol{f}\,]}{[\,\boldsymbol{u}\,]}\,. \tag{9}$$

For the Riemann problem, the left and right values u_L and u_R will be constants A and B. The shock speed s is the ratio between the jump $[\,f\,] = f(B) - f(A)$ and the jump $[\,u\,] = B - A$. Since this ration gives a constant slope, the shock line is straight. For other problems, the characteristics are carrying different values of u into the shock. So the shock speed s is not constant and the shock lines is curved.

The shock gives the solution when characteristics collide. With $f(u) = \frac{1}{2}u^2$ in Burger's equation, the shock speed is halfway between u_L and u_R:

Burger's equation Shock speed $s = \dfrac{1}{2}\dfrac{u_R^2 - u_L^2}{u_R - u_L} = \dfrac{1}{2}(u_R + u_L)\,.$ (10)

The Riemann problem has $u_L = A$ and $u_R = B$, and s is their average. Figure 5.14 shows how the integral form of Burger's equation is solved by the right placement of the shock.

Fans

You might expect a similar picture (just flipped) when $A < B$. *Wrong.* The integral form is still satisfied, but it is also satisfied by a **fan**. The choice between shock and fan is made by the "**entropy condition**" that as t increases, *characteristics must go into the shock.* The wave speed is faster than the shock speed on the left, and slower on the right:

Entropy condition $f'(u) > s > f'(u_R)$ (11)

Since Burger's equation has $f'(u) = u$, it only has shocks when u_L is larger than u_R. In the Riemann problem that means $A > B$. In the opposite case, the smaller value $u_L = A$ has to be connected to $u_R = B$ by the fan in Figure 5.14:

Fan (or rarefaction) $u = \dfrac{x}{t}$ for $At < x < Bt\,.$ (12)

use fig 6.28 p. 592 of IAM (reverse left and right figs)

Figure 5.14: Characteristics collide in a shock and separate in a fan.

Notice especially that in the traffic flow problem, the velocity $v(u)$ *decreases* as the density u increases. A good model is linear between $v = v_{\mathsf{max}}$ at zero density and $v = 0$ at maximum density. Then the flux $f(u) = u\,v(u)$ is a downward parabola (concave instead of Burger's convex $u^2/2$):

Traffic speed and flux $v(u) = v_{\mathsf{max}}\left(1 - \dfrac{u}{u_{\mathsf{max}}}\right)$ and $f(u) = v_{\mathsf{max}}\left(u - \dfrac{u^2}{u_{\mathsf{max}}}\right).$ (13)

Typical values for a single lane of traffic show a maximum flux of $f = 1600$ vehicles per hour, when the density is $u = 80$ vehicles per mile. This maximum flow rate

©2006 Gilbert Strang

is attained when the velocity f/u is $v = 20$ miles per hour! Small comfort at that speed, to know that other cars are getting somewhere too.

Problems ____ and ____ compute the solution when a light goes red (shock travels backward) and when a light goes green (fan moves forward). Please look at the figures, to see how the vehicle trajectories are entirely different form the characteristics.

A driver keeps adjusting the density to stay safely behind the car in front. (Hitting the car would give $u < 0$.) We all recognize the frustration of braking and accelerating from a series of shocks and fans. This traffic crawl happens when the green light is too short for the shock to make it through.

A Solution Formula for Burger's Equation

Let me comment on three nonlinear equations. They are useful models, quite special because each one has an exact solution formula:

Conservation law	$u_t + u\,u_x = 0$
Burger's with viscosity	$u_t + u\,u_x = \nu\,u_{xx}$
Korteweg-de Vries	$u_t + u\,u_x = -a\,u_{xxx}$

The conservation law can develop shocks. This won't happen in the second equation because the u_{xx} viscosity term prevents it. That term can stay small when the solution is smooth, but it dominates when a wave is about to break. The profile is steep but it stays smooth.

As starting function for the conservation law, I will pick a point source: $u(x,0) = \delta(x)$. We can guess a solution with a shock, and check the jump condition and entropy condition. Then we find an exact formula when $\nu\,u_{xx}$ is included, by a neat change of variables that produces $h_t = \nu\,h_{xx}$. When we let $\nu \to 0$, the limiting formula solves the conservation law—and we can check that the following solution is correct.

Solution with $u(x,0) = \delta(x)$ When $u(x,0)$ jumps upward, we expect a fan. When it drops we expect a shock. The delta function is an extreme case (very big jumps up and down, very close together!). So we look for a shock curve $x = X(t)$ immediately in front of a fan!

Expected solution $$u(x,t) = \frac{x}{t} \quad \text{for} \quad 0 \le x \le X(t); \quad \text{otherwise } u = 0. \tag{14}$$

The total mass at the start is $\int \delta(x)\,dx = 1$. This never changes, and already that locates the shock position $X(t)$:

$$\text{Mass at time } t = \int_0^X \frac{x}{t}\,dt = \frac{X^2}{2t} = 1 \qquad \text{so } X(t) = \sqrt{2t}\,. \tag{15}$$

Does the drop in u, from $X/t = \sqrt{2t}/t$ to zero, satisfy the jump condition?

$$\text{Shock speed } s = \frac{dX}{dt} = \frac{\sqrt{2}}{2\sqrt{t}} \quad \text{equals} \quad \frac{\text{Jump } [\,u^2/2\,]}{\text{Jump } [\,u\,]} = \frac{X^2/2t^2}{X/t} = \frac{\sqrt{2t}}{2t}\,.$$

The entropy condition $u_L > s > u_R = 0$ is also satisfied, and the solution ( ) looks good. It *is* good, but because of the delta function we check it another way.

Begin with $u_t + u\,u_x = \nu\,u_{xx}$, and solve that equation exactly. If $u(x)$ is $\partial U/\partial x$, then integrating our equation gives $U_t + \frac{1}{2}U_x^2 = \nu\,U_{xx}$. The initial value $U_0(x)$ is now a step function. Then the great change of variables $U = -2\nu \log h$ produces the heat equation $h_t = \nu\,h_{xx}$ (Problem ____). The initial value becomes $h(x,0) = e^{-U_0(x)/2\nu}$. Section 5.4 found the solution to the heat equation $u_t = u_{xx}$ from any starting function $h(x,0)$ and we just change t to νt:

$$U(x,t) = -2\nu \log h(x,t) = -2\nu \log\left[\frac{1}{\sqrt{4\pi\nu t}}\int_{-\infty}^{\infty} e^{-U_0(y)/2\nu} e^{-(x-y)^2/4\nu t}\,dy\right]. \tag{16}$$

It doesn't look easy to let $\nu \to 0$, but it can be done. That exponential has the form $e^{-B(x,y)/2\nu}$. This is largest when B is smallest. An asymptotic method called "steepest descent" shows that as $\nu \to 0$, the bracketed quantity in (16) approaches $c\,e^{-B_{\min}/2\nu}$. Taking its logarithm and multiplying by -2ν, (16) becomes $U = B_{\min}$ in the limit:

$$\lim_{\nu\to 0} U(x,t) = B_{\min} = \min_y\left[U_0(y) + \frac{1}{2t}(x-y)^2\right]. \tag{17}$$

This is the solution formula for $U_t + \frac{1}{2}U_x^2 = 0$. Its derivative $u = U_x$ solves the conservation law $u_t + u\,u_x = 0$. By including the viscosity $\nu\,u_{xx}$ with $\nu \to 0$, we are finding the $u(x,t)$ that satisfies the jump condition and the entropy condition.

Example Starting from $u(x,0) = \delta(x)$, its integral U_0 is a step function. The minimum of B is either at $y = x$ or at $y = 0$. Check each case:

$$U(t,x) = B_{\min} = min_y\left[\begin{array}{ll} 0 & (y \le 0) \\ 1 & (y > 0) \end{array} + \frac{(x-y)^2}{2t}\right] = \begin{cases} 0 & \text{for } x \le 0 \\ x^2/2t & \text{for } 0 \le x \le \sqrt{2t} \\ 1 & \text{for } x \ge \sqrt{2t} \end{cases}$$

The result $u = dU/dx$ is $\mathbf{0}$ or $\boldsymbol{x/t}$ or $\mathbf{0}$. This agrees with our guess in equation ( )—a fan rising from 0 and a shock back to 0 at $x = \sqrt{2t}$.

©2006 Gilbert Strang

5.7 Level Sets and the Fast Marching Method

The level sets of $f(x,y)$ are the sets on which the function is constant. For example $f(x,y) = x^2 + y^2$ is constant on circles around the origin. Geometrically, a level plane $z =$ constant will cut through the surface $z = f(x,y)$ on a level set. One attractive feature of working with level sets is that their topology can change (pieces of the level set can separate or come together) just by changing the constant.

Starting from one level set, the **signed distance function** $d(x,y)$ is especially important. It gives the *distance* to the level set, and also the sign: typically $d > 0$ outside and $d < 0$ inside. For the unit circle, $d = r - 1 = \sqrt{x^2+y^2} - 1$ will be the signed distance function. In the mesh generation algorithm of Section 2.____, it was convenient to describe the region by its distance function $d(x,y)$.

A fundamental fact of calculus: The gradient of $f(x,y)$ is perpendicular to its level sets. Reason: In the tangent direction t to the level set, $f(x,y)$ is not changing and $(\operatorname{grad} f) \cdot t$ is zero. So $\operatorname{grad} f$ is in the normal direction. For the function x^2+y^2, the gradient $(2x, 2y)$ points outward from the circular level sets. The gradient of $d(x,y) = \sqrt{x^2+y^2} - 1$ points the same way, and it has a special property: **The gradient of a distance function is a unit vector**. It is the *unit* normal $n(x,y)$ to the level sets. For the circles,

$$\operatorname{grad}(\sqrt{x^2+y^2} - 1) = (\frac{x}{r}, \frac{y}{r}) \qquad \text{and} \qquad |\operatorname{grad}|^2 = \frac{x^2}{r^2} + \frac{y^2}{r^2} = 1\,. \tag{1}$$

You could think of the level set $d(x,y) = 0$ as a wall of fire. This firefront will move normal to itself. If it has constant velocity 1 then at time T the fire will reach all points on the level set $d(x,y) = T$.

That "wall of fire" example brings out an important point when the zero level set has a corner (it might be shaped like a **V**). The points at distance d outside that set (the firefront at time d) will lie on lines parallel to the sides of the **V**, and also on a circular arc of radius d around the corner. For $d < 0$ the **V** moves *inward*. It remains a **V** (with no smoothing of the corner).

The central problem of the level set method is to **propagate a curve** like the firefront. A velocity field $v = (v_1, v_2)$ gives the direction and speed of each point for the movement. At time $t = 0$, the curve is the level set where $d(x,y) = 0$. At later times the curve is the zero level set of a function $\phi(x,y,t)$. The fundamental **level set equation** in its first form is

$$\frac{d\phi}{dt} + v \cdot \operatorname{grad} \phi = 0, \quad \text{with } \phi = d(x,y) \text{ at } t = 0\,. \tag{2}$$

In our wall of fire example, v would be the unit vector in the normal direction to the firefront: $v = n = \operatorname{grad}\phi / |\operatorname{grad}\phi|$. In all cases it is only the normal component $F = v \cdot n$ that moves the curve! Tangential movement (like rotating a circle around its center) gives no change in the curve as a whole. By rewriting $v \cdot \operatorname{grad}\phi$, the level

set equation takes a second form that is more useful in computation:

$$v \cdot \operatorname{grad}\phi = v \cdot \frac{\operatorname{grad}\phi}{|\operatorname{grad}\phi|}|\operatorname{grad}\phi| = F|\operatorname{grad}\phi| \quad \text{leads to} \quad \frac{d\phi}{dt} + F|\operatorname{grad}\phi| = 0\,. \tag{3}$$

We only need to know the velocity field v (and only its normal component F) near the *current location* of the level curve–not everywhere else. We are propagating a curve. The velocity field may be fixed (easiest case) or it may depend on the local shape of the curve (nonlinear case). An important example is **motion by mean curvature**: $F = -\kappa$. The neat property $|\operatorname{grad}\phi| = 1$ of distance functions simplifies the formulas for the normal n and curvature κ:

When ϕ is a distance function			
	$n = \dfrac{\operatorname{grad}\phi}{\lvert\operatorname{grad}\phi\rvert}$	becomes	$n = \operatorname{grad}\phi$
	$\kappa = \operatorname{div} n$	becomes	$\kappa = \operatorname{div}(\operatorname{grad}\phi) =$ Laplacian of ϕ

(4)

But here is an unfortunate point for $t > 0$. Constant speed ($F = 1$) in the normal direction does maintain the property $|\operatorname{grad}\phi| = 1$ of a distance function. Motion by mean curvature, and other motions, will destroy this property. To recover the simple formulas (4) for distance functions, the level set method often **reinitializes** the problem—restarting from the current time t_0 and computing the distance function $d(x, y)$ to the current level set $\phi(x, y, t_0) = 0$. This reinitialization was the **Fast Marching Method**, which finds distances from nearby meshpoints to the current level set.

We describe this quick method to compute distances to meshpoints, and then discuss the numerical solution of the level set equation (3) on the mesh.

Fast Marching Method

The problem is to march outward, computing distances from meshpoints to the interface (the current level set where $\phi = 0$). Imagine that we know these distances for the grid points adjacent to the interface. (We describe fast marching but not the full algorithm of reinitialization.) The key step is to compute the distance to the *next nearest meshpoint*. Then the front moves further outward with velocity $F = 1$. When the front crosses a new meshpoint, it will become the next nearest and its distance will be settled next.

So we accept one meshpoint at a time. Distances to further meshpoints are tentative (not accepted). They have to be recomputed using the newly accepted meshpoint and its distance. The Fast Marching Method must quickly take these steps recursively:

1. Find the tentative meshpoint p with smallest distance (to be accepted).

2. Update the tentative distances to all meshpoints adjacent to p.

©2006 Gilbert Strang

To speed up step **1**, we maintain a binary tree of unaccepted meshpoints and their tentative distances. The smallest distance is at the top of the tree, which identifies p. When that value is removed from the tree, others move up to form the new tree.

Recursively, each vacancy is filled by the smaller of the two distance values below it. Then step **2** updates those values at points adjacent to p. These updated values may have to move (a little) up or down to reset the tree. In general, the updated values should be smaller (they mostly move up, since they have the latest meshpoint p as a new candidate in finding the shortest route to the original interface).

The Fast Marching Method finds distances to N meshpoints in time $O(N \log N)$. The method applies when the front moves *in one direction only*. The underlying equation is $F|\nabla T| = 1$ (Eikonal equation with $F > 0$). The front never crosses a point twice (and the crossing time is T). If the front is allowed to move in both directions, and F can change sign, we need the initial value formulation (3).

Lagrangian versus Eulerian

A fundamental choice in analyzing and computing fluid flow is between Lagrange and Euler. For the minimizing function in optimization, they arrived at the same "Euler-Lagrange equation". In studying fluids, they chose *very different* approaches:

Lagrange follows the path of each particle of fluid. He moves.

Euler sees which particles pass through each point. He sits.

Lagrange is more direct. He "*tracks*" the front. At time zero, points on the front have positions $x(0)$. They move according to vector differential equations $dx/dt = V(x)$. If we mark and follow a finite set of points, equally spaced at the start, serious difficulties can appear. Their spacing can get very tight or very wide (forcing us to remove or add marker points). The initial curve can split apart or cross itself (changes of topology). The level set method escapes from these difficulties by going Eulerian.

For Euler, the x-y coordinate system is fixed. He "*captures*" the front implicitly, as a level set of $\phi(x, y, t)$. When the computational grid is also fixed, we are constantly interpolating to locate level sets and compute distance functions. Squeezing or stretching or tangling of the front appear as changes in ϕ, not as disasters for the mesh.

The velocity v *on the interface* determines its movement. When the level set method needs v at a meshpoint off the interface, a good candidate is the value of v at the nearest point on the interface.

Upwind Differencing

The level set finite difference method is properly developed in the books by its originators: Sethian [ ] and Osher and Fedkiw [ ]. Here we concentrate on an essential point:

upwind differencing. *Recall* from Section 1.____ the three simplest approximations **F**, **B**, **C**to the first derivative $d\phi/dx$–*Forward*, *Backward*, *Centered*:

$$\mathbf{F}\quad \frac{\phi(x+h)-\phi(x)}{h} \qquad \mathbf{B}\quad \frac{\phi(x)-\phi(x-h)}{h} \qquad \mathbf{C}\quad \frac{\phi(x+h)-\phi(x-h)}{2h}$$

Which do we use in the simple convection equation $d\phi/dt + a\, d\phi/dx = 0$? Its true solution is $\phi(x-at,0)$. *The choice of finite differences depends on the sign of* a. The flow moves left to right for $a<0$. Then the *backward* difference is natural—the "upwind" value $\phi(x-h,t)$ on the left should contribute to $\phi(x,t+\Delta t)$. The downwind value $\phi(x+h,t)$ on the right moves further downwind during the time step, and has no influence at x.

When the movement of the solution (and the wind) is right to left (with $a>0$), then the *forward* difference will use the appropriate upwind value $\phi(x+h,t)$ along with $\phi(x,t)$, in computing the new $\phi(x,t+\Delta t)$.

Notice the time-step limitation $|a|\Delta t \le h$. In time Δt, the "wind" will bring the true value of ϕ from $x+a\Delta t$ to the point x. If $a>0$ and finite differences reach upwind to $x+h$, that must be far enough to include information at $x+a\Delta t$. So the *Courant-Friedrichs-Lewy condition* is $a\Delta t \le h$. The numerical waves must propagate at least as fast as the physical waves (and in the right direction!). Downwind differencing is looking for that information on the wrong side of the point x, and is doomed to failure. Centered differencing in space is unstable for ordinary forward Euler.

By careful choice of the right finite differences, Osher has constructed higher-order essentially non-oscillatory (ENO) schemes. A central idea in nonlinear problems, where the differential equation has multiple solutions (see Section ____), is to choose the "*viscosity solution*." This physically correct solution appears in the limit as an extra ϵu_{xx} diffusion term goes to zero. With good differencing the viscosity solution is the one that appears as $\Delta x \to 0$.

At this point, the level set method does not appear in large production codes. In research papers it has successfully solved a great variety of difficult nonlinear problems.

Chapter 6

Solving Large Systems

6.1 Elimination with Reordering

Finite elements and finite differences produce large linear systems $KU = F$. *The matrix K is extremely sparse.* It has only a small number of nonzero entries in a typical row. In "physical space" those nonzeros are tightly clustered—they come from neighboring nodes and meshpoints. But we cannot number N^2 nodes in a plane in any way that keeps all neighbors close together! So in 2-dimensional problems, and even more in 3-dimensional problems, we meet three questions right away:

1. How best to number the nodes

2. How to use the sparseness of K (when nonzeros might be widely separated)

3. Whether to choose **direct elimination** or an **iterative method**.

That last point separates this section on elimination (where **node order** is important) from later sections on iterative methods (where **preconditioning** is crucial).

To fix ideas, we will create the n equations $KU = F$ from Laplace's difference equation in an interval, a square, and a cube. With N unknowns in each direction, K has order $n = N$ or N^2 or N^3. There are 3 or 5 or 7 nonzeros in a typical row of the matrix. Second differences in 1D, 2D, and 3D are shown in Figure 6.1.

Along an inside row of the matrix, the entries add to zero. In two dimensions this is $4 - 1 - 1 - 1 - 1 = 0$. This "zero sum" remains true for finite elements (the element shapes decide the exact numbers). It reflects the fact that $u = 1$ solves Laplace's equation and $U = \texttt{ones}(n, 1)$ has differences equal to zero. The constant vector solves $KU = 0$ *except near the boundaries.* When a neighbor is a boundary point, its known value moves onto the right side of $KU = F$. Then that row of K is *not zero sum.* Otherwise K would be singular, if $K * \texttt{ones}(n, 1) = \texttt{zeros}(n, 1)$.

Using block matrix notation, we can create the 2D matrix $\boldsymbol{K2D}$ from the familiar N by N second difference matrix K. We number the nodes of the square a row at

©2006 Gilbert Strang

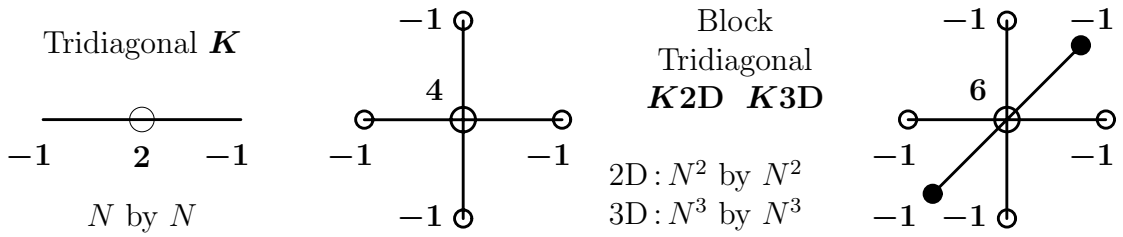


Figure 6.1: $3, 5, 7$-point difference molecules for $-u_{xx}, -u_{xx} - u_{yy}, -u_{xx} - u_{yy} - u_{zz}$.

a time (this "natural numbering" is not necessarily best). Then the -1's for the neighbors above and below are N positions away from the main diagonal of $\boldsymbol{K2D}$.

The 2D matrix in Figure 6.2a is *block tridiagonal with tridiagonal blocks*:

$$K = \begin{bmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & \cdot & \cdot & \cdot \\ & & -1 & 2 \end{bmatrix} \qquad \boldsymbol{K2D} = \begin{bmatrix} K+2I & -I & & \\ -I & K+2I & -I & \\ & \cdot & \cdot & \cdot \\ & & -I & K+2I \end{bmatrix} \tag{1}$$

Size N **Elimination in this order: Size $n = N^2$**

Time N **Bandwidth $w = N$, Space $nw = N^3$, Time $nw^2 = N^4$**

The matrix $K2D$ has 4's down the main diagonal. Its bandwidth $w = N$ is the distance from the diagonal to the nonzeros in $-I$. Many of the spaces in between are filled during elimination! Then the storage space required for the factors in $K2D = LU$ is of order $nw = N^3$. The time is proportional to $nw^2 = N^4$, when n rows each contain w nonzeros, and w nonzeros below the pivot require elimination.

Again, the operation count grows as nw^2. Each elimination step uses a row of length w. There can be nw nonzeros to eliminate below the diagonal. If some entries stay zero inside the band, elimination could be faster than nw^2—and this is our goal.

Those counts are not impossibly large in many practical 2D problems (and we show how they can be reduced). The horrifying N^7 count will come for elimination on $K3D$. Suppose the 3D cubic grid is numbered a plane at a time. In each plane we see a 2D square, and $K3D$ has blocks of order N^2 from those squares. With each square numbered as above, the blocks come from $\boldsymbol{K2D}$ and $\boldsymbol{I} = \boldsymbol{I2D}$:

$$\boldsymbol{K3D} = \begin{bmatrix} \boldsymbol{K2D}+\boldsymbol{2I} & -\boldsymbol{I} & & \\ -\boldsymbol{I} & \boldsymbol{K2D}+\boldsymbol{2I} & -\boldsymbol{I} & \\ & \cdot & \cdot & \cdot \\ & & -\boldsymbol{I} & \boldsymbol{K2D}+\boldsymbol{2I} \end{bmatrix}$$

3D Size $n = N^3$
Bandwidth $w = N^2$
Elimination space N^5
Elimination time N^7

The main diagonal of $K3D$ contains 6's, and "inside rows" have six -1's. Next to a face or edge or corner of the cube, we lose one or two or three of those -1's.

From any node to the node above it, we count N^2 nodes. The $-I$ blocks are far from the main diagonal and the bandwidth is $w = N^2$. Then $nw^2 = N^7$.

Fill-in Let me focus attention immediately on the key problem, when we apply elimination to sparse matrices. *The zeros "inside the band" may fill in with nonzeros.* Those nonzeros enter the triangular factors of $K = LL^{\mathrm{T}}$ (Figure 6.2a). When multiples of one pivot row are subtracted from other rows, a nonzero in that pivot row will infect all the others.

Sometimes a matrix has a full row, from an equation like $\sum U_j = 1$ (see Figure 6.3). That full row better come last!

A good way to visualize the nonzero structure (sparsity structure) of K is by a graph. The rows of K are nodes in the graphs. A nonzero entry K_{ij} produces an edge between nodes i and j. *The graph of $K2D$ is exactly the mesh of unknowns*! Filling in a nonzero adds a new edge (a diagonal in our square graph). Watch how fill-in happens in elimination:

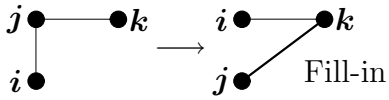
FILL-IN ***New nonzero in the matrix*** / ***New edge in the graph***

Suppose a_{ij} is eliminated. A multiple of row j later is subtracted from the row i. If a_{jk} is nonzero in row j, then a_{ik}^{new} becomes filled in row i:

Matrix nonzeros

$$\begin{matrix} a_{jj} & a_{jk} \\ a_{ij} & 0 \end{matrix} \longrightarrow \begin{matrix} a_{jj} & a_{jk} \\ 0 & a_{ik}^{\text{new}} \end{matrix}$$

Graph edges



Kronecker product One good way to create $K2D$ from K and I (N by N) is the $\texttt{kron}(A, B)$ command. This replaces each number a_{ij} by the block $a_{ij}B$. To take second differences in all columns at the same time, and all rows, use `kron` twice:

$$\boldsymbol{K2D} = \texttt{kron}(K, I) + \texttt{kron}(I, K) = \begin{bmatrix} 2I & -I & \cdot \\ -I & 2I & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} + \begin{bmatrix} K & & \\ & K & \\ & & \cdot \end{bmatrix} \tag{2}$$

This sum agrees with (1). Then a 3D box needs $K2D$ and $I2D = \texttt{kron}(I, I)$ in each plane. This easily adjusts to allow rectangles, with I's of different sizes. For a cube, take second differences inside all planes with $K2D$, plus differences in the z-direction:

$$\boldsymbol{K3D} = \texttt{kron}(K2D, I) + \texttt{kron}(I2D, K) \quad \text{has size} \quad (N^2)(N) = N^3\,. \tag{3}$$

Here K and $K2D$ and $K3D$ of size N and N^2 and N^3 are serving as *models of the type of matrices that we meet*. But we have to say that there are special ways to work with these particular matrices. The x, y, z directions are separable. We can use FFT-type methods in each direction. See Section 6.____ on *Fast Poisson Solvers*.

MATLAB should know that the matrices are sparse! If we create $I =$ `speye`(N) and K comes from `spdiags` as in Section 1.1, the `kron` command will preserve the sparse treatment. Before we describe reordering by minimum degree, allow me to display `spy`$(\boldsymbol{K2D})$ plus fill, and the reordered `spy`$(P\boldsymbol{K2D}P^{\mathrm{T}})$. *The latter will have much less fill-in.* Your eye may not see this so clearly, but the count is convincing!

Figure 6.2: (a) $K2D$ is filled in by elimination. (b) A good reordering of $K2D$.

Minimum Degree Algorithm

We now describe a useful way to reorder meshpoints and equations in $\boldsymbol{K2D}\,U = F$. The ordering achieves *approximate minimum degree* at each step—**the number of nonzeros below the pivot is almost minimized**. This is essentially the algorithm used in MATLAB's command $U = K\backslash F$, when K has been defined as a sparse matrix. We list some of the functions from the `sparfun` directory:

`speye`	(sparse identity I)	`nnz`	(number of nonzero entries)
`find`	(find indices of nonzeros)	`spy`	(visualize sparsity pattern)
`colamd` and `symamd`	(approximate minimum degree permutation of K)		

You can test and use the minimum degree algorithms without a careful analysis. The approximations are faster than the exact minimum degree permutations `colmmd` and `symmmd`. The speed (in two dimensions) and the roundoff errors are quite acceptable.

In the Laplace examples, the minimum degree ordering of nodes is irregular compared to "a row at a time." The final bandwidth is probably not decreased. But many nonzero entries are postponed as long as possible! That is the key.

The difference is shown in the **arrow matrix** of Figure 6.3. Its minimum degree ordering (one nonzero below each pivot) produces large bandwidth, but ***no fill-in***. Elimination will only change the last row and column. The triangular factors L and U have all the same zeros as A, keeping the arrow. The space for storage stays at $3n$, and elimination needs only n divisions and multiplications and subtractions. When a sparse matrix has a full row, *put it last.*

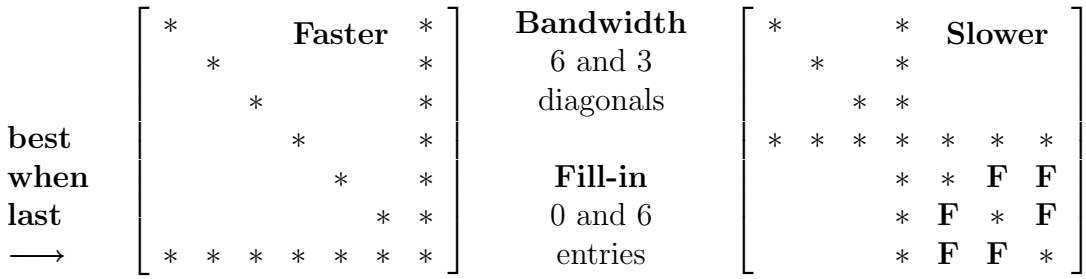


Figure 6.3: Minimum degree (arrow matrix) defeats minimum bandwidth.

The second ordering reduces the bandwidth from 6 to 3. But when row 4 is used as the pivot row, the entries indicated by **F** are *filled in*. That lower quarter of A becomes full, with $n^2/8$ nonzeros in each factor L and U. You see that the whole nonzero "profile" of the matrix decides the fill-in, not just the bandwidth.

Here is another example, from the **red-black ordering** of a square grid. Color the gridpoints like a checkerboard. Then all four neighbors or a red point are black, and vice versa. If we number the red points before the black points, the *permuted* $K2D$ has diagonal blocks of $4I$:

Red-black permuation

$$\boldsymbol{P}\,(\boldsymbol{K2D})\,\boldsymbol{P^{\mathrm{T}}} = \begin{bmatrix} 4\,I_{\mathsf{red}} & -1\text{'s} \\ -1\text{'s} & 4\,I_{\mathsf{black}} \end{bmatrix}. \tag{4}$$

This pushes the -1's and the fill-in into the last $N^2/2$ rows. Notice how P permutes the rows (the equations) and P^{T} permutes the columns (the unknowns). Together they keep 4's on the main diagonal.

Now the real thing. **Minimum degree algorithms** choose the $(k+1)$st pivot column, after the first k columns have been eliminated as usual below the diagonal. The algorithms look at the nonzeros in the lower right matrix of size $n-k$.

Symmetric case:	Choose the remaining meshpoint with the **fewest neighbors**.
Unsymmetric case:	Choose the remaining column with the **fewest nonzeros**.

The component of U corresponding to that column is renumbered $k+1$. So is the meshpoint in the finite difference grid. Of course elimination in that column will normally produce new nonzeros in the remaining columns. Some fill-in is unavoidable. The algorithm keeps track of the new positions of nonzeros, and also the actual entries. It is the ***positions*** that decide the ***ordering*** of unknowns (the permutation of columns). Then the *entries* in K decide the *numbers* in L and U.

The **degree of a node** is the number of connections to other nodes. This is the number of off-diagonal nonzeros in that column of K. In Figure 6.4 the corner nodes $1, 3, 4, 6$ all begin with degree 2. The side nodes 2 and 5 have degree 3. *The degrees change as elimination proceeds*! **Nodes connected to the pivot become connected to each other—and that entry of the matrix fills in.**

You will see how a renumbering of meshpoints preserves the symmetry of K. The rows and columns are reordered in the same way. Then $P\,K_{\mathsf{old}}P^{\mathrm{T}} = K_{\mathsf{new}} = K_{\mathsf{new}}^{\mathrm{T}}$.

Example 1 Figure 6.4 shows a small example of the minimal degree ordering, for Laplace's 5-point scheme. The problem starts with six meshpoints (they would be connected to boundary points where U is known) and a 6 by 6 matrix. K has two 3 by 3 tridiagonal blocks from horizontal links, and two 3 by 3 blocks with $-I$ from vertical links.

The first elimination step chooses row 1 as pivot row, because node 1 has minimum degree 2. (Any degree 2 node could come first, leading to different elimination orders.) The pivot is **P**, the other nonzeros in that row are boxed. When row 1 operates on rows 2

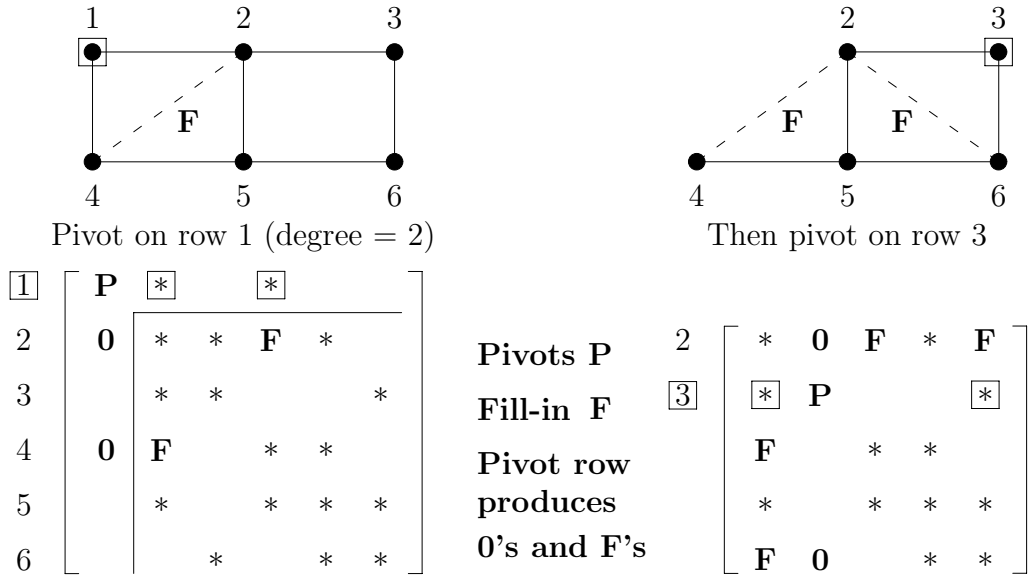


Figure 6.4: Minimum degree nodes 1 and 3 give pivots **P**. New diagonal edges 2–4 and 2–6 in the graph match the first four entries **F** that are filled in by elimination.

and 4, it changes six entries below it (left figure). *The two fill-in entries marked by* **F** *change to nonzeros*. This fill-in of the $(2,4)$ and $(4,2)$ entries corresponds to the dashed line connecting nodes 2 and 4 in the graph.

Elimination continues on the 5 by 5 matrix (and the graph with 5 nodes). Node 2 still has degree 3, so it is not eliminated next. If we break the tie by choosing node 3, elimination using the new pivot **P** will fill in the $(2,6)$ and $(6,2)$ positions. *Node 2 becomes linked to node 6 because they were both linked to the eliminated node 3.*

The problem is reduced to 4 by 4, for the unknown U's at the remaining nodes 2, 4, 5, 6. Problem ____ asks you to take the next step—choose a minimum degree node and reduce the system to 3 by 3. Figure 6.5 shows the start of a minimum degree ordering for a larger grid. Notice how fill-in (16 edges, 32 **F**'s) increases the degrees.

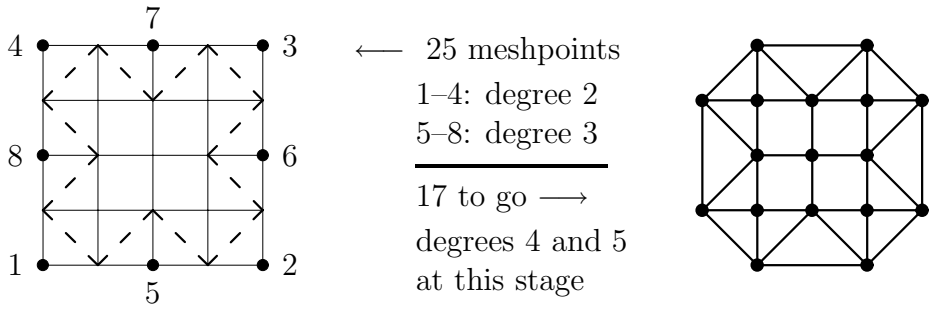


Figure 6.5: Nodes connected to an eliminated node become connected to each other.

 ©2006 Gilbert Strang

Storing the Nonzero Structure = Sparsity Pattern

A large system $KU = F$ needs a fast and economical storage of the node connections (the positions i, j of nonzeros in the matrix). The internal list of edges and nonzero positions will change as elimination proceeds. Normally we don't see that list.

Here we create the list for $N = 4$ by $[i, j, s] = \texttt{find}(K)$. This has $\texttt{nnz}(K) = 10$:

$$\boldsymbol{i} = \mathbf{1\ \ 2\ \ 1\ \ 2\ \ 3\ \ 2\ \ 3\ \ 4\ \ 3\ \ 4} \qquad \boldsymbol{j} = \mathbf{1\ \ 1\ \ 2\ \ 2\ \ 2\ \ 3\ \ 3\ \ 3\ \ 4\ \ 4}$$

$$\boldsymbol{s} = \mathbf{2\ \ -1\ \ -1\ \ 2\ \ -1\ \ -1\ \ 2\ \ -1\ \ -1\ \ 2}$$

The 10 entries are in s, so the fifth nonzero has $i = 3$, $j = 2$, and $s = K_{32} = -1$. The positions i, j of nonzeros in K are in lexicographical order.

You can see that the list j of column indices should be compressed. All we need are *pointers* to indicate when a new column appears on the list. In reality j is replaced by this short list of pointers to the list, including a last pointer to position 11 that signals the stop:

$$\textbf{point} = \mathbf{1\ \ 3\ \ 5\ \ 8\ \ 11}$$

can be updated by $\texttt{perm}(\textbf{point})$ when columns are reordered.

The sparse backslash command $U = K\backslash F$ uses an approximate minimum degree algorithm. First it checks the nonzero pattern to see if row and column permutations P_1 and P_2 can produce a **block triangular form**. The reordered system is $\boldsymbol{P_1 K P_2^{\mathrm{T}}(P_2 U) = P_1 F}$:

$$\textbf{Block triangular matrix} \qquad \boldsymbol{P_1 K P_2^{\mathrm{T}}} = \begin{bmatrix} B_{11} & B_{12} & \cdot & \cdot \\ 0 & B_{22} & \cdot & \cdot \\ 0 & 0 & \cdot & \cdot \\ 0 & 0 & 0 & B_{mm} \end{bmatrix}.$$

The reordered unknown P_2U and the right-hand side P_1F are similarly partitioned into m blocks. ***Block back-substitution*** starts with the smaller problem $B_{mm}U_m = F_m$. Working upwards, we only deal with the blocks B_{ii} on the diagonal (the smaller the better). Surprisingly often, a block triangular form is available.

To preserve symmetry we need $P_1 = P_2$. In the positive definite case, the Cholesky command `chol` is preferred to `lu`, since useless row exchanges can be safely omitted. If $\texttt{diag}(K)$ is positive, there is a chance (not a certainty!) of positive definiteness. Backslash will try `chol` until it is forced to accept row exchanges.

It is understood that MATLAB is not tuned for high performance. Use it for tests and experiments and adjustments. Established codes often turn to C^{++}.

Graph Separators

Here is another approach to ordering, different from minimum degree. The whole graph or mesh is separated into disjoint pieces by a cut. This separator goes through

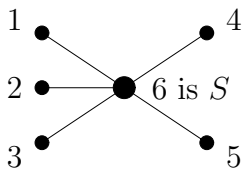
a small number of nodes or meshpoints. *It is a good idea to number the nodes in the separator last.* Elimination is relatively fast for the disjoint pieces P and Q. It only slows down at the end, for the (smaller) separator S.

The meshpoints in P have no direct connections to Q (both are connected to the separator S). Numbered in that order, the "block arrow" stiffness matrix has two blocks of zeros. Its $K = LU$ factorization looks like this:

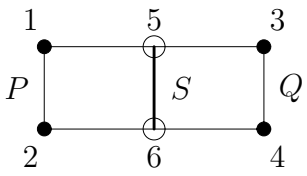
$$\boldsymbol{K} = \begin{bmatrix} K_{\mathrm{P}} & \mathbf{0} & K_{\mathrm{PS}} \\ \mathbf{0} & K_{\mathrm{Q}} & K_{\mathrm{QS}} \\ K_{\mathrm{SP}} & K_{\mathrm{SQ}} & K_{\mathrm{S}} \end{bmatrix} \quad \boldsymbol{L} = \begin{bmatrix} L_{\mathrm{P}} & & \\ \mathbf{0} & L_{\mathrm{Q}} & \\ X & Y & Z \end{bmatrix} \quad \boldsymbol{U} = \begin{bmatrix} U_{\mathrm{P}} & \mathbf{0} & A \\ & U_{\mathrm{Q}} & B \\ & & C \end{bmatrix} \quad (5)$$

The zero blocks in K give zero blocks in L and U. The submatrix K_{P} produces L_{P} and U_{P}. Then come the factors $L_{\mathrm{Q}}U_{\mathrm{Q}}$ of K_{Q}, followed by the connections through the separator. The major cost is often that last step, the solution of a fairly dense system. When the separator S has size N, a dense system costs N^3.

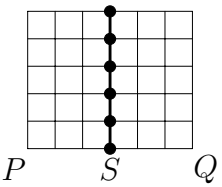
A separator illustrates the key idea of **domain decomposition**: *Cut the problem into smaller pieces.* This is natural for structural analysis of an airplane: Solve separately for the wings and the fuselage. The smaller system for the separator (where the pieces meet) is like the third row of equation (5). This expresses the requirement that the unknown and its normal derivative (stress or flux) match along the separator. We apologize that a full discussion of domain decomposition is impossible (see [–]).



Arrow matrix
(Figure 6.3)



Separator comes last
(Figure 6.4)



Blocks P, Q
Separator S

Figure 6.6: A graph separator numbered last produces a block arrow matrix K.

Figure 6.6 shows three examples, each with separators. The graph for a perfect arrow matrix has a one-point separator (very unusual). The 6-node rectangle has a two-node separator in the middle. Every N by N grid can be cut by an N-point separator (and N is much smaller than N^2).

On a rectangular grid, the best cut is down the middle in the shorter direction. Our model problem on a square is actually the hardest! A region shaped like a U (or maybe a radiator in 3D) might look difficult. But actually it allows very short separators and fast elimination. A tree needs *no* fill-in (Problem ____).

 ©2006 Gilbert Strang

Nested Dissection

You could say that the numbering of P then Q then S is **block minimum degree**. But one cut with one separator will not come close to an optimal numbering. It is natural to extend the idea to a nested sequence of cuts. P and Q have their own separators at the next level. This **nested dissection** continues until it is not productive to cut further. It is a strategy of "divide and conquer."

Figure 6.7 illustrates three levels of nested dissection on a 7 by 7 grid. The first separator is down the middle. Then two cuts go across and four cuts go down. Numbering the separators last within each stage, the matrix K of size 49 has arrows inside arrows inside arrows. The `spy` command will display the pattern of nonzeros.

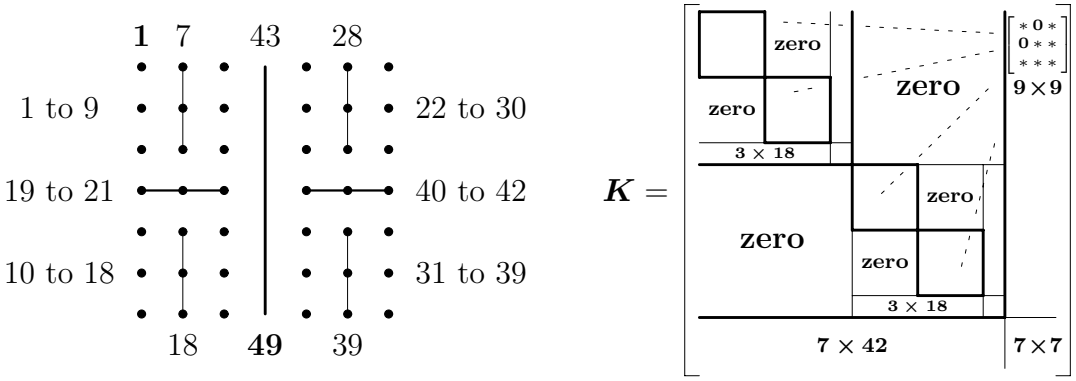


Figure 6.7: Three levels of separators. Still ____ nonzeros in K, only ____ in L.

Separators and nested dissection show how numbering strategies are based on the **graph of nodes**. Edges between nodes correspond to nonzeros in the matrix K. The fill-in created by elimination (entries **F** in L and U) corresponds to adjacent edges in the graph. In practice, there has to be a balance between simplicity and optimality in the numbering—in scientific computing simplicity is a very good thing!

Here are the complexity estimates for the Laplacian with N^2 or N^3 nodes:

Nested Separators	$n = N^2$ in 2D	$n = N^3$ in 3D
Space (nonzeros from fill-in)	$\boldsymbol{N^2 \log N}$	$\boldsymbol{N^4}$
Time (flops for elimination)	$\boldsymbol{N^3}$	$\boldsymbol{N^6}$

In the last century, nested dissection lost out (too slow) on almost all applications. Now larger problems are appearing and the asymptotics eventually give nested dissection an edge. All planar graphs have separators of size $\sqrt{n}$ into nearly equal pieces (P and Q have sizes at most $2n/3$). Of course a new idea for ordering could still win. We don't recommend the older algorithm of (reverse) Cuthill-McKee.

A reasonable compromise is the backslash command $U = K \backslash F$ that uses a nearly minimum degree ordering in Sparse MATLAB.

The text [GL] by George and Liu is the classic reference for this section on ordering of the nodes. The new book [,2007] by Davis has a superb description of his recent algorithms and how they are implemented (in MATLAB's backslash and in UMFPACK for sparse unsymmetric systems).

Problem Set 6.1

1 Create $K2D$ for a 4 by 4 square grid with $N^2 = 3^2$ interior mesh points (so $n = 9$). Print out its factors $K = LU$ (or its Cholesky factor $C = \texttt{chol}(K)$ for the symmetrized form $K = C^{\mathrm{T}}C$). How many zeros in these triangular factors? Also print out $\texttt{inv}(K)$ to see that it is full.

2 As N increases, what parts of the LU factors of $K2D$ are filled in ?

3 Can you answer the same question for $K3D$? In each case we really want an estimate cN^p of the number of nonzeros (the most important number is p).

4 Use the `tic; ...; toc` clocking command or the `cpu` command to compare the solution time for $K2Dx$ = random f in ordinary MATLAB and sparse MATLAB (where $K2D$ is defined as a sparse matrix). Above what value of N does the sparse routine $K\backslash f$ win ?

5 Compare ordinary vs. sparse solution times in the three-dimensional $K3Dx$ = random f. At which N does the sparse $K\backslash f$ begin to win ?

6 Incomplete LU

7 Draw the next step after Figure 6.4 when the matrix has become 4 by 4 and the graph has nodes 2–4–5–6. Which have minimum degree? Is there more fill-in?

8 Redraw the right side of Figure 6.4 if row number 2 is chosen as the second pivot row. Node 2 does not have minimum degree. Indicate new edges in the 5-node graph and new nonzeros **F** in the matrix.

9 Order a tree to avoid all fill-in.

10 Block triangular.

11 Suppose the unknowns U_{ij} on a square grid are stored in an N by N matrix, and not listed in a vector U of length N^2. Show that the vector result of $(K2D)U$ is now produced by the matrix $KU + UK$.

12 Elimination in minimum degree order is started for the square mesh in Figure 6.5. How large do the degrees become as elimination continues? (You could try a large graph, giving systematics instructions for ties in the minimum degree.)

13 Experiment with the red-black permutation in equation ( ), to see how much fill-in is produced by elimination. Is the red-black ordering superior to the original row-by-row ordering?

 ©2006 Gilbert Strang

6.2 Iterative Methods

New solution methods are needed when a problem $Ax = b$ is too large and expensive for ordinary elimination. We are thinking of *sparse matrices* A, so that multiplications Ax are relatively cheap. If A has at most p nonzeros in every row, then ***Ax needs at most pn multiplications***. Typical applications are to large finite difference or finite element equations, where we often write $A = K$.

We are turning from elimination to look at **iterative methods**. There are really two big decisions, the preconditioner P and the choice of the method itself:

1. A good preconditioner P is close to A but much simpler to work with.

2. Options include ***pure iterations*** (6.2), ***multigrid*** (6.3), and ***Krylov methods*** (6.4), including the conjugate gradient method.

Pure iterations compute each new x_{k+1} from $x_k - P^{-1}(Ax_k - b)$. This is called "*stationary*" because every step is the same. Convergence to $x_\infty = A^{-1}b$, studied below, will be fast when all eigenvalues of $M = I - P^{-1}A$ are small. It is easy to suggest a preconditioner, but not so easy to suggest an excellent P (*Incomplete LU is a success*). The older iterations of Jacobi and Gauss-Seidel are less favored (but they are still important, you will see good points and bad points).

Multigrid begins with Jacobi or Gauss-Seidel iterations, for the one job that they do well. They remove high frequency components (rapidly oscillating parts) to leave a smooth error. Then the central idea is to *move to a coarser grid*—where the rest of the error can be destroyed. Multigrid is often dramatically successful.

Krylov spaces contain all combinations of $b, Ab, A^2b, \ldots$ and Krylov methods look for the best combination. Combined with preconditioning, the result is terrific. When the growing subspaces reach the whole space $\mathbf{R}^n$, those methods give the exact solution $A^{-1}b$. But in reality we stop much earlier, long before n steps are complete. The *conjugate gradient method* (*for positive definite A, and with a good preconditioner*) has become truly important.

The goal of numerical linear algebra is clear: **Find a fast stable algorithm that uses the special properties of the matrix.** We meet matrices that are symmetric or triangular or orthogonal or tridiagonal or Hessenberg or Givens or Householder. Those are at the core of matrix computations. The algorithm doesn't need details of the entries (which come from the specific application). By concentrating on the matrix structure, numerical linear algebra offers major help.

Overall, elimination with good numbering is the first choice! But storage and CPU time can become excessive, especially in three dimensions. At that point we turn from elimination to iterative methods, which require more expertise than $K\backslash F$. The next pages aim to help the reader at this frontier of scientific computing.

©2006 Gilbert Strang

Stationary Iterations

We begin with old-style pure stationary iteration. The letter K will be reserved for "Krylov" so we leave behind the notation $KU = F$. The linear system becomes $Ax = b$. The large sparse matrix A is not necessarily symmetric or positive definite:

Linear system $Ax = b$ **Residual** $r_k = b - Ax_k$ **Preconditioner** $P \approx A$

The preconditioner P attempts to be close to A and still allow fast iterations. The Jacobi choice $P =$ diagonal of A is one extreme (fast but not very close). The other extreme is $P = A$ (too close). Splitting the matrix A gives a new form of $Ax = b$:

Splitting $$\boldsymbol{Px = (P - A)x + b}\,. \tag{1}$$

This form suggests an iteration, in which every vector x_k leads to the next x_{k+1}:

Iteration $$\boldsymbol{Px_{k+1} = (P - A)x_k + b}\,. \tag{2}$$

Starting from any x_0, the first step finds x_1 from $Px_1 = (P - A)x_0 + b$. The iteration continues to x_2 with the same matrix P, so it often helps to know its triangular factors in $P = LU$. Sometimes P itself is triangular, or L and U are approximations to the triangular factors of A. Two conditions on P make the iteration successful:

1. The new x_{k+1} must be quickly computable. Equation (2) must be fast to solve.

2. The errors $e_k = x - x_k$ should approach zero as rapidly as possible.

Subtract equation (2) from (1) to find the **error equation**. It connects e_k to e_{k+1}:

Error $Pe_{k+1} = (P - A)e_k$ which means $$\boldsymbol{e_{k+1} = (I - P^{-1}A)e_k = Me_k}\,. \tag{3}$$

The right side b disappears in this error equation. Each step multiplies the error vector e_k by M. The speed of convergence of x_k to x (and of e_k to zero) depends entirely on M. *The test for convergence is given by the eigenvalues of* M:

Convergence test Every eigenvalue of $M = I - P^{-1}A$ must have $|\lambda(M)| < 1$.

The largest eigenvalue (in absolute value) is the **spectral radius** $\boldsymbol{\rho(M)} = \max|\lambda(M)|$. Convergence requires $\rho(M) < 1$. The **convergence rate** is set by the largest eigenvalue. For a large problem, we are happy with $\rho(M) = .9$ and even $\rho(M) = .99$.

When the initial error e_0 happens to be an eigenvector of M, the next error is $e_1 = Me_0 = \lambda e_0$. At every step the error is multiplied by λ. *So we must have* $|\lambda| < 1$. Normally e_0 is a combination of all the eigenvectors. When the iteration multiplies by M, each eigenvector is multiplied by its own eigenvalue. After k steps those multipliers are λ^k, and the largest is $(\rho(M))^k$.

If we don't use a preconditioner then $M = I - A$. All the eigenvalues of A must be inside a unit circle centered at 1, for convergence. Our second difference matrices $A = K$ would fail this test ($I - K$ is too large). The first job of a preconditioner is to get the matrix decently scaled. Jacobi will now give $\rho(I - \frac{1}{2}K) < 1$, and a really good P will do more.

 ©2006 Gilbert Strang

Jacobi Iterations

For preconditioner we first propose a simple choice:

Jacobi iteration $\boldsymbol{P}$ **= diagonal part** $\boldsymbol{D}$ **of** $\boldsymbol{A}$

Typical examples have spectral radius $\rho(M) = 1 - cN^{-2}$, where N counts meshpoints in the longest direction. This comes closer and closer to 1 (too close) as the mesh is refined and N increases. But Jacobi is important, it does part of the job.

For our tridiagonal matrices K, Jacobi's preconditioner is just $P = 2I$ (the diagonal of K). **The Jacobi iteration matrix becomes** $\boldsymbol{M = I - D^{-1}A = I - \frac{1}{2}K}$:

Iteration matrix for a Jacobi step

$$M = I - \frac{1}{2}K = \frac{1}{2}\begin{bmatrix} 0 & 1 & & \\ 1 & 0 & 1 & \\ & 1 & 0 & 1 \\ & & 1 & 0 \end{bmatrix}. \tag{4}$$

Here is x^{new} from x^{old}, in detail. You see how Jacobi shifts the off-diagonal entries of A to the right-hand side, and divides by the diagonal part $D = 2I$:

$$\begin{aligned} 2x_1 - x_2 &= b_1 \\ -x_1 + 2x_2 - x_3 &= b_2 \\ -x_2 + 2x_3 - x_4 &= b_3 \\ -x_3 + 2x_4 &= b_4 \end{aligned} \quad \text{becomes} \quad \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}^{\text{new}} = \frac{1}{2}\begin{bmatrix} & x_2 & \\ x_1 & + & x_3 \\ x_2 & + & x_4 \\ x_3 & & \end{bmatrix}^{\text{old}} + \frac{1}{2}\begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}. \tag{5}$$

The equation is solved when $x^{\text{new}} = x^{\text{old}}$, but this only happens in the limit. The real question is the number of iterations to get close to convergence, and that depends on the eigenvalues of M. How close is $\lambda_{\max}$ to 1?

Those eigenvalues are simple cosines. In Section 1.5 we actually computed all the eigenvalues $\lambda(K) = 2 - 2\cos(\frac{j\pi}{N+1})$. Since $M = I - \frac{1}{2}K$, we now divide those eigenvalues by 2 and subtract from 1. The eigenvalues $\cos j\theta$ of M are less than 1!

Jacobi eigenvalues $$\lambda_j(M) = 1 - \frac{2 - 2\cos j\theta}{2} = \boldsymbol{\cos j\theta} \quad \text{with } \theta = \frac{\pi}{N+1}. \tag{6}$$

Convergence is safe (but slow) because $|\cos\theta| < 1$. Small angles have $\cos\theta \approx 1 - \frac{1}{2}\theta^2$. The choice $j = 1$ gives us the first (and largest) eigenvalue of M:

Spectral radius $$\lambda_{\max}(M) = \cos\theta \approx 1 - \frac{1}{2}\left(\frac{\pi}{N+1}\right)^2. \tag{7}$$

Convergence is slow for the lowest frequency. The matrix M in (4) has the four eigenvalues $\cos\frac{\pi}{5}, \cos\frac{2\pi}{5}, \cos\frac{3\pi}{5}$, and $\cos\frac{4\pi}{5}$ (which is $-\cos\frac{\pi}{5}$) in Figure 6.8.

There is an important point about those Jacobi eigenvalues $\lambda_j(M) = \cos j\theta$. The magnitude $|\lambda_j|$ at $j = N$ is the same as the magnitude at $j = 1$. This is not good for multigrid, where high frequencies need to be strongly damped. So **weighted Jacobi** has a valuable place, with a weighting factor ω in $M = I - \omega D^{-1}A$:

©2006 Gilbert Strang

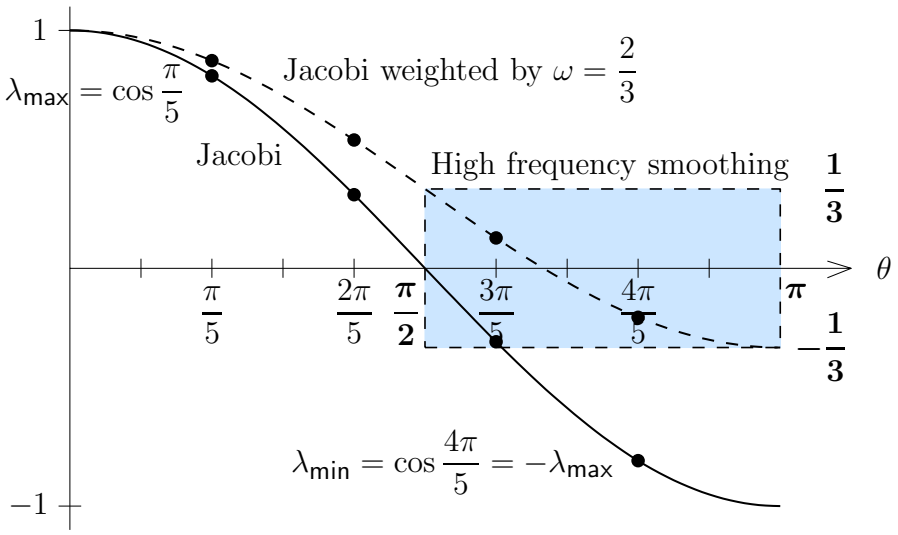


Figure 6.8: The eigenvalues of Jacobi's $M = I - \frac{1}{2}K$ are $\cos j\theta$, starting near $\lambda = 1$ and ending near $\lambda = -1$. Weighted Jacobi has $\lambda = 1 - \omega + \omega\cos j\theta$, ending near $\lambda = 1 - 2\omega$. Both graphs show $j = 1, 2, 3, 4$ and $\theta = \frac{\pi}{N+1} = \frac{\pi}{5}$ (with $\omega = \frac{2}{3}$).

Jacobi's iteration matrix $M = I - D^{-1}A$ changes to $M = I - \omega D^{-1}A$.

The preconditioner is now $P = D/\omega$. Here are the eigenvalues $\lambda(M)$ when $A = K$:

Weighted Jacobi

$$D = 2I \quad \text{and} \quad M = I - \frac{\omega}{2}A \quad \text{and} \quad \omega < 1$$
$$\lambda_j(M) = 1 - \frac{\omega}{2}(2 - 2\cos j\theta) = \mathbf{1 - \omega + \omega\cos j\theta}. \tag{8}$$

The dashed-line graph in Figure 6.8 shows these values $\lambda_j(M)$ for $\omega = \frac{2}{3}$. This ω is optimal in damping the high frequencies ($j\theta$ between $\pi/2$ and π) by at least $\frac{1}{3}$:

$$\text{At } j\theta = \frac{\pi}{2} \qquad \lambda(M) = 1 - \omega + \omega\cos\frac{\pi}{2} = 1 - \frac{2}{3} = \mathbf{\frac{1}{3}}$$

$$\text{At } j\theta = \pi \qquad \lambda(M) = 1 - \omega + \omega\cos\pi = 1 - \frac{4}{3} = \mathbf{-\frac{1}{3}}$$

If we move away from $\omega = \frac{2}{3}$, one of those eigenvalues will increase in magnitude. A weighted Jacobi iteration will be a good smoother within multigrid.

In two dimensions the picture is essentially the same. **The N^2 eigenvalues of $K2D$ are the sums $\lambda_j + \lambda_k$ of the N eigenvalues of K.** All eigenvectors are samples of $\sin j\pi x \sin k\pi y$. (In general, the eigenvalues of $\texttt{kron}(A, B)$ are $\lambda_j(A)\lambda_k(B)$. For $\texttt{kron}(K, I) + \texttt{kron}(I, K)$, sharing eigenvectors means we can add eigenvalues.)

Jacobi has $P = 4I$, from the diagonal of $K2D$. So $M2D = I2D - \frac{1}{4}K2D$:

$$\lambda_{jk}(M2D) = 1 - \frac{1}{4}\left[\lambda_j(K) + \lambda_k(K)\right] = \frac{1}{2}\cos j\theta + \frac{1}{2}\cos k\theta\,. \tag{9}$$

With $j = k = 1$, the spectral radius $\lambda_{\max}(M) = \cos\theta$ is the same $1 - cN^{-2}$ as in 1D.

Numerical Experiments

The multigrid method grew out of the slow convergence of Jacobi iterations. You have to see how typical error vectors e_k begin to decrease and then stall. For weighted Jacobi, the high frequencies disappear long before the low frequencies. Figure 6.___ shows a drop between e_0 and e_{--}, and then very slow decay toward $e_\infty = 0$. We have chosen the second difference matrix $A = K_{500}$ and started the iterations with the right-hand side $x_0 = b = \texttt{rand}(500, 1)$.

This unacceptably slow convergence is hidden if we only look at the residual $r_k = b - Ax_k$. Instead of measuring the error $x - x_k$ in the solution, r_k measures the error in the equation. That residual error *does* fall quickly. The key that led to multigrid is the rapid drop in r with such a slow drop in e.

Figure 6.9: The residual $b - Ax$ falls quickly but the solution error gets stalled.

Gauss-Seidel and the Red-Black Ordering

The Gauss-Seidel idea is to use the components of x^{new} as soon as they are computed. This cuts the storage requirement in half, since x^{old} is overwritten by x^{new}. The preconditioner $P = D + L$ becomes *triangular* instead of diagonal (still easy to use):

Gauss-Seidel iteration $\quad \boldsymbol{P}$ **= lower triangular part of** $\boldsymbol{A}$

Gauss-Seidel gives faster error reduction than ordinary Jacobi, because the Jacobi eigenvalues $\cos j\theta$ become $(\cos j\theta)^2$. The spectral radius is squared, so one Gauss-Seidel step is worth two Jacobi steps. The (large) number of iterations is cut in half, when the -1's below the diagonal stay with the 2's on the *left side* of Px^{new}:

Gauss-Seidel $\boldsymbol{Px^{\text{new}} = (P-A)x^{\text{old}} + b}$

$$-1\begin{bmatrix}0\\x_1\\x_2\\x_3\end{bmatrix}^{\text{new}} + 2\begin{bmatrix}x_1\\x_2\\x_3\\x_4\end{bmatrix}^{\text{new}} = \begin{bmatrix}x_2\\x_3\\x_4\\0\end{bmatrix}^{\text{old}} + \begin{bmatrix}b_1\\b_2\\b_3\\b_4\end{bmatrix}. \quad (10)$$

A new x_1 comes from the first equation, because P is triangular. Using x_1^{new} in the second equation gives x_2^{new}, which enters the third equation. Problem 1 shows that $x^{\text{new}} = (\cos j\theta)^2 x^{\text{old}}$, with the correct eigenvector x^{old}. All the Jacobi eigenvalues $\cos j\theta$ are squared for Gauss-Seidel, so they become smaller.

Symmetric Gauss-Seidel comes from a double sweep, reversing the order of components to make the combined process symmetric. By itself, $I - P^{-1}A$ is not symmetric for triangular P.

A **red-black ordering** produces a neat compromise between Jacobi and Gauss-Seidel. Imagine that a two-dimensional grid is a *checkerboard*. Number the red nodes before the black nodes. The numbering will not change Jacobi's method (which keeps

©2006 Gilbert Strang

all of x^{old} to create x^{new}). But Gauss-Seidel will be improved. In one dimension, Gauss-Seidel updates all the even (red) components x_{2j} using known black values. Then it updates the odd (black) components x_{2j+1} using the new red values:

$$x_{2j} \longleftarrow \frac{1}{2}\left(x_{2j-1} + x_{2j+1} + b_{2j}\right) \quad \text{and then} \quad x_{2j+1} \longleftarrow \frac{1}{2}\left(x_{2j} + x_{2j+2} + b_{2j+1}\right). \tag{11}$$

In two dimensions, $x_{i,j}$ is red when $i+j$ is even, and black when $i+j$ is odd. Laplace's five-point difference matrix uses four black values to update each center value (red). Then red values update black, giving one example of *block Gauss-Seidel*.

For **line Gauss-Seidel**, each row of grid values forms a block. The preconditioner P is *block triangular*. That is the right way to see P in 2D:

$$P_{\text{red}-\text{black}} = \begin{bmatrix} 4I & 0 \\ -1\text{'s} & 4I \end{bmatrix} \quad \text{and} \quad P_{\text{line G}-\text{S}} = \begin{bmatrix} K+2I & 0 & 0 \\ -I & K+2I & 0 \\ 0 & -I & K+2I \end{bmatrix}$$

A great feature is the option to compute all red values *in parallel*. They need only black values and can be updated in any order—there are no connections among red values (or within black values in the second half-step). For line Jacobi, the rows in 2D can be updated in parallel (and the plane blocks in 3D). Block matrix computations are efficient.

Overrelaxation (SOR) is a combination of Jacobi and Gauss-Seidel, using a factor ω that almost reaches 2. The preconditioner is $P = D + \omega L$. (By *overcorrecting* from x_k to x_{k+1}, hand calculators noticed that they could finish in a few weeks.) My earlier book and many other references show how ω is chosen to minimize the spectral radius $\rho(M)$, improving $\rho = 1 - cN^{-2}$ to $\rho(M) = 1 - cN^{-1}$. Then convergence is much faster (N steps instead of N^2, to reduce the error by a constant factor like e).

Incomplete *LU*

A different approach has given much more flexibility in constructing a good P. The idea is to compute an ***incomplete LU factorization*** of the true matrix A:

Incomplete *LU* $\quad \boldsymbol{P = (approximation\ to\ L)(approximation\ to\ U)}$ (12)

The exact $A = LU$ has fill-in. So does Cholesky's $A = R^{\mathrm{T}}R$. Zero entries in A become nonzero in L and U and R. But $P = L_{\text{approx}}U_{\text{approx}}$ can keep only the fill-in entries $\mathbf{F}$ above a fixed tolerance. The MATLAB commands for incomplete LU are

$$[\,L, U, \texttt{Perm}\,] = \texttt{luinc}(A, \texttt{tol}) \qquad \text{or} \qquad R = \texttt{cholinc}(A, \texttt{tol})\,.$$

If you set `tol` $= 0$, those letters `inc` have no effect. This becomes ordinary sparse LU (and Cholesky for positive definite A). A large value of `tol` will remove all fill-in.

 ©2006 Gilbert Strang

Difference matrices like $K2D$ can maintain zero row sums by adding entries below `tol` to the main diagonal (instead of destroying those entries completely). This **modified *ILU*** is a success (`mluinc` and `mcholinc`). The variety of options, and especially the fact that the computer can decide automatically how much fill-in to keep, has made incomplete LU a very popular starting point.

In the end, $Px_{k+1} = (P-A)x_k + b$ is too simple! Often the smooth (low frequency) errors decrease too slowly. *Multigrid will fix this.* And pure iteration is choosing one particular vector in a "Krylov subspace." With relatively little work we can make a much better choice of x_k. Multigrid methods and Krylov projections are the state of the art in today's iterative methods.

Problem Set 6.2

1 If $x^{\mathsf{old}} = (\cos\frac{k\pi}{N+1}\sin\frac{k\pi}{N+1}, \cos^2\frac{k\pi}{N+1}\sin\frac{2k\pi}{N+1}, \ldots, \cos^N\frac{k\pi}{N+1}\sin\frac{Nk\pi}{N+1})$ show that the Gauss-Seidel iteration (10) is satisfied with $x^{\mathsf{new}} = \left[\cos^2\frac{k\pi}{N+1}\right] x^{\mathsf{old}}$. This shows that M for Gauss-Seidel has $\lambda = \cos^2\frac{k\pi}{N+1}$ (squares of Jacobi eigenvalues).

©2006 Gilbert Strang

6.3 Multigrid Methods

The Jacobi and Gauss-Seidel iterations produce smooth errors. The error vector e has its high frequencies nearly removed in a few iterations. But low frequencies are reduced very slowly. Convergence requires $O(N^2)$ iterations—which can be unacceptable. The extremely effective **multigrid idea** is to change to a coarser grid, on which "smooth becomes rough" and low frequencies act like higher frequencies.

On that coarser grid a big piece of the error is removable. *We iterate only a few times before changing from fine to coarse and coarse to fine.* The remarkable result is that multigrid can solve many sparse and realistic systems to high accuracy in a **fixed number of iterations**, not growing with n.

Multigrid is especially successful for symmetric systems. The key new ingredients are the (rectangular!) matrices R and I that change grids:

1. A ***restriction matrix R*** transfers vectors from the fine grid to the coarse grid.

2. The return step to the fine grid is by an ***interpolation matrix*** $\boldsymbol{I = I_{2h}^h}$.

3. The original matrix A_h on the fine grid is approximated by $\boldsymbol{A_{2h} = RA_hI}$ on the coarse grid. You will see how this A_{2h} is smaller and easier and faster than A_h. I will start with interpolation (a 7 by 3 matrix I that takes 3 v's to 7 u's):

Interpolation $\boldsymbol{Iv = u}$
$\boldsymbol{u}$ on the fine ($\boldsymbol{h}$) grid from
$\boldsymbol{v}$ on the coarse ($\boldsymbol{2h}$) grid
values are the $\boldsymbol{u}$'s.

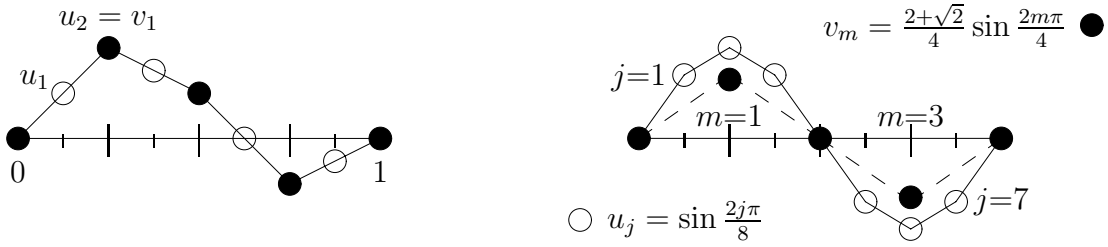
$$\frac{1}{2}\begin{bmatrix} 1 & & \\ 2 & & \\ 1 & 1 & \\ & 2 & \\ & 1 & 1 \\ & & 2 \\ & & 1 \end{bmatrix}\begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix} = \begin{bmatrix} v_1/2 \\ v_1 \\ v_1/2+v_2/2 \\ v_2 \\ v_2/2+v_3/2 \\ v_3 \\ v_3/2 \end{bmatrix} = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} \quad (1)$$

This example has $h = \frac{1}{8}$ on the interval $0 \le x \le 1$ with zero boundary conditions. The seven interior values are the u's. The grid with $2h = \frac{1}{4}$ has three interior v's.

Notice that u_2, u_4, u_6 from rows 2, 4, 6 are the same as v_1, v_2, v_3! Those coarse grid values v_j are just moved to the fine grid at the points $x = \frac{1}{4}, \frac{2}{4}, \frac{3}{4}$. The in-between values u_1, u_3, u_5, u_7 on the fine grid are coming from *linear interpolation* between $0, v_1, v_2, v_3, 0$:

Linear interpolation in rows 1, 3, 5, 7 $\qquad u_{2j+1} = \dfrac{1}{2}(v_j + v_{j+1})\,. \qquad (2)$

The odd-numbered rows of the interpolation matrix have entries $\frac{1}{2}$ and $\frac{1}{2}$. We almost always use grid spacings $h, 2h, 4h, \ldots$ with the convenient ratio 2. Other matrices I are possible, but linear interpolation is easy and effective. Figure 6.10a shows the new values u_{2j+1} (open circles) between the transferred values $u_{2j} = v_j$ (solid circles).



(a) Linear interpolation by $u = I_{2h}^h v$ (b) Restriction by $R_h^{2h} u = \frac{1}{2}(I_{2h}^h)^{\mathrm{T}} u$

Figure 6.10: Interpolation to the h grid (7 u's). Restriction to the $2h$ grid (3 v's).

When the v's represent smooth errors on the coarse grid (because Jacobi or Gauss-Seidel has been applied on that grid), interpolation gives a good approximation to the errors on the fine grid. A practical code can use 8 or 10 grids.

The second matrix we need is a **restriction matrix** $\boldsymbol{R_h^{2h}}$. It transfers u on a fine grid to v on a coarse grid. One possibility is the one-zero "injection matrix" that simply copies v from the values of u at the same points on the fine grid. This ignores the odd-numbered fine grid values u_{2j+1}. Another possibility (which we adopt) is the **full weighting operator** $\boldsymbol{R}$ that comes from transposing I_{2h}^h.

Fine grid h to coarse grid 2h by a restriction matrix $\boldsymbol{R_h^{2h} = \frac{1}{2}(I_{2h}^h)^{\mathrm{T}}}$

Full weighting $\boldsymbol{Ru = v}$
Fine grid $\boldsymbol{u}$ to coarse grid $\boldsymbol{v}$

$$\frac{1}{4}\begin{bmatrix} 1 & 2 & 1 & & & & \\ & & 1 & 2 & 1 & & \\ & & & & 1 & 2 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ u_5 \\ u_6 \\ u_7 \end{bmatrix} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}. \quad (3)$$

The effect of this restriction matrix is shown in Figure 6.10b. We intentionally chose the special case in which $u_j = \sin(2j\pi/8)$ on the fine grid (*open circles*). Then v on the coarse grid (*dark circles*) is also a pure sine vector. *But the frequency is doubled* (a full cycle in 4 steps). So a smooth oscillation on the fine grid becomes "half as smooth" on the coarse grid, which is the effect we wanted.

Interpolation and Restriction in Two Dimensions

Coarse grid to fine grid in two dimensions from bilinear interpolation: Start with values $v_{i,j}$ on a square or rectangular coarse grid. Interpolate to fill in $u_{i,j}$ by a sweep (interpolation) in one direction followed by a sweep in the other direction. We could allow two spacings h_x and h_y, but one meshwidth h is easier to visualize. A horizontal sweep along row i of the coarse grid (which is row $2i$ of the fine grid) will

©2006 Gilbert Strang

fill in values of u at odd-numbered columns $2j+1$ of the fine grid:

$$\textbf{Horizontal sweep} \qquad u_{2i,2j} = v_{i,j} \quad \text{and} \quad u_{2i,2j+1} = \frac{1}{2}(v_{i,j} + v_{i,j+1}) \quad \text{as in 1D.} \tag{4}$$

Now sweep vertically, up each column of the fine grid. Interpolation will keep those values (4) on even-numbered rows $2i$. It will average those values to find $\boldsymbol{u = I2D\, v}$ on the fine-grid odd-numbered rows $2i+1$:

$$\begin{aligned} &\textbf{Vertical sweep} && u_{2i+1,2j} &&= (v_{i,j} + v_{i+1,j})/2 \\ &\textbf{Averages of (4)} && u_{2i+1,2j+1} &&= (v_{i,j} + v_{i+1,j} + v_{i,j+1} + v_{i+1,j+1})/4\,. \end{aligned} \tag{5}$$

The entries in the tall thin coarse-to-fine interpolation matrix $I2\text{D}$ are 1, $\frac{1}{2}$, and $\frac{1}{4}$.

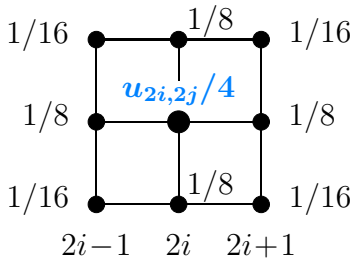
The full weighting fine-to-coarse restriction operator $R2\text{D}$ is the *transpose* $I2\text{D}^{\text{T}}$, multiplied by $\frac{1}{4}$. That factor is needed (like $\frac{1}{2}$ in one dimension) so that a constant vector of 1's will be restricted to a constant vector of 1's. (The entries along each row of the wide matrix R add to 1.) This restriction matrix has entries $\frac{1}{4}$, $\frac{1}{8}$, and $\frac{1}{16}$ and *each coarse-grid value v is a weighted average of nine fine-grid values u*:

Restriction matrix $\boldsymbol{R = \frac{1}{4}\, I^{\text{T}}}$

Row $\boldsymbol{i, j}$ of $\boldsymbol{R}$ produces $\boldsymbol{v_{i,j}}$

$\boldsymbol{v_{i,j}}$ uses $\boldsymbol{u_{2i,2j}}$ and 8 neighbors

The nine weights add to 1



You can see how a sweep along each row with weights $\frac{1}{4}, \frac{1}{2}, \frac{1}{4}$, followed by a sweep down each column, gives the nine coefficients in that "restriction molecule." Its matrix $R2\text{D}$ is an example of a *tensor product* or *Kronecker product* $\texttt{kron}(R, R)$. A 3 by 7 matrix R in one dimension becomes a 9 by 49 restriction matrix $R2\text{D}$ in two dimensions.

Now we can transfer vectors between grids. We are ready for the ***geometric multigrid*** method, when the geometry is based on spacings h and $2h$ and $4h$. The idea extends to triangular elements (each triangle splits naturally into four similar triangles). The geometry can be more complicated than our model on a square.

When the geometry becomes too difficult, or we are just given a matrix, we turn (in the final paragraph) to ***algebraic multigrid***. This will imitate the multi-scale idea, but it works directly with $Au = b$ and not with any underlying geometric grid.

A Two-Grid V-Cycle (a v-cycle)

Our first multigrid method only involves two grids. The iterations on each grid can use Jacobi's $I - D^{-1}A$ (possibly weighted by $\omega = 2/3$ as in the previous section) or Gauss-Seidel. For the larger problem on the fine grid, iteration converges slowly to

the low frequency smooth part of the solution u. *The multigrid method transfers the current residual $r_h = b - Au_h$ to the coarse grid.* We iterate a few times on that $2h$ grid, to approximate the coarse-grid error by E_{2h}. Then interpolate back to E_h on the fine grid, make the correction to $u_h + E_h$, and begin again.

This fine-coarse-fine loop is a **two-grid V-cycle**. We call it a **v-cycle** (small v). Here are the steps (remember, the error solves $A_h(u - u_h) = b_h - A_h u_h = r_h$):

1. **Iterate** on $A_h u = b_h$ to reach u_h (say 3 Jacobi or Gauss-Seidel steps).

2. **Restrict** the residual $r_h = b_h - A_h u_h$ to the coarse grid by $r_{2h} = R_h^{2h} r_h$.

3. **Solve** $A_{2h} E_{2h} = r_{2h}$ (or come close to E_{2h} by 3 iterations from $E = 0$).

4. **Interpolate** E_{2h} back to $E_h = I_{2h}^h E_{2h}$. Add E_h to u_h.

5. **Iterate** 3 more times on $A_h u = b_h$ starting from the improved $u_h + E_h$.

Steps **2-3-4** give the restriction-coarse solution-interpolation sequence that is the heart of multigrid. Recall the three matrices we are working with:

$$\begin{aligned} \boldsymbol{A} &= \boldsymbol{A_h} &&= \textit{original matrix} \\ \boldsymbol{R} &= \boldsymbol{R_h^{2h}} &&= \textit{restriction matrix} \\ \boldsymbol{I} &= \boldsymbol{I_{2h}^h} &&= \textit{interpolation matrix}. \end{aligned}$$

Step **3** involves a fourth matrix A_{2h}, to be defined now. A_{2h} is square and it is smaller than the original A_h. In words, we want to "project" the larger matrix A_h onto the coarse grid. There is a natural choice! The variationally correct A_{2h} comes directly and beautifully from R and A and I:

$$\textit{The coarse grid matrix is } \boldsymbol{A_{2h} = R_h^{2h} A_h I_{2h}^h = RAI}. \tag{6}$$

When the fine grid has $N = 7$ interior meshpoints, the matrix A_h is 7 by 7. Then the coarse grid matrix RAI is (3 by 7)(7 by 7)(7 by 3) = **3 by 3**.

Example In one dimension, $A = A_h$ might be the second difference matrix K/h^2. Our first example came from $h = \frac{1}{8}$. Now choose $h = \frac{1}{6}$, so that multigrid goes from five meshpoints inside $0 < x < 1$ to two meshpoints (I is 5 by 2 and R is 2 by 5): The neat multiplication (we will use it again later) is $RA_h = RK_5/h^2$:

$$RA = \frac{1}{4}\begin{bmatrix} 1 & 2 & 1 & & \\ & & 1 & 2 & 1 \end{bmatrix} \frac{1}{h^2} \begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & -1 & 2 & -1 & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{bmatrix} = \frac{1}{(2h)^2}\begin{bmatrix} 0 & \mathbf{2} & 0 & \mathbf{-1} & 0 \\ 0 & \mathbf{-1} & 0 & \mathbf{2} & 0 \end{bmatrix}. \tag{7}$$

©2006 Gilbert Strang

A natural choice for A_{2h} on the coarse grid is $K_2/(2h)^2$ and multigrid makes this choice:

$$\textbf{Coarse grid matrix} \qquad A_{2h} = RAI = \frac{1}{(2h)^2}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}. \tag{8}$$

The reader will appreciate that the $I^{\mathrm{T}}AI$ rule preserves symmetry and positive definiteness, when A has those properties. The rule arises naturally in Galerkin methods [–,page –], including the finite element method. Notice how the restriction operator R with the factor $\frac{1}{4}$ automatically adjusts $1/h^2$ to $1/(2h)^2$.

Steps **1** and **5** are necessary, but they are really outside the essential multigrid idea. The **smoother** is step **1**, the **post-smoother** is step **5**. Those are normal iterations for which weighted Jacobi or Gauss-Seidel is satisfactory.

The Errors e_h and E_h

Suppose we solve the coarse grid equation exactly at step **3**. Is the multigrid error correction E_h then equal to the true fine-grid error $e_h = u - u_h$? No, that is too much to expect! We have only solved the smaller problem on the coarse grid, not the full problem. But the connection between E_h and e_h is simple and crucial for understanding multigrid. We now track down the steps from E to e.

Four matrices multiply e. At step **2**, the residual $r = b - Au_h = A(u - u_h) = Ae$ multiplies by A. The restriction is multiplication by R. The solution step **3** multiplies by $A_{2h}^{-1} = (RAI)^{-1}$. The interpolation step **4** multiplies by I to find the correction E. *Altogether, E is* $\boldsymbol{IA_{21}^{-1}RA_he}$:

$$E = I(RAI)^{-1}RAe \quad \text{and we call this} \quad \boldsymbol{E = Se}\,. \tag{9}$$

When I is 5 by 2 and R is 2 by 5, that matrix S on the right side is 5 by 5. It can't be the identity matrix, since RAI and its inverse are only 2 by 2 (rank two). But $S = I(RAI)^{-1}RA$ has the remarkable property $S^2 = S$. This says that S *is the identity matrix on its 2-dimensional column space.* (Of course S is the zero matrix on its 3-dimensional nullspace.) $S^2 = S$ is easy to check:

$$S^2 = (I(RAI)^{-1}RA)(I(RAI)^{-1}RA) = S \quad \text{because } (RAI)^{-1}RAI \text{ disappears.} \tag{10}$$

So the multigrid correction $E = Se$ is not the whole error e, *it is a projection of* e. The new error is $e - E = e - Se = (I - S)e$. ***This matrix $\boldsymbol{I - S}$ is the two-grid operator.*** $I - S$ plays the same fundamental role in describing the multigrid steps **2**–**4** that the usual $M = I - P^{-1}A$ plays for each iteration in steps **1** and **5**:

$$\textbf{v-cycle matrix} = \boldsymbol{I - S} \qquad \textbf{iteration matrix} = \boldsymbol{I - P^{-1}A}\,.$$

Example (continued). The 5 by 5 matrix $A_h = K_5/h^2$ and the rectangular I and R led in (8) to $A_{2h} = K_2/(2h)^2$. To find $S = IA_{2h}^{-1}RA_h$, we multiply (7) by IA_{2h}^{-1}:

$$A_{2h}^{-1}RA_h = \begin{bmatrix} 0 & \mathbf{1} & 0 & \mathbf{0} & 0 \\ 0 & \mathbf{0} & 0 & \mathbf{1} & 0 \end{bmatrix}$$

Now multiply by I to find S

$$S = \begin{bmatrix} 0 & 1/2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1/2 & 0 & 1/2 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1/2 & 0 \end{bmatrix}. \qquad (11)$$

The eigenvalues of this S are $1, 1, 0, 0, 0$. If you square S, you recover $S^2 = S$. With its three columns of zeros, the nullspace of S contains all fine-grid vectors of the form $(e_1, 0, e_3, 0, e_5)$. Those are vectors that don't appear on the coarse grid. If the error e had this form, then $E = Se$ would be zero (no improvement from multigrid). *But we don't expect a large component of those high frequency vectors in e, because of the smoothing.*

The column space of S contains column 2 $= (\frac{1}{2}, 1, \frac{1}{2}, 0, 0)$ and column 4 $= (0, 0, \frac{1}{2}, 1, \frac{1}{2})$. These are "mixed-frequency vectors." We do expect them to appear in e, because the smoothing step didn't remove them. But these are vectors for which $E = Se = e$ and they are the errors that multigrid catches! After step **4** they are gone.

Let me say this in another way. Because $S^2 = S$, the only eigenvectors are $\lambda = 0$ and $\lambda = 1$. (If $Su = \lambda u$ we always have $S^2u = \lambda^2 u$. Then $S^2 = S$ gives $\lambda^2 = \lambda$.) Our example has $\lambda = 1, 1, 0, 0, 0$. The eigenvalues of $I - S$ are $0, 0, 1, 1, 1$. The eigenvectors e reveal what multigrid is doing:

E = Se = 0 In this case multigrid gives no improvement. The correction E added to u_h in step **4** is zero. IN the example, this happens for errors $e = (e_1, 0, e_3, 0, e_5)$ that are zero on the coarse grid where step **3** is working.

E = Se = e In this case multigrid is perfect. The correction E_h added to u_h in step **4** is the whole error e_h. In the example, two eigenvectors of S for $\lambda = 1$ are $e = (1, 2, 2, 2, 1)$ and $e = (1, 2, 0, -2, -1)$. Those have large low-frequency components. They oscillate up and down only once and twice. They are in the column space of I. They are no perfect sines, but an important part of the low-frequency error is caught and removed. The number of independent vectors with $Se = e$ is the number of coarse gridpoints (here 2). That measures the A_{2h} problem that step **3** deals with. It is the rank of S and also I. The other $5 - 2$ gridpoints account for the nullspace of S, where $E = Se = 0$ means no improvement from multigrid.

Note The "high-frequency" vectors $(u_1, 0, u_3, 0, u_5)$ with $Su = 0$ are *not exactly* combinations of the last three discrete sines y_3, y_4, y_5. The frequencies are mixed by S, as equations (18–19) will clearly show. The exact statements are *column space of S = column space of I* and *nullspace of S = nullspace of RA*. The mixing of frequencies does not affect our main point: **Iteration handles the high frequencies and multigrid handles the low frequencies.**

You can see that a perfect smoother followed by perfect multigrid (exact solution at step **3**) would leave no error. In reality, this will not happen. Fortunately, a careful

©2006 Gilbert Strang

(not so simple) analysis will show that a multigrid cycle with good smoothing can reduce the error by a constant factor ρ that is **independent of h**:

$$\|\textbf{error after step 5}\| \le \rho \|\textbf{error before step 1}\| \quad \text{with} \quad \rho < 1\,. \tag{12}$$

A typical value is $\rho = \frac{1}{10}$. Compare with $\rho = .99$ for Jacobi alone. This is the Holy Grail of numerical analysis, to achieve a convergence factor ρ (a spectral radius of the overall iteration matrix) that *does not move up to* 1 *as* $h \to 0$. We can achieve a given relative accuracy in a fixed number of cycles. Since each step of each cycle requires only $O(n)$ operations on sparse problems of size n, **multigrid is an $O(n)$ algorithm**. This does not change in higher dimensions.

There is a further point about the number of steps and the accuracy. The user may want the solution error e to be as small as the discretization error (when the original differential equation was replaced by $Au = b$). In our examples with second differences, this demands that we continue until $e = O(h^2) = O(N^{-2})$. In that case we need more than a fixed number of v-cycles. To reach $\rho^k = O(N^{-2})$ requires $k = O(\log N)$ cycles. Multigrid has an answer for this too.

Instead of repeating v-cycles, or nesting them into V-cycles or W-cycles, it is better to use **full multigrid**: FMG cycles are described below. Then the operation count comes back to $O(n)$ even for this higher required accuracy $e = O(h^2)$.

V-Cycles and W-Cycles and Full Multigrid

Clearly multigrid need not stop at two grids. If it did stop, it would miss the remarkable power of the idea. The lowest frequency is still low on the $2h$ grid, and that part of the error won't decay quickly until we move to $4h$ or $8h$ (or a very coarse $512h$).

The two-grid v-cycle extends in a natural way to more grids. It can go down to coarser grids $(2h, 4h, 8h)$ and back up to $(4h, 2h, h)$. This nested sequence of v-cycles is a **V-cycle** (capital V). *Don't forget that coarse grid sweeps are much faster than fine grid sweeps.* Analysis shows that time is well spent on the coarse grids. So the **W-cycle** that stays coarse longer (Figure 6.11b) is generally superior to a V-cycle.

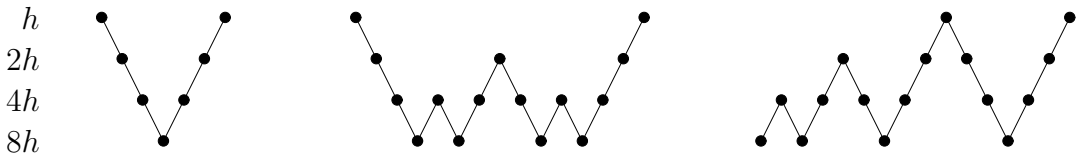


Figure 6.11: V-cycles and W-cycles and FMG use several grids several times.

The **full multigrid cycle** in Figure 6.11c is asymptotically better than V or W. *Full multigrid starts on the coarsest grid.* The solution on the $8h$ grid is interpolated to provide a good initial vector u_{4h} on the $4h$ grid. A v-cycle between $4h$ and $8h$ improves it. Then interpolation predicts the solution on the $2h$ grid, and a deeper

V-cycle makes it better (using $2h, 4h, 8h$). Interpolation of that improved solution onto the finest grid gives an excellent start to the last and deepest V-cycle.

The operation counts for a deep V-cycle and for full multigrid are certainly greater than for a two-grid v-cycle, but *only by a constant factor*. That is because the count is divided by a power of 2 every time we move to a coarser grid. For a differential equation in d space dimensions, we divide by 2^d. The cost of a V-cycle (as deep as we want) is less than a fixed multiple of the v-cycle cost:

$$\textbf{V-cycle cost} < \left(1 + \frac{1}{2^d} + \left(\frac{1}{2^d}\right)^2 + \cdots\right)\textbf{v-cycle cost} = \frac{2^d}{2^d - 1}\,\textbf{v-cycle cost}\,. \quad (13)$$

Full multigrid is no more than a series of inverted V-cycles, beginning on a very coarse mesh. By the same reasoning that led to (13),

$$\textbf{Full multigrid cost} \;<\; \frac{2^d}{2^d - 1}\,\textbf{V-cycle cost} \;<\; \left(\frac{2^d}{2^d - 1}\right)^2 \textbf{v-cycle cost}\,. \quad (14)$$

And the method works in practice. But good programming is required.

Multigrid Matrices

For a 3-grid V-cycle, what matrix S_3 corresponds to the 2-grid v-cycle projection $S = I(RAI)^{-1}RA$? No smoothing is involved here. S and S_3 only project to coarser problems. By itself, S_3 will not be a good solver without smoothers.

To construct A_{4h}, replace the matrix $A_{2h} = RAI$ on the middle grid by using a coarse restriction $R_c = R_{2h}^{4h}$ that transfers down to the $4h$ grid, and an interpolation $I_c = I_{4h}^{2h}$ that comes back to $2h$:

$$\textbf{Very coarse matrix} \qquad \boldsymbol{A_{4h} = R_c A_{2h} I_c}\,. \quad (15)$$

If $h = \frac{1}{16}$, that product is $(3 \times 7)(7 \times 7)(7 \times 3)$. The 3 by 3 problem uses A_{4h} on the $4h = \frac{1}{4}$ grid (with three interior unknowns). Then $S_3 = (S_3)^2$ is the matrix that goes down two grids, solves the very coarse problem by A_{4h}^{-1}, and comes back up:

$$\boldsymbol{E_3 = S_3 e} = \textbf{Error removed} \qquad S_3 = I_{2h}^h I_{4h}^{2h} A_{4h}^{-1} R_{2h}^{4h} R_h^{2h} A\,. \quad (16)$$

The error that remains after an unsmoothed V-cycle will be $(I - S_3)e$. On the $h = \frac{1}{16}$ grid, there are 15 frequencies in the error e. Only the lowest 3 are (approximately) removed by $S_3 e$. We have only solved a 3 by 3 problem with A_{4h}. It is the *smoothers*, on the fine h grid and the middle $2h$ grid, that reduce the high and middle frequency errors in the solution.

Note to myself: $S_{4h} = I_c A_{4h}^{-1} R_c A_{2h} = I_c A_{4h}^{-1} R_c RAI = 7 \times 7$ has I on right instead of left. Still $S_{4h}^2 = S_{4h}$.

©2006 Gilbert Strang

Numerical Experiments

The real test of multigrid effectiveness is numerical! Its k-step approximation e_k should approach zero and the graphs will show how quickly this happens. An initial guess u_0 includes an error e_0. Whatever iteration we use, we are trying to drive u_k to u, and e_k to zero. The multigrid method jumps between two or more grids, so as to converge more quickly, and our graphs will always show the error on the fine grid.

We can work with the equation $Ae = 0$, whose solution is $e = 0$. Following [–], Figure 6.____a displays an initial error with low and high frequencies:

$$(e_0)_j = \sin 4j\pi h + \sin 20j\pi h \quad \text{with} \quad h = \frac{1}{32}\,.$$

We draw continuous functions $\sin 4\pi x$ and $\sin 20\pi x$ rather than 31 discrete values. Figure 6.____b shows the error vector e_3 after three fine-grid sweeps of weighted Jacobi (with $\omega = \frac{2}{3}$). As expected, the high frequency component has greatly decayed. The error $e_3 = (I - P^{-1}A)^3 e_0$ is much smoother than e_0. For our second difference matrix A_h (better known as K), the Jacobi preconditioner from Section 6.2 simply has $P^{-1} = \frac{1}{2}\omega I$. We can ignore the factors h^2 that divide K and P, because they cancel.

Figure 6.12: The initial error and the error e_3 after three weighted Jacobi relaxations.

Now multigrid begins. The current fine-grid residual is $r_h = -A_h e_3$. After restriction to the coarse grid it becomes r_{2h}. Three weighted Jacobi iterations on the coarse grid error equation $A_{2h}E_{2h} = r_{2h}$ start with the guess $E_{2h} = 0$. That produces...

ADD V-CYCLE AND FMG EXPERIMENTS

Figure 6.13: The fine-grid error after 3 sweeps on the coarse grid (a) and then the fine grid (b).

Eigenvector Analysis

The reader will recognize that one matrix (like $I - S$ for a v-cycle) can describe each multigrid method. The eigenvalues of a full multigrid matrix would be nice to know, but they are usually impractical to find. Numerical experiments build confidence. Computation also provides a *diagnostic* tool, to locate where convergence is stalled and a change is needed (often in the boundary conditions). If we want a *predictive* tool, the best is **modal analysis**.

The key idea is to watch the Fourier modes. In our example those are discrete sines, because of the boundary conditions $u(0) = u(1) = 0$. We will push this model problem all the way, to see what the multigrid matrix $I - S$ does to those sine vectors.

The final result in (18–19) shows why multigrid works and it also shows how *pairs of frequencies are mixed*. The eigenvectors are mixtures of two frequencies.

The frequencies that mix together are k and $N+1-k$. The discrete sines with those frequencies are y_k and Y_k:

$$y_k = \left(\sin\frac{k\pi}{N+1}, \sin\frac{2k\pi}{N+1}, \dots\right) \text{ and } Y_k = \left(\sin\frac{(N+1-k)\pi}{N+1}, \sin\frac{2(N+1-k)\pi}{N+1}, \dots\right).$$

Where y_k goes up and down k times, Y_k does that $N+1-k$ times.

There is something neat you have to see. **Y_k and y_k have the same components except for alternating signs!** Cancel $N+1$ with $N+1$ in those components of Y_k:

$$Y_k = \left(\sin\left[\pi - \frac{k\pi}{N+1}\right], \sin\left[2\pi - \frac{2k\pi}{N+1}\right], \dots\right) = \left(+\sin\frac{k\pi}{N+1}, -\sin\frac{2k\pi}{N+1}, \dots\right) \quad (17)$$

For our model problem with second differences, we can report the result of multiplying by $S = I(RAI)^{-1}RA$. In fact the true multigrid matrix is $I - S$, to give the error $e - E = (I - S)\,e$ that remains after steps **2**, **3**, **4**. Seeing $I - S$ is even better:

One v-cycle ($2k \le N+1$)
Smooth errors reduced
Frequencies mixed

$$(I - S)\,y_k = \frac{1}{2}\left(1 - \cos\frac{k\pi}{N+1}\right)(y_k + Y_k) \quad (18)$$

$$(I - S)Y_k = \frac{1}{2}\left(1 + \cos\frac{k\pi}{N+1}\right)(y_k + Y_k) \quad (19)$$

Such beautiful formulas can't be expected in more complicated problems, and we seize this chance to make four key points:

1. **Low frequencies like $k = 1, 2, 3$ are greatly reduced by multigrid.** The cosine in equation (18) will be near 1. The factor $(1 - \cos\frac{k\pi}{N+1})$ is very small, of order $(k/N)^2$. This shows how multigrid is powerful in nearly killing the low frequencies. These are exactly the frequencies on which Jacobi and Gauss-Seidel will stall in the smoothing iterations.

2. **The pair of sines y_k and Y_k is mixed together by multigrid.** We will see that the restriction R and interpolation I are responsible. Those are not like the square matrix A, which keeps frequencies separate (because the sines are its eigenvectors). Aliasing appears for rectangular matrices!

3. **The combinations $e = y_k + Y_k$ are eigenvectors of $I - S$ with $\lambda = 1$.** Just add equations (18) and (19), to get $(I - S)e = e$. Since Y_k has the same components as y_k with alternating signs, $y_k + Y_k$ has the correct form $(e_1, 0, e_3, 0, \dots)$. Those are the vectors that we discovered earlier in the nullspace ($\lambda = 0$) of S. They are not touched by $I - S$ since $Se = 0$.

©2006 Gilbert Strang

4. The other eigenvectors of $I-S$ are just Sy_k. We know that $(I-S)Sy_k = 0$ because $S = S^2$. In our example with $N = 5$, the vectors Sy_1 and Sy_2 are multiples of $(1,2,2,2,1)$ and $(1,2,0,-2,1)$ that we found explicitly. Multigrid removes them from the error.

According to (18), these vectors Sy_k are combinations of y_k and Y_k. To find a good combination, multiply (18) and (19) by $(1+\cos\frac{k\pi}{N+1})$ and $(1-\cos\frac{k\pi}{N+1})$. The right-hand sides are now the same and we subtract:

$$\boldsymbol{S^* = e^*}\ (I-S)e^* = (I-S)\left[\left(1+\cos\frac{k\pi}{N+1}\right)y_k - \left(1-\cos\frac{k\pi}{N+1}\right)Y_k\right] = 0 \quad (20)$$

These mixtures e^* in square brackets are completely killed ($\lambda = 0$) by multigrid. A smooth error vector e_h (after Jacobi iterations have reduced its high frequency components) has only small components along the eigenvectors $y_k + Y_k$. Multigrid doesn't touch those pieces ($\lambda = 1$). The larger components along the Z_k will die.

The Restriction R Produces Aliasing

To complete this analysis we have to see where and how a pair of frequencies is mixed. The aliasing comes from the restriction matrix $R = R_h$, when both vectors y_k^h and Y_k^h lead to multiples of the same output vector y_k^{2h}:

$$Ry_k^h = \left(\frac{1}{2} - \frac{1}{2}\cos\frac{k\pi}{N+1}\right)y_k^{2h} \quad \text{and} \quad RY_k^h = \left(-\frac{1}{2} - \frac{1}{2}\cos\frac{k\pi}{N+1}\right)y_k^{2h}. \qquad (21)$$

You see the aliasing by R. We cannot hope to decide the input y_k or Y_k from these outputs. This is normal for a short wide matrix. (The matrix R is 3 by 7 and 2 by 5 in our examples.) The coarse mesh output has only about half as many components as the fine mesh input.

The transpose of R does the opposite. Where R mixes two inputs y_k^h and Y_k^h into one output, the interpolation matrix I sends one input frequency on the coarse grid into a pair of frequencies on the fine grid:

$$2\,I_{2h}^h y_k^{2h} = \left(1+\cos\frac{k\pi}{N+1}\right)y_k^h - \left(1-\cos\frac{k\pi}{N+1}\right)Y_k^h. \qquad (22)$$

Interpolation of a smooth vector (low k) on a coarse grid will excite an oscillatory mode Y_k^h (high k) on the fine grid. But those oscillations have small amplitude, because the cosine of $k\pi/(N+1)$ is near 1.

The key formulas (18) and (19) that describe multigrid come from assembling (21) for R, (22) for I, and the known eigenvalues for A_h and A_{2h}. I think the calculation of S in (11) shows this best. Its zero columns put $y_k + Y_k$ in its nullspace. The nonzero columns in S come from the interpolation matrix I. So Se captures a part of the whole error e. *Multigrid solves a projection of the original problem.*

Example completed It would be useless to repeat steps **2**, **3**, **4** with no smoothing in between. Nothing would change! The untouched vectors $e = y_k + Y_k$ with $(I - S)e = e$ will still be untouched. It is the smoothing matrix $M = I - P^{-1}A$ that must reduce these errors.

If we apply Jacobi with weight $w = \frac{2}{3}$ at steps **1** and **5**, then $M = I - \frac{1}{3}A$. The overall matrix for all steps **1** to **5** will be $M(I - S)M$. The eigenvalues of that matrix will decide the success (or not) of multigrid. To my amazement, the 5 by 5 matrix $M(I - S)M$ has a *triple eigenvalue of* $\frac{1}{9}$!

The three eigenvalues $\lambda = 1$ of $I - S$ are reduced to $\lambda = \dfrac{1}{9}$ for $M(I - S)M$.

The largest eigenvalue of M is .91—you see the value of multigrid. I hope you will try $\texttt{eig}(M * M * (I - S) * M * M)$ with double smoothing (see Problems 9–12).

Fourier Modal Analysis

Pure modal analysis neglects the boundaries completely. It assumes an infinite grid! The vectors y_k in the example were sines because of the boundary conditions. When boundaries are gone, the y_k are replaced by infinitely long vectors y_ω coming from complex exponentials. Now there is a continuum of frequencies ω:

$$y_\omega = (\dots, e^{-2i\omega}, e^{-i\omega}, 1, e^{i\omega}, e^{2i\omega}, \dots) \quad \text{with} \quad -\pi \le \omega \le \pi\,. \tag{23}$$

We need infinite matrices K_∞ to multiply these infinite vectors. Second differences $-1, 2, -1$ appear on *all rows forever*. The key is that each y_ω is an eigenvector of K_∞, with eigenvalue $\lambda = 2 - 2\cos\omega$:

$$K_\infty\, y_\omega = (2 - 2\cos\omega)\, y_\omega \quad \text{because} \quad -e^{i\omega(n+1)} - e^{i\omega(n-1)} = -2\cos\omega\, e^{i\omega n}\,. \tag{24}$$

This tells us the action of $A_h = K_\infty/h^2$. It also tells us about A_{2h}, when the coarse mesh changes h^2 to $(2h)^2$ and ω to 2ω.

The restriction matrix R still introduces aliasing. The frequencies that mix together are now ω and $\omega + \pi$. Notice how increasing ω by π produces a factor $e^{i\pi n} = (-1)^n$ with alternating signs. This is exactly what we saw for y_k and Y_k. This time it is $y_\omega - y_{\omega+\pi}$ that has all zeros in its even-numbered components.

This pure Fourier analysis will go all the way to formulas for the infinite $(I - S)y_\omega$ and $(I - S)y_{\omega+\pi}$, just like equations (18) and (19). In that finite case, those equations explained why multigrid succeeds. They do the same in the infinite case.

Let me emphasize why this pure modal analysis (with no boundaries and constant coefficients) was mentioned. It allows Fourier to work freely. The differential equation $\text{div}(c(x, y)\,\text{grad}\,u) = f(x, y)$ on a general region would lead to giant complications in the eigenvectors for multigrid—impossible to find them. But if we fix $c =$ constant and ignore boundaries, those "interior eigenvectors" return to simple combinations of y_ω and $y_{\omega+\pi}$. That leaves difficulties associated with the boundary conditions, which Fourier doesn't easily resolve.

©2006 Gilbert Strang

Algebraic Multigrid

We close this section with a few words about **algebraic multigrid**, when the problem comes as a system of equations $Au = b$. *There is no grid in the background.* We need to find replacements for the key ideas on which geometric multigrid was based: *smooth vectors, connected nodes, coarse meshes.* Replacing the first two is fairly easy, rescaling A_h to A_{2h} on a (nonexistent) coarse mesh is less clear.

1. **Smooth vectors.** In geometric multigrid these are vectors for which the norms of u and Au are comparable. That will be our indication of a smooth vector. High frequencies in u would be greatly amplified by A (just as the second derivative amplifies $\sin kt$ by k^2). For low frequencies, Multigrid works on smooth errors and Jacobi doesn't.

2. **Connected nodes.** On a grid, neighboring nodes are reflected by a nonzero entry in A. When there is no grid, we look directly at the matrix. Its significant nonzero entries A_{ij} (magnitude greater than αA_{ii}, say with $\alpha = \frac{1}{10}$) tell us when we should think of the "nodes" i and j as being connected.

3. **Coarse subset of nodes.** Each significant entry A_{ij} indicates that the value of u_j *strongly influences* the value of u_i. Probably the errors e_j and e_i are comparable, when the error is smooth. We don't need both i and j in the "coarse set C" of nodes. On a grid they would be neighbors, not both in C.

   But if i is *not* in C, then every j that strongly influences i should either be in C or be strongly influenced by another J that *is* in C. This heuristic rule is discussed more fully in [–], with an algorithm for constructing C. (In general, too many coarse unknowns are better than too few.)

The excellent book [–] also constructs the coarse-to-fine interpolation matrix I. This starts with the errors E_j for j in C, and leaves them unchanged. If i is not in C, the interpolated value E_i at step **2** of multigrid will be a *weighted combination* of the E_j that do have j in C. In our one-dimensional model problem, that weighted combination was the average of the two neighbors of i. That model problem certainly had a grid!

The interpolating combination will give greatest weight to the e_j for which j in C strongly influences i. But there may be smaller entries A_{ij} that cannot be completely ignored. The final decision on the weights for each interpolated value is more subtle than a simple average. Algebraic multigrid is more expensive than geometric multigrid, but it applies to a much wider range of sparse matrices A (and the software can control AMG without us). We still expect a "smoothing + multigrid" combination that is close to $O(n)$ steps for an accurate solution of $Au = b$.

Let me mention an important contrast between solid mechanics and fluid mechanics. For solids, the original finite element grid is already relatively coarse. We are typically looking for "engineering accuracy" and we don't have fine scale motions to resolve. Multigrid is not such a common choice for structural problems (elimination

is simpler). Fluids do have fine scales so that multigrid becomes a natural idea, not only numerically but physically. Of course fluids don't generally present symmetric matrices, because of the convective terms, and finite element methods may require "upwind adjustments." The analysis of multigrid convergence becomes harder for fluids, just when a multiscale approach becomes attractive.

Problem Set 6.3

1 What is the 3 by 7 matrix $R_{\textbf{injection}}$ that copies v_1, v_2, v_3 from u_2, u_4, u_6 and ignores u_1, u_3, u_5, u_7?

2 Write down a 9 by 4 bilinear interpolation matrix I that uses the four grid values at the corners of a square (side $2h$) to produce nine values at the (h) gridpoints. What constant should multiply the transpose of I to give a one-element restriction matrix R?

3 In Problem 2, the four small squares (side h) subdivide into 16 smaller squares (side $h/2$). How many rows and columns in the interpolation matrix $I_{h/2}$?

4 If A is the 5-point discrete Laplace matrix (with $-1, -1, 4, -1, -1$ on a typical row) what is a typical row of $A_{2h} = RAI$ using bilinear interpolation as in Problems 2–3?

5 Suppose A comes from the 9-point stencil (8/3 surrounded by eight entries of $-1/3$). What is now the stencil for $A_{2h} = RAI$?

6 Verify $Ry_k^h = \frac{1}{2}(1+\cos\frac{k\pi}{N+1})y_k^{2h}$ in equation (23) for the linear restriction matrix R applied to discrete sines y_k^h with $k \le \frac{1}{2}(N+1)$.

7 Show that $RY_k^h = \frac{1}{2}(-1-\cos\frac{k\pi}{N+1})y_k^{2h}$ in equation (23) for the "complementary" discrete sines $Y_k^h = y_{N+1-k}$. Now $N+1-k > \frac{1}{2}(N+1)$.

8 Verify equation (24) for linear interpolation applied to the discrete sines y_k^h with $k \le \frac{1}{2}(N+1)$.

9 With $h = \frac{1}{8}$, use the 7 by 3 and 3 by 7 matrices I and R in equations (1–2) to find the 3 by 3 matrix $A_{2h} = RAI$ with $A = K_7/h^2$.

10 Continue Problem 9 to find the 7 by 7 matrix $S = I(RAI)^{-1}RA$. Verify that $S^2 = S$, and that S has the same nullspace as RA and the same column space as I. What are the seven eigenvalues of S?

11 Continue Problem 10 to find (by MATLAB) the multigrid matrix $I - S$ and the presmoothed/postsmoothed matrix MSM, where M is the Jacobi smoother $I - \omega D^{-1}A = I - \frac{1}{3}K_7$ with $\omega = \frac{2}{3}$. Find the eigenvalues of MSM!

12 Continue Problem 11 to find the eigenvalues of M^2SM^2 with two Jacobi smoothers ($\omega = \frac{2}{3}$) before and after the grid changes.

©2006 Gilbert Strang

13 With unweighted Jacobi ($\omega = 1$ and $M = I - \frac{1}{2}K_7$) find MSM and its eigenvalues. Is the weighting useful?

14 Starting from $h = \frac{1}{16}$ and the second difference matrix $A = K_{15}/h^2$, compute the projected 7 by 7 matrix $A_{2h} = RAI$ and the doubly projected 3 by 3 matrix $A_{4h} = R_c A_{2h} I_c$. Use linear interpolation matrices I and I_c, and restriction matrices $R = \frac{1}{2}I^{\mathrm{T}}$ and $R_c = \frac{1}{2}I_c^{\mathrm{T}}$.

15 Use A_{4h} from Problem 14 to compute the projection matrix S_3 in (16). Verify that this 15 by 15 matrix has rank 3, and that $(S_3)^2 = S_3$.

16 The weighted Jacobi smoothers are $M_h = I_{15} - \frac{1}{3}K_{15}$ and $M_{2h} = I_7 - \frac{1}{3}K_7$. With MATLAB, compute the smoothed error reduction matrix V_3 and its eigenvalues:

$$V_3 = M_h I M_{2h}(I_7 - S_{2h})M_{2h} R A M_h \, .$$

$S_{2h} = I_c A_{4h}^{-1} R_c A_{2h}$ is the projection matrix for the v-cycle within the V-cycle.

17 Compute the $(15 \times 7)(7 \times 3)$ linear interpolation matrix $I_{2h}^h I_{4h}^{2h}$. What is the restriction matrix?

 ©2006 Gilbert Strang

6.4 Krylov Subspaces and Conjugate Gradients

Our original equation is $Ax = b$. *The preconditioned equation is* $P^{-1}Ax = P^{-1}b$. When we write P^{-1}, we never intend that an inverse will be explicitly computed. P may come from Incomplete LU, or a few steps of a multigrid iteration, or "domain decomposition." Entirely new preconditioners are waiting to be invented.

The residual is $\boldsymbol{r_k = b - Ax_k}$**.** This is the error in $Ax = b$, not the error in x itself. An ordinary preconditioned iteration corrects x_k by the vector $P^{-1}r_k$:

$$Px_{k+1} = (P-A)x_k + b \quad \text{or} \quad Px_{k+1} = Px_k + r_k \quad \text{or} \quad \boldsymbol{x_{k+1} = x_k + P^{-1}r_k}\,. \tag{1}$$

In describing Krylov subspaces, I should work with $P^{-1}A$. **For simplicity I will only write** A**!** I am assuming that P has been chosen and used, and the preconditioned equation $P^{-1}Ax = P^{-1}b$ is given the notation $Ax = b$. The preconditioner is now $P = I$. Our new A is probably better than the original matrix with that name.

With $x_1 = b$, look first at two steps of the pure iteration $x_{j+1} = (I - A)x_j + b$:

$$x_2 = (I-A)b + b = \boldsymbol{2b - Ab} \qquad x_3 = (I-A)x_1 + b = \boldsymbol{3b - 3Ab + A^2b}. \tag{2}$$

My point is simple but important: $\boldsymbol{x_j}$ ***is a combination of*** $\boldsymbol{b, Ab, \ldots, A^{j-1}b}$**.** We can compute those vectors quickly, multiplying each time by a sparse A. Every iteration involves only one matrix-vector multiplication. Krylov gave a name to ***all*** combinations of those vectors, and he suggested that there might be better combinations than the particular choices x_j in (2).

Usually a different combination will come closer to the desired $x = A^{-1}b$.

Krylov subspaces

The linear combinations of $b, Ab, \ldots, A^{j-1}b$ form the jth Krylov subspace. This space depends on A and b. Following convention, I will write $\boldsymbol{\mathcal{K}_j}$ for that subspace and $\mathbf{K_j}$ for the matrix with those basis vectors in its columns:

$$\begin{aligned} &\textbf{Krylov matrix} \quad \mathbf{K_j} = [\ \boldsymbol{b} \quad \boldsymbol{Ab} \quad \boldsymbol{A^2b} \quad \ldots \quad \boldsymbol{A^{j-1}b}\ ]\,. \\ &\textbf{Krylov subspace} \quad \boldsymbol{\mathcal{K}_j} = \boldsymbol{\text{all combinations of } b, Ab, \ldots, A^{j-1}b}. \end{aligned} \tag{3}$$

Thus $\boldsymbol{\mathcal{K}_j}$ is the column space of $\mathbf{K_j}$. We want to choose the ***best combination*** as our improved x_j. Various definitions of "best" will give various x_j. Here are four different approaches to choosing a good x_j in $\boldsymbol{\mathcal{K}_j}$—this is the important decision:

1. The residual $r_j = b - Ax_j$ is orthogonal to $\boldsymbol{\mathcal{K}_j}$ (**Conjugate Gradients**).

2. The residual r_j has minimum norm for x_j in $\boldsymbol{\mathcal{K}_j}$ (**GMRES** and **MINRES**).

3. r_j is orthogonal to a different space $\boldsymbol{\mathcal{K}_j}(A^{\mathrm{T}})$ (**BiConjugate Gradients**).

©2006 Gilbert Strang

4. The error e_j has minimum norm (**SYMMLQ**).

In every case we hope to compute the new x_j quickly from the earlier x's. If that step only involves x_{j-1} and x_{j-2} (**short recurrence**) it is especially fast. Short recurrences happen for conjugate gradients and symmetric positive definite A. The BiCG method is a natural extension of short recurrences to unsymmetric A (using two Krylov spaces). A stabilized version called **BiCGStab** chooses x_j in $A^{\mathrm{T}}\boldsymbol{\mathcal{K}_j}(A^{\mathrm{T}})$.

As always, computing x_j can be very unstable until we choose a decent basis.

Vandermonde Example

To follow each step of orthogonalizing the basis, and solving $Ax = b$ by conjugate gradients, we need a good example. It has to stay simple! I am happy with this one:

$$A = \begin{bmatrix} 1 & & & \\ & 2 & & \\ & & 3 & \\ & & & 4 \end{bmatrix} \qquad b = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \qquad Ab = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \qquad A^{-1}b = \begin{bmatrix} 1/1 \\ 1/2 \\ 1/3 \\ 1/4 \end{bmatrix}. \tag{4}$$

That constant vector b spans the Krylov subspace $\boldsymbol{\mathcal{K}_1}$. Then Ab, A^2b, and A^3b are the other basis vectors in $\boldsymbol{\mathcal{K}_4}$. They are the columns of $\mathbf{K_4}$, which we will name V:

Vandermonde matrix $$\mathbf{K_4} = V = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 \\ 1 & 3 & 9 & 27 \\ 1 & 4 & 16 & 64 \end{bmatrix}. \tag{5}$$

Those columns are constant, linear, quadratic, and cubic. The column vectors are independent but not at all orthogonal. The best measure of non-orthogonality starts by computing *inner products of the columns* in the matrix $V^{\mathrm{T}}V$. When columns are orthonormal, their inner products are 0 or 1. (The matrix is called Q, and the inner products give $Q^{\mathrm{T}}Q = I$.) Here $V^{\mathrm{T}}V$ is far from the identity matrix!

$$V^{\mathrm{T}}V = \begin{bmatrix} 4 & 10 & 30 & 100 \\ 10 & 30 & 100 & 354 \\ 30 & 100 & 354 & 1300 \\ 100 & 354 & 1300 & 4890 \end{bmatrix} \qquad \begin{aligned} 10 &= 1 + 2 + 3 + 4 \\ 30 &= 1^2 + 2^2 + 3^2 + 4^2 \\ 100 &= 1^3 + 2^3 + 3^3 + 4^3 \\ 1300 &= 1^5 + 2^5 + 3^5 + 4^5 \end{aligned}$$

The eigenvalues of this inner product matrix (*Gram matrix*) tell us something important. The extreme eigenvalues are $\lambda_{\max} \approx 5264$ and $\lambda_{\min} \approx .004$. Those are the squares of σ_4 and σ_1, the largest and smallest **singular values of $\boldsymbol{V}$**. The key measure is their ratio σ_4/σ_1, the **condition number of $\boldsymbol{V}$**:

$$\mathrm{cond}(V^{\mathrm{T}}V) \approx \frac{5264}{.004} \approx 10^6 \qquad \mathrm{cond}(V) = \sqrt{\frac{\lambda_{\max}}{\lambda_{\min}}} \approx 1000\,.$$

For such a small example, 1000 is a poor condition number. For an orthonormal basis with $Q^{\mathrm{T}}Q = I$, all eigenvalues = singular values = condition number = 1.

We could improve the condition by rescaling the columns of V to unit vectors. Then $V^{\mathrm{T}}V$ has ones on the diagonal, and the condition number drops to 263. But when the matrix size is realistically large, that rescaling will not save us. In fact we could extend this Vandermonde model from constant, linear, quadratic, and cubic vectors to the functions $1, x, x^2, x^3$. (A multiplies by x.) Please look at what happens!

Continuous Vandermonde matrix $$V_c = \begin{bmatrix} 1 & x & x^2 & x^3 \end{bmatrix}. \tag{6}$$

Again, those four functions are far from orthogonal. The inner products in $V_c^{\mathrm{T}}V_c$ change from sums to *integrals*. Working on the interval from 0 to 1, the integrals are $\int_0^1 x^i x^j\,dx = 1/(i+j-1)$. They appear in the ***Hilbert matrix***:

Continuous inner products $$V_c^{\mathrm{T}}V_c = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \frac{1}{6} \\ \frac{1}{4} & \frac{1}{5} & \frac{1}{6} & \frac{1}{7} \end{bmatrix}. \tag{7}$$

The extreme eigenvalues of this Hilbert matrix are $\lambda_{\max} \approx 1.5$ and $\lambda_{\min} \approx 10^{-4}$. As always, those are the squares of the singular values $\sigma_{\max}$ and $\sigma_{\min}$ of V_c. *The condition number of the power basis* $1, x, x^2, x^3$ *is the ratio* $\sigma_{\max}/\sigma_{\min} \approx 125$. If you want a more impressive number (a numerical disaster), go up to x^9. The condition number of the 10 by 10 Hilbert matrix is $\lambda_{\max}/\lambda_{\min} \approx 10^{13}$, and $1, x, \ldots, x^9$ is a very poor basis.

To reduce that unacceptably large number, Legendre orthogonalized the basis. He chose the interval from -1 to 1, so that even powers would be automatically orthogonal to odd powers. The first ***Legendre polynomials*** are $1, x, x^2 - \frac{1}{3}, x^3 - \frac{3}{5}x$. Our point is that the Vandermonde matrix example (as we follow it below) will be completely parallel to the famous functions of Legendre.

In particular, the **three-term recurrence** in the Arnoldi-Lanczos orthogonalization is exactly like Legendre's classical three-term recurrence for his polynomials. They appear for the same reason—the symmetry of A.

Orthogonalizing the Krylov Basis

The best basis $q_1, \ldots, q_j$ for the Krylov subspace $\boldsymbol{\mathcal{K}_j}$ is orthonormal. Each new q_j comes from orthogonalizing $t = Aq_{j-1}$ to the basis vectors $q_1, \ldots, q_{j-1}$ that are already chosen. The iteration to compute these orthonormal q's is **Arnoldi's method**.

This method is essentially the Gram-Schmidt idea (called *modified* Gram-Schmidt when we subtract the projections of t onto the q's one at a time, for numerical stability). We display one Arnoldi cycle for the Vandermonde example that has $b = [1 \quad 1 \quad 1 \quad 1]'$ and $A = \texttt{diag}([1 \quad 2 \quad 3 \quad 4])$:

Arnoldi's orthogonalization of $\boldsymbol{b, Ab, \ldots, A^{n-1}b}$**:**

©2006 Gilbert Strang

0	$\boldsymbol{q_1} = b/\|b\|$;	% Normalize b to $\|q_1\| = 1$	$\boldsymbol{q_1} = [1 \quad 1 \quad 1 \quad 1]'/2$
	for $j = 1, \ldots, n-1$	% Start computation of q_{j+1}	
1	$t = \boldsymbol{Aq_j}$;	% one matrix multiplication	$\boldsymbol{Aq_1} = [1 \quad 2 \quad 3 \quad 4]'/2$
	for $i = 1, \ldots, j$	% t is in the space $\boldsymbol{\mathcal{K}_{j+1}}$	
2	$\boldsymbol{h_{ij}} = q_i^{\mathrm{T}} t$;	% $h_{ij} q_i$ = projection of t on q_i	$\boldsymbol{h_{11}} = 5/2$
3	$\boldsymbol{t} = t - h_{ij} q_i$;	% Subtract that projection	$\boldsymbol{t} = [-3 \quad -1 \quad 1 \quad 3]'/4$
	end;	% t is orthogonal to $q_1, \ldots, q_j$	
4	$\boldsymbol{h_{j+1,j}} = \|t\|$;	% Compute the length of t	$\boldsymbol{h_{21}} = \sqrt{5}/2$
5	$\boldsymbol{q_{j+1}} = t/h_{j+1,j}$;	% Normalize t to $\|q_{j+1}\| = 1$	$\boldsymbol{q_2} = [-3 \quad -1 \quad 1 \quad 3]'/\sqrt{20}$
	end	% $q_1, \ldots, q_n$ are orthonormal	

You might like to see the four orthonormal vectors in the Vandermonde example. Those columns q_1, q_2, q_3, q_4 of Q are still constant, linear, quadratic, and cubic. I can also display the matrix H of numbers h_{ij} that produced the q's from the Krylov vectors b, Ab, A^2b, A^3b. (Since Arnoldi stops at $j = n-1$, the last column of H is not actually computed. It comes from a final command $H(:, n) = Q' * A * Q(:, n)$.) This H turns out to be *symmetric and tridiagonal*, and we will look for a reason.

Arnoldi's method for the Vandermonde example V gives Q and H:

$$Q = \begin{bmatrix} 1 & -3 & 1 & -1 \\ 1 & -1 & -1 & 3 \\ 1 & 1 & -1 & -3 \\ 1 & 3 & 1 & 1 \end{bmatrix} \begin{matrix} /2 \\ /\sqrt{20} \\ /2 \\ /\sqrt{20} \end{matrix} \qquad H = \begin{bmatrix} 5/2 & \sqrt{5}/2 & & \\ \sqrt{5}/2 & 5/2 & \sqrt{.80} & \\ & \sqrt{.80} & 5/2 & \sqrt{.45} \\ & & \sqrt{.45} & 5/2 \end{bmatrix}$$

Please notice that H is not upper triangular. The usual QR factorization of the original Krylov matrix $\mathbf{K}$ (which was V in our example) has this same Q, but Arnoldi does not compute R. Even though the underlying idea copies Gram-Schmidt (at every step q_{j+1} is a unit vector orthogonal to the previous j columns), there is a difference. The vector t that Arnoldi orthogonalizes against the previous $q_1, \ldots, q_j$ is $t = Aq_j$. This is not column $j+1$ of $\mathbf{K}$, as in Gram-Schmidt. Arnoldi is factoring AQ!

Arnoldi factorization $AQ = QH$ for the final subspace $\mathcal{K}_n$:

$$AQ = \begin{bmatrix} & & \\ Aq_1 & \cdots & Aq_n \\ & & \end{bmatrix} = \begin{bmatrix} & & \\ q_1 & \cdots & q_n \\ & & \end{bmatrix} \begin{bmatrix} h_{11} & h_{12} & \cdot & h_{1n} \\ h_{21} & h_{22} & \cdot & h_{2n} \\ \mathbf{0} & h_{23} & \cdot & \cdot \\ \mathbf{0} & \mathbf{0} & \cdot & h_{nn} \end{bmatrix}. \tag{8}$$

This matrix H is upper triangular plus one lower diagonal, which makes it "*upper Hessenberg.*" The h_{ij} in step **2** go down column j as far as the diagonal. Then

 ©2006 Gilbert Strang

$h_{j+1,j}$ in step **4** is below the diagonal. We check that the first column of $AQ = QH$ (multiplying by columns) is Arnoldi's first cycle that produces q_2:

$$\textbf{Column 1} \quad Aq_1 = h_{11}q_1 + h_{21}q_2 \qquad \text{which is} \qquad q_2 = (Aq_1 - h_{11}q_1)/h_{21}\,. \tag{9}$$

That subtraction is step **3** in Arnoldi's algorithm. The division by h_{21} is step **5**.

Unless more of the h_{ij} are zero, the cost is increasing at every iteration. The vector updates in step **3** for $j = 1, \dots, n-1$ give nearly $n^2/2$ updates and $n^3/2$ flops. A *short recurrence* means that most of these h_{ij} are zero, and the count of floating point operations drops to $O(n^2)$. That happens when $A = A^{\mathrm{T}}$.

Arnoldi Becomes Lanczos

The matrix H is symmetric and therefore tridiagonal when A is symmetric. This fact is the foundation of conjugate gradients. For a matrix proof, multiply $AQ = QH$ by Q^{T}. The left side $Q^{\mathrm{T}}AQ$ is always symmetric when A is symmetric. Since H has only one lower diagonal, it has only one upper diagonal. This tridiagonal H has only three nonzeros in its rows and columns. So computing q_{j+1} only involves q_j and q_{j-1}:

$$\textbf{Arnoldi when } \boldsymbol{A = A^{\mathrm{T}}} \qquad Aq_j = h_{j+1,j}\,q_{j+1} + h_{j,j}\,q_j + h_{j-1,j}\,q_{j-1}\,. \tag{10}$$

This is the ***Lanczos iteration***. Each new $q_{j+1} = (Aq_j - h_{j,j}q_j - h_{j-1,j}q_{j-1})/h_{j+1,j}$ involves one multiplication Aq_j, two dot products for h's, and two vector updates.

Allow me an important comment on the *symmetric eigenvalue problem* $Ax = \lambda x$. We have seen that $H = Q^{\mathrm{T}}AQ$ is tridiagonal, and $Q^{\mathrm{T}} = Q^{-1}$ from the orthogonality $Q^{\mathrm{T}}Q = I$. The matrix $H = Q^{-1}AQ$ has the same eigenvalues as A:

$$\textbf{Same } \boldsymbol{\lambda} \qquad Hy = Q^{-1}AQy = \lambda y \quad \text{gives} \quad Ax = \lambda x \;\text{ with }\; x = Qy\,. \tag{11}$$

It is much easier to find the eigenvalues λ for a tridiagonal H than for the original A.

For a *large* symmetric matrix, we often stop the Arnoldi-Lanczos iteration at a tridiagonal H_k with $k < n$. The full n-step process to reach H_n is too expensive, and often we don't need all n eigenvalues. So we compute the k eigenvalues of H_k instead of the n eigenvalues of H. These computed $\lambda_{1k}, \dots, \lambda_{kk}$ (sometimes called Ritz values) can provide good approximations to the first k eigenvalues of A. And we have an excellent start on the eigenvalue problem for H_{k+1}, if we decide to take a further step.

This **Lanczos method** will find, approximately and iteratively and quickly, the leading eigenvalues of a large symmetric matrix. For its inner loop (the eigenvalues of the tridiagonal H_k) we use the "QR method" described in section ____.

©2006 Gilbert Strang

The Conjugate Gradient Method

We return to iterative methods for $Ax = b$. The Arnoldi algorithm produced orthonormal basis vectors $q_1, q_2, \ldots$ for the growing Krylov subspaces $\boldsymbol{\mathcal{K}_1}, \boldsymbol{\mathcal{K}_2}, \ldots$. Now we select vectors x_k in $\boldsymbol{\mathcal{K}_k}$ that approach the exact solution to $Ax = b$.

We concentrate on the *conjugate gradient method when A is symmetric positive definite*. Symmetry gives a short recurrence. Definiteness prevents division by zero.

The rule for x_k in conjugate gradients is that the residual $r_k = b - Ax_k$ should be orthogonal to all vectors in $\boldsymbol{\mathcal{K}_k}$. Since r_k will be in $\boldsymbol{\mathcal{K}_{k+1}}$, it must be a multiple of Arnoldi's next vector q_{k+1}! Each residual is therefore orthogonal to all previous residuals (which are multiples of the previous q's):

Orthogonal residuals $$r_i^{\mathrm{T}} r_k = 0 \quad \text{for} \quad i < k\,. \tag{12}$$

The difference between r_k and q_{k+1} is that the q's are normalized, as in $q_1 = b/\|b\|$.

Similarly r_{k-1} is a multiple of q_k. Then the difference $r_k - r_{k-1}$ is orthogonal to each subspace $\boldsymbol{\mathcal{K}_i}$ with $i < k$. Certainly $x_i - x_{i-1}$ lies in that $\boldsymbol{\mathcal{K}_i}$. So Δr is orthogonal to earlier Δx's:

$$(x_i - x_{i+1})^{\mathrm{T}}(r_k - r_{k-1}) = 0 \quad \text{for} \quad i < k\,. \tag{13}$$

These differences Δx and Δr are directly connected, because the b's cancel in Δr:

$$r_k - r_{k-1} = (b - Ax_k) - (b - Ax_{k-1}) = -A(x_k - x_{k-1})\,. \tag{14}$$

Substituting (14) into (13), ***the updates Δx are "A-orthogonal" or conjugate***:

Conjugate directions $$(x_i - x_{i-1})^{\mathrm{T}} A(x_k - x_{k-1}) = 0 \quad \text{for } i < k\,. \tag{15}$$

Now we have all the requirements. Each conjugate gradient step ends with a "search direction" d_{k-1} for the next update $x_k - x_{k-1}$. Steps **1** and **2** compute the correct multiple $\alpha_k d_{k-1}$ to move to x_k. Using (14), step **3** finds the new r_k. Steps **4** and **5** orthogonalize r_k against the search direction just used, to find the next d_k.

The constants β_k in the search direction and α_k in the update come from (12) and (13) for $i = k-1$. For symmetric A, orthogonality will be automatic for $i < k-1$, as in Arnoldi. We have a "short recurrence" for the new x_k and r_k.

Here is one cycle of the algorithm, starting from $x_0 = 0$ and $r_0 = b$ and $d_0 = r_0$. Steps **1** and **3** involve the same matrix-vector multiplication Ad.

Conjugate Gradient Method for Symmetric Positive Definite A

Example: $\boldsymbol{A} = \texttt{diag}([1\ \ 2\ \ 3\ \ 4])$ **and** $\boldsymbol{b} = [1\ \ 1\ \ 1\ \ 1]'$

1	$\boldsymbol{\alpha_k} = r_{k-1}^{\mathrm{T}} r_{k-1} / d_{k-1}^{\mathrm{T}} A d_{k-1}$	% Step length to next x_k	$\boldsymbol{\alpha_1} = 4/10 = 2/5$
2	$\boldsymbol{x_k} = x_{k-1} + \alpha_k d_{k-1}$	% Approximate solution	$\boldsymbol{x_1} = [2\ \ 2\ \ 2\ \ 2]'/5$
3	$\boldsymbol{r_k} = r_{k-1} - \alpha_k A d_{k-1}$	% New residual from (14)	$\boldsymbol{r_1} = [3\ \ 1\ \ -1\ \ -3]'/5$
4	$\boldsymbol{\beta_k} = r_k^{\mathrm{T}} r_k / r_{k-1}^{\mathrm{T}} r_{k-1}$	% Improvement this step	$\boldsymbol{\beta_1} = 1/5$
5	$\boldsymbol{d_k} = r_k + \beta_k d_{k-1}$	% Next search direction	$\boldsymbol{d_1} = [4\ \ 2\ \ 0\ \ -2]'/5$

The formulas for α_k and β_k are explained briefly below—and fully by Trefethen-Bau [–] and Shewchuk [–] and many other good references.

When there is a preconditioner P (to use even fewer CG steps for an accurate x), step **3** uses $P^{-1}A$ and the inner products in steps **1** and **4** include an extra factor P^{-1}.

Different Viewpoints on Conjugate Gradients

I want to describe the (same!) conjugate gradient method in two different ways:

1. It solves a tridiagonal system $Hy = f$ recursively, for Arnoldi's H.

2. It minimizes the energy $\frac{1}{2}x^{\mathrm{T}}Ax - x^{\mathrm{T}}b$ recursively. This is important.

How does $Ax = b$ change to the tridiagonal $Hy = f$? Those are connected by Arnoldi's orthonormal columns $q_1, \dots, q_n$ in Q, with $Q^{\mathrm{T}} = Q^{-1}$ and $Q^{\mathrm{T}}AQ = H$:

$$Ax = b \quad \text{is} \quad (Q^{\mathrm{T}}AQ)(Q^{\mathrm{T}}x) = Q^{\mathrm{T}}b \quad \text{which is} \quad Hy = f = (\|b\|, 0, \dots, 0)\,. \tag{16}$$

Since q_1 is $b/\|b\|$, the first component of $f = Q^{\mathrm{T}}b$ is $q_1^{\mathrm{T}}b = \|b\|$. The other components of f are $q_i^{\mathrm{T}}b = 0$ because q_i is orthogonal to q_1. The conjugate gradient method is implicitly computing the symmetric tridiagonal H. When the method finds x_k, it also finds $y_k = Q_k^{\mathrm{T}}x_k$ (but it doesn't say so). Here is the third step:

Tridiagonal system $Hy = f$ Implicitly solved by CG

$$H_3y_3 = \begin{bmatrix} h_{11} & h_{12} & \\ h_{21} & h_{22} & h_{23} \\ & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} \\ y_3 \\ \\ \end{bmatrix} = \begin{bmatrix} \|b\| \\ 0 \\ 0 \end{bmatrix}. \tag{17}$$

This is the equation $Ax = b$ *projected by* Q_3 onto the third Krylov subspace $\boldsymbol{\mathcal{K}_3}$.

These h's never appear in conjugate gradients. We don't want to do Arnoldi too! It is the LDL^{T} factors of H that CG is somehow computing—two new numbers α and β at each step. Those give a fast update from y_{k-1} to y_k. The iterates $x_k = Q_k y_k$ from conjugate gradients approach the exact solution $x_n = Q_n y_n$ which is $x = A^{-1}b$.

Energy By seeing conjugate gradients as an energy minimizing algorithm, we can extend it to nonlinear problems and use it in optimization. For our linear equation $Ax = b$, the energy is $E(x) = \frac{1}{2}x^{\mathrm{T}}Ax - x^{\mathrm{T}}b$. Minimizing $E(x)$ is the same as solving $Ax = b$, when A is positive definite—this was the main point of Section 1.6. **The CG iteration minimizes $\boldsymbol{E(x)}$ on the growing Krylov subspaces**.

The first subspace $\boldsymbol{\mathcal{K}_1}$ is the line in the direction $d_0 = r_0 = b$. Minimization of the energy $E(x)$ for the vectors $x = \alpha b$ produces the number α_1:

$$E(\alpha b) = \frac{1}{2}\alpha^2 b^{\mathrm{T}}Ab - \alpha b^{\mathrm{T}}b \quad \text{is minimized at} \quad \alpha_1 = \frac{b^{\mathrm{T}}b}{b^{\mathrm{T}}Ab}\,. \tag{18}$$

This α_1 is the constant chosen in step **1** of the first conjugate gradient cycle.

©2006 Gilbert Strang

The gradient of $E(x) = \frac{1}{2}x^{\mathrm{T}}Ax - x^{\mathrm{T}}b$ is exactly $Ax - b$. *The steepest descent direction at x_1 is along the negative gradient, which is r_1!* This sounds like the perfect direction d_1 for the next move. But the great difficulty with steepest descent is that this r_1 can be too close to the first direction d_0. Little progress that way. So step **5** adds the right multiple $\beta_1 d_0$, in order that the new $d_1 = r_1 + \beta_1 d_0$ will be A-orthogonal to the first direction d_0.

Then we move in this conjugate direction d_1 to $x_2 = x_1 + \alpha_2 d_1$. This explains the name *conjugate gradients*. The pure gradients of steepest descent would be too nearly parallel, and we would take small steps across a valley instead of a good step to the bottom (the minimizing x in Figure 6.15). Every cycle of CG chooses α_k to minimize $E(x)$ in the new search direction $x = x_{k-1} + \alpha d_{k-1}$. The last cycle (if we go that far) gives the overall minimizer $x_n = x = A^{-1}b$.

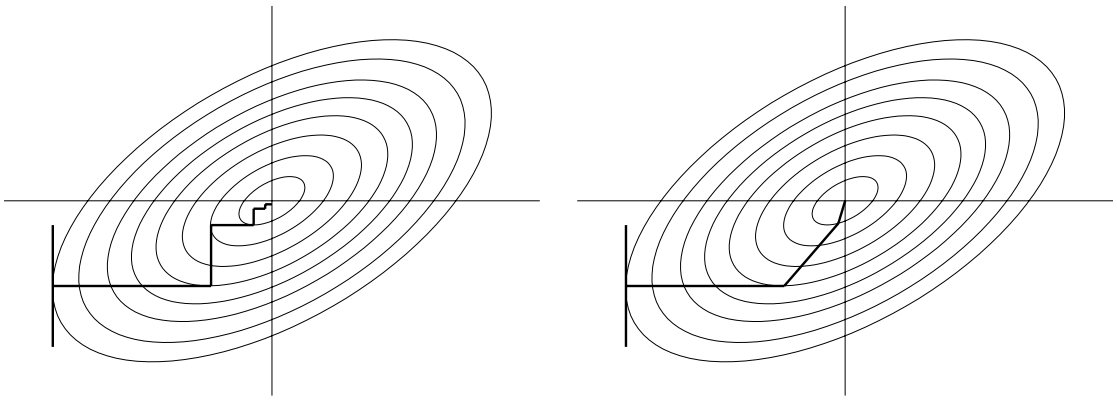


Figure 6.14: Steepest descent vs. conjugate gradient.

The main point is always this. **When you have orthogonality, projection and minimizations can be computed one direction at a time.**

Example

$$\text{Suppose} \quad Ax = b \quad \text{is} \quad \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ -1 \\ -1 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \\ 0 \end{bmatrix}.$$

From $x_0 = (0, 0, 0)$ and $r_0 = d_0 = b$ the first cycle gives $\alpha_1 = \frac{1}{2}$ and $x_1 = \frac{1}{2}b = (2, 0, 0)$. The new residual is $r_1 = b - Ax_1 = (0, -2, -2)$. Then the CG algorithm yields

$$\beta_1 = \frac{8}{16} \quad d_1 = \begin{bmatrix} 2 \\ -2 \\ -2 \end{bmatrix} \quad \alpha_2 = \frac{8}{16} \quad x_2 = \begin{bmatrix} 3 \\ -1 \\ -1 \end{bmatrix} = A^{-1}b!$$

The correct solution is reached in two steps, where normally CG will take $n = 3$ steps. The reason is that this particular A has only two distinct eigenvalues 4 and 1. In that case $A^{-1}b$ is a combination of b and Ab, and this best combination x_2 is found at cycle 2. The residual r_2 is zero and the cycles stop early—very unusual.

Energy minimization leads in [ ] to an estimate of the convergence rate for the error $e = x - x_k$ in conjugate gradients, using the A-norm $\|e\|_A = \sqrt{e^{\mathrm{T}} A e}$:

$$\textbf{Error estimate} \qquad \|x - x_k\|_A \le 2 \left(\frac{\sqrt{\lambda_{\max}} - \sqrt{\lambda_{\min}}}{\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\min}}} \right)^k \|x - x_0\|_A \,. \tag{19}$$

This is the best-known error estimate, although it doesn't account for any clustering of the eigenvalues of A. It involves only the condition number $\lambda_{\max}/\lambda_{\min}$. Problem ____ gives the "optimal" error estimate but it is not so easy to compute. That optimal estimate needs all the eigenvalues of A, while (19) uses the extreme eigenvalues $\lambda_{\max}(A)$ and $\lambda_{\min}(A)$—which in practice we can bound above and below.

Minimum Residual Methods

When A is not symmetric positive definite, CG is not guaranteed to solve $Ax = b$. We lose control of $d^{\mathrm{T}} A d$ in computing α. We will follow van der Vorst [ ] in briefly describing the ***minimum norm residual*** approach, leading to MINRES and GMRES.

These methods choose x_j in the Krylov subspace $\boldsymbol{\mathcal{K}_j}$ so that $\|b - Ax_j\|$ is minimal. The first orthonormal vectors $q_1, \ldots, q_j$ go in the columns of Q_j, so $Q_j^{\mathrm{T}} Q_j = I$. As in (16) we set $x_j = Q_j y$, to express the solution as a combination of those q's: using (8) is

$$\textbf{Norm of residual} \qquad \|r_j\| = \|b - Ax_j\| = \|b - AQ_j y\| = \|b - Q_{j+1} H_{j+1,j} y\|. \tag{20}$$

Here I used the first j columns of Arnoldi's formula $AQ = QH$. Since the jth column of H is zero after entry $j+1$, we only need $j+1$ columns of Q on the right side:

$$\text{First } j \text{ columns of } QH = \begin{bmatrix} q_1 & \cdots & q_{j+1} \end{bmatrix} \begin{bmatrix} h_{11} & \cdots & h_{1j} \\ h_{12} & \ddots & \vdots \\ & \ddots & h_{jj} \\ & & h_{j+1,j} \end{bmatrix}. \tag{21}$$

The norm in (20) is not changed when we multiply by Q_{j+1}^{T}. Our problem becomes:

$$\textbf{Choose } y \textbf{ to minimize} \quad \|r_j\| = \|Q_{j+1}^{\mathrm{T}} b - H_{j+1,j}\, y\| \,. \tag{22}$$

This is an ordinary least squares problem with only $j+1$ equations and j unknowns. The right side $Q_{j+1}^{\mathrm{T}} b$ is $(\|r_0\|, 0, \ldots, 0)$ as in (16). The rectangular matrix $H_{j+1,j}$ is Hessenberg in (21). We face a completely typical problem of numerical linear algebra: *Use zeros in H and $Q_{j+1}^{\mathrm{T}} b$ to find a fast algorithm that computes y.* The two favorite algorithms for this least squares problem are closely related:

MINRES A is symmetric (likely indefinite, or we use CG) and H is tridiagonal.

GMRES A is *not* symmetric and the upper triangular part of H can be full.

©2006 Gilbert Strang

In both cases we want to clear out that nonzero diagonal below the main diagonal of H. The natural way to do that, one entry at a time, is by "*Givens rotations*." These plane rotations are so useful and simple (the essential part is only 2 by 2) that we complete this section by explaining them.

Givens Rotations

The direct approach to the least squares solution of $Hy = f$ constructs the normal equations $H^{\mathrm{T}}H\widehat{y} = H^{\mathrm{T}}f$. That was the central idea in Chapter 1, but you see what we lose. If H is Hessenberg, with many good zeros, $H^{\mathrm{T}}H$ is full. Those zeros in H should simplify and shorten the computations, so we don't want the normal equations.

The other approach to least squares is by Gram-Schmidt. **We factor H into orthogonal times upper triangular**. Since the letter Q is already used, the orthogonal matrix will be called G (after Givens). The upper triangular matrix is $G^{-1}H$. The 3 by 2 case shows how a rotation in the 1–2 plane can clear out h_{21}:

$$G_{21}^{-1}H = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \\ 0 & h_{32} \end{bmatrix} = \begin{bmatrix} * & * \\ \mathbf{0} & * \\ 0 & \boldsymbol{*} \end{bmatrix}. \tag{23}$$

That bold zero entry requires $h_{11}\sin\theta = h_{21}\cos\theta$, which determines the rotation angle θ. A second rotation G_{32}^{-1}, in the 2-3 plane, will zero out the $3,2$ entry. Then $G_{32}^{-1}G_{21}^{-1}H$ is a square upper triangular matrix U above a row of zeros!

The Givens orthogonal matrix is $G_{21}G_{32}$ but there is no reason to do this multiplication. We use each G_{ij} as it is constructed, to simplify the least squares problem. Rotations (and all orthogonal matrices) leave the lengths of vectors unchanged:

$$\|Hy - f\| = \|G_{32}^{-1}G_{21}^{-1}Hy - G_{32}^{-1}G_{21}^{-1}f\| = \left\| \begin{bmatrix} U \\ 0 \end{bmatrix} y - \begin{bmatrix} F \\ e \end{bmatrix} \right\|. \tag{24}$$

This length is what MINRES and GMRES minimize. The row of zeros below U means that the last entry e is the error—we can't reduce it. But we get all the other entries exactly right by solving the j by j system $Uy = F$ (here $j = 2$). This gives the best least squares solution y. Going back to the original problem of minimizing the residual $\|r\| = \|b - Ax_j\|$, the best x_j in the jth Krylov space is $Q_j y$.

For non-symmetric A (GMRES rather than MINRES) the recurrence is not short. The upper triangle in H can be full, and *step j becomes expensive*. Possibly it is inaccurate as j increases. So we may change "full GMRES" to GMRES(m), which restarts the algorithm every m steps. It is not so easy to choose a good m. But GMRES is an important algorithm for unsymmetric A.

Problem Set 6.4

1 When the tridiagonal K is preconditioned by $P = T$ (second difference matrix with $T_{11} = 1$) show that $\boldsymbol{T^{-1}K = I + \ell e_1^{\mathrm{T}}}$ with $e_1^{\mathrm{T}} = [\,1\ 0\ \ldots\ 0\,]$. Start from

$K = T + e_1 e_1^{\mathrm{T}}$. Then $T^{-1}K = I + (T^{-1}e_1)e_1^{\mathrm{T}}$. Verify that $T^{-1}e_1 = \ell$ from:

$$\boldsymbol{T\ell = e_1} = \begin{bmatrix} 1 & -1 & & \\ -1 & 2 & -1 & \\ & \cdot & \cdot & \cdot \\ & & -1 & 2 \end{bmatrix} \begin{bmatrix} N \\ N-1 \\ \cdot \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ \cdot \\ 0 \end{bmatrix}.$$

Second differences of this linear vector ℓ are zero. Multiply $I + \ell e_1^{\mathrm{T}}$ times $I - (\ell e_1^{\mathrm{T}})/(N+1)$ to establish that this is the inverse matrix $K^{-1}T$.

2 For the model of a square grid with separator down the middle, create the reordered matrix K in equation (4). Use `spy`(K) to print its pattern of nonzeros.

3 Arnoldi expresses each Aq_j as $h_{j+1,j}q_{j+1} + h_{j,j}q_j + \cdots + h_{1,j}q_1$. Multiply by q_i^{T} to find $h_{i,j} = q_i^{\mathrm{T}}Aq_j$. *If A is symmetric* this is $(Aq_i)^{\mathrm{T}}q_j$. Explain why $(Aq_i)^{\mathrm{T}}q_j = 0$ for $i < j-1$ by expanding Aq_i into $h_{i+1,i}q_{i+1} + \cdots + h_{1,i}q_1$. We have a *short recurrence if* $A = A^{\mathrm{T}}$ (only $h_{j+1,j}$ and $h_{j,j}$ and $h_{j-1,j}$ are nonzero).

4 (This is Problem 3 at the matrix level) The Arnoldi equation $AQ = QH$ gives $H = Q^{-1}AQ = Q^{\mathrm{T}}AQ$. Therefore the entries of H are $h_{ij} = q_i^{\mathrm{T}}Aq_j$.

(a) Which Krylov space contains Aq_j? What orthogonality gives $h_{ij} = 0$ when $i > j+1$? Then H is upper Hessenberg.

(b) If $A^{\mathrm{T}} = A$ then $h_{ij} = (Aq_i)^{\mathrm{T}}q_j$. Which Krylov space contains Aq_i? What orthogonality gives $h_{ij} = 0$ when $j > i+1$? Now H is tridiagonal.

5 Test the `pcg`$(A,\ \ )$ MATLAB command on the $-1, 2, -1$ second difference matrix $A = K$. As preconditioner use $P = T$, when $T_{11} = 1$.

6 If $K = [\,b \quad Ab \quad \ldots \quad A^{n-1}b\,]$ is a Krylov matrix with $A = A^{\mathrm{T}}$, why is the inner product matrix $K^{\mathrm{T}}K$ a **Hankel matrix**? This means constant entries down each *antidiagonal* (the opposite of Toeplitz). Show that $(K^{\mathrm{T}}K)_{ij}$ depends on $i + j$.

7 These are famous names associated with linear algebra (and a lot of other mathematics too). All dead. Write one sentence on what they are known for.

Arnoldi	Gram	Jacobi	Schur
Cholesky	Hadamard	Jordan	Schwartz
Fourier	Hankel	Kronecker	Seidel
Frobenius	Hessenberg	Krylov	Toeplitz
Gauss	Hestenes-Stiefel	Lanczos	Vandermonde
Gershgorin	Hilbert	Markov	Wilkinson
Givens	Householder	Schmidt	Woodbury

`solution: Jacobi (matrices), Gauss (elimination, numerical integration), Lanczos [-1,2,-1 with q_1=(1,0,-)], zeros of cosines/convergence rate?, Another q_1? Berresford.`

©2006 Gilbert Strang

6.5 The Saddle Point Stokes Problem

So far the matrix C has been diagonal—no trouble to invert. This section jumps to a fluid flow problem that is still linear (simpler than Navier-Stokes). But now C^{-1} represents the positive definite Laplacian $(-\Delta)$ in the continuous problem, and it is a finite difference or finite element discretization in the matrix problem. The matrix C^{-1} is sparse, but C itself involves a serious boundary value problem.

In this case we don't reduce the computation all the way to $A^{\mathrm{T}}CA$! Instead we stay with the sparse matrix whose blocks are C^{-1}, A, A^{T}, and 0. This is an *indefinite* matrix, with that zero block on its diagonal. The solution v, p is a *saddle point*.

This section also discusses the preconditioning of indefinite systems (not only Stokes).

The Stokes Problem

The unknowns in the two-dimensional Stokes problem are the velocity components in $v = (v_1(x,y), v_2(x,y))$ and the pressure $p(x,y)$. The flow is *incompressible* so the velocity vector is constrained by $\operatorname{div} v = 0$. Therefore A^{T} is (minus) the divergence and A must be the gradient. Notice the different letters in our framework (u is changed to p, w is changed to v, and b is changed to f):

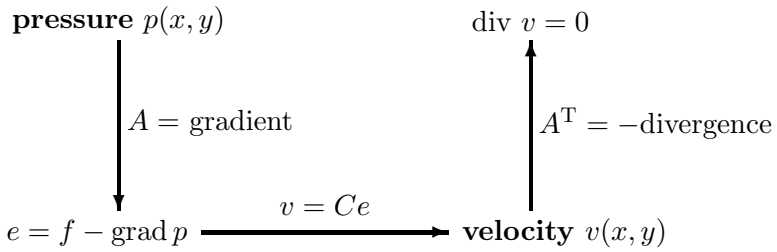


Figure 6.15: The three-step $A^{\mathrm{T}}CA$ framework for the Stokes problem.

The real novelty is in $e = C^{-1}v = (-\Delta v_1, -\Delta v_2)$. The Stokes problem asks us to find a velocity vector v and a pressure p that solve $-\Delta v = f - \operatorname{grad} p$ and $\operatorname{div} v = 0$:

$$-\left(\frac{\partial^2 v_1}{\partial x^2} + \frac{\partial^2 v_1}{\partial y^2}\right) + \frac{\partial p}{\partial x} = f_1(x,y) \tag{1a}$$

$$-\left(\frac{\partial^2 v_2}{\partial x^2} + \frac{\partial^2 v_2}{\partial y^2}\right) + \frac{\partial p}{\partial y} = f_2(x,y) \tag{1b}$$

$$\frac{\partial v_1}{\partial x} + \frac{\partial v_2}{\partial y} = 0 \tag{1c}$$

This describes **slow viscous flow**. It is not for aeronautics! The full Navier-Stokes equations have extra nonlinear terms from the motion of the underlying fluid. The Stokes problem could come into its own in biological applications, not for large-scale blood flow in the heart but for small-scale movements in capillaries (or even in cells). This linear problem is simpler than Navier-Stokes, and needs to be understood first. We keep it simple by omitting discussion of the boundary conditions.

In the language of optimization, the pressure $p(x,y)$ is the Lagrange multiplier that imposes the incompressibility constraint div $v = 0$ when the energy is minimized:

$$\text{Minimize} \iint \left(|\operatorname{grad} v_1|^2 + |\operatorname{grad} v_2|^2 - 2f \cdot \operatorname{grad} v \right) dx\, dy \text{ subject to } \operatorname{div} v = 0\,.$$

The Lagrangian $L(v,p)$ adds $\int\int \delta L/\delta v = 0$ and $\delta L/\delta p = 0$ are exactly the Stokes equations (1). Chapter 6 will develop this primal-dual block form to find the saddle point of L—here we only need to know A and C.

The key point is that we do not eliminate v to reach $K = A^{\mathrm{T}}CA$, because C is *the inverse of the Laplacian*. It is diagonal only in frequency space, not in physical x, y space. The Stokes problem stays in its block form (1) with two unknowns v and p:

$$\textbf{Block form (continuous)} \quad \begin{bmatrix} C^{-1} & A \\ A^{\mathrm{T}} & 0 \end{bmatrix} \begin{bmatrix} v \\ p \end{bmatrix} = \begin{bmatrix} -\Delta & \operatorname{grad} \\ -\operatorname{div} & 0 \end{bmatrix} \begin{bmatrix} v \\ p \end{bmatrix} = \begin{bmatrix} f \\ 0 \end{bmatrix}. \tag{2}$$

In the discrete case, that block matrix will be symmetric but ***indefinite*** (as usual). It will have positive and also negative eigenvalues and pivots (positive pivots from C^{-1}, negative pivots from $-A^{\mathrm{T}}CA$). Finite elements for velocity v and pressure p have to be carefully matched to ensure that the discrete problem has a good solution. The approximating polynomials are often *one degree higher* for v than for p.

The Inf-Sup Condition

Restricting a positive definite operator K to a subspace (by a projection P) never destroys positive definiteness. All eigenvalues of PKP^{T} lie safely between $\lambda_{\min}(K)$ and $\lambda_{\max}(K)$. But an indefinite operator has a *negative* $\lambda_{\min}$, so this interval includes zero: not safe! We must take particular care that the block matrix is safely invertible.

These **mixed problems**, or **saddle point problems**, are partly positive and partly negative. The new requirement for a bounded inverse is an *inf-sup condition* (also known as a Babuska-Brezzi condition): **For every p there must be a v so that**

$$v^{\mathrm{T}}Ap \ge \beta \sqrt{v^{\mathrm{T}}C^{-1}v}\ \sqrt{p^{\mathrm{T}}p} \quad \text{for a fixed } \beta > 0\,. \tag{3}$$

This "compatibility" of velocities and pressures produces $\|(A^{\mathrm{T}}CA)^{-1}\| \le 1/\beta^2$.

Condition (3) will immediately fail if there is a nonzero pressure p with $Ap = 0$. In the discrete case, this means that A must have full column rank. Otherwise the

©2006 Gilbert Strang

last columns of the block matrix in (2) are not independent, and the block matrix is not invertible. In particular, the rectangular A cannot be short and wide.

More exactly, the lower bound (3) on $v^{\mathrm{T}}Ap$ requires that a nonzero pressure p may not be orthogonal to every $A^{\mathrm{T}}v$. The space of all $A^{\mathrm{T}}v$'s must have *at least* the dimension of the space of p's. Thinking of A^{T} as divergence, this indicates why the velocity finite elements are often one polynomial degree above the pressure finite elements. But dimensions of spaces are not sufficient to confirm that (3) is actually true. Each choice of finite elements for v and p requires its own analysis.

Now we explain the highlighted statement after equation (3): $A^{\mathrm{T}}CA$ is bounded below by β^2 and $\|(A^{\mathrm{T}}CA)^{-1}\| \le 1/\beta^2$ when the inf-sup condition holds. Begin by introducing $w = C^{-1/2}v$. The continuous and discrete Laplacians, both called C^{-1}, have positive definite square roots $C^{-1/2}$ (never to be computed). Then (3) reads:

$$\text{For every } p \text{ there must be a } w \text{ so that} \quad \frac{w^{\mathrm{T}}C^{1/2}Ap}{\|w\|} \ge \beta\|p\|\,. \tag{4}$$

This is where we take the maximum over all w (the *sup*). On the left, the maximum of $w^{\mathrm{T}}z/\|w\|$ is exactly $\|z\|$. This is attained when $w = z$ (then the cosine is one). Since (4) has $z = C^{1/2}Ap$, this best choice of w reduces (3) to

$$\text{For every } p,\ \|z\| = \|C^{1/2}Ap\| \ge \beta\|p\|\,. \tag{5}$$

Square both sides, and minimize their ratio over all p (the *inf*). The inequality

$$\|C^{1/2}Ap\|^2 \ge \beta^2\|p\|^2 \quad \text{or} \quad p^{\mathrm{T}}A^{\mathrm{T}}CAp \ge \beta^2 p^{\mathrm{T}}p \tag{6}$$

is tightest at the lowest eigenvector and eigenvalue, where $A^{\mathrm{T}}CAp = \beta^2 p$. The largest eigenvalue of $(A^{\mathrm{T}}CA)^{-1}$ is $1/\beta^2$. The matrix is symmetric so its norm is also $1/\beta^2$.

A key goal in replacing $\|(A^{\mathrm{T}}CA)^{-1}\| \le 1/\beta^2$ by the inf-sup condition is to work with C^{-1} and not C. We can apply (3) to the continuous problem (it holds, so Stokes is well-posed). We can also test (3) on subspaces V_h and P_h of finite element trial functions. It is a *compatibility condition*: for every pressure p_h in P_h, there must be a velocity v_h in V_h satisfying (3). If and when this is established, with a bound β_h that stays away from zero, the **mixed finite element method** will be stable.

Correction to $p^{\mathrm{T}}p$: Finite elements are functions $p_h(x,y)$, not just vectors v and p. So inner products and norms of v_h and p_h come from integrals and not sums. The L^2 norm of a pressure trial function $p_h = \sum p_j\phi_j(x,y)$ is connected to the discrete coefficient vector p through a positive definite "pressure mass matrix" Q:

$$\|p_h\|^2 = \iint (p_h)^2\,dx\,dy = \sum\sum p_i p_j \iint \phi_i\phi_j\,dx\,dy = p^{\mathrm{T}}Qp\,. \tag{7}$$

The correct inf-sup condition (3) changes $p^{\mathrm{T}}p$ to $p^{\mathrm{T}}Qp$. When satisfied for the finite elements $\mathbf{3}, \mathbf{5}, \mathbf{8}$ in the list below, the inequality (6) involves $p^{\mathrm{T}}Qp$:

Correction to (6) $$p^{\mathrm{T}}A^{\mathrm{T}}CAp \ge \beta^2 p^{\mathrm{T}}Qp \tag{8}$$

This makes Q (or even its diagonal part) a simple and useful candidate as preconditioner. Not the best, but it is directly available and it scales the problem correctly.

 ©2006 Gilbert Strang

Testing the Inf-Sup Condition

The inf-sup condition can be delicate. The books by Brezzi-Fortin and Elman-Silvester-Wathen are excellent references. From the latter book we report on typical results, writing P_0, P_1, P_2 for constant, linear, and quadratic polynomials on triangles, and Q_0, Q_1, Q_2 for constant, bilinear, and biquadratic elements on quadrilaterals:

1. Velocities in P_1, pressures in P_0: failure

2. Velocities in P_1, pressures in P_1: failure

3. Velocities in P_2, pressures in P_1: **success**

4. Velocities in Q_1, pressures in Q_1: failure

5. Velocities in Q_2, pressures in Q_1: **success** (Figure 6.17)

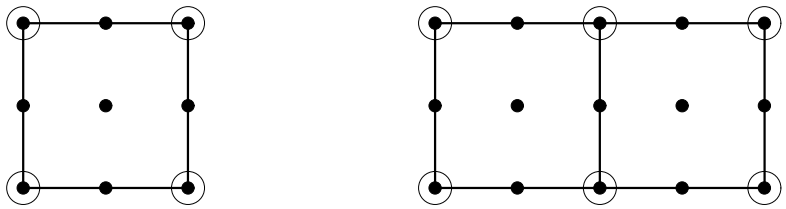


Figure 6.16: Q_2 velocities • and Q_1 pressures ◯: failure on one square, success on two.

For $Q_2 - Q_1$ on one square, A is 2 by 4 (four pressures, two equations $\operatorname{div} v = 0$ at the center). There will surely be non-constant solutions to $Ap = 0$: failure.

For two squares, A is 6 by 6 (six pressures, $\operatorname{div} v = 0$ at three internal nodes). Only a constant pressure solves $Ap = 0$, and this *hydrostatic solution* is removed by the boundary conditions (like grounding a node, here a better choice would be $\iint p\, dx\, dy = 0$). A related "serendipity" element is also successful, by removing the node inside the square and the x^2y^2 term in the velocities. That leaves 8 nodes matching 8 terms in the polynomial.

The list of element pairs also includes pressures P_{-1} and Q_{-1} that are *not continuous* between elements (this is permitted since delta functions don't appear):

6. Velocities in P_2, pressures in P_{-1}: failure

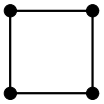
7. Velocities in Q_2, pressures in Q_{-1}: failure

8. Velocities in Q_2, pressures in P_{-1}: **success**

9. Velocities in Q_1, pressures in Q_0: failure (Figure 6.18)

©2006 Gilbert Strang

In each square
$v = a + bx + cy + dxy$ in Q_1
$p =$ *constant* in Q_0



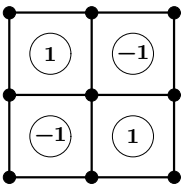


Figure 6.17: Bilinear velocities •, constant pressures ◯ (unstable checkerboard mode).

The failure of $Q_1 - Q_0$ is especially important. Q_0 means a constant pressure in each square. This is the simplest pair of elements that are *conforming* (no delta functions in grad v because Q_1 elements $a + bx + cy + dxy$ are continuous across edges).

Four squares are not stable because that pressure vector $p = (1, -1, 1, -1)$ satisfies $Ap = 0$. More squares don't help—this checkerboard pattern with "white squares $= 1$" and "black squares $= -1$" continues to be a spurious mode satisfying $Ap = 0$. The columns of A, even after $p =$ constant is removed, are still not independent.

This $Q_1 - Q_0$ element is so simple and potentially useful that it is often *stabilized* by relaxing the incompressibility equation $A^{\mathrm{T}}v = 0$. We insert a new anti-checkerboard matrix $-\alpha E$, exactly chosen to eliminate $(1, -1, 1, -1)$ from the nullspace!

Stabilized The block matrix becomes $\begin{bmatrix} C^{-1} & A \\ A^{\mathrm{T}} & -\alpha E \end{bmatrix}$ with $E = \begin{bmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix}$.

Our zero block is replaced by the negative semidefinite matrix $-\alpha E$. Elimination leaves $-A^{\mathrm{T}}CA - \alpha E$ in that block, which is more strongly negative. The $Q_1 - Q_0$ construction is now uniformly invertible, since E times the checkerboard p is not zero.

In this special case we know the guilty p. In other cases E is constructed with only the constant pressure in its nullspace (which isn't harmful). The new αE is taken small enough so that its positive eigenvalues roughly match those of $A^{\mathrm{T}}CA$. The zero eigenvalue of $A^{\mathrm{T}}CA$ (produced by the checkerboard pressure) becomes positive.

Solving the Discrete Saddle Point Problem

Don't forget that this is the fundamental problem of scientific computing!

$$\begin{bmatrix} C^{-1} & A \\ A^{\mathrm{T}} & 0 \end{bmatrix} \begin{bmatrix} w \\ u \end{bmatrix} = \begin{bmatrix} b \\ f \end{bmatrix} \quad \text{or} \quad A^{\mathrm{T}}CAu = f - A^{\mathrm{T}}Cb. \tag{9}$$

In this section, w has become v and u is p. In earlier sections, when C was a diagonal matrix (small-block diagonal, in a multidimensional problem), the reduction to $A^{\mathrm{T}}CA$ was very reasonable. Now the Stokes problem illustrates that C^{-1} can be sparse (coming from finite differences or finite elements) while C is full. In that case

we may prefer to compute with the sparse block form, even though its matrix is not at all positive definite like $A^{\mathrm{T}}CA$.

A direct method (elimination) is appropriate up to $n = 10^4$ or 10^5. It will be used in most problems! Of course an indefinite matrix may require row exchanges to avoid small pivots. A good code will decide that without our intervention.

The pivoting can be organized using 2 by 2 blocks, or we may reorder the unknowns as in Section ____ to keep the L and U factors as sparse as possible. If we eliminate in the original order, subtracting $A^{\mathrm{T}}C$ times one block row from the other, then $-K = -A^{\mathrm{T}}CA$ will appear as the **Schur complement** in the lower right block.

For very large systems, a direct method demands too much computation (and memory). We'll look briefly at iterative methods, remembering that the conjugate gradient method expects positive definiteness. It could be used with $A^{\mathrm{T}}CA$, but it is not safe for the indefinite problem. Each CG step applies $A^{\mathrm{T}}CA$ (possibly preconditioned) to a vector. The matrix-vector multiplications Ap and $A^{\mathrm{T}}v$ are fast. In between, multiplying by C amounts to solving the system $C^{-1}v = Ap$. In practice this will be two linear systems for the velocity components v_1 and v_2, with the same Laplace matrix—two "Poisson solves" at each step. This is not impossible but some inner iteration like multigrid is likely to be used.

A number of iterative methods compute both v and p, based on the block form:

1. The **Uzawa method** has $C^{-1}v^{k+1} = f - Ap^k$ and $p^{k+1} = p^k + \alpha A^{\mathrm{T}}v^{k+1}$
2. **Penalty methods** are described in Chapter ____ on optimization
3. The **augmented Lagrangian method** is presented in detail by Glowinski-Le Tallec.

These and more are discussed by Quarteroni-Valli for the Stokes problem. They all involve the "Poisson solve" associated with C. We turn instead to a closer look at *preconditioners for the general saddle point problem*, whether it arises in fluid flow or optimization or network analysis or elsewhere.

Preconditioning Saddle Point Problems

The basic methods for large sparse indefinite problems are MINRES and GMRES (symmetric and nonsymmetric). Our focus here is on constructing a good preconditioner.

When we only know the matrix entries in C^{-1} and A, the code will have to construct a good preconditioner "blindly" from that information. Recall the fundamental dividing line mentioned in the preface, to be *given a matrix or given a problem*. When we know the underlying problem (like Stokes), we can devise a preconditioner by hand. There is a similar separation between algebraic multigrid and geometric multigrid (the computer thinks or we do).

©2006 Gilbert Strang

Look first at the saddle point matrix M and its block factorization $M = LDL^{\mathrm{T}}$:

$$M = \begin{bmatrix} C^{-1} & A \\ A^{\mathrm{T}} & 0 \\ m & n \end{bmatrix} = \begin{bmatrix} I & 0 \\ A^{\mathrm{T}}C & I \\ m & n \end{bmatrix} \begin{bmatrix} C^{-1} & 0 \\ 0 & -A^{\mathrm{T}}CA \\ m & n \end{bmatrix} \begin{bmatrix} I & CA \\ 0 & I \\ m & n \end{bmatrix}. \tag{10}$$

M^{-1} clearly involves C and $(A^{\mathrm{T}}CA)^{-1}$ from the inverse of that middle matrix. Our preconditioners will keep this m, n block structure. To come close to M^{-1}, as preconditioners are intended to do, we could approximate the blocks of M and invert or we could directly approximate C and $(A^{\mathrm{T}}CA)^{-1}$.

Three approximations to M are suggested by Elman-Silvester-Wathen:

Preconditioners $P_1 = \begin{bmatrix} \widehat{C}^{-1} & A \\ A^{\mathrm{T}} & 0 \end{bmatrix}$ $P_2 = \begin{bmatrix} \widehat{C}^{-1} & 0 \\ 0 & \widehat{K} \end{bmatrix}$ $P_3 = \begin{bmatrix} \widehat{C}^{-1} & A \\ 0 & \widehat{K} \end{bmatrix}$

The matrix A (often relatively simple) is kept unchanged. $\widehat{C}$ approximates the Poisson solver C, and $\widehat{K}$ approximates $K = A^{\mathrm{T}}CA$. We must choose $\widehat{C}$ and $\widehat{K}$.

A key point about P_2 and P_3: If $\widehat{C} = C$ and $\widehat{K} = K$, then $P_2^{-1}M$ has only *three* different eigenvalues and $P_3^{-1}M$ has only the eigenvalue $\lambda = 1$. In this case. In that case MINRES and GMRES converge in three or two steps. So those changes from P_1 add very few iterations. Wathen begins with simple suggestions for the preconditioner P_2^{-1}:

1. Replace the Poisson solver C by a single multigrid cycle $\widehat{C}$

2. Replace K^{-1} by four conjugate gradient steps preconditioned by Q or $\mathsf{diag}(Q)$.

The multigrid cycle applies a basic iteration on fine and also coarse meshes, to reduce both high and low frequencies in the error residual (Section ____). The matrix Q that preconditions K is the pressure mass matrix in (7). Its energy $p^{\mathrm{T}}Qp$ gives a lower (and also upper) bound to $p^{\mathrm{T}}Kp$. Usually $\mathsf{diag}(Q)$ also has this "equispectral property", scaling the system correctly even if the approximation to K is not really close.

$\widehat{C}$ and $\widehat{K}$ are defined algorithmically by multigrid and preconditioned conjugate gradient cycles. This is simpler than trying to express them by a matrix.

Nonsymmetric Problems and Model Reduction

We move now toward problems more general than Stokes. They might be associated with the Navier-Stokes equations (div v is still zero) and its linearizations. In that case C will be unsymmetric, from the first derivative terms in the linearization. *Convection joins diffusion* (also in many problems from chemical engineering).

A key point is the relative strength of those two terms, inertia versus viscosity. It is measured by the **Reynolds number** Re = (density)(velocity)(distance)/(viscosity).

Stokes flow is the limiting case Re = 0 for high viscosity and low velocity. The high speed Euler equation $dv/dt = -\operatorname{grad} p + F$ is at the other extreme Re $\to \infty$.

Beyond computational fluid dynamics are other differential equations with their own constraints, and then all the algebraic problems of constrained optimization. Our discussion is moving from "given a problem" toward "given a matrix." The constraints make it a saddle point matrix—possibly unsymmetric, with A^{T} changed to B. We look to preconditioners of the same general forms, as a guide.

Always the key is an approximate inverse to the *Schur complement* $-K = -BCA$:

$$\text{Elimination reduces } \begin{bmatrix} C^{-1} & A \\ B & 0 \end{bmatrix} \text{ to } \begin{bmatrix} C^{-1} & A \\ 0 & -BCA \end{bmatrix} = \begin{bmatrix} C^{-1} & A \\ 0 & -K \end{bmatrix}. \tag{11}$$

Normally K will be dense. If it is also large, we look at iterative methods. Multigrid for C will become *algebraic multigrid* [Briggs-McCormick-van Hensen]. The approximation is constructed directly from the matrix, not using the differential equation that may be in the background.

For $K = BCA$, a "black box" approach to preconditioning is suggested in [____]: Reduce C to a smaller model matrix $\widehat{C}$, satisfying

$$A\tilde{C} \approx CA \quad \text{and then} \quad \widehat{K} = BA\widehat{C} \approx BCA. \tag{12}$$

Notice that C is m by m (large) while $\widehat{C}$ is n by n. The model reduction $CA = A\widehat{C}$ will not be exactly solvable (more equations than unknowns). The meaning of $\approx$ is to be decided by the user! We may apply weighted least squares to compute $\widehat{C}^{-1}$ from $A\widehat{C}^{-1} \approx C^{-1}A$, since the saddle point problem gives us C^{-1} and not C.

I expect new ideas will keep coming, for this fundamental indefinite problem.

Advection-Diffusion Problems

The Stokes problem involves the Laplacian $-\Delta v$. The corresponding matrix C^{-1} is "essentially symmetric." Perfect symmetry holds in the interior and could only be lost at the boundary (it shouldn't be). Now, linearizing the $v \cdot \operatorname{grad} v$ term in the Navier-Stokes equation produces a first derivative "advection" term, and symmetry is definitely lost.

Chapter 7

Optimization and Minimum Principles

7.1 Two Fundamental Examples

Within the universe of applied mathematics, optimization is often a world of its own. There are occasional expeditions to other worlds (like differential equations), but mostly the life of optimizers is self-contained: ***Find the minimum of $F(x_1, \ldots, x_n)$.*** That is not an easy problem, especially when there are many variables x_j and many constraints on those variables. *Those constraints may require $Ax = b$ or $x_j \geq 0$ or both or worse.* Whole books and courses and software packages are dedicated to this problem of ***constrained minimization***.

I hope you will forgive the poetry of worlds and universes. I am trying to emphasize the importance of optimization—a key component of engineering mathematics and scientific computing. This chapter will have its own flavor, but it is strongly connected to the rest of this book. To make those connections, I want to begin with two specific examples. **If you read even just this section**, you will see the connections.

Least Squares

Ordinary least squares begins with a matrix A whose n columns are independent. The rank is n, so $A^{\mathrm{T}}A$ is symmetric positive definite. The input vector b has m components, the output $\widehat{u}$ has n components, and $\boldsymbol{m > n}$:

Least squares problem	Minimize $\|Au - b\|^2$	(1)
Normal equations for best $\widehat{u}$	$A^{\mathrm{T}}A\widehat{u} = A^{\mathrm{T}}b$	

Those equations $A^{\mathrm{T}}A\widehat{u} = A^{\mathrm{T}}b$ say that the error residual $e = b - A\widehat{u}$ solves $\boldsymbol{A^{\mathrm{T}}e = 0}$. Then e is perpendicular to columns $1, 2, \ldots, n$ of A. Write those zero inner products

©2006 Gilbert Strang

as $\mathbf{(column)^T(e) = 0}$ to find $A^TA\widehat{u} = A^Tb$:

$$\begin{bmatrix} (\text{column } 1)^T \\ \vdots \\ (\text{column } n)^T \end{bmatrix} \begin{bmatrix} \\ e \\ \\ \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix} \quad \text{is} \quad \begin{matrix} A^Te = 0 \\ A^T(b - A\widehat{u}) = 0 \\ A^TA\widehat{u} = A^Tb\,. \end{matrix} \tag{2}$$

Graphically, Figure 7.1 shows $A\widehat{u}$ as the *projection of* b. It is the combination of columns of A (the point in the column space) that is nearest to b. We studied least squares in Section 2.3, and now we notice that **a second problem is solved at the same time**.

This second problem (*dual problem*) does not project b down onto the column space. Instead it projects b across onto the perpendicular space. In the 3D picture, that space is a line (its dimension is $3 - 2 = 1$). In m dimensions that perpendicular subspace has dimension $m - n$. *It contains the vectors that are perpendicular to all columns of* A. The line in Figure 7.1 is the **nullspace of $\boldsymbol{A^T}$**.

One of the vectors in that perpendicular space is $e =$ projection of b! Together, e and $\widehat{u}$ solve the two linear equations that express exactly what the figure shows:

Primal-Dual
Saddle Point
Kuhn-Tucker (KKT)

$$\begin{matrix} e & + & A\widehat{u} & = & b & m \text{ equations} \\ A^Te & & & = & 0 & n \text{ equations} \end{matrix} \tag{3}$$

We took this chance to write down three names for these very simple but so fundamental equations. I can quickly say a few words about each name.

Primal-Dual The primal problem is to minimize $\|Au - b\|^2$. This produces $\widehat{u}$. The dual problem is to minimize $\|w - b\|^2$, under the condition that $A^Tw = 0$. This produces e. We can't solve one problem without solving the other. They are solved together by equation (3), which finds the projections in both directions.

Saddle Point The block matrix S in those equations is not positive definite!

Saddle point matrix

$$\boldsymbol{S = \begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix}} \tag{4}$$

The first m pivots are all 1's, from the matrix I. When elimination puts zeros in place of A^T, *it puts the negative definite* $-A^TA$ *into the zero block.*

Multiply row 1 by $\boldsymbol{A^T}$
Subtract from row 2

$$\begin{bmatrix} I & 0 \\ -A^T & I \end{bmatrix} \begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} = \begin{bmatrix} I & A \\ 0 & \boldsymbol{-A^TA} \end{bmatrix}. \tag{5}$$

That elimination produced $-A^TA$ in the $(2,2)$ block (the "Schur complement"). So the final n pivots will all be negative. S is **indefinite**, with pivots of both signs.

S doesn't produce a pure minimum or maximum, positive or negative definite. It leads to a saddle point $(\widehat{u}, e)$. When we get up more courage, we will try to draw this.

Kuhn-Tucker These are the names most often associated with equations like (3) that solve optimization problems. Because of an earlier Master's Thesis by Karush, you often see "KKT equations." In continuous problems, for functions instead of vectors, the right name would be "Euler-Lagrange equations." When the constraints include inequalities like $w \geq 0$ or $Bu = d$, Lagrange multipliers are still the key. For those more delicate problems, Kuhn and Tucker earned their fame.

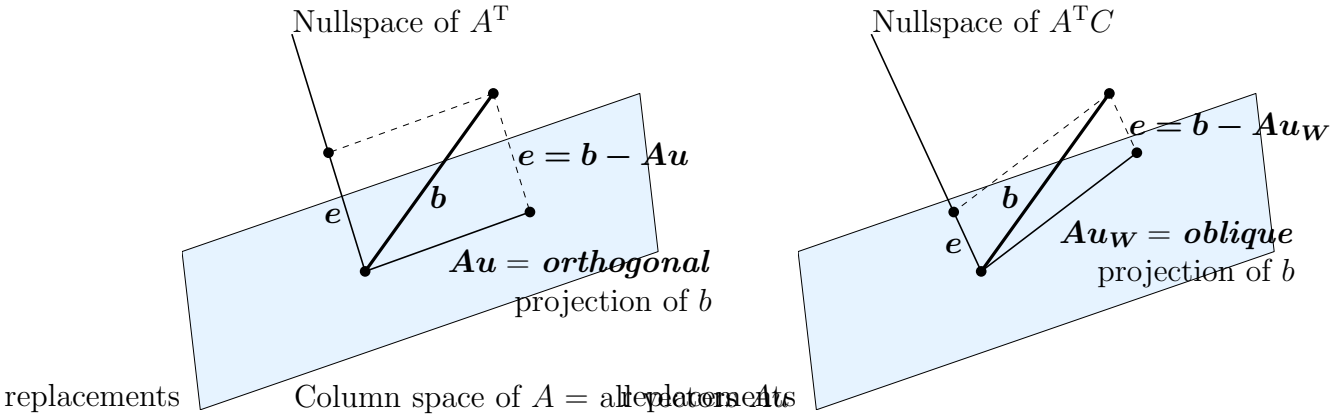


Figure 7.1: Ordinary and weighted least squares: $\min \|b - Au\|^2$ and $\|Wb - WAu\|^2$.

Weighted Least Squares

This is a small but very important extension of the least squares problem. It involves the same rectangular A, and a square weighting matrix W. Instead of $\widehat{u}$ we write $\widehat{u}_W$ (this best answer changes with W). You will see that the symmetric positive definite combination $C = W^{\mathrm{T}}W$ is what matters in the end.

Weighted least squares	Minimize $\|WAu - Wb\|^2$	(6)
Normal equations for $\widehat{u}_W$	$(WA)^{\mathrm{T}}(WA)\,\widehat{u}_W = (WA)^{\mathrm{T}}(Wb)$	

No new mathematics, just replace A and b by WA and Wb. The equation has become

$$A^{\mathrm{T}}W^{\mathrm{T}}WA\,\widehat{u}_W = A^{\mathrm{T}}W^{\mathrm{T}}Wb \quad \text{or} \quad A^{\mathrm{T}}CA\,\widehat{u}_W = A^{\mathrm{T}}Cb \quad \text{or} \quad A^{\mathrm{T}}C(b - A\,\widehat{u}_W) = 0. \tag{7}$$

In the middle equation is that all-important matrix $A^{\mathrm{T}}CA$. In the last equation, $\boldsymbol{A^{\mathrm{T}}e = 0}$ **has changed to** $\boldsymbol{A^{\mathrm{T}}Ce = 0}$. When I made that change in Figure 7.1, I lost the 90° angles. The line is no longer perpendicular to the plane, and the projections are no longer orthogonal. We are still splitting b into two pieces, $A\widehat{u}_W$ in the column space and e *in the nullspace of* $A^{\mathrm{T}}C$. The equations now include this $C = W^{\mathrm{T}}W$:

e is "C-orthogonal"	e	$+$	$A\widehat{u}_W$	$=$	b
to the columns of A	$A^{\mathrm{T}}Ce$			$=$	0

(8)

With a simple change, the equations (8) become symmetric! **Introduce $\boldsymbol{w = Ce}$ and $\boldsymbol{e = C^{-1}w}$ and shorten $\widehat{\boldsymbol{u}}_{\boldsymbol{W}}$ to $\boldsymbol{u}$:**

Primal-Dual Saddle Point Kuhn-Tucker

$$\begin{array}{rcrcl} C^{-1}w & + & Au & = & b \\ A^{\mathrm{T}}w & & & = & 0 \end{array} \tag{9}$$

This weighted saddle point matrix replaces I by C^{-1} (still symmetric):

Saddle point matrix

$$\boldsymbol{S} = \begin{bmatrix} \boldsymbol{C^{-1}} & \boldsymbol{A} \\ \boldsymbol{A^{\mathrm{T}}} & \boldsymbol{0} \end{bmatrix} \begin{matrix} m \text{ rows} \\ n \text{ rows} \end{matrix}$$

Elimination produces m positive pivots from C^{-1}, and n negative pivots from $-A^{\mathrm{T}}CA$:

$$\begin{bmatrix} C^{-1} & A \\ A^{\mathrm{T}} & 0 \end{bmatrix} \begin{bmatrix} w \\ u \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix} \longleftrightarrow \begin{bmatrix} C^{-1} & A \\ \boldsymbol{0} & \boldsymbol{-A^{\mathrm{T}}CA} \end{bmatrix} \begin{bmatrix} w \\ u \end{bmatrix} = \begin{bmatrix} b \\ \boldsymbol{-A^{\mathrm{T}}Cb} \end{bmatrix}. \tag{10}$$

The "Schur complement" that appears in the $2,2$ block becomes $-A^{\mathrm{T}}CA$. We are back to $A^{\mathrm{T}}CAu = A^{\mathrm{T}}Cb$ and all its applications.

Two more steps will finish this overview of optimization. We show how a different vector f can appear on the right side. The bigger step is also taken by our second example, coming now. *The dual problem (for w not u) has a constraint.* At first it was $A^{\mathrm{T}}e = 0$, now it is $A^{\mathrm{T}}w = 0$, and in the example it will be $A^{\mathrm{T}}w = f$.

How do you minimize a function of e or w when these constraints are enforced? Lagrange showed the way, with his multipliers.

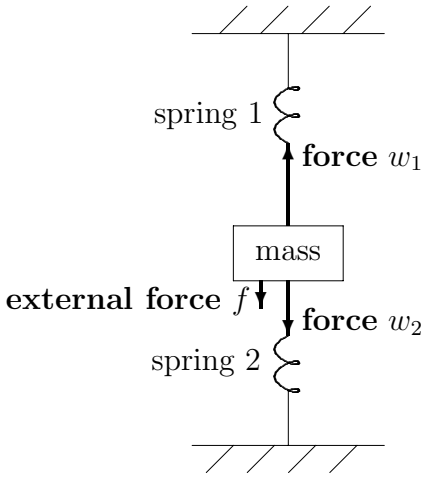
Minimizing with Constraints

The second example is a line of *two springs and one mass*. The function to minimize is the energy in the springs. The constraint is the balance $A^{\mathrm{T}}w = f$ between internal forces (in the springs) and the external force (on the mass). I believe you can see in Figure 7.2 the fundamental problem of constrained optimization. The forces are drawn as if both springs are stretched with forces $f > 0$, pulling on the mass. Actually spring 2 will be compressed (w_2 is negative).

As I write those words—*spring*, *mass*, *energy*, *force balance*—I am desperately hoping that you won't just say "this is not my area." Changing the example to another area of science or engineering or economics would be easy, the problem stays the same in all languages.

All of calculus trains us to minimize functions: **Set the derivative to zero**! But the basis calculus course doesn't deal properly with constraints. We are minimizing an energy function $\mathrm{E}(w_1, w_2)$, but we are constrained to stay on the line $w_1 - w_2 = f$. *What derivatives do we set to zero?*

A direct approach is to solve the constraint equation. Replace w_2 by $w_1 - f$. That seems natural, but I want to advocate a different approach (which leads to the same result). Instead of looking for w's that satisfy the constraint, the idea of Lagrange is



Internal energy in the springs
$E(w) = E_1(w_1) + E_2(w_2)$

Balance of internal/external forces
$w_1 - w_2 = f$ (*the constraint*)

Constrained optimization problem
Minimize $E(w)$ *subject to* $w_1 - w_2 = f$

Figure 7.2: Minimum spring energy $E(w)$ subject to balance of forces.

to *build the constraints into the function.* Rather than removing w_2, we will add a new unknown u. It might seem surprising, but this second approach is better.

With n constraints on m unknowns, Lagrange's method has $m+n$ unknowns. The idea is to add a **Lagrange multiplier** for each constraint. (Books on optimization call this multiplier λ or π, we will call it u.) Our Lagrange function L has the constraint $w_1 - w_2 - f = 0$ built in, and multiplied by $-u$:

Lagrange function $L(w_1, w_2, u) = E_1(w_1) + E_2(w_2) - u(w_1 - w_2 - f)\,.$

Calculus can operate on L, by setting derivatives (*three partial derivatives*!) to zero:

Kuhn-Tucker optimality equations

$$\frac{\partial L}{\partial w_1} = \frac{dE_1}{dw}(w_1) - u = 0 \tag{11a}$$

$$\frac{\partial L}{\partial w_2} = \frac{dE_2}{dw}(w_2) + u = 0 \tag{11b}$$

Lagrange multiplier u

$$\frac{\partial L}{\partial u} = -(w_1 - w_2 - f) = 0 \tag{11c}$$

Notice how the third equation $\partial L/\partial u = 0$ automatically brings back the constraint—because it was just multiplied by $-u$. If we add the first two equations to eliminate u, and substitute $w_1 - f$ for w_2, we are back to the direct approach with one unknown.

But we don't want to eliminate u! That Lagrange multiplier is an important number with a meaning of its own. In this problem, u is the displacement of the mass. In economics, u is the selling price to maximize profit. In all problems, u measures the **sensitivity of the answer** (the minimum energy E_{min}) to a change in the constraint. We will see this sensitivity dE_{min}/df in the linear case.

Linear Case

The force in a linear spring is proportional to the elongation e, by Hooke's Law $w = ce$. Each small stretching step requires work = (force)(movement) = $(ce)(\Delta e)$. Then the integral $\frac{1}{2}ce^2$ that adds up those small steps gives the energy stored in the spring. We can express this energy in terms of e or w:

Energy in a spring
$$E(w) = \frac{1}{2}c\,e^2 = \frac{1}{2}\frac{w^2}{c}\,. \tag{12}$$

Our problem is to minimize a quadratic energy $E(w)$ subject to a linear balance equation $w_1 - w_2 = f$. This is the model problem of optimization.

Minimize $E(w) = \frac{1}{2}\frac{w_1^2}{c_1} + \frac{1}{2}\frac{w_2^2}{c_2}$ **subject to** $w_1 - w_2 = f\,.$ (13)

We want to solve this model problem by geometry and then by algebra.

Geometry In the plane of w_1 and w_2, draw the line $w_1 - w_2 = f$. Then draw the ellipse $E(w) = E_{\mathbf{min}}$ that *just touches this line*. The line is tangent to the ellipse. A smaller ellipse from smaller forces w_1 and w_2 will not reach the line—those forces will not balance f. A larger ellipse will not give minimum energy. This ellipse touches the line at the point (w_1, w_2) that minimizes $E(w)$.

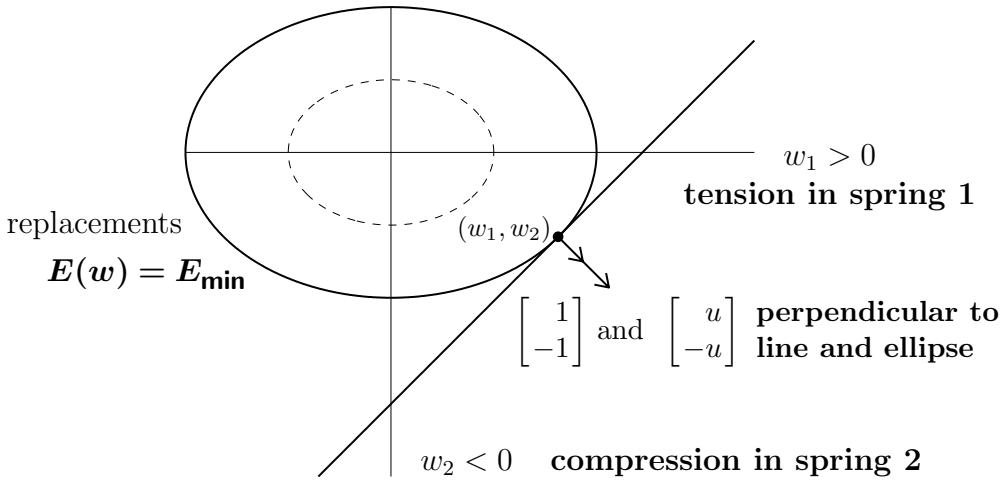


Figure 7.3: The ellipse $E(w) = E_{\mathbf{min}}$ touches $w_1 - w_2 = f$ at the solution (w_1, w_2).

At the touching point in Figure 7.3, the perpendiculars $(1, -1)$ and $(u, -u)$ to the line and the ellipse are parallel. The perpendicular to the line is the vector $(1, -1)$ from the partial derivatives of $w_1 - w_2 - f$. The perpendicular to the ellipse is $(\partial E/\partial w_1, \partial E/\partial w_2)$, from the gradient of $E(w)$. By the optimality equations (11a) and (11b), this is exactly $(u, -u)$. Those parallel gradients at the solution are the algebraic statement that **the line is tangent to the ellipse**.

Algebra To find (w_1, w_2), start with the derivatives of $w_1^2/2\,c_1$ and $w_2^2/2\,c_2$:

$$\textbf{Energy gradient} \qquad \frac{\partial E}{\partial w_1} = \frac{w_1}{c_1} \quad \text{and} \quad \frac{\partial E}{\partial w_2} = \frac{w_2}{c_2}\,. \tag{14}$$

Equations (11a) and (11b) in Lagrange's method become $w_1/c_1 = u$ and $w_2/c_2 = -u$. Now the constraint $w_1 - w_2 = f$ yields $(c_1 + c_2)u = f$ (*both* w's *are eliminated*):

$$\text{Substitute } w_1 = c_1 u \text{ and } w_2 = -c_2 u. \quad \text{Then } (c_1 + c_2)u = f\,. \tag{15}$$

I don't know if you recognize $c_1 + c_2$ as our stiffness matrix $A^{\mathrm{T}}CA$! This problem is so small that you could easily miss $K = A^{\mathrm{T}}CA$. The matrix A^{T} in the constraint equation $A^{\mathrm{T}}w = w_1 - w_2 = f$ is only 1 by 2, so *the stiffness matrix* K *is* 1 *by* 1:

$$A^{\mathrm{T}} = \begin{bmatrix} 1 & -1 \end{bmatrix} \text{ and } K = A^{\mathrm{T}}CA = \begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} c_1 & \\ & c_2 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} c_1 + c_2 \end{bmatrix}. \tag{16}$$

The algebra of Lagrange's method has recovered $Ku = f$. Its solution is the movement $u = f/(c_1{+}c_2)$ of the mass. Equation (15) eliminated w_1 and w_2 using (11a) and (11b). Now back substitution finds those energy-minimizing forces:

$$\textbf{Spring forces} \qquad w_1 = c_1 u = \frac{c_1 f}{c_1 + c_2} \quad \text{and} \quad w_2 = -c_2 u = \frac{-c_2 f}{c_1 + c_2}\,. \tag{17}$$

Those forces (w_1, w_2) are on the ellipse of minimum energy $E_{\mathbf{min}}$, tangent to the line:

$$E(w) = \frac{1}{2}\frac{w_1^2}{c_1} + \frac{1}{2}\frac{w_2^2}{c_2} = \frac{1}{2}\frac{c_1 f^2}{(c_1 + c_2)^2} + \frac{1}{2}\frac{c_2 f^2}{(c_1 + c_2)^2} = \boldsymbol{\frac{1}{2}\frac{f^2}{c_1 + c_2} = E_{\mathrm{min}}}\,. \tag{18}$$

This $E_{\mathbf{min}}$ must be the same minimum value $\frac{1}{2}f^{\mathrm{T}}K^{-1}f$ as in Section _____. It is.

We can directly verify the mysterious fact that u measures the sensitivity of $E_{\mathbf{min}}$ to a small change in f. Compute the derivative $dE_{\mathbf{min}}/df$:

$$\textbf{Lagrange multiplier = Sensitivity} \qquad \frac{d}{df}\left(\frac{1}{2}\frac{f^2}{c_1 + c_2}\right) = \frac{f}{c_1 + c_2} = u\,. \tag{19}$$

This sensitivity is linked to the observation in Figure 7.3 that one gradient is u times the other gradient. From (11a) and (11b), that stays true for nonlinear springs.

A Specific Example

I want to insert $c_1 = c_2 = 1$ in this model problem, to see the saddle point of L more clearly. The Lagrange function with built-in constraint depends on w_1 and w_2 and u:

$$\boldsymbol{L = \frac{1}{2}w_1^2 + \frac{1}{2}w_2^2 - uw_1 + uw_2 + uf}\,. \tag{20}$$

The equations $\partial L/\partial w_1 = 0$ and $\partial L/\partial w_2 = 0$ and $\partial L/\partial u = 0$ produce a beautiful symmetric saddle-point matrix S:

$$\begin{array}{rl} \partial L/\partial w_1 = & w_1 - u = 0 \\ \partial L/\partial w_2 = & w_2 + u = 0 \\ \partial L/\partial u = & -w_1 + w_2 = f \end{array} \quad \text{or} \quad \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \\ -1 & 1 & 0 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \\ u \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ f \end{bmatrix}. \tag{21}$$

Is this matrix S positive definite? *No.* It is invertible, and its pivots are $1, 1, -2$. That -2 destroys positive definiteness—it means a saddle point:

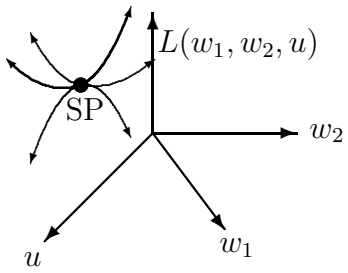
$$\textbf{Elimination} \quad \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \\ -1 & 1 & 0 \end{bmatrix} \longrightarrow \begin{bmatrix} \mathbf{1} & 0 & -1 \\ & \mathbf{1} & 1 \\ & & \mathbf{-2} \end{bmatrix} \quad \text{with } L = \begin{bmatrix} 1 & & \\ 0 & 1 & \\ -1 & 1 & 1 \end{bmatrix}.$$

On a symmetric matrix, elimination equals "completing the square." The pivots $1, 1, -2$ are outside the squares. The entries of L are inside the squares:

$$\frac{1}{2}w_1^2 + \frac{1}{2}w_2^2 - uw_1 + uw_2 = \frac{1}{2}\left[1(w_1 - u)^2 + 1(w_2 + u)^2 - 2(u)^2\right]. \tag{22}$$

The first squares $(w_1 - u)^2$ and $(w_2 + u)^2$ go "upwards," but $-2u^2$ goes down. **This gives a saddle point SP = (w_1, w_2, u) in Figure 7.4.**

The eigenvalues of a symmetric matrix have the same signs as the pivots, and the same product (which is $\det S = -2$). Here the eigenvalues are $\lambda = 1, 2, -1$.



$L = \frac{1}{2}\left[(w_1 - u)^2 + (w_2 + u)^2 - 2u^2\right] + uf$

Four dimensions make it a squeeze

Saddle point SP $= (w_1, w_2, u) = \dfrac{(c_1 f, -c_2 f, f)}{c_1 + c_2}$

Figure 7.4: $(w_1 - u)^2$ and $(w_2 + u)^2$ go up, $-2u^2$ goes down from the saddle point SP.

The Fundamental Problem

May I describe the full linear case with $w = (w_1, \ldots, w_m)$ and $A^{\mathrm{T}}w = (f_1, \ldots, f_n)$? The problem is to minimize the total energy $E(w) = \frac{1}{2}w^{\mathrm{T}}C^{-1}w$ in the m springs. The n constraints $A^{\mathrm{T}}w = f$ are built in by Lagrange multipliers $u_1, \ldots, u_n$. Multiplying the force balance on the kth mass by $-u_k$ and adding, all n constraints are built into the dot product $u^{\mathrm{T}}(A^{\mathrm{T}}w - f)$. For mechanics, we use a minus sign in L:

$$\textbf{Lagrange function} \qquad L(w, u) = \frac{1}{2}w^{\mathrm{T}}C^{-1}w - u^{\mathrm{T}}(A^{\mathrm{T}}w - f). \tag{23}$$

To find the minimizing w, set the $m+n$ first partial derivatives of L to zero:

Kuhn-Tucker optimality equations

$$\partial L/\partial w = C^{-1}w - Au = 0 \tag{24a}$$

$$\partial L/\partial u = -A^{\mathrm{T}}w + f = 0 \tag{24b}$$

This is the main point, that Lagrange multipliers lead exactly to the linear equations $w = CAu$ and $A^{\mathrm{T}}w = f$ that we studied in the first chapters of the book. By using $-u$ in the Lagrange function L and introducing $e = Au$, we have the plus signs that appeared for springs and masses:

$$e = Au \qquad w = Ce \qquad f = A^{\mathrm{T}}w \qquad \Longrightarrow \qquad A^{\mathrm{T}}CAu = f\,.$$

Important Least squares problems have $e = b - Au$ (minus sign from voltage drops). **Then we change to $+\boldsymbol{u}$ in $\boldsymbol{L}$.** The energy $E = \frac{1}{2}w^{\mathrm{T}}C^{-1}w - b^{\mathrm{T}}w$ now has a term involving b. When Lagrange sets derivatives of L to zero, he finds S!

$$\begin{aligned} \partial L/\partial w &= C^{-1}w + Au - b = 0 \\ \partial L/\partial u &= A^{\mathrm{T}}w \qquad\quad - f = 0 \end{aligned} \quad \text{or} \quad \begin{bmatrix} \boldsymbol{C^{-1}} & \boldsymbol{A} \\ \boldsymbol{A^{\mathrm{T}}} & \boldsymbol{0} \end{bmatrix} \begin{bmatrix} \boldsymbol{w} \\ \boldsymbol{u} \end{bmatrix} = \begin{bmatrix} \boldsymbol{b} \\ \boldsymbol{f} \end{bmatrix}. \tag{25}$$

This system is my top candidate for the fundamental problem of scientific computing.

You could eliminate $w = C(b - Au)$ *but I don't know if you should.* If you do it, $K = A^{\mathrm{T}}CA$ will appear. Usually this is a good plan:

Remove $\boldsymbol{w}$ $A^{\mathrm{T}}w = A^{\mathrm{T}}C(b - Au) = f$ which is $A^{\mathrm{T}}CAu = A^{\mathrm{T}}Cb - f\,.$ (26)

Duality and Saddle Points

We minimize the energy $E(w)$ but we do *not* minimize Lagrange's function $L(w, u)$. It is true that $\partial L/\partial w$ and $\partial L/\partial u$ are zero at the solution. But the matrix of second derivatives of L is not positive definite. **The solution w, u is a saddle point**. It is a minimum of L over w, and at the same time it is a maximum of L over u. A saddle point has something to do with a horse... It is like the lowest point in a mountain range, which is also the highest point as you go across.

The *minimax theorem* states that we can minimize first (over w) or maximize first (over u). Either order leads to the unique saddle point, given by $\partial L/\partial w = 0$ and $\partial L/\partial u = 0$. The minimization removes w from the problem, and it corresponds exactly to eliminating w from the equations (11) for "first derivatives = 0." Every step will be illustrated by examples (linear case first).

Allow me to use the A, C, A^{T} notation that we already know, so I can point out the fantastic idea of **duality**. The Lagrangian is $L(w, u) = \frac{1}{2}w^{\mathrm{T}}C^{-1}w - u^{\mathrm{T}}(A^{\mathrm{T}}w - f)$, leaving out b for simplicity. We compare minimization first and maximization first:

Minimize over w $\partial L/\partial w = 0$ when $C^{-1}w - Au = 0$. Then $w = CAu$ and $L = -\frac{1}{2}(Au)^{\mathrm{T}}C(Au) + u^{\mathrm{T}}f$. This is to be maximized.

Maximize over u Key point: $L_{\mathbf{max}} = +\infty$ if $A^{\mathrm{T}}w \neq f$. Minimizing over w will keep $A^{\mathrm{T}}w = f$ to avoid that $+\infty$. Then $L = \frac{1}{2}w^{\mathrm{T}}C^{-1}w$.

The maximum of a linear function like $5u$ is $+\infty$. But the maximum of $0u$ is 0.

Now come the two second steps, maximize over u and minimize over w. **Those are the dual problems**. Often one problem is called the "primal" and the other is its "dual"—but they are reversible. Here are the two dual principles for linear springs:

1. Choose u to maximize $-\frac{1}{2}(Au)^{\mathrm{T}}C(Au) + u^{\mathrm{T}}f$. Call this function $-P(u)$.
2. Choose w to minimize $\frac{1}{2}w^{\mathrm{T}}C^{-1}w$ keeping the force balance $A^{\mathrm{T}}w = f$.

Our original problem was **2**. In the language of mechanics, we were minimizing the complementary energy $E(w)$. Its dual problem is **1**. This minimizes the potential energy $P(u)$, by maximizing $-P(u)$. Most finite element systems choose that "displacement method." They work with u because it avoids the constraint $A^{\mathrm{T}}w = f$.

The dual problems **1** and **2** involve the same inputs A, C, f but they look entirely different. *Equality between minimax(L) and maximin(L) gives the duality principle and the saddle point*:

Duality of 1 and 2
$$\max_{\text{all } u}(-P(u)) = \min_{A^{\mathrm{T}}w = f} E(w)\,. \tag{27}$$

That is the big theorem of optimization—**the maximum of one problem equals the minimum of its dual**. We can find those numbers, and see that they are equal, because the derivatives are linear. Maximizing $-P(u)$ will minimize $P(u)$, which is the problem we solved in Chapter 1. Write u^* and w^* for the minimizer and maximizer:

1. $P(u) = \frac{1}{2}u^{\mathrm{T}}A^{\mathrm{T}}CAu - u^{\mathrm{T}}f$ is $P_{\mathbf{min}} = -\frac{1}{2}f^{\mathrm{T}}(A^{\mathrm{T}}CA)^{-1}f$ when $u^* = (A^{\mathrm{T}}CA)^{-1}f$
2. $E(w) = \frac{1}{2}w^{\mathrm{T}}C^{-1}w$ is $E_{\mathbf{min}} = \frac{1}{2}f^{\mathrm{T}}(A^{\mathrm{T}}CA)^{-1}f$ when $w^* = CAu^*$.

So $u^* = K^{-1}f$ and $w^* = CAu^*$ give the saddle point (w^*, u^*) of L. This is where $E_{\mathbf{min}} = -P_{\mathbf{min}}$.

Problem Set 7.1

1 Our model matrix M in (21) has eigenvalues $\lambda_1 = 1, \lambda_2 = 2, \lambda_3 = -1$:

$$M = \begin{bmatrix} C^{-1} & A \\ A^{\mathrm{T}} & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \\ -1 & 1 & 0 \end{bmatrix}$$

The trace $1 + 1 + 0$ down the diagonal of M equals the sum of λ's. Check that $\det M$ = product of λ's. Find eigenvectors x_1, x_2, x_3 of unit length for those eigenvalues $1, 2, -1$. The eigenvectors are orthogonal!

2 The quadratic part of the Lagrangian $L(w_1, w_2, u)$ comes directly from M:

$$\frac{1}{2}\begin{bmatrix} w_1 & w_2 & u \end{bmatrix}\begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & 1 \\ -1 & 1 & 0 \end{bmatrix}\begin{bmatrix} w_1 \\ w_2 \\ u \end{bmatrix} = \frac{1}{2}\left(w_1^2 + w_2^2 - 2uw_1 + 2uw_2\right).$$

Put the unit eigenvectors x_1, x_2, x_3 inside the squares and $\lambda = 1, 2, -1$ outside:

$$w_1^2 + w_2^2 - 2uw_1 + 2uw_2 = 1(____)^2 + 2(____)^2 - 1(____)^2 .$$

The first parentheses contain $(1w_1 - 1w_2 + 0w_3)/\sqrt{2}$ because x_1 is $(1, -1, 0)/\sqrt{2}$. Compared with (22), these squares come from orthogonal eigenvector directions. We are using $A = Q\Lambda Q^{\mathrm{T}}$ instead of $A = LDL^{\mathrm{T}}$.

3 *Weak duality* Half of the duality theorem is $\max -P(u) \le \min E(w)$. This is surprisingly easy to prove. Show that $-P(u)$ is *always smaller* than $E(w)$, for every u and w with $A^{\mathrm{T}}w = f$.

$$-\frac{1}{2}u^{\mathrm{T}}A^{\mathrm{T}}CAu + u^{\mathrm{T}}f \le \frac{1}{2}w^{\mathrm{T}}C^{-1}w \quad \text{whenever } A^{\mathrm{T}}w = f .$$

Set $f = A^{\mathrm{T}}w$. Verify that (right side) $-$ (left side) $= \frac{1}{2}(w - CAu)^{\mathrm{T}}C^{-1}(w - CAu) \ge 0$.

Equality holds and $\max(-P) = \min(E)$ when $w = CAu$. That is equation (11)!

4 Suppose the lower spring in Figure 7.2 is not fixed at the bottom. A mass at that end adds a new force balance constraint $w_2 - f_2 = 0$. Build the old and new constraints into the Lagrange function $L(w_1, w_2, u_1, u_2)$ to minimize the energy $E_1(w_1) + E_2(w_2)$. Write down *four equations* like (11a)–(11c): partial derivatives of L are zero.

5 For spring energies $E_1 = \frac{1}{2}w_1^2/c_1$ and $E_2 = \frac{1}{2}w_2^2/c_2$, find A in the block form

$$\begin{bmatrix} C^{-1} & A \\ A^{\mathrm{T}} & 0 \end{bmatrix}\begin{bmatrix} w \\ u \end{bmatrix} = \begin{bmatrix} 0 \\ f \end{bmatrix} \quad \text{with } w = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, u = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}, f = \begin{bmatrix} f_1 \\ f_2 \end{bmatrix}.$$

Elimination subtracts $A^{\mathrm{T}}C$ times the first block row from the second. With $c_1 = c_2 = 1$, what matrix $-A^{\mathrm{T}}CA$ enters the zero block? Solve for $u = (u_1, u_2)$.

6 Continuing Problem 5 with $C = I$, write down $w = CAu$ and compute the energy $E_{\mathbf{min}} = \frac{1}{2}w_1^2 + \frac{1}{2}w_2^2$. Verify that its derivatives with respect to f_1 and f_2 are the Lagrange multipliers u_1 and u_2 (sensitivity analysis).

Solutions 7.1

4. The Lagrange function is $L = E_1(w_1) + E_2(w_2) - u_1(w_1 - w_2 - f_1) - u_2(w_2 - f_2)$. Its first partial derivatives at the saddle point are

$$\begin{aligned} \frac{\partial L}{\partial w_1} &= \frac{dE_1}{dw}(w_1) - u_1 = 0 \\ \frac{\partial L}{\partial w_2} &= \frac{dE_2}{dw}(w_2) + u_1 - u_2 = 0 \\ \frac{\partial L}{\partial u_1} &= -(w_1 - w_2 - f) = 0 \\ \frac{\partial L}{\partial u_2} &= -(w_2 - f_2) = 0\,. \end{aligned}$$

5. $$\begin{bmatrix} C^{-1} & A \\ A^{\mathrm{T}} & 0 \end{bmatrix} = \left[\begin{array}{cc|cc} c_1^{-1} & & -1 & 0 \\ & c_2^{-1} & 1 & -1 \\ \hline -1 & 1 & & \\ 0 & -1 & & \end{array}\right]$$

With $C = I$ elimination leads to

$$\begin{bmatrix} I & A \\ 0 & -A^{\mathrm{T}}A \end{bmatrix} \quad \text{with} \quad -A^{\mathrm{T}}A = -\begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}.$$

The equation...

 ©2006 Gilbert Strang

7.2 Calculus of Variations

One theme of this book is the relation of equations to minimum principles. ***To minimize P is to solve $P' = 0$.*** There may be more to it, but that is the main point. For a quadratic $P(u) = \frac{1}{2}u^{\mathrm{T}}Ku - u^{\mathrm{T}}f$, there is no difficulty in reaching $P' = Ku - f = 0$. The matrix K is symmetric positive definite at a minimum.

In a continuous problem, the "derivative" of P is not so easy to find. The unknown $u(x)$ is a function, and $P(u)$ is usually an integral. Its derivative $\delta P/\delta u$ is called the ***first variation***. ***The "Euler-Lagrange equation" $\delta P/\delta u = 0$ has a weak form and a strong form.*** For an elastic bar, P is the integral of $\frac{1}{2}c(u'(x))^2 - f(x)u(x)$. The equation $\delta P/\delta u = 0$ is linear and the problem will have boundary conditions:

Weak form $\displaystyle\int cu'v'\,dx = \int fv\,dx$ for every v **Strong form** $-(cu')' = f(x)$.

Our goal in this section is to get beyond this first example of $\delta P/\delta u$.

The basic idea should be simple and it is: ***Perturb $u(x)$ by a test function $v(x)$***. Comparing $P(u)$ with $P(u+v)$, the linear term in the difference yields $\delta P/\delta u$. *This linear term must be zero for every admissible v (weak form).* This program carries ordinary calculus into the calculus of variations. We do it in several steps:

1. One-dimensional problems $P(u) = \int F(u, u')\,dx$, not necessarily quadratic

2. Constraints, not necessarily linear, with their Lagrange multipliers

3. Two-dimensional problems $P(u) = \iint F(u, u_x, u_y)\,dx\,dy$

4. Time-dependent equations in which $u' = du/dt$.

At each step the examples will be as familiar (and famous) as possible. In two dimensions that means Laplace's equation, and minimal surfaces in the nonlinear case. In time-dependent problems it means Newton's Laws, and relativity in the nonlinear case. In one dimension we rediscover the straight line and the circle.

This section is also the opening to ***control theory***—the modern form of the calculus of variations. Its constraints are differential equations, and Pontryagin's maximum principle yields solutions. That is a whole world of good mathematics.

Remark To go from the strong form to the weak form, *multiply by v and integrate*. For matrices the strong form is $A^{\mathrm{T}}CAu = f$. The weak form is $v^{\mathrm{T}}A^{\mathrm{T}}CAu = v^{\mathrm{T}}f$ for all v.

For functions with $Au = u'$, this exactly matches $\int cu'v'\,dx = \int fv\,dx$ above.

©2006 Gilbert Strang

One-dimensional Problems

The basic problem is to minimize $P(u)$ with a boundary condition at each end:

$$P(u) = \int_0^1 F(u, u')\, dx \quad \text{with} \quad u(0) = a \quad \text{and} \quad u(1) = b\,.$$

The best u defeats every other candidate $u+v$ that satisfies these boundary conditions. Then $(u+v)(0) = a$ and $(u+v)(1) = b$ require that $v(0) = v(1) = 0$. When v and v' are small the correction terms come from $\partial F/\partial u$ and $\partial F/\partial u'$. They don't involve v^2:

$$\textbf{Inside the integral} \qquad F(u+v, u'+v') = F(u,u') + v\frac{\partial F}{\partial u} + v'\frac{\partial F}{\partial u'} + \cdots$$

$$\textbf{After integrating} \qquad P(u+v) = P(u) + \int_0^1 \left(v\frac{\partial F}{\partial u} + v'\frac{\partial F}{\partial u'} \right) dx + \cdots$$

That integrated term is the "first variation". We have already reached $\delta P/\delta u$:

$$\textbf{Weak form} \qquad \frac{\delta P}{\delta u} = \int_0^1 \left(v\frac{\partial F}{\partial u} + v'\frac{\partial F}{\partial u'} \right) dx = 0 \quad \text{for every } v\,. \tag{1}$$

This is the equation for u. The derivative of P in each direction v must be zero. Otherwise we can make $\delta P/\delta u$ negative, which would mean $P(u+v) < P(u)$: *bad*.

The strong form looks for a single derivative which—if it is zero—makes all these directional derivatives zero. It comes from integrating $\delta P/\delta u$ by parts:

$$\textbf{Weak form / by parts} \qquad \int_0^1 \left(v\,\frac{\partial F}{\partial u} - v\frac{d}{dx}\left(\frac{\partial F}{\partial u'}\right)\right) dx + \left[v\,\frac{\partial F}{\partial u'} \right]_0^1 = 0\,.$$

The boundary term vanishes because $v(0) = v(1) = 0$. To guarantee zero for *every* $v(x)$ in the integral, the function multiplying v must be zero (**strong form**):

$$\textbf{Euler-Lagrange equation for } \boldsymbol{u} \qquad \frac{\partial F}{\partial u} - \frac{d}{dx}\left(\frac{\partial F}{\partial u'}\right) = 0\,. \tag{2}$$

Example Find the shortest path $u(x)$ between two points $(0,a)$ and $(1,b)$.

By Pythagoras, $\sqrt{(dx)^2 + (du)^2}$ is a short step on the path. So $P(u') = \int \sqrt{1+(u')^2}\, dx$ is the length of the path between the points. This square root $F(u')$ depends only on u' and $\partial F/\partial u = 0$. The derivative $\partial F/\partial u'$ brings the square root into the denominator:

$$\textbf{Weak form} \quad \frac{\partial F}{\partial u} = \int_0^1 v'\frac{u'}{\sqrt{1+(u')^2}}\, dx = 0 \quad \text{for every } v \text{ with } v(0) = v(1) = 0\,. \tag{3}$$

If the quantity multiplying v' is a constant, then (3) is satisfied. The integral is certain to be zero because $v(0) = v(1) = 0$. The strong form *forces* $\partial F/\partial u'$ to be

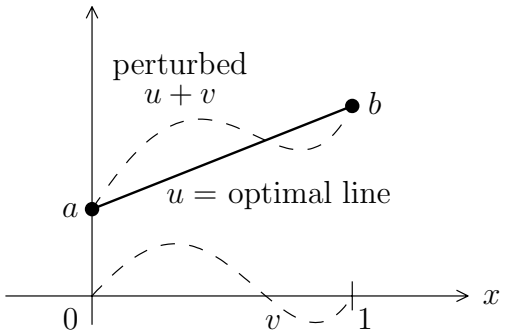
constant: the Euler-Lagrange equation (2) is

$$-\frac{d}{dx}\left(\frac{\partial F}{\partial u'}\right) = -\frac{d}{dx}\frac{u'}{\sqrt{1+(u')^2}} = 0 \quad \text{or} \quad \frac{u'}{\sqrt{1+(u')^2}} = c\,. \tag{4}$$

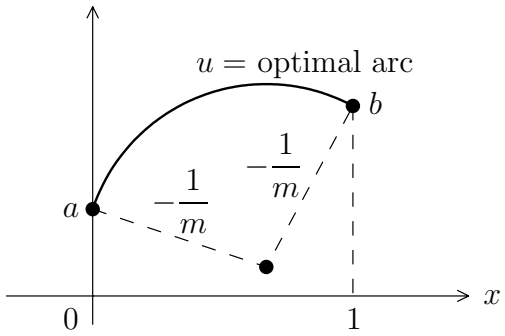
That integration is always possible when F depends only on u' ($\partial F/\partial u = 0$). It leaves the equation $\partial F/\partial u' = c$. Squaring both sides, u is seen to be *linear*:

$$(u')^2 = c^2(1+(u')2) \quad \text{and} \quad u' = \frac{c}{\sqrt{1-c^2}} \quad \text{and} \quad u = \frac{c}{\sqrt{1-c^2}}x + d\,. \tag{5}$$

replacements The constants c and d are chosen to match $u(0) = a$ and $u(1) = b$. *The shortest curve connecting two points is a straight line.* No surprise! The length $P(u)$ is a minimum, not a maximum or a saddle point, because the second derivative F'' is positive.



typical perturbation $v(x)$



area below arc is A

Figure 7.5: Shortest paths from a to b: straight line and circular arc (constrained).

Constrained Problems

Suppose we cannot go in a straight line because of a constraint. When the constraint is $\int u(x)\,dx = A$, we look for ***the shortest curve that has area A below it***:

$$\text{Minimize} \quad P(u) = \int_0^1 \sqrt{1+(u')^2}\,dx \quad \text{with} \quad u(0) = a,\ u(1) = b,\ \int_0^1 u(x)\,dx = A\,.$$

The area constraint should be built into P by a Lagrange multiplier—here called m. The multiplier is a number and not a function, because there is one overall constraint rather than a constraint at every point. The Lagrangian$L$ builds in $\int u\,dx = A$:

Lagrangian $L(u,m) = P + (\text{multiplier})(\text{constraint}) = \int (F+mu)\,dx - mA\,.$

The Euler-Lagrange equation $\delta L/\delta u = 0$ is exactly like $\delta P/\delta u = 0$ in (2):

$$\frac{\partial(F+mu)}{\partial u} - \frac{d}{dx}\left[\frac{\partial(F+mu)}{\partial u'}\right] = m - \frac{d}{dx}\frac{u'}{\sqrt{1+(u')^2}} = 0\,. \tag{6}$$

©2006 Gilbert Strang

Again this equation is favorable enough to be integrated:

$$mx - \frac{u'}{\sqrt{1+(u')^2}} = c \quad \text{which gives} \quad u' = \frac{mx-c}{\sqrt{1-(mx-c)^2}}\,.$$

After one more integration we reach the equation of a circle in the x-u plane:

$$u(x) = \frac{-1}{m}\sqrt{1-(mx-c)^2} + d \quad \text{and} \quad (mx-c)^2 + (mu-d)^2 = 1\,. \tag{7}$$

The shortest path is a circular arc! It goes high enough to enclose area A. The three numbers m, c, d are determined by the conditions $u(0) = a, u(1) = b$, and $\int u\,dx = A$. The arc is drawn in Figure 7.5 (and m is negative).

We now summarize the one-dimensional case, allowing F to depend also on u''. That introduces v'' into the weak form and needs *two* integrations by parts to reach the Euler-Lagrange equation. When F involves a varying coefficient $c(x)$, the form of the equation does not change, because it is u and not x that is perturbed.

The first variation of $\boldsymbol{P(u) = \int_0^1 F(u, u', u'', x)\,dx}$ is zero at a minimum:

Weak form
$$\frac{\delta P}{\delta u} = \int_0^1 \left(v\frac{\partial F}{\partial u} + v'\frac{\partial F}{\partial u'} + v''\frac{\partial F}{\partial u''} \right) dx = 0 \text{ for all } v.$$

The Euler-Lagrange equation from integration by parts determines $\boldsymbol{u(x)}$:

Strong form
$$\frac{\partial F}{\partial u} - \frac{d}{dx}\left(\frac{\partial F}{\partial u'}\right) + \frac{d^2}{dx^2}\left(\frac{\partial F}{\partial u''}\right) = 0\,.$$

Constraints on $\boldsymbol{u}$ bring Lagrange multipliers and saddle points of $\boldsymbol{L}$.

Applications are everywhere, and we mention one (of many) in sports. What angle is optimal in shooting a basketball? The force of the shot depends on the launch angle—line drives or sky hooks need the most push. The force is minimized at 45° if the ball leaves your hand ten feet up; for shorter people the angle is about 50°. What is interesting is that the same angle solves a second optimization problem: to have the largest margin of error and still go through the hoop.

The condition is $P' = 0$ in basketball (one shot) and $\delta P/\delta u = 0$ in track—where the strategy to minimize the time $P(u)$ has been analyzed for every distance.

Two-dimensional Problems

In two dimensions the principle is the same. The starting point is a quadratic $P(u)$, without constraints, representing the potential energy over a plane region S:

$$\textbf{Minimize} \quad P(u) = \iint_S \left[\frac{c}{2}\left(\frac{\partial u}{\partial x}\right)^2 + \frac{c}{2}\left(\frac{\partial u}{\partial y}\right)^2 - f(x,y)\,u(x,y) \right] dx\,dy\,.$$

If this energy has its minimum at $u(x,y)$, then $P(u+v) \geq P(u)$ for every $v(x,y)$. We mentally substitute $u+v$ in place of u, and look for the term that is linear in v. That term is the first variation $\delta P/\delta u$, which must be zero for every $v(x,y)$:

Weak form
$$\frac{\delta P}{\delta u} = \iint_S \left[c\frac{\partial u}{\partial x}\frac{\partial v}{\partial x} + c\frac{\partial u}{\partial y}\frac{\partial v}{\partial y} - fv \right] dx\,dy = 0\,. \tag{8}$$

This is the **equation of virtual work**. It holds for all admissible functions $v(x,y)$, and it is the weak form of Euler-Lagrange. The strong form requires as always an integration by parts (Green's formula), in which the boundary conditions take care of the boundary terms. Inside S, that integration moves derivatives away from $v(x,y)$:

Integrate by parts
$$\iint_S \left[-\frac{\partial}{\partial x}\left(c\frac{\partial u}{\partial x}\right) - \frac{\partial}{\partial y}\left(c\frac{\partial u}{\partial y}\right) - f \right] v\,dx\,dy = 0\,. \tag{9}$$

Now the strong form appears. This integral is zero for every $v(x,y)$. By the "fundamental lemma" of the calculus of variations, the term in brackets is forced to be zero everywhere:

Strong form
$$-\frac{\partial}{\partial x}\left(c\frac{\partial u}{\partial x}\right) - \frac{\partial}{\partial y}\left(c\frac{\partial u}{\partial y}\right) = f(x,y) \quad \text{throughout } S\,. \tag{10}$$

This is the Euler-Lagrange equation $A^{\mathrm{T}}CA = f$, or $-\nabla \cdot c\nabla u = f$. For constant c it is Poisson. If the y variable is removed, we are back to a one-dimensional rod.

With no extra effort we can go backwards to $P(u)$ from any linear equation:

Second-order equation
$$a\frac{\partial^2 u}{\partial x^2} + 2b\frac{\partial^2 u}{\partial x \partial y} + c\frac{\partial^2 u}{\partial y^2} = 0\,. \tag{11}$$

When a, b, and c are constant, the corresponding quadratic "energy" is

$$P(u) = \frac{1}{2}\iint \left[a\left(\frac{\partial u}{\partial x}\right)^2 + 2b\left(\frac{\partial u}{\partial x}\right)\left(\frac{\partial u}{\partial y}\right) + c\left(\frac{\partial u}{\partial y}\right)^2 \right] dx\,dy\,.$$

If we minimize P we expect to reach (11) as its Euler equation. But there is more to it than that. To *minimize* P it should be *positive definite*. Inside the integral is an ordinary 2 by 2 quadratic $au_x^2 + 2bu_xu_y + cu_y^2$. The test for positive-definiteness is still $ac > b^2$, as it was in Chapter 1. (We can make $a > 0$ in advance.) That test decides whether or not equation (11) can be solved with arbitrary boundary values on $u(x,y)$.

In this positive definite case the equation is called "elliptic" and minimization is justified. There are three fundamental classes of partial differential equations:

©2006 Gilbert Strang

The partial differential equation $au_{xx} + 2bu_{xy} + cu_{yy} = 0$ is **elliptic** or **parabolic** or **hyperbolic**, according to the matrix $\begin{bmatrix} a & b \\ b & c \end{bmatrix}$:

E	$ac > b^2$	elliptic boundary-value problems	(**steady state equations**)
P	$ac = b^2$	parabolic initial-value problems	(**heat/diffusion equations**)
H	$ac < b^2$	hyperbolic initial-value problems	(**wave/convection equations**)

Laplace's equation $u_{xx} + u_{yy} = 0$ is elliptic, with $a = c = 1$ producing the identity matrix. The heat equation $u_{xx} - u_t = 0$ is parabolic, with $b = c = 0$; the matrix is singular and its determinant is zero. That parabolic borderline between elliptic and hyperbolic needs help from lower-order terms. The wave equation $u_{xx} - u_{tt} = 0$ has $a = 1$ and $c = -1$. It asks for initial values—two conditions on part of the boundary and no conditions on another part, instead of one condition everywhere.

Here we stay with elliptic equations and minima of $P(u)$. ***The boundary conditions can specify*** u, ***or they can specify the normal component*** $w \cdot n$. Comparing (8) with (9), the boundary term comes from Green's formula in Section ____:

$$\iint_S c\nabla u \cdot \nabla v \, dx \, dy = -\iint_S (\nabla \cdot (c\nabla u))v \, dx \, dy + \int_C (c\nabla u \cdot n)v \, ds \, . \qquad (12)$$

Both sides equal $\iint fv \, dx \, dy$; that is the weak form. The first term on the right yields the strong form $-\operatorname{div}(c \operatorname{grad} u) = f(x, y)$. A zero integral over the boundary c in (12) will be achieved by the boundary conditions. Strictly speaking, the boundary conditions on $u(x, y)$ are part of the strong form.

There are two ways to make this boundary integral of $(c\nabla u \cdot n)v$ safely zero. If boundary values $u = u_0$ are given, and $u + v$ shares those values, then $v = 0$ on the boundary. That kills the integral. When u is not given and v is free, we must impose the natural boundary condition $c\nabla u \cdot n = w \cdot n = 0$.

A natural condition on w goes with A^{T}. The essential condition $u = u_0$ goes with A.

The Minimal Surface Problem

Now we are ready for ***nonlinear*** partial differential equations. The corresponding energy will not be a quadratic $P(u)$. It will be the exact energy $E(u)$, from which P originally came as an approximation. If there is a thin membrane covering the set S—like a soap bubble—then stretching this membrane requires energy. The energy is proportional to the ***surface area of the soap bubble***. In the right units the material constant will be $c = 1$, and the problem is to ***minimize the surface***

area $E(u)$:

$$\textbf{Choose } \boldsymbol{u(x,y)} \textbf{ to minimize } E(u) = \iint_S \left[1 + \left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial u}{\partial y}\right)^2\right]^{1/2} dx\,dy\,. \quad (13)$$

When $u = 0$, the bubble is flat. The expression in brackets reduces to 1. That is the minimum if $u = 0$ is admissible. But suppose the bubble is created on a piece of wire that goes around S at a varying height $u_0(x, y)$. The bubble has to stick to the wire, so the trivial solution $u = 0$ is not allowed. The bent wire imposes a boundary condition $u = u_0(x, y)$ at the edge of S, and the ***minimal surface problem*** is to find the smallest area $E(u)$. Physically, surface tension makes the area a minimum.

The test for a minimum is still $E(u) \le E(u + v)$. To compute the term $\delta E/\delta u$ that is linear in v, look at the part A from u alone, and the correction B involving v:

$$A = 1 + \left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial u}{\partial y}\right)^2 \quad \text{and} \quad B = 2\,\frac{\partial u}{\partial x}\frac{\partial v}{\partial x} + 2\,\frac{\partial u}{\partial y}\frac{\partial v}{\partial y} + O(v^2)\,.$$

For small v, the square root is $\sqrt{A + B} = \sqrt{A} + B/2\sqrt{A} + \cdots$. Integrate both sides:

$$E(u + v) = E(u) + \iint_S \frac{1}{\sqrt{A}}\left(\frac{\partial u}{\partial x}\frac{\partial v}{\partial x} + \frac{\partial u}{\partial y}\frac{\partial v}{\partial y}\right) dx\,dy + \cdots\,. \quad (14)$$

Now $\delta E/\delta u$ is exposed. It is the integral in (14) and it is zero. That is the weak form of the minimal surface equation. Because of the square root of A, it is not linear in u. (It is always linear in v; that is the whole point of the first variation!) Integrating by parts to remove the derivatives from v produces the Euler equation in its strong form:

Minimal surface equation

$$-\frac{\partial}{\partial x}\left(\frac{1}{\sqrt{A}}\frac{\partial u}{\partial x}\right) - \frac{\partial}{\partial y}\left(\frac{1}{\sqrt{A}}\frac{\partial u}{\partial y}\right) = 0\,. \quad (15)$$

This is not easy to solve, because of the square root in the denominator. For nearly flat bubbles, linearization approximates $\sqrt{A}$ by 1. *The result is Laplace's equation.* Perhaps it is only natural that the most important nonlinear equation in geometry should reduce to the most important linear equation, but still it is beautiful.

Nonlinear Equations

The shortest distance and minimal surface problems are typical of the general case. The variational problem starts with an integral $E = \iint F\,dx\,dy$. Then F depends on x and y and u and derivatives like $\partial u/\partial x$:

$$F = F(x, y, u, D_1u, D_2u, \ldots)\,.$$

For an elastic bar there was only $D_1u = \partial u/\partial x$. For a soap bubble there is also $D_2u = \partial u/\partial y$. Higher derivatives are allowed, and we can think of u itself as D_0u.

©2006 Gilbert Strang

The comparison of $E(u)$ with the nearby $E(u+v)$ starts from an ordinary expansion like $F(u+v) = F(u) + F'(u)v + O(v^2)$. When F depends on several derivatives of u, this expansion has more terms from $F(x, y, D_0u + D_0v, D_1u + D_1v, \ldots)$:

Inside the integral $$F(u+v) = F(u) + \sum \frac{\partial F}{\partial D_i u} D_i v + \cdots . \tag{16}$$

We take the derivatives of F ***with respect to*** u ***and*** u_x ***and any other*** $D_i u$.

The weak form involving v deals with those linear terms: (integral$= 0$ for all v). The strong form lifts each derivative D_i from v and puts it (as D_i^{T}) onto the part involving u:

Weak to Strong $$\iint \left(\frac{\partial F}{\partial D_i u}\right)(D_i v)\,dx\,dy \longrightarrow \iint \left[D_i^{\mathrm{T}}\left(\frac{\partial F}{\partial D_i u}\right)\right] v\,dx\,dy$$

The transpose is $D_i^{\mathrm{T}} = -D_i$ for derivatives of odd order (with an odd number of integrations by parts and minus signs). For even derivatives $D_i^{\mathrm{T}} = +D_i$.

Buried inside the calculus of variations is the real source of $A^{\mathrm{T}}CA$! The derivatives $D_i u$ give Au. Their "transposes" D_i^{T} give A^{T}. In between, C *is normally nonlinear.* But when F is a pure quadratic $\frac{1}{2}c(Du)^2$, then $D^{\mathrm{T}}\partial F/\partial Du$ becomes $D^{\mathrm{T}}(cDu)$—which is exactly the linear $A^{\mathrm{T}}CAu$ the we know so well.

7G Each problem in the calculus of variations can be stated in three forms:

Variational form Minimize $E(u) = \iint_S F(x, y, u, D_1u, D_2u, \ldots)\,dx\,dy$

Weak form $$\frac{\delta E}{\delta u} = \iint_S \left(\sum \frac{\partial F}{\partial D_i u}\right)(D_i v)\,dx\,dy = 0 \text{ for all } v$$

Euler-Lagrange strong form $$\sum D_i^{\mathrm{T}}\left(\frac{\partial F}{\partial D_i u}\right) = 0.$$

Example 1 $F = u^2 + u_x^2 + u_y^2 + u_{xx}^2 + u_{xy}^2 + u_{yy}^2 = (D_0u)^2 + \cdots + (D_5u)^2$

The derivatives of F (a pure quadratic) are $2u, 2u_x, 2u_y, \ldots, 2u_{yy}$. They are derivatives with respect to u and u_x and the other $D_i u$, *not* with respect to x!

Weak form $$2\iint [uv + u_x v_x + u_y v_y + u_{xx} v_{xx} + u_{xy} v_{xy} + u_{yy} v_{yy}]\,dx\,dy = 0\,.$$

We integrate every term by parts to give the strong form (times v):

Strong form $$2\,[u - u_{xx} - u_{yy} + u_{xxxx} + u_{xyxy} + u_{yyyy}] = 0\,.$$

This is linear because F is quadratic; the minus signs come with odd derivatives in F.

Example 2 $F = (1 + u_x^2)^{1/2}$ or $F = (1 + u_x^2 + u_y^2)^{1/2}$

The derivatives with respect to u_x and u_y bring the square root into the denominator. The shortest path equation and the minimal surface equation are

$$-\frac{d}{dx}\frac{u_x}{(1+u_x^2)^{1/2}} = 0 \quad \text{and} \quad -\frac{\partial}{\partial x}\left(\frac{u_x}{F}\right) - \frac{\partial}{\partial y}\left(\frac{u_y}{F}\right) = 0\,.$$

Every term fits into the pattern of $A^{\mathrm{T}}CA$, and **the framework becomes nonlinear**:

$$e = Au \underset{\text{nonlinear}}{\longrightarrow} w = C(e) = \frac{\partial F}{\partial e} \quad \text{leads to} \quad A^{\mathrm{T}}w = A^{\mathrm{T}}C(Au) = f\,.$$

Nonlinear $C(Au)$ from Nonquadratic Energies

That last line was worth a chapter of words. It is the shortest path to nonlinear equilibrium equations. In a linear spring, $w = ce$ is proportional to e. The internal strain energy is $F = \int ce\,de = \frac{1}{2}ce^2$. ***In a nonlinear spring the constitutive law is*** $w = C(e)$. The relation of force to stretching, or current to voltage, or flow to pressure, is no longer a straight line Ce. We need parentheses in $C(e)$!

The energy density is still $F = \int C(e)\,de =$ force times movement. Minimizing the total energy (integrate F to find E) still gives the equilibrium equation:

The energy $P(u) = \displaystyle\int [F(Au) - fu]\,dx$ **is minimized when** $A^{\mathrm{T}}C(Au) = f\,.$

The first variation of E leads to $\int [C(Au)(Av) - fv]\,dx = 0$ for every v (weak form). Then $A^{\mathrm{T}}C(Au) = f$ is the Euler equation (strong form).

For the nonlinear equivalent of positive definiteness, *the function $C(e)$ should be increasing.* The line $w = ce$ had a constant slope $c > 0$. Now that slope $C' = dC/de$ is changing—but it is still positive. That makes the energy $E(u)$ a convex function. The Euler equation is elliptic—we have a minimum.

An example is the *power law* $w = C(e) = e^{p-1}$ with $p > 1$. The energy density is its integral $F = e^p/p$. The stretching is $e = Au = du/dx$. The equilibrium equation is $A^{\mathrm{T}}C(Au) = (-d/dx)(du/dx)^{p-1} = f$. This is linear for $p = 2$. Otherwise nonlinear.

Complementary Energy

The complementary energy is a function of w instead of e. It starts with the *inverse constitutive law* $e = C^{-1}(w)$—in our example $e = w^{1/(p-1)}$. The strain e comes from the stress w; the arrow in our framework is reversed. Graphically, we are looking at Figure 7.6a from the side. ***The area under that curve is the complementary energy density*** $F^*(w) = \int C^{-1}(w)\,dw$. The twin equations come from F and F^*:

Constitutive Laws $$w = C(e) = \frac{\partial F}{\partial e} \quad \text{and} \quad e = C^{-1}(w) = \frac{\partial F^*}{\partial w}\,. \tag{17}$$

©2006 Gilbert Strang

The symmetry is perfect and the dual minimum principle applies to $Q(w) = \int F^*(w)\,dx$:

The complementary energy $Q(w)$ **is a minimum subject to** $A^{\mathrm{T}}w = f(x)$.

A Lagrange multiplier $u(x)$ takes Q to $L(w,u) = \int [F^*(w) - uA^{\mathrm{T}}w + uf]\,dx$, with the constraint $A^{\mathrm{T}}w = f$ built in. Its derivatives recover the two equations of equilibrium, now nonlinear:

$$\begin{aligned} \partial L/\partial w = 0 \quad &\text{is} \quad C^{-1}(w) - Au = 0 \\ \partial L/\partial u = 0 \quad &\text{is} \quad A^{\mathrm{T}}w \qquad\quad = f\,. \end{aligned}$$

The first equation gives $w = C(Au)$ and then the second is $A^{\mathrm{T}}C(Au) = f$.

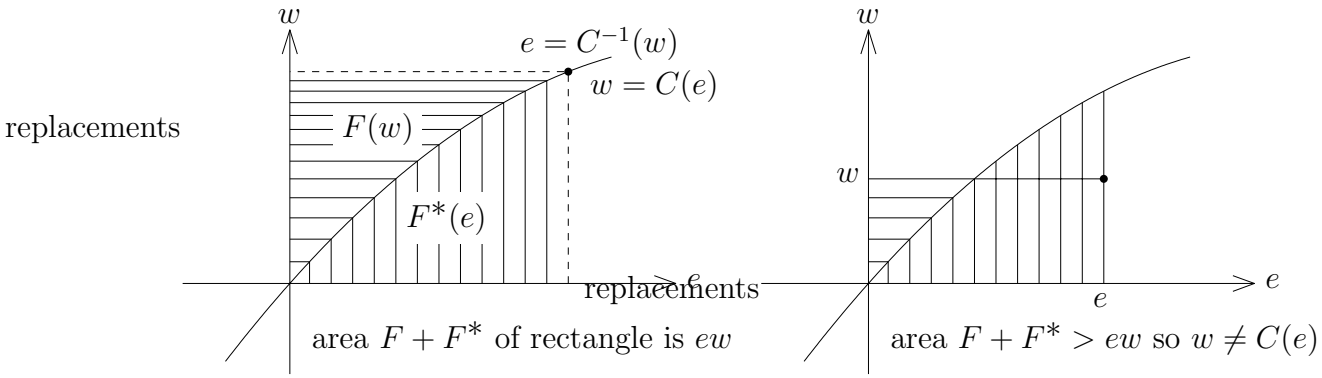


Figure 7.6: The graphs of $w = C(e)$ and $e = C^{-1}(w)$, and their areas F^* and F.

Since these nonlinear things are in front of us, why not take the last step? It is never seen in advanced calculus, but there is nothing so incredibly difficult. It is the direct link between F and F^*, known as the ***Legendre-Fenchel transform***:

$$F^*(w) = \max_e\,[ew - F(e)] \quad \text{and} \quad F(e) = \max_w\,[ew - F^*(w)]\,. \tag{18}$$

For the first maximum, differentiate with respect to e. That brings back $w = \partial F/\partial e$, which is the correct $C(e)$. The maximum itself is $F^* = e\partial F/\partial e - F$. The figures show graphically that the areas satisfy $F^* = ew - F$ on the curve and $ew - F < F^*$ off the curve. So the maximum of $ew - F$ is F^* as desired.

Similarly, the second maximum in (18) leads to $e = \partial F^*/\partial w$. That is the constitutive law in the other direction, $e = C^{-1}(w)$. The whole nonlinear theory is there, provided the material laws are conservative—the energy in the system should be constant. This conservation law seems to be destroyed by dissipation, or more spectacularly by fission, but in some ultimate picture of the universe it must remain true.

F ***will be the Lagrangian and*** F^* ***is the Hamiltonian.*** The equations $mu_{tt} + cu_t + ku = 0$ and $u_{xx} = u_t$ include friction and damping and diffusion, but we hesitate to stretch our framework that far. Feynman's wonderful lectures made "least action" the starting point for physics, and the next paragraphs.

The Legendre transform reappears at full strength in constrained optimization. There F and F^* are more general convex functions (with nonnegative second derivatives) and ***we recognize that*** F^{**} ***is*** F. Here we compute $F^*(w)$ for the power law and verify that it agrees with the integral of $C^{-1}(w)$.

Example Find $F^*(w)$ for the power law $F(e) = e^p/p$ ($e > 0$ and $w > 0$ and $p > 1$)

Differentiating $ew - F(e)$ gives $w - e^{p-1} = 0$. Then the conjugate $F^*(w)$ in (18) is w^q/q:

$$F^* = ew - \frac{1}{p}e^p = w^{1/(p-1)}w - \frac{1}{p}w^{p/(p-1)} = \frac{p-1}{p}w^{p/(p-1)} = \frac{1}{q}w^q\,.$$

$F^*(w) = w^q/q$ is also a power law, with *dual exponent* $q = p/(p-1)$. This matches the area under $C^{-1}(w) = w^{1/(p-1)}$, because integration will increase that exponent to $1 + 1/(p-1) = q$. The symmetric relation between the powers is $p^{-1} + q^{-1} = 1$.

Dynamics and Least Action

Fortunately or unfortunately, the world is not in equilibrium. The energy stored in springs and beams and nuclei and people is waiting to be released. When the external forces change, the equilibrium is destroyed. Potential energy is converted to kinetic energy, the system becomes dynamic, and it may or may not find a new steady state.

When the system is conservative, the transients will not grow or decay. The energy changes from potential to kinetic to potential to kinetic, but the total energy $K + P$ remains constant. It is like the earth around the sun or a child on a frictionless swing. The force dP/du is no longer zero, and the system oscillates. The problems are dynamic instead of static.

To describe the motion we need an equation or a variational principle. Numerically we mostly work with equations (Newton's laws and conservation laws). This section derives those laws from the ***principle of least action***:

The actual path $u(t)$ minimizes the action integral $A(u)$ between the initial state $u(t_0)$ and the final state $u(t_1)$:

$$A(u) = \int_{t_0}^{t_1} (\textbf{kinetic energy} - \textbf{potential energy})\, dt\,.$$

It is better to claim only that $\delta A/\delta u = 0$—the path is always a stationary point but not in every circumstance a minimum. We have a difference of energies, and positive definiteness can be lost (a saddle point). Laplace's equation will be overtaken by the wave equation. First come three examples to show how the ***global law*** of least action (the variational principle of least action) produces Newton's ***local law*** $F = ma$.

Example 3 A ball of mass m is attracted by the Earth's gravity

©2006 Gilbert Strang

The only degree of freedom is the ball's height $u(t)$. The energies are K and P:

$$K = \textbf{kinetic energy} = \frac{1}{2}m\left(\frac{du}{dt}\right)^2 \quad \text{and} \quad P = \textbf{potential energy} = mgu\,.$$

The action is $A = \int(\frac{1}{2}m(u')^2 - mgu)\,dt$. Then $\delta A/\delta u$ follows from the rules of this section—with the time variable t replacing the space variable x. The true path u is compared to its neighbors $u + v$. The linear part of $A(u+v) - A(u)$ gives the first variation $\delta A/\delta u = 0$:

$$\frac{\delta A}{\delta u} = \int_{t_0}^{t_1}(mu'v' - mgv)\,dt = 0 \quad \text{for every } v\,.$$

That is the weak form of Newton's law. You recognize the momentum mu' as the derivative of $\frac{1}{2}m(u')^2$ with respect to the velocity u'. The strong form is the Euler equation:

$$\textbf{Newton's law} \qquad -\frac{d}{dt}\left(m\frac{du}{dt}\right) - mg = 0 \quad \text{which is} \quad ma = F\,. \tag{19}$$

The action integral is minimized by the path that follows Newton's law.

The 3-step framework of applied mathematics is not changed. The place of A is taken by d/dt and A^{T} is $-d/dt$. The material constant is m and the external force is $f = mg$. The balance between $A^{\mathrm{T}}CAu$ and f is Newton's law—a balance of **inertial forces** instead of mechanical forces. Figure 7.7 identifies the variables as the velocity and momentum.

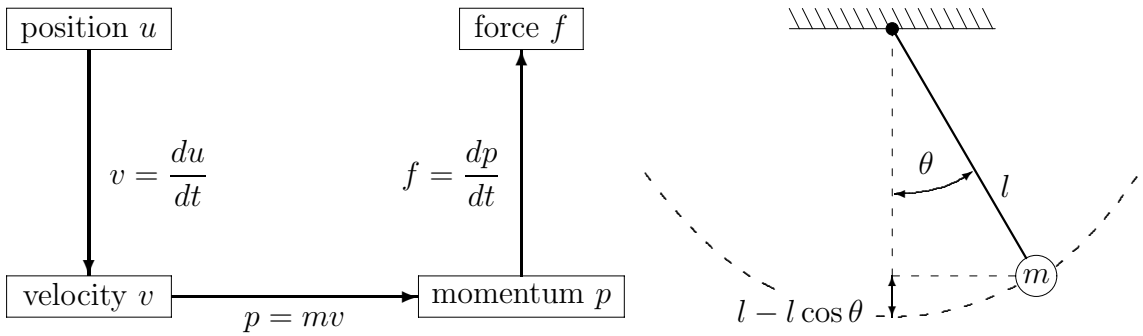


Figure 7.7: Newton's law for a ball. A pendulum also fits the framework.

Example 4 A simple pendulum with mass m and length l

The state variable u is the angle θ from the vertical. The height $l - l\cos\theta$ still enters the potential energy and the velocity is $v = l\,d\theta/dt$:

$$K = \textbf{kinetic energy} = \frac{1}{2}ml^2\left(\frac{d\theta}{dt}\right)^2 \quad \text{and} \quad P = \textbf{potential energy} = mg(l - l\cos\theta)$$

In this problem the equation will no longer be linear. K is quadratic but P involves $\cos\theta$. The Euler equation follows the rule for an integral $\int F(\theta,\theta')\,dt$, with $F = K - P$:

Euler equation $$\frac{\partial F}{\partial \theta} - \frac{d}{dt}\left(\frac{\partial F}{\partial \theta'}\right) = 0 \quad \text{or} \quad -mgl\sin\theta - \frac{d}{dt}\left(ml^2\frac{d\theta}{dt}\right) = 0\,.$$

This is the equation of a simple pendulum. The mass cancels out; clocks keep time!

Pendulum equation $$\frac{d^2\theta}{dt^2} + \frac{g}{l}\sin\theta = 0\,. \tag{20}$$

When the angle is small and $\sin\theta$ is approximated by θ, the equation becomes linear. The period changes a little. A linear clock keeps time, but not the right time.

Example 5 A vertical line of springs and masses has $Mu'' + Ku = 0$

The potential energy in the springs is $P = \frac{1}{2}u^{\mathrm{T}}A^{\mathrm{T}}CAu$. The kinetic energy has $\frac{1}{2}m_i(du_i/dt)^2$ from each mass. Then $K - P$ goes into the action integral, and there is an Euler equation $\delta A/\delta u_i = 0$ for each mass. In matrix form this is the basic equation of mechanical engineering:

Undamped oscillation $$Mu'' + A^{\mathrm{T}}CAu = 0\,.$$

M is the diagonal mass matrix and $A^{\mathrm{T}}CA$ is the positive definite stiffness matrix. The system oscillates around equilibrium and the total energy $K + P$ is constant.

Example 6 Waves in an elastic bar—*a continuum of masses and springs*.

The action integral in this continuous case has an integral instead of a sum:

Action $$A(u) = \int_{t_0}^{t_1}\int_{x=0}^{1}\left[\frac{1}{2}m\left(\frac{du}{dt}\right)^2 - \frac{1}{2}c\left(\frac{du}{dx}\right)^2\right]dx\,dt\,.$$

The Euler-Lagrange rules for $\delta A/\delta u = 0$ cover this case of a double integral:

Wave equation $$\frac{\delta A}{\delta u} = -\frac{\partial}{\partial t}\left(m\frac{\partial u}{\partial t}\right) - \frac{\partial}{\partial x}\left(-c\frac{\partial u}{\partial x}\right) = 0\,.$$

That is the **wave equation** $mu_{tt} = cu_{xx}$. With constant density m and elastic constant c, the wave speed is $\sqrt{c/m}$—faster when the bar is stiffer and lighter.

Staying with the calculus of variations, there are two important comments:

1. When u is independent of time (no motion), the kinetic energy is $K = 0$ and the least action principle reduces to $\delta P/\delta u = 0$. The dynamic problem goes back to the static problem of $A^{\mathrm{T}}CAu = f$, without a time derivative.

©2006 Gilbert Strang

2. The dual variable w, from all the evidence, should be found in its usual place in Figure 7.7. It is the ***momentum*** $p = mv = m\,du/dt$. We can rewrite K:

$$\textbf{The kinetic energy} \quad K = \frac{1}{2}mv^2 \quad \textbf{becomes} \quad \frac{1}{2m}p^2\,.$$

That is the "complementary" kinetic energy, expressed in terms of p instead of v. Note that m moves to the denominator, just as c did in the elastic energy $w^2/2c$. Hamilton found the right total energy, using the momentum p and the displacement u. Those variables (not the velocity!) are at the primal and dual corners of our framework:

$$\textbf{Hamiltonian} \qquad H = K + P = \frac{1}{2m}p^2 + mgu\,. \tag{21}$$

The Hamiltonian $H(p,u)$ takes us directly to the equations of motion:

$$\textbf{Hamilton's equations} \qquad \frac{\partial H}{\partial p} = \frac{du}{dt} \quad \text{and} \quad \frac{\partial H}{\partial u} = -\frac{dp}{dt}\,. \tag{22}$$

The derivative dH/dt is zero so *the total energy is constant*:

$$\textbf{Chain rule} \qquad \frac{dH}{dt} = \frac{\partial H}{\partial p}\frac{dp}{dt} + \frac{dH}{du}\frac{du}{dt} = u'p' - p'u' = 0\,. \tag{23}$$

This is the essence of classical mechanics. It is tied to Hamilton and not to Newton. For that reason it survived the revolution brought by Einstein. We will see that H has a relativistic form and even a quantum mechanical form. Comparing a falling ball with an oscillating spring, the key difference in H is between u and u^2:

$$(\textbf{ball}) \quad H = \frac{1}{2m}p^2 + mgu \qquad (\textbf{spring}) \quad H = \frac{1}{2m}p^2 + \frac{1}{2}cu^2$$

Hamilton's equations $\partial H/\partial p = u'$ and $\partial H/\partial u = -p'$ yield Newton's Laws:

$$(\textbf{ball}) \qquad \frac{p}{m} = u' \quad \text{and} \quad mg = -p', \quad \text{or} \quad mu'' = -mg$$

$$(\textbf{spring}) \qquad \frac{p}{m} = u' \quad \text{and} \quad cu = -p', \quad \text{or} \quad mu'' + cu = 0\,.$$

The mass passes through equilibrium at top speed (all energy in K). The force reverses to slow it down and stop it (all energy in P). In the $u-p$ plane (*the phase plane*) the motion stays on the energy surface $H =$ constant, which is the ellipse in Fig. 7.8. Each oscillation of the spring is a trip around the ellipse.

With more springs there are $2n$ axes $u_1, \dots, u_n, p_1, \dots, p_n$ and the ellipse becomes an ellipsoid. Hamilton's equations are $\partial H/\partial p_i = du_i/dt$ and $\partial H/\partial u_i = -dp_i/dt$. They lead again to $Mu'' + Ku = 0$, and to the wave equation in the continuous case.

©2006 Gilbert Strang

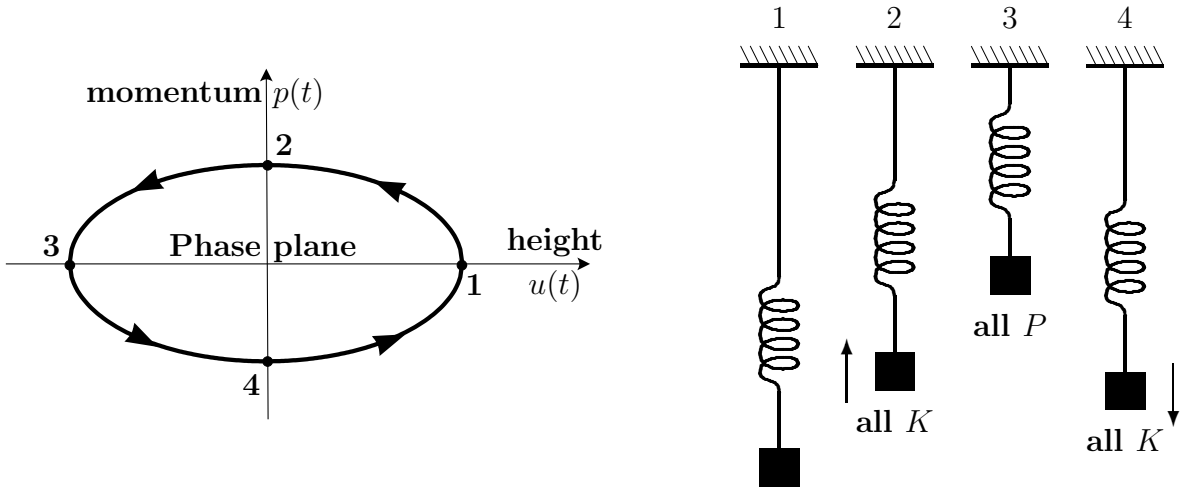


Figure 7.8: **1** tension **2** motion **3** compression **4** motion: Constant energy $H = K + P$.

Relativity and Quantum Mechanics

These few paragraphs are an attempt, by a total amateur, to correct the action integral by the rules of relativity. Feynman's lectures propose the term $-mc^2\sqrt{1-(v/c)^2}$ in the Lagrangian $K - P$. At $v = 0$ we see Einstein's formula $e = mc^2$ for the energy in a mass m at rest. It becomes part of the potential energy P.

As the velocity increases from zero there is also a part corresponding to K. For small x the square root of $1-x$ is approximately $1-\frac{1}{2}x$, which linearizes the problem and brings back Newton's $\frac{1}{2}mv^2$—just as linearizing the minimal surface equation brought back Laplace. Relativity mixes together K and P!

Lagrangian $$F(v) = -mc^2\sqrt{1-(v/c)^2} \approx -mc^2\left(1 - \frac{1}{2}\frac{v^2}{c^2}\right).$$

Now, trusting in duality, we look for the conjugate function F^* in the Hamiltonian. It will be a function of p, not v. The first step is to find that momentum p. Before relativity, F was $\frac{1}{2}mv^2$ and its derivative was $p = mv$. Always p is $\partial F/\partial v$:

Momentum $$p = \frac{\partial F}{\partial v} = \frac{mv}{\sqrt{1-(v/c)^2}}. \tag{24}$$

This becomes infinite as v approaches the speed of light. According to (18), F^* is

Hamiltonian $$F^*(p) = \max_v \left[pv - F(v)\right] = mc^2\sqrt{1+(p/mc)^2}\,. \tag{25}$$

This maximum occurs at (24)—which we solved for v and substituted into $pv - F(v)$.

The new Hamiltonian F^* is again Einstein's energy $e = mc^2$ when the system is at rest. As p increases from zero, the next contribution is Newton's $p^2/2m$:

$$F^* \approx mc^2\left(1 + \frac{1}{2}\frac{p^2}{m^2c^2}\right) = \text{ rest energy } + \frac{p^2}{2m}.$$

©2006 Gilbert Strang

Newton found the low-order term in the energy F^* that Einstein computed exactly! Perhaps the universe is like a minimal surface in space-time. To Laplace and Newton it looked flat (linearized), but to Einstein it became curved.

It is risky to add anything about quantum mechanics, which works with probabilities. It is a mixture of differential equations (Schrödinger) and matrix equations (Heisenberg). The event at which $\delta A/\delta u = 0$ almost always occurs. Feynman gave each possible trajectory of the system a phase factor $e^{iA/h}$ multiplying its probability amplitude. The small number h (Planck's constant) means that a slight change in the action A completely alters the phase. There are strong canceling effects from nearby paths unless *the phase is stationary*. In other words $\delta A/\delta u = 0$ at the most probable path.

This prediction of "stationary phase" applies equally to light rays and particles. Optics follows the same principles as mechanics, and light travels by the fastest route: ***least action becomes least time***. If Planck's constant could go to zero, the deterministic principles of least action and least time would appear and the path of least action would be not only probable but certain.

Problem Set 7.2

1 What are the weak form and the strong form of the linear beam equation—the Euler equation for $P = \int [\frac{1}{2}c(u'')^2 - fu]\,dx$?

2 Minimizing $P = \int (u')^2 dx$ with $u(0) = a$ and $u(1) = b$ also leads to the straight line through these points. Write down the weak form and the strong form.

3 Find the Euler equations (strong form) for

$$\text{(a)} \int [(u')^2 + e^u]\,dx \qquad \text{(b)} \int uu'\,dx \qquad \text{(c)} \int x^2(u')^2 dx$$

4 If $F(u, u')$ is independent of x, as in almost all our examples, show from the Euler equation and the chain rule that $H = u'\partial F/\partial u' - F$ is constant. This is dual to the fact that $\partial F/\partial u'$ is constant when F is independent of u.

5 If the speed is x the travel time of a light ray is

$$T = \int_0^1 \frac{1}{x}\sqrt{1 + (u')^2}\,dx \quad \text{with} \quad u(0) = 0 \quad \text{and} \quad u(1) = 1\,.$$

(a) From equation $\delta T/\delta u = 0$ what quantity is constant (Snell's law)?

(b) Can you integrate once more to find the optimal path $u(x)$?

6 With the constraints $u(0) = u(1) = 0$ and $\int u\,dx = A$, show that the minimum value of $P = \int (u')^2 dx$ is $12A^2$. Introduce a multiplier m, solve the Euler equation for u, and verify that $A = -m/24$. Then the derivative $dP/dA = 24A$ equals the multiplier $-m$ as the sensitivity theory predicts.

 ©2006 Gilbert Strang

7 For the shortest path constrained by $\int u\,dx = A$, what is unusual about the solution in Fig. 7.5 as A becomes large?

8 Suppose the constraint is $\int u\,dx \geq A$, with inequality allowed. Why does the solution remain a straight line as A becomes small? Where does the multiplier m remain? This is typical of inequality constraints: either the Euler equation is satisfied or the multiplier is zero.

9 Suppose the constrained problem is reversed, and we *maximize* the area $P = \int u\,dx$ subject to fixed length $I = \int \sqrt{1+(u')^2}\,dx$, with $u(0) = a$ and $u(1) = b$.

(a) Form the Lagrangian and solve its Euler equation for u.

(b) How is the multiplier M related to m in the text?

(c) When do the constraints eliminate all functions u?

10 Find by calculus the shortest broken-line path between $(0,1)$ and $(1,1)$ that goes first to the horizontal axis $y = 0$ and bounces back. Show that the best path treats this axis like a mirror: angle of incidence = angle of reflection.

11 The principle of maximum entropy selects the probability distribution that maximizes $H = -\int u \log u\,dx$. Introduce Lagrange multipliers for the constraints $\int u\,dx = 1$ and $\int xu\,dx = 1/a$, and find by differentiation an equation for u. On the interval $0 < x < \infty$ show that the most likely distribution is $u = ae^{-ax}$.

12 If the second moment $\int x^2 u\,dx$ is also known show that Gauss wins again: the maximizing u is the exponential of a quadratic. If only $\int u\,dx = 1$ is known, the most likely distribution is $u =$ constant. The *least* information comes when only one outcome is possible, say $u(6) = 1$, since $u \log u$ is then identically zero.

13 A helix climbs around a cylinder with $x = \cos\theta, y = \sin\theta, z = u(\theta)$:

$$\text{its length is} \quad L = \int \sqrt{dx^2 + dy^2 + dz^2} = \int \sqrt{1+(u')^2}\,d\theta\,.$$

Show that $u' =$ constant satisfies Euler's equation. The helix is regular.

14 Multiply the nonlinear equation $-u'' + \sin u = 0$ by v and integrate the first term by parts to find the weak form. What integral P is minimized by u?

15 Find the Euler equations (strong form) for

(a) $P(u) = \dfrac{1}{2}\displaystyle\iint \left[\left(\frac{\partial^2 u}{\partial x^2}\right)^2 + 2\left(\frac{\partial^2 u}{\partial x \partial y}\right)^2 + \left(\frac{\partial^2 u}{\partial y^2}\right)^2\right] dx\,dy$

(b) $P(u) = \dfrac{1}{2}\displaystyle\iint (yu_x^2 + u_y^2)\,dx\,dy$ (c) $E(u) = \displaystyle\int u\sqrt{1+(u')^2}\,dx$

(d) $P(u) = \dfrac{1}{2}\displaystyle\iint (u_x^2 + u_y^2)\,dx\,dy$ with $\displaystyle\iint u^2\,dx\,dy = 1.$

©2006 Gilbert Strang

16 Show that the Euler equations for these integrals are the same:

$$\iint \frac{\partial^2 u}{\partial x^2}\frac{\partial^2 u}{\partial y^2}\,dx\,dy \quad \text{and} \quad \iint \left(\frac{\partial^2 u}{\partial x \partial y}\right)^2 dx\,dy$$

Presumably the two integrals are equal if the boundary conditions are zero.

17 Sketch the graph of $p^2/2m + mgu = \text{constant}$ in the $u-p$ plane. Is it an ellipse, parabola, or hyperbola? Mark the point where the ball reaches maximum height and begins to fall.

18 Draw a second spring and mass hanging from the first. If the masses are m_1, m_2 and the spring constants are c_1, c_2, the energy is

$$H = K + P = \frac{1}{2m_1}p_1^2 + \frac{1}{2m_2}p_2^2 + \frac{1}{2}c_1u_1^2 + \frac{1}{2}c_2(u_2-u_1)^2\,.$$

Find the four Hamilton's equations $\partial H/\partial p_i = du_i/dt, \partial H/\partial u_i = -dp_i/dt$, and the matrix equation $Mu'' + Ku = 0$.

19 The Hamiltonian for a pendulum (with $u = \theta$) is $H = p^2/2m + mgl(1-\cos u)$. Write out Hamilton's equations (22) and eliminate p to find the equation of a pendulum.

20 Verify that the energy $\frac{1}{2}e^{\mathrm{T}}Ce$ and the complementary energy $\frac{1}{2}w^{\mathrm{T}}C^{-1}w$ are conjugate. As in equation (18), this means that $\frac{1}{2}w^{\mathrm{T}}C^{-1}w = \max[e^{\mathrm{T}}w - \frac{1}{2}e^{\mathrm{T}}Ce]$.